

Modern C++ 工程学习手册

Modern C++ Engineering Handbook

CPP-HANDBOOK

v1.0.0

PUBLICATION EDITION

权威内容来源为 Markdown。本件为 v1.0.0 正式阅读版。发布身份与校验摘要见随附发布记录。

阅读与版本说明

PUBLICATION EDITION · 正式阅读版 · v1.0.0

本件从已接受的 Markdown 冻结提交取源。正文中的“待审核”保留提交时的历史文字；内容接受不等于 PDF 或正式发布批准。标题、章节编号、代码与 Gate 不因出版改写。封面、目录和代码续页标签是生成的阅读辅助。

制品标识 CPP-HANDBOOK

版本 1.0.0

状态 PUBLICATION EDITION

阅读密度 balanced · Visual Profile v1.0, 已冻结；出版版本 v1.0.0

基准 **commit** 8f479deaf660533b2ad82e1f721eb41a363112b6

准备标识 8bbde993ed7cb74ad676ce2e

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。完整输入以准备记录为准，可能包含未提交变更；正文原有编号不重新连续编号。

G0 v1.2.1 ACCEPTED CONTENT

G1 v1.2.1 ACCEPTED CONTENT

G2 v1.2.1 ACCEPTED CONTENT

G3 v1.2.1 ACCEPTED CONTENT

G4 v1.2.1 ACCEPTED CONTENT

G5 v1.1.1 ACCEPTED CONTENT

G6 v1.1.1 ACCEPTED CONTENT

G7 v1.1.1 ACCEPTED CONTENT

G8 v1.1.1 ACCEPTED CONTENT

G9 v1.1.1 ACCEPTED CONTENT

G10 v1.1.1 ACCEPTED CONTENT

G11 v1.1.1 ACCEPTED CONTENT

G12 v1.1.1 ACCEPTED CONTENT

目录

阅读与版本说明	2
G0 · 原生工具链与机器边界	12
阅读入口	12
1. 从三个文件理解分离编译	12
2. 实验 G0-L1: 缺少的定义在哪里	13
3. 五层诊断: 先找失败的阶段	14
4. 目标文件、库与加载器	15
5. API、ABI 与跨语言边界	15
6. 进程、栈与寄存器	16
7. Final Gate	18
8. Final Gate · 参考答案与常见误判	19
G1 · 对象模型与生命周期	20
阅读入口	20
1. 同一个地址, 不等于同一个活对象	20
2. 指针、引用与别名	21
3. 数组、退化与借用	22
4. 实验: 对象可以先于存储结束	23
5. 表达式值类别: 先看表达式, 不给对象贴标签	24
6. 跨语言回查与检查工具	25
7. Final Gate	26
8. Final Gate · 参考答案与常见误判	27
G2 · RAII、资源与所有权	28
阅读入口	28
1. 资源与清理模型	29
2. 独占所有权与值成员	31
3. 接口与所有权合同	36
4. 特殊成员与失败安全	39
5. 实验 · G2-L1: 清理不是只覆盖快乐路径	44
6. 回滚与共享所有权	45

7. 实验 · G2-L2: 弱观察不持有资源, lock 暂时持有	49
8. 共享所有权的适用边界	50
9. 异步边界与停机	51
10. 架构与跨语言回查	55
11. 工程审查与常见误判	57
12. 术语与统一模型	62
13. Final Gate	64
14. Final Gate · 参考答案与常见误判	66
15. 工程原则回查	66
16. G2 → G3	67
G3 · 值语义、移动与成本模型	68
阅读入口	68
1. 逻辑值、表示与复制成本	69
2. 实验 · G3-L1: 复制的是完整值, 还是访问关系?	71
3. 平凡复制与对象表示	72
4. 移动、异常保证与复制消除	73
5. 实验 · G3-L2 ~ L4: 转换、重载与结果对象	78
6. 返回值、参数与借用边界	80
7. 容器迁移与稳定身份	86
8. 对象表示策略	90
9. 值导向架构与生命周期解耦	94
10. 成本图与接口决策	97
11. 跨语言回查	101
12. 工程审查与常见误判	102
13. 术语与统一模型	106
14. Final Gate	109
15. Final Gate · 参考答案与常见误判	111
16. 工程原则回查	111
17. G3 → G4	112
G4 · STL、Ranges 与数据抽象	113
阅读入口	113
1. 标准库的抽象模型	114

2. 序列容器与失效	115
3. 连续、多维与字符视图	120
4. 迭代器、算法与 Ranges	126
5. 关联表示与容器选择	133
6. 状态建模	140
7. SignalBatch: 完整组件实验	142
8. 数据修改与 API 边界	145
9. 内存资源与性能决策	151
10. 实验与观察	154
11. 工程审查与常见误判	158
12. 跨语言回查	161
13. 工程原则回查	163
14. Final Gate	165
15. Final Gate · 参考答案与常见误判	167
16. 统一模型	167
17. 接下来怎么用	169
G5 · 泛型、类型推导与编译期抽象	170
阅读入口	170
1. 泛型源代码与实例化模型	171
2. 类型推导: 实参与形参	172
3. 引用折叠、转发与消费边界	173
4. 类模板与编译期状态	177
5. 约束与语义合同	179
6. 常量求值与编译期分支	181
7. 特化与运行时分派	183
8. 编译模型与实例化控制	184
9. 泛型边界与工程预算	185
10. 跨语言回查	189
11. 工程审查与反模式	190
12. 统一模型与术语	192
13. 实验与验证	194
14. Final Gate	197

15. Final Gate · 参考答案与常见误判	199
16. 工程原则回查	200
17. 参考与验证入口	200
G6 · 内存布局、机器成本与性能测量	201
阅读入口	201
1. 阅读模型	202
2. 对象布局	203
3. 缓存与局部性	206
4. 数据布局: AoS、SoA 与 AoSoA	210
5. 分配、区域与内存资源	213
6. 虚拟内存与地址翻译	217
7. 分支预测与控制流	220
8. 缓存一致性与伪共享	223
9. 测量、剖析与证据	225
10. 统一性能审查	229
11. 跨语言回查	232
12. 术语回查	233
13. 实验与验证	234
14. Final Gate	239
15. Final Gate · 参考答案与常见误判	242
16. 工程原则回查	243
17. G1 ~ G6 的统一系统模型	244
18. 从机器观察转向语言并发模型	245
19. 参考与验证入口	245
G7 · 并发协议与 C++ 内存模型	246
阅读入口	246
1. 阅读模型	247
2. 数据竞争与语言内存模型	247
3. 顺序与同步关系	249
4. 原子性与内存序	251
5. 互斥锁与条件变量	254
6. 原子状态转换与 CAS	257

7. ABA、进展保证与安全回收	258
8. 并发队列的交接协议	261
9. 线程生命周期与停机	263
10. 任务执行、线程池与背压	266
11. 并发架构与写入权限	268
12. 跨层关系图	271
13. 统一并发审查	271
14. 并发反模式	273
15. 工程原则回查	275
16. 实验与验证	276
17. Final Gate	280
18. Final Gate · 参考答案与常见误判	283
19. G0 ~ G7 的统一模型	284
20. 实际代码的跨层审查	285
21. 本章范围与后续阅读	286
22. 参考与验证入口	286
G8 · ABI、原生库与 C 互操作	287
阅读入口	287
1. API 与二进制合同	288
2. 符号、链接属性与 C 入口	289
3. 调用约定与数据模型	289
4. 对象布局、PImpl 与继承	290
5. RTTI 与异常边界	291
6. 分配域、标准库与不透明句柄	292
7. 结构版本与存储表示	292
8. 可见性、库与插件协商	293
9. 兼容性变更与构建配置	295
10. C 接口设计规则与错误输出	295
11. 回调、并发与销毁	296
12. 包装层、跨语言与借用	297
13. 二进制检查与语义合同	298
14. 库接口的分层实现	299

15. 兼容策略与边界分级	299
16. 接口审查清单	300
17. 实践路线与验证选择	300
18. 常见误判	300
19. C++、Zig 与 Rust 对照	301
20. 完整实验：符号与 C 二进制接口	301
21. 边界复核协议	304
22. Final Gate	306
23. Final Gate · 参考答案与常见误判	308
24. 工程原则与全链回查	310
25. 参考资料与验证边界	311
G9 · 构建、打包与原生生态	312
阅读入口	312
1. 构建图与三棵目录树	313
2. Target 与依赖传播	313
3. 产物类型、语言要求与文件集	314
4. 构建阶段与生成器	315
5. 预设、工具链与交叉编译	316
6. 依赖发现与供应策略	317
7. 安装、导出与包配置	318
8. 代码生成与模块依赖	319
9. 构建优化、告警与检测配置	319
10. 开发工具、测试与加载路径	320
11. 项目组织与安装库示例	321
12. 构建反模式	322
13. 可复现性、矩阵与构建成本	322
14. 工程配置的五层职责	323
15. 实践路线与验证选择	323
16. 常见误判	324
17. C++、Zig 与 Rust 生态对照	324
18. 完整实验：安装消费与生成依赖	324
19. 构建复核协议	329

20. Final Gate	331
21. Final Gate · 参考答案与常见误判	333
22. 工程原则与全链回查	335
23. 参考资料与验证边界	336
G10 · 系统运行时工程	337
阅读入口	337
1. 项目合同：从语言机制到有界数据流	338
2. 不变量与生命周期状态	338
3. 载荷、借用与工作线程私有存储	339
4. 有界队列的状态与线性化点	340
5. 工作线程、Sink 与 happens-before 论证	340
6. 启动、接收与错误模型	341
7. 背压预算与关闭协议	341
8. 指标与时延的测量口径	342
9. 表示演进、PImpl 与 C 边界	343
10. 工程布局与构建配置	343
11. 验证按协议命题组织	344
12. 从性能观察到 profile 假设	344
13. 优化阶梯与配置发布	345
14. 可观察性、异常保证与交付	345
15. 实现阶段与工程记录	346
16. 常见架构误判及其反例	346
17. 验收层次与本章的完成含义	346
18. 完整实验：运行时协议、安装与观察	347
19. 跨组件复核记录	363
20. Final Gate	364
21. Final Gate · 参考答案与常见误判	366
22. 参考资料与验证边界	368
G11 · 机器人系统工程	369
阅读入口	369
1. 物理闭环与实时合同	370
2. 时间域、数据年龄与控制步长	370

3. 调度、干扰与优先级反转	371
4. 内存阶段与故障表示	371
5. 单位、坐标系与数值表示	372
6. 传感器采集、通道语义与快照回收	372
7. 估计、控制与硬件边界	373
8. 看门狗、故障锁存与命令仲裁	373
9. ROS 2、QoS 与借出缓冲区	374
10. 大载荷、配置事务与安全过滤	375
11. 故障隔离、观测与测试阶梯	375
12. 执行等级与集成拓扑	376
13. 输出不变量与工程分层	376
14. 可运行学习案例的范围	377
15. 常见误判与语言选择	377
16. 完整实验：逻辑仿真、快照与时序	378
17. 机器人路径复核记录	384
18. Final Gate	386
19. Final Gate · 参考答案与常见误判	388
20. 参考资料与验证边界	390
G12 · 统一系统模型与工程审查	391
阅读入口	391
1. 全书的终点：分配证明责任	392
2. 对象、存储与清理边界	392
3. 所有权、借用、别名与值传递	393
4. 分配与失败：机制不等于事务	393
5. 泛型、分派与容器抽象	394
6. 机器成本与并发协议	394
7. 二进制与构建：保证到哪里结束	395
8. 编译期元数据与外部生成	396
9. 时限、嵌入式与性能证据	396
10. 安全抽象、状态与 API 合同	397
11. 子系统选择与跨语言成本	397
12. 可迁移的系统审查协议	398

13. 四类项目的对照推理	398
14. 长期维护与常见错误推论	399
15. 三类论证与六张关系图	399
16. 跨章应用与语言反思	400
17. 可带入新项目的最终检查	400
18. 审查练习：把同一套模型用于三个案例	401
19. 全书回查与证据分层	402
20. Final Gate	403
21. Final Gate · 参考答案与常见误判	405
22. 参考资料与验证边界	407

G0 · 原生工具链与机器边界

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.2.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。主实验适用 Clang / Apple Clang；macOS / Apple Silicon 机器观察不代表通用 ABI。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

全系列导航 · 下一章：G1

阅读入口

本章围绕一个问题：为什么一份源码能编译成目标文件，却不能链接成程序？

读前只需能看懂函数声明和一次调用。首次读第 1~3 节并完成 G0-L1；第 4~6 节是二进制与机器边界的回查材料。学完应能指出错误发生在哪一阶段，以及下一步检查哪个工件，而不是笼统地说“编译器坏了”。

- 1. 从三个文件理解分离编译
- 2. 实验 G0-L1：缺少的定义在哪里
- 3. 五层诊断
- 4. 目标文件、库与加载器
- 5. API、ABI 与跨语言边界
- 6. 进程、栈与寄存器
- 7. Final Gate

1. 从三个文件理解分离编译

1.1 问题：编译器没看到函数体，凭什么生成调用？

普通、非模板函数的声明 `int add(int, int);` 足够让当前翻译单元（translation unit, TU）检查调用的参数和结果类型。编译器按照目标应用二进制接口（application binary interface, ABI）约定生成调用所需信息，把尚未解决的符号交给链接阶段。它不需要此刻读到另一个 `.cpp` 的函数体。

例如，把声明放入 `math.hpp`，实现放入 `math.cpp`，调用放入 `main.cpp`：

```

main.cpp + math.hpp → main.o ┐
                             └─ linker → executable → loader → process
math.cpp + math.hpp → math.o ┘

```

这是传统头文件模型，不是 Modules 的完整说明。`#include` 是文本包含；预处理结果组成翻译单元 (TU)。编译每个 TU 时，不能假定编译器天然掌握其他 TU 的定义。

1.2 为什么实现文件也包含自己的头文件？

如果调用方手写一份声明，实现方又写另一份，二者可能不一致。让 `math.cpp` 也包含 `math.hpp`，能让同一 TU 内的声明与定义接受类型检查。头文件提供共同声明，**不是保证 ABI 永不变化的证书**。

声明描述实体；定义完成相应实体的定义。不要把“定义”机械理解成“一定生成一段机器码”：内联、常量求值、优化和未使用代码都可能影响实际工件。模板、`inline` 函数以及完整类型的需求也不能套用“只见声明就够”的普通函数例子。

1.3 编译器驱动器在调度什么？

`clang++` 是驱动器，不只是语法检查器。常用停止点如下；具体后端是否分成独立进程由实现决定。

命令选项	到哪一步	你可以检查什么
<code>-E</code>	预处理	头文件是否找到、宏展开
<code>-S</code>	生成汇编	目标指令与调用约定
<code>-c</code>	生成目标文件	定义/未定义符号、重定位
不指定上述停止选项	通常继续链接	是否把需要的目标文件和库交给链接器
执行生成文件	加载并运行	动态库依赖、运行时行为

2. 实验 G0-L1: 缺少的定义在哪里

先预测：只编译 `main.cpp` 能成功吗？只链接 `main.o` 呢？加上 `math.o` 后应输出什么？

下面三个文件是完整实验。执行器分别编译两个 TU，先验证缺失 `add` 定义的链接失败，再链接完整程序；只有失败诊断对应缺失符号、完整程序返回 0 且输出精确匹配，才算通过。

[完整实验 · G0-L1 · math.hpp]

```

#pragma once
int add(int a, int b);

```

[完整实验 · G0-L1 · math.cpp]

```

#include "math.hpp"
int add(int a, int b) { return a + b; }

```

[完整实验 · G0-L1 · main.cpp]

```
#include "math.hpp"
#include <iostream>

int main() {
    const int result = add(3, 4);
    if (result != 7) return 1;
    std::cout << "sum=" << result << '\n';
}
```

手动练习时，把上面代码保存到一个新的实验目录，在该目录执行：

```
clang++ -std=c++23 -E main.cpp -o main.ii
clang++ -std=c++23 -c main.cpp -o main.o
clang++ -std=c++23 -c math.cpp -o math.o
nm -C main.o
clang++ main.o -o missing-definition
clang++ main.o math.o -o demo
./demo
```

其中 `missing-definition` 的链接失败是预期反例，其他命令不应因此被解释成失败；不要用 `&&` 把预期失败后的正例跳过。自动执行方式见实验说明。

应该看到什么： `main.o` 使用了 `add` 但没有定义它；在常见 `nm` 输出中它标为 `U`。这对目标文件合法，对当前缺少定义的最终链接不成立。补上 `math.o` 后输出 `sum=7`。符号装饰和报错排版随平台变化，测试不绑定整段诊断文本。

改变条件： 把头文件声明改成 `long add(int, int);`，保留实现中的 `int` 并重新编译实现。此时应在编译阶段发现冲突，而不必等链接。这个改动是手动练习，不在 `G0-L1` 的固定判据内。

边界： 链接成功只说明本次符号与链接要求得到满足，不证明函数语义正确、运行时依赖可用或所有 ABI 合同一致。

3. 五层诊断：先找失败的阶段

现象	阶段	首先检查
找不到 <code>math.hpp</code>	预处理	文件位置、 <code>include</code> 路径、大小写
参数类型不匹配	编译	当前 TU 看见的声明与表达式
Undefined / Duplicate symbol	链接	缺失定义、重复定义、参与链接的工件
Library not loaded: @rpath/...	加载	动态库位置、依赖记录、运行路径
错误结果、异常、越界	运行	输入、状态、生命周期和工具报告

“编译报错”可以口语化，但排查时必须落到具体阶段。构建日志中的最后一行不一定是根因；先找第一条有意义的诊断，再检查实际执行命令。

构建系统（CMake、Ninja、Make 等）管理的是输入到工件的依赖关系和执行规则。它们不能消除 C++ 的 TU、单一定义规则（One Definition Rule, ODR）和 ABI 要求；缓存了错误配置，照样可能稳定地产生错误工件。

4. 目标文件、库与加载器

4.1 三种 Object 不要混淆

`int x = 42;` 中的 object 是语言实体；`main.o` 是目标文件；object code 是机器代码。目标文件通常还包含节、符号表、重定位、常量和调试信息，不是裸指令串。

常见格式是 macOS 的 Mach-O、Linux 的 ELF、Windows 目标文件的 COFF 与可执行镜像的 PE。Mach-O 的 `__text` 等 section 组织细粒度内容；segment 用于更大的映射布局与权限组织。不要把平台的节名、权限细节当作通用语言规定。

4.2 符号和重定位解决什么？

未定义符号记录“我使用了哪个实体”；重定位记录“某处编码需要根据最终布局或符号绑定修补”。常规链接可能解决一部分，动态加载还可能处理剩余绑定；不是所有地址都在一次静态链接中固定下来。

C++ name mangling 支持在二进制层区分重载等实体。Itanium C++ ABI 中 `add(int,int)` 的编码示例为 `_Z3addii`；Mach-O 显示时还可能带平台前缀。不要手写猜测导出符号，使用工具查看并 demangle。

4.3 静态库和动态库的差异

工件	内容与通常行为	最容易误判的点
<code>.a</code> 静态库	一组目标文件归档，通常按未解析符号需要抽取成员	不是把所有成员机械复制；整包加载等选项会改变抽取策略
<code>.dylib/.so</code>	独立镜像，可执行文件记录动态依赖，加载器映射和绑定	构建时找到库，不代表部署时还能找到
可执行文件	连接后的程序镜像及相关依赖信息	磁盘文件还不是正在运行的进程

已经静态链接进去的那部分代码不在运行时依赖原 `.a` 文件；这不意味着整个程序从此没有任何动态库或数据文件依赖。macOS 的 `@rpath` 是运行时路径机制，具体部署问题留到 G9。

5. API、ABI 与跨语言边界

应用编程接口（application programming interface, API）决定源码如何调用；ABI 决定独立编译的机器码怎样协同，包括调用约定、数据布局、符号与语言运行时规则。重新编译后源码仍可用，不等于旧二进制还能使用新库。

例如下面是布局观察片段，不是跨平台常量：

[机制片段 · 不承诺独立编译]

```
struct Packet {
    char tag;
    int value;
    short code;
};
// 可用 sizeof(Packet)、alignof(Packet)、offsetof(Packet, value) 观察。
```

常见 Apple ARM64 配置会给出大小 12、对齐 4，成员间及末尾有 padding；标准没有要求所有 C++ 平台都如此。重排字段改变偏移，旧客户端按旧布局读取就可能出错。

Apple ARM64 的简单整数/指针参数常使用 `x0 ~ x7`（32 位视图为 `w0 ~ w7`），浮点参数常使用 `v0 ~ v7`；简单结果常从 `x0` 或 `v0` 返回。聚合、间接返回、可变参数等有专门规则，**不能用这两行速记实现完整 FFI**。详见 [Apple ARM64 ABI 说明](#)。

`extern "C"` 指定语言链接，常见工具链据此采用 C 的符号约定；它不会把函数体变成 C，也不会自动转换异常。常用边界形状是 `opaque handle`：

[反例片段 · 依邻近说明判定失效条件；不要用于生产代码]

```
// 接口形状示意：实际头文件还需处理 C/C++ 兼容声明和错误合同。
struct engine_handle;
extern "C" engine_handle* engine_create() noexcept;
extern "C" void engine_destroy(engine_handle*) noexcept;
```

由创建方提供匹配销毁函数，可以隐藏布局及分配器细节。要求是遵守同一资源协议，不混用不兼容的运行时分派/释放机制。`noexcept` 不是错误转换器：异常逃出会终止程序；实现必须另外设计捕获、错误表示及失败时的后备策略。参见 [FM-8 边界](#)。

6. 进程、栈与寄存器

6.1 局部变量一定在栈上吗？

不是。`automatic storage duration` 是语言概念；寄存器、栈槽或完全消除是实现选择。`as-if` 规则约束可观察行为，不要求每个源码变量对应一个物理栈地址。

程序是工件；进程是运行实例，涉及虚拟地址空间、线程、句柄等状态。每次普通嵌套调用通常有相应调用状态，但内联、尾调用和优化会改变观察到的栈形状。

6.2 ARM64 回查卡

名称	作用	不要推出的结论
SP	栈指针；使用时须遵守 ABI 对齐约束	栈上每个源码变量都有独立槽
x30 / LR	BL 写入返回地址；RET 常用它返回	所有函数都用同样序言保存 LR
x29 / FP	帧指针约定，辅助栈回溯	所有优化后函数都保留完整链
x19 ~ x28	通用被调用者保存寄存器	任意调用都不会改变其他寄存器
x0 ~ x7、x9 ~ x15	参数/临时寄存器中的常用部分	这是完整的 ARM64 寄存器分配合同

caller-saved 意味着调用方若需跨调用保留值，要按约定安排保存；callee-saved 意味着被调用方改变它后须恢复。寄存器不足等原因可导致 **spill**。叶函数可能不建立普通栈帧；非叶函数需要以 ABI 合法方式保留后续返回所需状态，不能据源码猜固定指令序列。

深递归可能耗尽栈，但是否优化、栈大小、故障表现都依赖实现，不能承诺某个递归次数必报某条 macOS 错误。

6.3 为什么旧栈字节不代表旧对象仍存在？

[反例片段 · 依邻近说明判定失效条件；不要用于生产代码]

```
// 错误接口片段：禁止通过返回结果读取 x。
int* bad() {
    int x = 42;
    return &x;
}
```

返回后看到旧 bits，既不能延长 x 的生命周期，也不能使后续读取合法。这正是下一章要区分的“存储迹象”与“对象语义”。

7. Final Gate

先闭卷回答，再对照参考答案；不要只复述名词。

1. 画出 `.cpp` 到运行进程的路径。G0-L1 第一次失败在哪一阶段？
2. 为什么只见普通函数声明就能生成 `main.o`？它还欠缺什么？
3. 未定义符号与重定位分别记录什么？
4. 静态库链接后可删除，能否推出程序不再依赖外部文件？
5. 为什么重排 `public struct` 可能源码兼容却 ABI 不兼容？
6. LR 与 FP 做什么？为什么不能从局部变量数推断栈帧大小？
7. 返回局部变量地址后读到了 42，能否说明接口有效？

8. Final Gate · 参考答案与常见误判

1. 预处理得到 TU，编译/汇编形成目标文件，链接形成镜像，加载后运行。反例败在链接，不是语法或类型检查；常见误判是把所有非零退出码叫“编译失败”。
2. 声明足够检查本次普通调用并按 ABI 生成调用信息，定义由其他工件提供。常见误判是把此结论推广给所有模板和 inline 用法。
3. 符号说明实体的定义/使用，重定位说明布局或绑定确定后哪里需要修补。它们相关但不是同一张“函数地址表”。
4. 只能推出被静态纳入的内容不再需要原归档。动态库、配置和数据仍可有依赖；删库试验不是完整部署验证。
5. 新源码可按新偏移重新编译，旧二进制仍按旧偏移访问。常见误判是用“能重新编译”证明“二进制兼容”。
6. LR 保存返回地址，FP 遵守帧指针约定；优化、寄存器分配、spill 和调用约定共同影响栈。常见误判是源码一行对应一个栈槽。
7. 不能。对象已结束生命周期，旧 bits 不是合法读取凭证。没崩溃也不是 UB 的反证。

完成标准：不看答案复现缺失定义的链接失败，补齐后运行成功；能把每个工件的责任讲给未来的自己听。汇编细节暂时看不懂，不阻塞进入 G1。

参考：Clang 驱动器选项；机器层规则按具体 ABI 查证。G0-L1 不涵盖动态部署、LTO、Modules 或所有 ODR 违规。二进制接口的完整合同转入 G8 §1，构建与安装依赖转入 G9 §1；首次阅读先沿 G1 §1 区分存储迹象与对象生命周期。

G1 · 对象模型与生命周期

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.2.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 的语言层与机器层区分；本章建立合法访问模型。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G0 · 全系列导航 · 下一章：G2

阅读入口

本章的问题是：指针非空、地址可读，为什么还可能不能合法访问？

首次按第 1~5 节阅读，完成两个实验，再做 Gate。第 6 节的跨语言对照只作回查，不要求先学 Zig 或 Rust。本章先建立合法访问模型；清理责任转入 G2 §1，复制和移动的成本转入 G3 §4，这里不重复两章的操作与成本推导。

- 1. 同一个地址，不等于同一个活对象
- 2. 指针、引用与别名
- 3. 数组、退化与借用
- 4. 实验：对象可以先于存储结束
- 5. 表达式值类别
- 6. 跨语言回查与检查工具
- 7. Final Gate

1. 同一个地址，不等于同一个活对象

1.1 问题与例子

[机制片段 · 不承诺独立编译]

```
int a = 42;
int b = a;
```

值相同不代表对象相同。上例中两个独立的普通 int 对象有各自身份；修改 b 不改变 a。但若第二行改成 `int& b = a;`，b 就是另一个名字访问同一对象。

概念	用 <code>int a = 42;</code> 解释	常见混淆
类型 <code>type</code>	<code>int</code> 规定可用操作与表示约束	类型不是地址
名称 <code>name</code>	<code>a</code> 是源码标识符	没有名字也可以有对象
对象 <code>object</code>	本次初始化形成的 <code>int</code> 实体	<code>object</code> 不只指 <code>class</code> 实例
值 <code>value</code>	当前为 42	相等的值可属于不同对象
存储 <code>storage</code>	对象表示占用的字节空间	字节仍在不证明对象存活
生命周期 <code>lifetime</code>	对象存在的运行时区间	不等于名字可见的文本范围

这些概念需要区分，但并非互不相关的“六个正交公理”。类型、初始化和存储等一起决定对象语义；编译器仍可按 `as-if` 规则省去不影响可观察行为的物理操作。

1.2 生命周期与存储期

通常要有足够大小和正确对齐的存储，并完成相应初始化，生命周期才开始。对 `class`，析构调用**开始时**生命周期结束，构造/析构期间的访问另有专门规则。释放或复用存储也可能结束原对象的生命周期。精确边界见 N4950: `basic.life`。

存储期有 `automatic`、`static`、`thread`、`dynamic` 四类；作用域描述名字可见范围。静态存储并不意味着对象从程序第一条指令前到最后一条指令后都已完成初始化且可任意访问。

把资源操作拆成“取得存储 → 初始化 → 清理 → 归还存储”有助于推理，但**分配一定不创建对象**不是普遍规则：C++23 有 `implicit-lifetime` 对象的隐式创建机制。本章不尝试用四步图代替全部低层对象规则。

1.3 表示不是访问许可

`00 00 80 3F` 在常见小端、32 位 IEEE 754 `float` 表示下可对应 1.0；这不是所有平台的字约定，更不允许把任意 `int*` 强转为 `float*` 后读取。需要位级转换时，检查类型与表示条件，使用合适的 `std::bit_cast` 等机制；“bits 看起来正确”不能绕过类型可访问性和生命周期要求。

2. 指针、引用与别名

[机制片段 · 不承诺独立编译]

```
int x = 1;
int y = 2;
int* p = &x;
int& r = x;
r = y;           // 给 x 赋值；r 没有改绑到 y。
p = &y;          // p 这个指针对象改存另一指针值。
```

`p` 自己是对象，有地址 `&p` 和生命周期；`r` 是引用，不是独立的对象，`&r` 得到的是 `&x`。引用不提供“自动追踪存活”的能力，一样可能悬挂。

声明	可以重定向 p	可以通过 p 修改目标
T* p	是	是
T* const p	否	是
const T* p	是	否
const T* const p	否	否

`const T*` 限制这条访问路径，不证明其他别名不能修改同一对象。裸指针类型也不编码“由我 delete”还是“只是借用”；接口必须说明。

2.1 临时对象延寿不是传染能力

[机制片段 · 不承诺独立编译]

```
const std::string& text = std::string{"hello"};
```

这种直接绑定可延长临时对象的生命周期；把引用传过函数、再保存函数返回的引用，不会一概获得同样延寿。遇到引用返回值，必须追到真正的对象及其结束时间，而不是看到 `const&` 就判安全。

2.2 每次访问前问什么？

有有效存储吗？目标对象是否活着？类型访问、对齐、边界、可修改性是否满足？并发访问是否有必要的同步？这些是审查问题，不是编译器会替你验证的乘法公式。`p != nullptr` 只排除空指针值；若指针自身都未初始化，连读取它也不能拿来作安全证明。

3. 数组、退化与借用

`int values[4]{};` 是四个 `int` 子对象组成的数组对象，大小是 `4 * sizeof(int)`，不是由语言固定为 16 字节。

多数需要指针的表达式中，数组可转换为首元素指针；数组没有“变成指针对象”。`values + 1` 前进一个 `int`，`&values + 1` 前进整个数组。尾后指针可作区间终点，不能当作可读取的元素。

函数形参 `void f(int a[100])` 会调整为指针形参，不强制调用方提供 100 个元素。若要保留数组长度，使用合适的数组引用、模板或带长度的接口。

[机制片段 · 不承诺独立编译]

```
// 接口片段：同步读取，不保存借用。
int sum(std::span<const int> values);
```

`std::span` 携带范围信息，不拥有元素，也不会阻止 `vector` 扩容或销毁。正确约束是：**每次使用借用访问元素时，目标元素必须存活，且期间没有使该访问路径失效的操作。**

不是“view 对象的生命周期必须小于 owner”的机械时间比较：一个已失效但不再被用于访问的 `span` 对象可以仍然存在；反过来，`owner` 还活着，扩容或 `erase` 也可能已经使借用失效。G4 再逐项解释失效规则。

4. 实验：对象可以先于存储结束

4.1 G1-L1：reserve 没有构造元素

预测：reserve 后、emplace 后、clear 后，活对象数分别是多少？clear 后 capacity 会不会变为 0？

完整运行例：

[完整实验 · G1-L1 · main.cpp]

```
#include <iostream>
#include <type_traits>
#include <vector>

struct Probe {
    static inline int live = 0;
    Probe() noexcept { ++live; }
    Probe(const Probe&) noexcept { ++live; }
    Probe(Probe&&) noexcept { ++live; }
    ~Probe() { --live; }
};

int main() {
    int values[4]{};
    static_assert(sizeof(values) == 4 * sizeof(int));
    static_assert(std::is_same_v<decltype(values), int[4]>);
    static_assert(std::is_same_v<decltype(&values), int (*)[4]>);
    int& alias = values[0];
    alias = 9;
    if (values[0] != 9 || &alias != &values[0]) return 1;

    std::vector<Probe> items;
    items.reserve(4);
    const auto capacity = items.capacity();
    if (items.size() != 0 || Probe::live != 0 || capacity < 4) return 2;
    items.emplace_back();
    if (items.size() != 1 || Probe::live != 1) return 3;
    items.clear();
    if (!items.empty() || Probe::live != 0 ||
        items.capacity() != capacity) return 4;
    std::cout << "live=0,1,0; capacity-preserved\n";
}
```

原因：capacity 表示可容纳元素的空间，size 表示现有元素数。reserve 不增加 size；clear 销毁元素但保留 capacity。由此可以看到“还有存储”和“还有元素”是两回事。

改变条件：把 emplace_back() 改成 resize(3)，先写出需要改变的判据；不要只改预期输出让测试变绿。此扩展没有包含在固定实验的结果中。

边界：实验只证明本例中的构造/析构与容量关系，不证明所有分配操作都不创建任何对象，也不验证自定义 allocator 或分配失败。

4.2 G1-L2：一个明确的错误读取

这是**必须被拒绝的未定义行为（undefined behavior, UB）反例**，不是建议写法。只在新建临时目录的独立进程中运行，并启用 AddressSanitizer。

[反例 · 未定义行为：仅限隔离检测 · G1-L2 · main.cpp]

```
int main() {
    int* p = new int{42};
    delete p;
    return *p; // 故意读取已释放对象，验证工具能否指出这个具体错误。
}
```

预期是 ASan 报告 heap-use-after-free。执行器先检查 ASan 的编译/正常运行能力，再要求反例出现这个诊断并以指定的 86 退出；普通编译失败、超时或任意崩溃都不计通过。测试配置关闭外部符号化，不校验源码行栈；诊断中的地址和栈偏移不固定。

为什么不能用“退出码是 42”验收？因为发生 UB 后语言不保证任何结果。ASan 提供的是本工具在本次执行中检测到错误的证据；未报告不代表没有 UB。ASan 官方说明

4.3 手工存储复用只作进阶回查

[机制片段 · 不承诺独立编译]

```
// 示意片段：需要 <cstdlib>、<new> 以及实际 T 定义。
alignas(T) std::byte storage[sizeof(T)];
T* p = ::new (storage) T(/* arguments */);
p->~T(); // storage 仍在，不能再按原来的活 T 访问。
```

普通 new T(...) 把取得存储和初始化组合起来；delete 通常组合析构与释放。不要用手写 placement new 代替本来能由 value member、vector 或智能指针表达的所有权。

5. 表达式值类别：先看表达式，不给对象贴标签

基本类别

含义与常见例子

lvalue

glvalue 中非 xvalue 的类别；普通表达式 x、*p

xvalue

标识可被复用资源的对象；对象表达式 std::move(x)

prvalue

初始化结果对象或计算操作数的值；42、返回 T 的普通调用

glvalue = lvalue 或 xvalue; rvalue = prvalue 或 xvalue。xvalue 同时在两组里，不要画成互不相交的两棵树。精确定义见 N4950: basic.lval。

[机制片段 · 不承诺独立编译]

```
void use(T&& r) {
    consume(r); // 通常的这个参数表达式 r 是 lvalue。
    consume(std::move(r)); // 对象表达式转换成 xvalue，供重载决议使用。
}
```


这里 `T` 是已知对象类型，不是推导中的 `forwarding reference`。C++23 的 `move-eligible return` 等特定上下文另有隐式移动规则，不能把“具名就永远是 `lvalue`”当作无例外规则。

`std::move` 本身是转换，不执行资源搬运。后续可能选移动、复制或因没有可行重载而失败；`const` 对象通常不能绑定到普通 `T&&` 移动构造参数。一个对象可以仍在生命周期内、但已被移走资源；其可用操作由类型合同决定，不能等同悬挂，也不能保证“什么都能做”。G3 有完整对照实验。

返回 `T`、`T&`、对象类型 `T&&` 的函数调用分别通常得到 `prvalue`、`lvalue`、`xvalue`；这决定表达式类别，不证明返回引用指向的对象活着。

6. 跨语言回查与检查工具

C++ 的 `std::span<T>`、Zig 的 `[]T` 与 Rust 的 `&[T]` 都可表达连续元素的借用，但检查机制不同。Rust 的普通引用受借用检查和安全接口约束；Rust 原始指针不是非空或自动安全引用。Zig `*T` 与可空 `?*T` 区分非空性，但不能据此消除悬挂。C++ 的裸指针同样不能证明生命周期。

以下为 **Zig 0.15.2** 语法与清理顺序示意，未在本批编译；`T` 的 `init/deinit` 是用户约定，不是语言级构造/析构：

```
const p = try allocator.create(T);
defer allocator.destroy(p); // 先登记，最后执行。
p.* = try T.init();
defer p.deinit();           // 后登记，先执行逻辑清理。
```

`defer` 按逆序执行，不能把 `destroy` 登记在 `deinit` 之后而让存储先释放。若 `init` 返回错误，上面只执行 `destroy`；若 `init` 自身取得多个资源，它还须自行处理部分失败。**Zig 0.15.2**: `defer`

Zig 类型对照回查：`[N]T` 数组、`*[N]T` 数组指针、`[*]T` 未携带长度的多项指针、`[]T` 切片。Zig 不采用 C++ 的引用声明机制，但也不要从这个差异推出所有调用成本都能从语法直接读出。

建议的诊断起点：

```
clang++ -std=c++23 -O1 -g -Wall -Wextra -Wpedantic \
-fsanitize=address,undefined -fno-omit-frame-pointer program.cpp -o program
```

Warnings、ASan、UBSan、测试和语言规则互相补充。UBSan 不是完整的 `lifetime/aliasing` 检查器；Sanitizer 不能证明未执行路径安全。并发问题还需适合的同步推理与工具。

7. Final Gate

1. `int a = 1; int b = a; int& c = a;` 有几个 `int` 对象？修改谁会影响谁？
2. `G1-L1` 在 `reserve` 后、`clear` 后各说明了什么？可以用 `items[0]` 查看旧内容吗？
3. `p != nullptr` 能证明什么，不能证明什么？
4. `int a[4]` 与函数形参 `int a[100]` 中的长度含义一样吗？
5. `vector` 活着时，指向它的 `span` 为什么仍可能失效？
6. `T&& r` 的类型和表达式 `r` 的类别为什么不矛盾？`move` 是否保证转移？
7. 把 `Zig` 两个 `defer` 的登记顺序倒过来，会发生什么？
8. `ASan` 没有报告，能否关闭整个生命周期正确性问题？

8. Final Gate · 参考答案与常见误判

1. 两个 `int` 对象；`c` 是 `a` 的引用。`b` 独立，`c` 写入 `a`。误判：把每个变量声明都数成一个独立对象。
2. `reserve` 不创建元素，`clear` 销毁元素但保留容量。`size` 为 0 时下标读取不合法。误判：`capacity` 是可读取的元素数。
3. 对已正确初始化的指针值，只排除空值；目标可能已释放、越界或不满足类型/同步条件。误判：加一次 `if` 就“修复悬挂”。
4. 实际数组有四个元素；普通数组形参调整为指针，100 不强制长度。误判：把形参写得长就获得边界检查。
5. 扩容、`erase`、`clear` 等可能让存储或元素身份失效。需要验证每次使用时的有效性，而不只是比较两个对象的寿命长短。
6. 类型描述声明，值类别描述表达式；普通具名参数表达式是 `lvalue`，转换后可参与不同重载决议。`move` 本身不搬运。误判：`T&&` 是“对象已经被移动”的状态。
7. 逆序执行会先释放存储，再通过 `p` 清理，顺序错误。误判：`defer` 按书写顺序执行。
8. 不能。只覆盖工具能检测且此次执行走到的行为。误判：把工具证据提升为全程序证明。

完成标准：能画出“owner → 存储/元素 ← 借用”的关系，指出何时失效；对每个实验先给预测，后解释结果。下一步用 G2 的类型设计把清理责任固定下来。

G2 · RAII、资源与所有权

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.2.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G1；覆盖资源、独占/共享所有权、失败安全与异步清理；Zig 仅作对照。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G1 · 全系列导航 · 下一章：G3

阅读入口

本章只抓住一个问题：同一资源经过提前返回、异常和异步交接后，谁仍负有清理责任？

- 主阅读线：资源与 RAII → 独占所有权 → 失败安全 → 清理实验 → 共享所有权 → 共享实验 → Final Gate。
- 回查层：按下面的主题目录查找接口、架构、跨语言和审查材料，不要求一次连续读完。
- 前置自测：能否从 G1 解释“unique_ptr 活着，get() 得到的旧指针仍可能失效”？若不能，先画 reset 前后的对象关系。
- 完成目标：为一个 owner 标出获取、交接、借用失效和清理点，并能解释失败路径。只读懂智能指针 API 不算完成。

旧主题编号保留为回查锚点，正文按主题重新分层。只有带 g-lab 标记的程序是本批提取运行的完整实验，其余块按上下文视为片段或设计示意。跨语言对照未在本批编译。公式用于提示审查维度，不是形式化安全证明。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 资源与清理模型
- 2. 独占所有权与值成员
- 3. 接口与所有权合同
- 4. 特殊成员与失败安全
- 5. 实验 · G2-L1：清理不是只覆盖快乐路径
- 6. 回滚与共享所有权

- 7. 实验 · G2-L2: 弱观察不持有资源, lock 暂时持有
- 8. 共享所有权的适用边界
- 9. 异步边界与停机
- 10. 架构与跨语言回查
- 11. 工程审查与常见误判
- 12. 术语与统一模型
- 13. Final Gate
- 14. Final Gate · 参考答案与常见误判
- 15. 工程原则回查
- 16. G2 → G3

1. 资源与清理模型

1.1 本章的工程问题

G1 问“访问是否合法”，G2 问“谁让资源活着，谁负责最终清理”。先画最小关系，再选智能指针：

```
owner — manages —> resource <— accesses — borrower
      cleanup duty                no cleanup duty
```

value、unique、shared 是不同的责任组织方式，不是智能指针 API 的难度等级。优先用最简单、足以表达实际生命周期的结构。

1.2 Resource、Handle 与 Ownership

Resource 不等于 Memory

文件描述符、socket、锁、线程、GPU buffer、数据库事务、映射区域、临时文件和帧缓冲 lease，都有取得、使用和释放协议。清理动作不一定是 delete：它可以是 close、unlock、rollback、munmap 或归还池。

Handle 不等于 Resource

[反例片段 · 依邻近说明判定失效条件；不要用于生产代码]

```
// POSIX 接口片段，省略头文件和错误处理。
int fd = ::open("data.bin", O_RDONLY);
```

fd 是一个 int 对象，内核里的打开文件状态才是被管理资源。复制这个整数不会复制资源，也不会替你决定哪一份负责 close。同理，T* 只表示一条访问路径，不能独自说明所有权。

Ownership 的工程定义

拥有者 (owner) 负有按协议结束资源使用并完成最终清理的责任。独占 owner 负责这项责任；共享 owner 共同决定管理资源何时释放；借用者 (borrower) 可以访问但不能擅自清理。资源可以临时为空，责任也可以按类型合同转移。

Aliasing ≠ Ownership

[机制片段 · 不承诺独立编译]

```
int value{42};
int* p = &value;
int* q = &value;
int& r = value;
```

三条别名路径访问同一对象，并没有产生三个 owner。因此“很多组件都用它”不是采用 shared_ptr 的充分理由；应先问它们是否真的需要独立延长生命。

1.3 Manual Cleanup 为什么难维护

单一路径很容易遗漏隐藏条件

[机制片段 · 不承诺独立编译]

```
// 演示手工清理路径，不是完整的文件处理实现。
int process() {
    int fd = ::open("data.bin", O_RDONLY);
    if (fd == -1) return -1;
    // use fd
    ::close(fd);
    return 0;
}
```

正常路径看起来成对，但还没有处理实际读取失败、close 结果等合同。此处只讨论“是否到达清理”，不把它当作可靠文件事务。

一个提前返回就改变清理路径

若在 use fd 处插入 if (!validate(fd)) return -2;，成功打开的描述符就越过 close。每增加分支，都要求作者重新检查所有已取得资源。

多资源形成手工 Cleanup Stack

取得 A 后取得 B，再取得 C；若 C 失败，必须清理已成功取得的 B、A，不能清理未取得的 C。返回、异常、部分初始化和回调会增加路径数。RAII 的价值是把这份状态记录交给对象结构，而不是复制多份 cleanup 分支。

1.4 RAII

把资源协议放进类型

资源获取即初始化（Resource Acquisition Is Initialization，RAII）把资源管理与对象的初始化/清理绑定。对象可以在创建时取得资源，也可表示空状态再按合同取得；核心是任何已拥有状态都有相应清理责任。

确定性清理及其边界

[机制片段 · 不承诺独立编译]

```
// 接口片段：假定 FileDescriptor 已正确定义清理协议。
{
    FileDescriptor file{fd};
    work();
}
```

正常退出、提前 `return`，以及实际发生的异常展开，会按规则销毁已构造的局部 `owner`。它不依赖稍后某次 GC 才清理，但也不承诺 **terminate**、强制结束进程或断电会完整展开。失败细节见 FM-3。

RAII 不等于 Smart Pointer

`vector` 管元素和存储，`lock_guard` 管解锁，`FileDescriptor` 管句柄，`FrameLease` 管归还使用权。设计类型时问“析构履行什么协议”，而不是“能不能套一层 `unique_ptr`”。需要向调用方报告的 `flush/close` 失败通常应有显式操作，析构承担不抛出的后备清理。

C++ 与 Zig 的责任放置不同

C++ 把清理协议放进类型的析构；Zig 的常见做法是在调用作用域登记 `defer`。概念片段如下，未在本批编译：

```
fn run() !void {
    var file = try openFile();
    defer file.deinit();
    try work();
}
```

两者都能组织成对清理，但一个跟随 `owner` 对象，一个跟随词法作用域。跨作用域交接资源时，仍须明确谁接手责任；`defer` 的逆序规则见 G1 的版本化示例。

2. 独占所有权与值成员

2.1 Unique Ownership

Unique Ownership Invariant

```
At most one final owner
    |
    ▼
Resource
```

可以有：

```
many borrowers
```

但只能有：一个最终 `cleanup responsibility owner`。

为什么不能复制同一资源的清理责任？

问题不是“`unique owner` 这个词禁止一切复制”，而是两份句柄如果都对同一个资源执行最终清理，就会 `double close / double free`。

因此，类似文件描述符所有者通常禁用复制：

[机制片段 · 不承诺独立编译]

```
// 接口片段
FileDescriptor(const FileDescriptor&) = delete;
FileDescriptor& operator=(const FileDescriptor&) = delete;
```

另一种合法设计是复制时创建独立资源：例如 `vector` 的元素副本，或明确定义 `clone` 的资源包装。这样每份对象仍唯一拥有自己的资源。能否复制应由资源语义决定；不能把“复制一个地址”冒充“复制完整值”。

为什么允许 **Move** ？

需要交接清理责任的 `owner` 可以设计为可移动；固定地址、不可迁移的 `owner` 也可以禁止移动。可移动 `owner` 的目标是：

```
Owner A
  |
  | move
  ▼
Owner B
```

保持：

exactly one owner

因此：

[机制片段 · 不承诺独立编译]

```
FileDescriptor(FileDescriptor&& other) noexcept;
```

通常会把：

```
source:
Owning(resource)
→ Empty

destination:
Empty/New
→ Owning(resource)
```

2.2 `std::unique_ptr<T>`

正确定位

[机制片段 · 不承诺独立编译]

```
std::unique_ptr<T>
```

不是：更聪明的 `T*`。

而是：**unique-owning RAII object whose resource happens to be pointer-shaped**。

它编码：


```

unique ownership
destruction
copy forbidden
move transfer
nullable/empty state
pointer-like borrow access

```

默认创建方式

[机制片段 · 不承诺独立编译]

```
auto robot = std::make_unique<Robot>();
```

优先于:

[机制片段 · 不承诺独立编译]

```
std::unique_ptr<Robot> robot{new Robot};
```

因为:

```

allocation
↓
immediately enters owner abstraction

```

减少 raw owning pointer 暴露。

核心 API 按 Ownership 理解

Operation	Ownership Meaning
make_unique	建立 unique ownership
destructor	销毁 owned resource
operator*, ->	borrow pointee
get()	暴露 borrowed raw pointer
release()	放弃 ownership, 不销毁 resource
reset()	结束当前 ownership / adopt 新 pointer
move constructor	transfer ownership
move assignment	replace current ownership via transfer
copy	forbidden

get() vs release()

[机制片段 · 不承诺独立编译]

```
T* view = owner.get();
```

:

```

owner remains owner
view only borrows

```

而:

[机制片段 · 不承诺独立编译]

```
T* raw = owner.release();
```

:

owner → empty

raw handle now carries cleanup responsibility by convention

必须记: **get() borrows; release() transfers responsibility out.**

unique_ptr 不证明 Borrow Lifetime

[反例片段 · 依邻近说明判定失效条件; 不要用于生产代码]

```
auto owner = std::make_unique<Robot>();
```

```
Robot* view = owner.get();
```

```
owner.reset();
```

```
view->update(); // dangling
```

所以:

unique_ptr

solves ownership uniqueness

does not solve borrow lifetime

G1 仍然有效。

unique_ptr<T[]>

[机制片段 · 不承诺独立编译]

```
auto values = std::make_unique<int[]>(count);
```

表达: unique ownership of dynamically allocated array。

但普通动态序列通常优先:

[机制片段 · 不承诺独立编译]

```
std::vector<T>
```

因为 vector 同时管理:

length

capacity

element lifetime

iteration

copy/move

Custom Deleter

完整模型:

[机制片段 · 不承诺独立编译]

```
std::unique_ptr<T, Deleter>
```

因此可以管理:

```
FILE*
C library handles
foreign runtime handles
custom allocated objects
```

只要 deleter 能正确表达 cleanup protocol。

例如：

[机制片段 · 不承诺独立编译]

```
struct FileCloser {
    void operator()(std::FILE* file) const noexcept {
        if (file != nullptr) {
            static_cast<void>(std::fclose(file));
        }
    }
};

using File = std::unique_ptr<std::FILE, FileCloser>;
```

此处是后备清理片段，忽略 fclose 返回值不等于证明写入/关闭成功。需要向调用方报告 I/O 失败时，提供显式 finish/close 协议，再保留不抛出的析构兜底。

非 Pointer-shaped Resource 不要硬套 unique_ptr

POSIX：

[机制片段 · 不承诺独立编译]

```
int fd;
```

更自然：

[机制片段 · 不承诺独立编译]

```
class FileDescriptor;
```

原因：

```
fd is integer handle
not naturally pointer-shaped
```

所以：dedicated RAII owner type 往往比强行使用 unique_ptr 更准确。

2.3 Value Ownership 优先

Unique Ownership 不等于 unique_ptr

[机制片段 · 不承诺独立编译]

```
class RobotRuntime {
private:
    Logger logger_;
};
```

这里：

```
RobotRuntime
owns Logger
```

已经是 unique/value ownership。

不需要：

```
[机制片段 · 不承诺独立编译]
std::unique_ptr<Logger>
```

Value Ownership 为什么通常最好推理？

如果 Logger 的存在与父对象紧密绑定，直接写 `Logger logger_;`，就不必再为这个成员引入可空指针、单独分配和间接访问。成员仍可能在内部拥有动态缓冲区；“value member”不等于“整个对象零堆分配”。

成员按声明顺序初始化、逆序销毁；它们的生命周期也不是与父对象每个瞬间完全相同。这里的设计意图是让子资源的管理嵌入父对象结构，而不是手工维护另一套外部 owner。

选择顺序可从 value member 开始；确有可选存在、独立动态生命周期、多态等需要时考虑 unique ownership；只有多个独立参与者确需共同延长生命时再考虑 shared ownership。这是设计起点，不是禁止其他表示的语言规则。

3. 接口与所有权合同

3.1 Ownership Taxonomy

可以把常见关系统一成：

Semantic Relationship	C++ 常见表达
Value ownership	T
Unique dynamic ownership	std::unique_ptr<T>
Required mutable borrow	T&
Required read-only borrow	const T&
Optional borrow/observer	T*, const T*
Sequence borrow	std::span<T>
Shared ownership	std::shared_ptr<T>
Shared-domain weak observer	std::weak_ptr<T>
Pool-backed temporary right	dedicated move-only Lease
Raw OS resource owner	dedicated RAII type

五个独立 API 维度

任何参数/返回类型都可以问：

```

Ownership
Nullability
Mutability
Extent
Lifetime / Invalidation

```

例如：

[机制片段 · 不承诺独立编译]

```
const T&
```

```
:
```

```

Ownership    = borrow
Nullability   = required
Mutability    = read-only
Extent        = one object

```

而：

[机制片段 · 不承诺独立编译]

```
std::span<const T>
```

```
:
```

```

Ownership    = borrow
Mutability    = read-only
Extent        = runtime bounded sequence

```

3.2 Function Signature 就是 Ownership Contract

T

[机制片段 · 不承诺独立编译]

```
void process(T value);
```

通常：callee 获得自己的 value。

参数对象有自己的生命周期，但若 T 是 span、指针或带借用成员的类型，底层数据仍依赖 caller 的资源；按值传参不自动消除别名。

T&

[机制片段 · 不承诺独立编译]

```
void process(T& value);
```

通常：required mutable borrow。

const T&

[机制片段 · 不承诺独立编译]

```
void process(const T& value);
```

通常：required read-only borrow。

T*

[机制片段 · 不承诺独立编译]

```
void process(T* value);
```

信息不足。

可能是：

```
optional borrow
legacy API
output parameter
C-compatible handle
array start
```

所以需要额外 contract。

std::unique_ptr<T>

[机制片段 · 不承诺独立编译]

```
void consume(std::unique_ptr<T> value);
```

非常清晰：callee takes unique ownership。

std::shared_ptr<T>

[机制片段 · 不承诺独立编译]

```
void schedule(std::shared_ptr<T> value);
```

应读成：callee acquires a shared ownership stake。

而不是：传一个 pointer。

不要泄漏 Ownership Representation

如果函数只是：

```
read T
```

不要因为 caller 恰好有：

[机制片段 · 不承诺独立编译]

```
std::unique_ptr<T>
```

就写：

[机制片段 · 不承诺独立编译]

```
void inspect(const std::unique_ptr<T>&);
```

更合理：

[机制片段 · 不承诺独立编译]

```
void inspect(const T&);
```

原则：**Accept the least ownership-aware abstraction the function actually needs.**

4. 特殊成员与失败安全

4.1 Rule of Three / Five / Zero

为什么 **Raw Owner** 会触发 **Special Member** 问题？

[机制片段 · 不承诺独立编译]

```
class Buffer {  
private:  
    int* data_{nullptr};  
    std::size_t size_{0};  
};
```

如果 `data_` owning:

默认 `memberwise copy`:

copy pointer value

而不是:

copy allocation ownership semantically correctly

于是可能 `double delete`。

Rule of Three

C++98:

Destructor
Copy Constructor
Copy Assignment

如果一个 `raw-resource class` 需要自己管理其中一个，通常必须系统审查三个。

Rule of Five

C++11 以后再加入:

Move Constructor
Move Assignment

所以 `raw resource owner` 必须审查:

destruction
copy construct
copy assign
move construct
move assign

Rule of Five 不是目标

不是: 每个 `class` 都手写五个函数。

而是: 如果你已经直接管理 `raw resource`, 就必须完整定义 `resource state transition`。

Rule of Zero 才是现代默认目标

[机制片段 · 不承诺独立编译]

```
class Robot {
private:
    std::string name_;
    std::vector<Sensor> sensors_;
    std::unique_ptr<Engine> engine_;
};
```

如果 members 已经拥有正确：

```
copy
move
destruction
```

语义，那么外层 class 不需要自己写这些机制。

这就是：**semantic composition**。

Rule of Zero 的本质

```
Raw Resource
  ↓
Small Proven RAII Owner
  ↓
Higher-Level Domain Types
  ↓
Rule of Zero
```

复杂度集中在：少数真正的 resource boundary types。

而不是散布到每个业务 class。

= default 不等于“不声明”

[机制片段 · 不承诺独立编译]

```
~Widget() = default;
```

仍然是：user-declared destructor。

这会影响某些 implicit move-generation rules。

所以：没有必要就不要为了“显式”机械写 special members。

Member Type 语义向上传播

如果：

[机制片段 · 不承诺独立编译]

```
class Robot {
private:
    std::unique_ptr<Engine> engine_;
};
```

那么：


```
unique_ptr non-copyable
↓
Robot copy also unavailable
```

```
unique_ptr movable
↓
Robot can be implicitly movable
```

前提是没有其它 user-declared special-member 规则阻止生成。

这就是 C++ ownership composition 很强的一点。

4.2 Failure Safety

Exception Safety 真正研究什么？

不是：

```
try/catch
```

而是：如果 **operation** 中途失败，剩下的 **program state** 仍满足什么 **guarantee**？

因此更广义应该理解：Failure Safety。

即使使用：

```
std::expected
Zig error union
C error code
```

同样需要。

Object Invariant

一个 class 应保证：

```
constructor success
→ invariant holds

operation success
→ invariant holds

operation failure
→ invariant still holds
```

资源不能：

```
leak
double cleanup
half-own
dangling internally
```

Basic Guarantee

失败后：

```
no resource leak
object invariants remain valid
object may have changed state
```

即：valid but state may change。

Strong Guarantee

失败时：state 保持原样。

```
success → commit S1
failure → rollback S0
```

即：commit-or-rollback。

No-throw Guarantee

Operation 保证：不允许 exception 逃出。这不等于操作一定完成：noexcept 函数仍可能终止或返回错误状态；事务提交需要另证 no-fail 条件。

典型：

```
destructor
swap
simple resource move
cleanup primitive
```

4.3 Transactional Update

高质量 mutation 常采用：

```
Prepare
↓
Validate
↓
Perform failure-prone work on temporary state
↓
No-throw Commit
↓
Automatic cleanup of old state
```

而不是：

```
Destroy old state
↓
Start risky work
↓
Hope it succeeds
```

先准备，再在可证明的提交点替换

问题：先 reset 旧模型，再构建新模型，构建失败后还有可用模型吗？

[机制片段 · 不承诺独立编译]

```
// 不佳片段：失败前已经销毁旧值。
model_.reset();
model_ = build_model(config);
```

把可能失败的构建放在临时状态中：

[机制片段 · 不承诺独立编译]

```
auto next = build_model(config);
model_ = std::move(next);
```

只有在 `build_model` 不改变旧模型、替换确实不会失败且旧值清理遵守合同等前提下，才能据此论证强保证。例如明确使用适合的 `unique_ptr` 与不抛出的 `deleter`，并把外部副作用另行处理。对未知 `T`，移动赋值未必是不失败的提交点。

边界：一个 `noexcept` 标注只禁止异常逃出，函数仍可能终止或通过其他渠道失败。强保证必须说明保护的状态范围，检查构建、验证、提交、返回和清理；参见 [FM-4](#)。

Copy-and-Swap 的真正意义

[机制片段 · 不承诺独立编译]

```
T copy{other};
swap(copy);
```

不是一个语法技巧。

而是：

Prepare temporary

↓

may fail

`noexcept swap`

↓

Commit

temporary destructor

↓

Cleanup old state

这是事务更新的候选结构。还必须证明 `swap`、提交后的返回和清理不会破坏所承诺的结果；单独看到 `noexcept swap` 不足以完成证明。

4.4 Constructor Failure

非委托构造失败与成员清理

如果 `constructor` 中：

```
member A constructed
member B constructed
member C throws
```

那么：

```
destroy B
destroy A
```

对这种非委托构造失败，完整对象没有构造完成，不调用它自身的析构函数，已完成的子对象按规则清理。不要推广成“所有构造函数体抛异常都不析构完整对象”：委托构造的目标已成功完成、委托体随后抛出时，完整对象会析构。[N4950: except.ctor](#)

为什么 Member RAI 如此重要？

Raw resource:

[机制片段 · 不承诺独立编译]

```
class Session {
    int fd_;
    std::byte* buffer_;
};
```

constructor 中途 throw: 很容易 leak。

RAII members:

[机制片段 · 不承诺独立编译]

```
class Session {
private:
    FileDescriptor fd_;
    std::vector<std::byte> buffer_;
};
```

construction failure: 已构造 members 自动 rollback。

5. 实验 · G2-L1: 清理不是只覆盖快乐路径

问题：三个退出路径是否都释放资源？构造函数体中途失败时，是否需要完整对象的析构函数才能清理成员？

先预测四次调用后，资源的获取/释放数，以及完整 Session 的析构次数。这里用计数器模拟资源，避免把 OS 的关闭语义混入最小实验。

[完整实验 · G2-L1 · main.cpp]

```
#include <iostream>

struct Resource {
    static inline int acquired = 0, released = 0, live = 0;
    Resource() { ++acquired; ++live; }
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
    ~Resource() noexcept { ++released; --live; }
};

void work(int mode) {
    Resource resource;
```

完整实验 · G2-L1 · main.cpp · continued

```

    if (mode == 1) return;
    if (mode == 2) throw 7;
}

struct Session {
    static inline int destructors = 0;
    Resource member;
    Session() { throw 9; } // 非委托构造; member 已完成。
    ~Session() { ++destructors; }
};

int main() {
    work(0);
    if (Resource::live != 0 || Resource::released != 1) return 1;
    work(1);
    if (Resource::live != 0 || Resource::released != 2) return 2;
    bool work_caught = false;
    try { work(2); }
    catch (int value) { work_caught = value == 7; }
    if (!work_caught || Resource::live != 0 ||
        Resource::released != 3) return 3;
    bool session_caught = false;
    try { Session session; }
    catch (int value) { session_caught = value == 9; }
    if (!session_caught || Resource::acquired != 4 ||
        Resource::released != 4 || Resource::live != 0 ||
        Session::destructors != 0) return 4;
    std::cout << "acquired=4 released=4 live=0 session-dtors=0\n";
}

```

解释：局部 owner 在正常退出、return 和本例的异常展开中清理；Session 未完成，但其 Resource 成员已经完成，所以成员单独被清理。无需依赖 Session 析构函数。

改一个条件：去掉 Resource 析构中的释放计数，应触发测试失败，而非只看到进程正常结束。另可写委托构造作为手动扩展，预测完整对象析构为何不同。

边界：这是模拟资源和已实际展开的异常路径，不证明 terminate、进程强制退出、断电或 OS close 失败都可自动恢复。析构不抛出也不等于外部数据已经持久化。

6. 回滚与共享所有权

6.1 C++ vs Zig Failure Rollback

C++:

[机制片段 · 不承诺独立编译]

```
class Session {
    Buffer buffer_;
    Socket socket_;
};
```

如果 `socket_ constructor` throws:

```
buffer_ destructor runs automatically
```

Zig:

```
const buffer = try allocator.alloc(u8, 4096);
errdefer allocator.free(buffer);

const socket = try Socket.connect();
errdefer socket.close();
```

Zig 显式:

```
errdefer rollback
```

C++:

```
subobject lifetime rollback
```

可以总结:

```
C++
type/object-driven rollback

Zig
scope/control-flow-driven rollback
```

6.2 Shared Ownership

什么是真正 **Shared Ownership** ?

```
Owner A ┌
Owner B ┤ ──> Resource
Owner C └
```

三个 owner 都参与: resource 必须保持 alive。

只有:

```
last strong owner gone
```

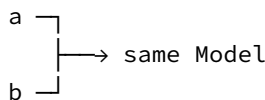
时 resource 才销毁。

shared_ptr Copy 不复制 Pointee

[机制片段 · 不承诺独立编译]

```
auto a = std::make_shared<Model>();
auto b = a;
```

结果:



复制的是: ownership handle。

不是 Model value。

Control Block

典型实现需要:

```
strong/reference count
weak count
deleter
allocation metadata
```

概念:



Resource 与 Control Block Lifetime

最后一个 strong owner 释放时执行被管理资源的释放协议; 只要仍有 strong owner 或 weak observer, 控制信息就还需要存在。只有两类关联都结束后, 控制块才可释放, 不能仅凭“外部 weak_ptr 数为 0”就推导控制块释放。

还要区分 managed resource 与 stored pointer: shared_ptr 的别名构造可让保存的指针与真正管理的对象不同。管理对象活着, 不自动证明任意别名目标仍有效。下方实验只使用普通 make_shared, 不覆盖自定义 deleter 和别名构造。

Shared Ownership 最大成本: Non-local Lifetime

Unique:

```
one owner
→ destruction point easy to reason
```

Shared:

```
many owners across program
→ last-owner point may be non-local
```

所以: deterministic rule, 不等于 locally obvious destruction。

6.3 std::weak_ptr

Weak Observer

weak_ptr: 不增加 strong ownership count, 因此不延长 pointee lifetime。

它观察：shared ownership domain。

lock()

[机制片段 · 不承诺独立编译]

```
if (auto model = weak.lock()) {
    model->run();
}
```

含义：

```
if object alive
→ acquire temporary strong ownership
→ safe to use during this owner lifetime
```

```
if expired
→ return empty shared_ptr
```

因此：lock() = lifetime re-acquisition attempt。

为什么不先 expired() ?

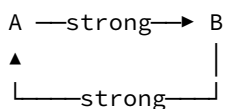
因为：

```
check alive
↓
time passes
↓
last owner disappears
↓
use
```

存在 TOCTOU race。

正确：直接 lock() 获得 lifetime right。

6.4 Shared Ownership Cycles



外部 owner 消失以后：

```
A count >= 1
B count >= 1
```

永不归零。

结果：leak。

Reference counting 无法自动收集 strong cycles。

打破 Cycle

典型：


```
Parent —strong→ Child
Parent ◀ · · weak · · — Child
```

反向关系: observation

而不是 ownership。

所以用:

[机制片段 · 不承诺独立编译]

```
std::weak_ptr<Parent>
```

或者普通 borrow, 取决于 topology。

6.5 Shared Ownership ≠ Thread Safety

shared_ptr control-block bookkeeping 对独立 shared handles 的并发 copy/destruction 提供相应线程安全语义。

但:

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<std::vector<int>>
```

并不会自动让:

[机制片段 · 不承诺独立编译]

```
values->push_back(...)
```

并发安全。

必须区分:

```
shared lifetime safety
≠
pointee data-race safety
```

7. 实验 · G2-L2: 弱观察不持有资源, lock 暂时持有

预测: 原 owner reset 后, weak 是否马上 expired? 先看局部 lease 是否还在。完整运行例:

[完整实验 · G2-L2 · main.cpp]

```
#include <iostream>
#include <memory>

struct Model {
    static inline int live = 0;
    Model() { ++live; }
    ~Model() { --live; }
};

int main() {
```

完整实验 · G2-L2 · main.cpp · continued

```

std::weak_ptr<Model> observer;
{
    auto owner = std::make_shared<Model>();
    observer = owner;
    auto lease = observer.lock();
    if (!lease || lease.get() != owner.get() || Model::live != 1) return 1;
    owner.reset();
    if (observer.expired() || Model::live != 1) return 2;
    lease.reset();
    if (!observer.expired() || Model::live != 0) return 3;
}
if (observer.lock()) return 4;
std::cout << "lease-keeps-alive; expired-after-release\n";
}

```

原因：weak 不增加 strong ownership；成功 lock 得到的 shared_ptr 会增加它。最后一份 strong owner 释放后，本例对象销毁，weak 仍可安全询问是否过期。

改变条件：在内层作用域额外保存 shared_ptr 副本，再观察 expired 何时成立。不要把“某个 owner reset”误认为“最后一个 owner reset”。

边界：本例不涉及并发，也不证明 Model 的读写线程安全；不同 shared_ptr 实例的控制块协作不等于同一个 shared_ptr 变量可以无同步并发改写。

8. 共享所有权的适用边界

8.1 Shared Ownership 的使用准则

优先只有在：

```

multiple independent components
really need to keep same resource alive

```

时使用。

典型：

```

independent async tasks
immutable model/config generations
callback lifetime
event fan-out

```

不要因为：

```

很多函数都需要访问
不知道谁 delete
传起来方便
担心 raw pointer

```

就直接升级到 shared_ptr。

9. 异步边界与停机

9.1 Async Boundary

同步：

[机制片段 · 不承诺独立编译]

```
void process(const Frame& frame);
```

caller lexical lifetime 可以保证：

Frame alive through call

异步：

[机制片段 · 不承诺独立编译]

```
executor.submit(...)
```

function 已返回。

因此：**Async boundary** 经常也是 **ownership boundary**。

Raw this

[机制片段 · 不承诺独立编译]

```
executor.submit([this] {
    work();
});
```

只 borrow。

必须保证：

*this outlives callback

Shared Self

[机制片段 · 不承诺独立编译]

```
executor.submit([self] {
    self->work();
});
```

callback 成为 owner。

Weak Self

[机制片段 · 不承诺独立编译]

```
executor.submit([weak] {
    if (auto self = weak.lock()) {
        self->work();
    }
});
```

仅保存 weak_ptr 的 callback 不在两次调用之间延长目标生命周期；成功 lock() 获得的局部 shared_ptr 会在本次使用期间延长它。

object 不存在时：no-op / cancel / domain-specific behavior。

9.2 enable_shared_from_this

如果 object 已经属于某个 shared ownership domain:

[机制片段 · 不承诺独立编译]

```
class Session : public std::enable_shared_from_this<Session> {
    ...
};
```

shared_from_this(): 获取属于同一 control block 的新 shared owner。

千万不要:

[机制片段 · 不承诺独立编译]

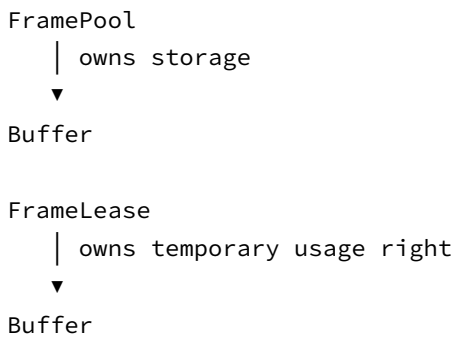
```
std::shared_ptr<Session>{this};
```

因为这会创建独立 control block, 可能造成: double deletion。

9.3 Pool + Lease

并不是所有 multi-consumer resource 都应该 shared-own。

例如 Frame buffer:



Consumer 不负责:

```
delete Buffer
```

而负责:

```
release lease
↓
return buffer to pool
```

所以 Lease 是另一种 ownership semantics。

RAII Lease

[机制片段 · 不承诺独立编译]

```
class FrameLease {
public:
    FrameLease(const FrameLease&) = delete;
    FrameLease& operator=(const FrameLease&) = delete;

    FrameLease(FrameLease&&) noexcept;
    FrameLease& operator=(FrameLease&&) noexcept;

    ~FrameLease() noexcept;

private:
    FramePool* pool_{nullptr};
    Frame* frame_{nullptr};
};
```

语义：

```
copy forbidden
move transfers lease
destructor returns resource to pool
```

RAII 依然成立。

9.4 Shutdown Architecture

Ownership 不只是 startup。

Shutdown 是 ownership/dependency graph 的镜像。

假设：

```
WorkerPool borrows Logger
WorkerPool borrows ModelManager
WorkerPool holds FrameLease
```

那么必须：

```
stop workers
↓
drain/cancel work
↓
join threads
↓
destroy tasks
↓
release FrameLeases
↓
destroy FramePool / ModelManager
↓
destroy Logger
```

原则：**Borrowers must end before their owners/backing resources。**

Member Declaration Order

C++ members:

[机制片段 · 不承诺独立编译]

```
class Runtime {  
private:  
    Logger logger_;  
    ModelManager models_;  
    FramePool frames_;  
    WorkerPool workers_;  
};
```

构造:

```
logger  
models  
frames  
workers
```

析构:

```
workers  
frames  
models  
logger
```

所以 member order 可以编码: destruction dependency。

不要用 Shared Ownership 掩盖 Shutdown Bug

如果 domain 要求:

```
Worker must stop before Logger
```

正确方案: 修 Worker shutdown/join protocol。

不是:

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<Logger>
```

让 Logger 神秘地继续活。

必须记: **Do not use shared ownership to hide an ordering bug.**

9.5 Healthy Ownership Graph

一个成熟系统通常希望:

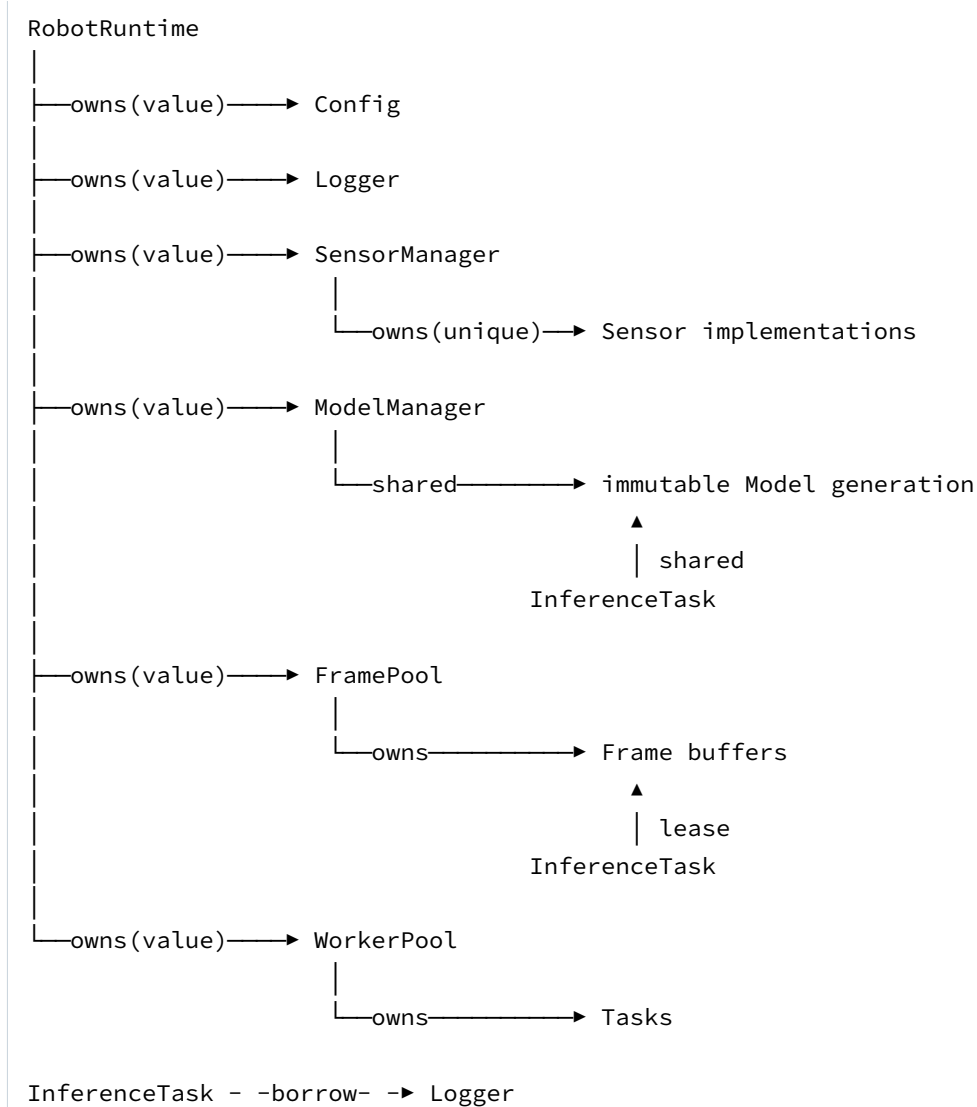
Broad Value / Unique Ownership Tree
 +
 Explicit Borrow Edges
 +
 Narrow Shared-Lifetime Islands
 +
 Lease / Pool where needed

而不是：

Everything is shared_ptr

10. 架构与跨语言回查

10.1 Robot Runtime 参考模型



这张图基本概括了 G2 的最终目标。

10.2 C++ vs Zig Ownership Architecture

维度	C++	Zig
Resource cleanup	destructor / RAI	defer / deinit
Unique owner type	可强编码	多依赖 API discipline
禁止 copy owner	= delete / member semantics	无直接等价语言机制
Ownership transfer	move semantics	explicit data-flow convention
Value ownership	强	强
Allocation policy	常由 owner abstraction 封装	allocator 通常显式
Borrow	T&, T*, span	*T, slice
Borrow lifetime proof	弱	弱
Shared ownership	shared_ptr	需要具体 abstraction
Weak observer	weak_ptr	需要具体 abstraction
Failure rollback	RAII / unwinding	defer / errdefer
Pool/Lease	dedicated RAI type	explicit lease + defer

核心差异：

C++
更多 ownership protocol
进入 type / object semantics
Zig
更多 allocation / cleanup protocol
留在显式 control flow

10.3 Rust 第三坐标

Rust：

Box<T>
Rc<T>
Arc<T>
Weak<T>
&T
&mut T

不仅表达 owner 类型，还静态检查更多 borrow/lifetime relationship。

但 Rust 仍不能替架构师决定：


```
Value or Box?
Unique or Arc?
Pool or Arc?
Generation or mutation?
Who should own whom?
```

所以：Ownership topology 始终是 architecture problem。

11. 工程审查与常见误判

11.1 Code Review Protocol

以后审查任何资源型 C++ 系统，按以下顺序。

Step 1 — Enumerate Resources

```
memory
fd
socket
thread
GPU buffer
model
frame
lock
transaction
```

Step 2 — Identify Natural Owner

问：谁应该执行最终 cleanup?

Step 3 — Prefer Value Ownership

如果 lifetime 与 parent 完全一致：

```
[机制片段 · 不承诺独立编译]
T member_;
```

Step 4 — Identify Real Dynamic Lifetime

若有：

```
runtime polymorphism
PImpl
stable address
dynamic tree
independent lifetime
```

才进入：

```
[机制片段 · 不承诺独立编译]
std::unique_ptr<T>
```

等 dynamic owner。

Step 5 — Mark Borrow Edges

```
T&
const T&
T*
span
string_view
iterator
```

并回答：谁必须 outlive 谁？

Step 6 — Find Ownership Transfer Boundaries

例如：

```
factory
queue submission
task creation
manager insertion
```

判断：

```
value
move
unique ownership transfer
shared ownership acquisition
lease transfer
```

Step 7 — Find Invalidators

```
destroy
erase
clear
reallocation
reload
pool reuse
shutdown
generation replacement
```

Step 8 — Find Shared Islands

问：多个独立参与者是否真的需要共同延长 lifetime？

Step 9 — Find Strong Cycles

```
A owns B
B owns A
```

决定：

```
weak edge
borrow edge
or redesign
```

Step 10 — Review Failure Paths

问：

```
partial construction?
allocation failure?
early return?
exception?
expected/error return?
```

是否：

```
no leak
invariants preserved
rollback semantics 明确
```

Step 11 — Review Async Captures

```
[this]
[raw_ptr]
[shared]
[weak]
[unique = std::move(...)]
```

每一种 capture 都是 lifetime semantics。

Step 12 — Review Shutdown Order

确保：borrowers/tasks end before resources they borrow。

11.2 高频 Smells

看到：

[机制片段 · 不承诺独立编译]

```
T* member_;
```

先问：owner or borrow?

看到：

[机制片段 · 不承诺独立编译]

```
delete member_;
```

问：为什么没有 RAII owner?

看到：

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<T>
```

问：为什么 domain 真需要 shared ownership?

看到：

[机制片段 · 不承诺独立编译]

```
const std::unique_ptr<T>&
```

问：函数真的需要 ownership handle，还是只需要 const T&?

看到：

```
[机制片段 · 不承诺独立编译]
[this]
```

进入 async callback：立即审查 object lifetime。

看到：

```
[机制片段 · 不承诺独立编译]
~Foo();
```

问：为什么 Foo 需要 user-declared destructor？是否影响 implicit move？

看到：

```
[机制片段 · 不承诺独立编译]
owner.reset();
owner = build_new();
```

问：为什么不先 prepare new state，再 commit？

看到：

```
[机制片段 · 不承诺独立编译]
std::move(x)
```

问：哪个后续 move operation 真正消费 xvalue？

11.3 高频错误直觉

错误

```
RAII = smart pointer。
```

正确：RAII 是 acquire/release protocol 与 object lifetime 的绑定。

错误

```
Unique ownership = unique_ptr。
```

正确：value member 本身经常就是最佳 unique owner。

错误

```
多个 users = shared ownership。
```

正确：多数 users 只是 borrowers。

错误

```
shared_ptr 是更安全的 unique_ptr。
```

正确：它表达不同、且更复杂的 ownership topology。

错误

`unique_ptr` 解决 dangling。

正确：它解决 owner uniqueness，不证明 borrow lifetime。

错误

Rule of Five 是现代 class 标准模板。

正确：高层 class 应尽量 Rule of Zero。

错误

`try/catch` = exception safety。

正确：exception/failure safety 是 failure 后的 state guarantee。

错误

`shared_ptr` thread-safe = T thread-safe。

错误。

错误

zero-copy = `shared_ptr`。

错误。

可能更正确的是：

Pool + Lease

错误

raw pointer 出现就是坏 C++。

错误。

Raw non-owning pointer 可以非常合理。

12. 术语与统一模型

12.1 核心术语表

English	核心含义
Resource	必须遵守 acquire/release protocol 的实体
Handle	对 resource 的标识/访问表示
Ownership	最终 lifetime/cleanup responsibility
Borrow	不拥有 resource 的临时访问关系
Unique Ownership	单一最终 owner
Shared Ownership	多 owner 共同延长 lifetime
Observer	非拥有访问角色
Weak Observer	不延长 shared lifetime 的 observer
RAII	resource lifetime → object lifetime
Move-only Type	copy 非法、move 转移状态/ownership
Rule of Five	raw-resource 类型的 special-member 全面审查
Rule of Zero	通过 RAII members 组合，外层无需手写 special members
Basic Guarantee	failure 后仍合法，无 leak，state 可改变
Strong Guarantee	commit-or-rollback
No-throw Guarantee	exception 不逃出 operation
Control Block	shared ownership bookkeeping
Lease	对 resource temporary usage right 的 owner
Ownership Graph	决定 resource destruction topology 的关系图
Invalidation	某操作使 borrow/view 不再合法
Shutdown Order	resource dependency 的反向 teardown 顺序

12.2 最终统一公式

G1 可以写：

```

Valid Access
=
Valid Storage
x
Active Object Lifetime
x
Correct Type
x
Bounds
x
Alignment
x
Access Rules

```

G2 补上:

```

Object Lifetime Stability
depends on
Ownership Architecture

```

最终:

```

Correct Native Resource Access
=
G1 Access Validity
+
G2 Ownership / Lifetime Coordination

```

也就是:

```

Can I access it?
  ▲
  |
  | depends on
  |
Will its owner keep it alive?

```

13. Final Gate

13.1 Final Gate

你应能够独立回答：

RAII

1. Resource 与 handle 为什么不同？
2. RAII 为什么不等于 smart pointer？
3. RAII 如何处理 early return 与 exception？
4. C++ RAII 与 Zig defer 的根本差异是什么？

Unique Ownership

1. Ownership 到底是什么责任？
2. 为什么不能复制同一资源的清理责任？深复制新资源为何是另一回事？
3. `std::move` 与真正 ownership transfer 分别在哪里发生？
4. `unique_ptr::get()` 和 `release()` 区别是什么？
5. 为什么 `unique_ptr` 仍可能产生 dangling borrow？

Rule of Zero

1. Rule of Three、Five、Zero 的区别是什么？
2. 为什么业务 class 手写完整 Rule of Five 常是 smell？
3. 为什么 `std::vector` / `std::string` / `unique_ptr` 能让外层 class 回归 Rule of Zero？
4. 为什么 `~T() = default` 仍值得审查 move generation？

Failure Safety

1. Basic Guarantee 与 Strong Guarantee 区别是什么？
2. 为什么 `prepare` → `commit` 是重要 mutation pattern？
3. constructor 中途 throw 时哪些 objects 会销毁？
4. 为什么 destructor 不应该让 exception 逃出？
5. Zig errdefer 与 C++ partial-construction rollback 有什么对应关系？

Shared Ownership

1. Aliasing 和 shared ownership 有什么区别？
2. control block 为什么存在？
3. strong count 降到 0 后为什么 control block 可能还存在？
4. 为什么 shared ownership destruction point 是 non-local？
5. 为什么 strong cycle 会 leak？
6. `weak_ptr lock()` 在 lifetime 上做了什么？

7. 为什么 `shared_ptr<T>` 不意味着 `T` thread-safe?

Architecture

1. 为什么 value member 应优先于 `unique_ptr`?
2. 多个 users 为什么不等于多个 owners?
3. Pool + Lease 与 `shared_ptr` 的语义差异是什么?
4. 为什么 shared ownership 应尽量限制成 narrow islands?
5. 为什么 shutdown order 是 ownership architecture 的组成部分?
6. 为什么 async boundary 经常也是 ownership boundary?
7. 为什么 `shared_ptr` 不能用于掩盖错误的 join/shutdown protocol?

14. Final Gate · 参考答案与常见误判

- **RAII 1 ~ 4:** 资源是受协议约束的能力，handle 只是标识；RAII 把协议绑定到对象清理，所以锁、文件和 lease 都能使用。已构造局部对象在正常退出与实际展开时清理，不保证终止路径展开。Zig defer 绑定词法作用域，C++ RAII 绑定对象；误判是把二者当作同一种隐式 GC。
- **Unique 1 ~ 5:** owner 负最终清理责任；不能浅复制同一资源的独占责任，但可明确 clone 独立资源。std::move 仅转换，转移在实际操作中发生。get 借用，release 交出指针并放弃管理而不释放资源；随后责任须有人接住。reset/销毁等仍可使 borrow 悬挂。
- **Rule of Zero 1 ~ 4:** Three 关注析构和复制，Five 加入移动，Zero 用成员类型承载这些语义。业务类手写五个成员会重复低层协议；并非禁止真正 owner 手写。成员语义能组合向外传播；用户声明的默认析构仍会影响隐式移动生成。误判是把 = default 当作“没有声明”。
- **Failure 1 ~ 5:** 基本保证保不变量与资源，强保证还保约定状态不变。prepare/commit 隔离可失败工作，但提交和清理要另证。非委托构造失败清理已完成子对象；委托构造目标成功后体抛出会析构完整对象。析构逃逸会破坏清理，展开时再次抛出还会终止。errdefer 按错误退出回滚，RAII 按子对象完成状态展开，机制不能完全互换。
- **Shared 1 ~ 7:** alias 只是访问，共享拥有才延长管理资源生命；控制块维护强/弱关联及释放信息。strong 归零后 weak 仍需观察元数据。最终释放位置由最后 owner 决定；强环让彼此无法归零。lock 临时取得 strong owner；引用计数安全不保护 pointee 的可变数据。误判是“用了 shared_ptr 就不需要锁”。
- **Architecture 1 ~ 7:** value member 少一层独立生命周期，非所有场景最优。多个用户可以只借用；pool 管存储，lease 管临时使用权。缩小共享区域便于定位最终释放者。shutdown 要先停生产者、终止并等待工作、再撤销依赖，具体顺序由依赖图决定。异步跨越调用作用域，必须重选拥有或借用协议；shared_ptr 不能替代停止、唤醒和 join。

完成标准：解释两项实验全部判据，并为一个异步 callback 写出 owner、borrow 和停机顺序；进阶题答不出时返回对应主题，而不是扩大本次必读范围。

15. 工程原则回查

15.1 如果几个月后只记住十五条

1. **Resource** 不等于 **memory**；**file**、**socket**、**lock**、**GPU buffer** 都有 **lifetime protocol**。
2. **Ownership** 是最终 **cleanup responsibility**，不是“谁拿着 **pointer**”。
3. **RAII** 把 **resource lifetime** 映射到 **C++ object lifetime**。
4. **Value ownership** 是最简单的 **ownership**，应优先考虑。
5. **unique_ptr** 表达 **dynamic unique ownership**，而不是 **unique ownership** 的唯一形式。
6. **Borrow** 可以有很多份，但 **borrower** 不决定 **resource lifetime**。
7. **get()** 是 **borrow**；**release()** 是把 **ownership responsibility** 重新降级为 **raw convention**。
8. **Rule of Zero** 是现代高层 **class** 的默认目标；**Rule of Five** 主要属于真正的低层 **raw-resource owner**。

9. **Failure-safe mutation** 应优先 **prepare → validate → no-throw commit → automatic cleanup**。
10. **Shared ownership** 表示多个独立 **owner** 共同延长 **lifetime**，而不是多个 **users**。
11. **weak_ptr** 不延长 **lifetime**，而通过 **lock()** 在使用前临时取得 **strong lifetime**。
12. **Shared ownership cycle** 是 **reference counting** 的结构性弱点。
13. **Pool owns storage; Lease owns temporary usage right**——两种 **responsibility** 不应混淆。
14. 健康的大型系统通常是 **value/unique ownership tree + explicit borrows + narrow shared islands**。
15. 语言可以帮助执行 **ownership contract**，但“谁应该拥有谁”始终是 **architecture decision**。

16. G2 → G3

本章的关键实验已纳入定向检查；进阶片段不因此变成全面验证的工程基线。

我们已经回答：**谁拥有 object/resource，它如何安全地活着和死去？**

G3 §1 接过这张所有权图，分析复制、移动、返回和容器迁移改变了哪些状态、产生哪些成本。本章已经说明的清理责任不再重新定义；异常与事务保证的细节回查 **FM-4**。

主线将进入：

```

Value Semantics
  ↓
Copy
  ↓
Move
  ↓
Copy Elision
  ↓
RVO / NRVO
  ↓
Pass by Value vs const&
  ↓
Container Relocation
  ↓
SSO / SBO
  ↓
Object Size / Allocation / Indirection
  ↓
Cache Locality
  ↓
Performance-aware API Design

```

也就是从：**Ownership Correctness**

正式进入：**Value Semantics + Performance Cost Model**。

G3 · 值语义、移动与成本模型

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.2.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G1/G2；覆盖值、复制/移动、返回、参数与表示成本；Zig/Rust 仅作参考对照。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G2 · 全系列导航 · 下一章：G4

阅读入口

本章的问题是：同样写 `auto b = a;`，为什么有时复制数据、有时只多一条访问路径？写 `std::move` 又改变了什么？

- 主阅读线：值与表示 → 复制实验 → 移动与消除 → 移动与返回实验 → 参数合同 → 容器迁移 → Final Gate。
- 回查层：按下面的主题目录查找接口、架构、跨语言和审查材料，不要求一次连续读完。
- 前置自测：能否画出 `vector`、`span`、`shared_ptr` 分别拥有和借用什么？若还混淆，先返回 G2。
- 完成目标：遇到复制/移动表达式，按“类型合同 → 候选操作 → 表示 → 实际成本”逐层判断。

带 `g-lab` 的块为完整定向实验。其余代码保留为片段和设计素材；成本公式是分析维度，不能代替测量。跨语言部分不在本批编译范围。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 逻辑值、表示与复制成本
- 2. 实验 · G3-L1：复制的是完整值，还是访问关系？
- 3. 平凡复制与对象表示
- 4. 移动、异常保证与复制消除
- 5. 实验 · G3-L2 ~ L4：转换、重载与结果对象
- 6. 返回值、参数与借用边界
- 7. 容器迁移与稳定身份
- 8. 对象表示策略

- 9. 值导向架构与生命周期解耦
- 10. 成本图与接口决策
- 11. 跨语言回查
- 12. 工程审查与常见误判
- 13. 术语与统一模型
- 14. Final Gate
- 15. Final Gate · 参考答案与常见误判
- 16. 工程原则回查
- 17. G3 → G4

1. 逻辑值、表示与复制成本

1.1 本章的工程问题

G1 §1 检查访问有效性，G2 §1 检查清理责任，G3 追问值在系统流动时支付了什么成本。正确性是前提，但“用了智能指针”或“写了 move”都不能直接推出性能。

本章按三步阅读表达式：类型表示什么逻辑值；操作允许改变哪些状态；表示如何映射到分配、元素操作和间接访问。

1.2 Logical Value 与 Representation

先问类型的逻辑值是什么

类型	逻辑值/责任	常见表示模型，非 ABI 保证
<code>vector<int></code>	拥有一串 <code>int</code>	外部元素存储，加指针/长度/容量等状态
<code>span<int></code>	借用一段连续 <code>int</code>	指针与范围；静态 <code>extent</code> 可编码在类型中
<code>unique_ptr<T></code>	独占资源或为空	指针及 <code>deleter</code> 状态
<code>shared_ptr<T></code>	共享拥有关系及保存的指针	对象指针与控制信息关联
<code>array<int, N></code>	内嵌 <code>N</code> 个 <code>int</code>	元素作为对象组成部分

这里的“常见表示”帮助估计成本，不允许据此访问私有布局、假定固定 `sizeof` 或跨 ABI 序列化。

Representation Size 不等于 Payload Size

`vector` 对象本身可以很小，拥有的序列却很大。`auto copy = values;` 的复制工作随元素及 `allocator` 合同变化，而不只处理 `sizeof(vector)` 个字节。先分清 `wrapper` 与 `payload`，再谈便宜还是昂贵。

1.3 Value Semantics

相同值，不是相同对象

[机制片段 · 不承诺独立编译]

```
std::string a{"hello"};
std::string b = a;
```

`b` 是新对象，初始字符串内容与 `a` 相同；修改 `b` 的字符不会因此修改 `a`。对一般 `T`，先查其复制合同，不能从赋值语法推导这种独立性。

Value Semantics 不等于 memcpy

复制操作要保持类型的语义合同。它可能逐成员复制、分配新 `payload`、更新引用计数或复制 `view` 元数据。`trivially copyable` 的表示复制规则也不赋予任意资源 `owner` 用 `memcpy` 克隆清理责任的许可。

View 的复制不是数据深复制

`span` 的逻辑值是访问范围，复制后仍指向同一底层数据。因此两个 `view` 对象独立，但其元素访问存在别名。说“复制成功”时必须说明复制的是 `view` 还是被观察的数据。

Shared Owner 的复制

[机制片段 · 不承诺独立编译]

```
auto a = std::make_shared<Model>();
auto b = a;
```

新建的是 `b` 这个 `shared_ptr` 对象，而不是第二个 `Model`。两个句柄共同参与管理同一资源；若需独立模型，应定义模型复制/clone 的操作，不要把复制 `shared_ptr` 当作快照。

Move-only 仍可表达完整的值抽象

`FileDescriptor`、`unique_ptr`、`FrameLease` 可以有明确的空/拥有状态和转移协议，只是不允许复制同一责任。不可复制不是“没有值语义”；是否能深复制独立资源仍由类型合同决定。

1.4 Copy Cost Model

同一语法对应不同工作

`int` 复制标量；`array<T, N>` 处理内嵌元素；`vector<T>` 复制时可能分配并复制 `N` 个元素；`span` 复制范围；`shared_ptr` 复制管理关系并更新计数。语法越相似，越需要回到类型。

sizeof 只描述对象本体

对内嵌定长数据，复制成本可能与大小相关；对外部拥有者，`payload` 规模可能远大于对象本体。也不能把更大的 `sizeof` 直接解释成更慢：需要结合访问模式、编译器优化与硬件。

Inline 与 Indirect

`inline` 把数据放在对象内部，减少访问跳转，但迁移对象需要处理内嵌数据。`indirect` 多一层访问，却可能仅交接外部存储句柄。所谓“移动便宜”常来自这一表示选择，而不是关键字本身。

成本递归组合

[机制片段 · 不承诺独立编译]

```
struct RobotState {
    std::string name;
    std::vector<float> joints;
    Pose pose; // 片段: Pose 由应用定义。
};
```

分析其复制，要分别检查三个成员；`vector<T>` 又要检查每个 `T`。可用“分配+元素操作+同步+间接访问”列账，但它不是可直接相加的物理耗时公式，缓存与优化会改变结果。

复制构造与复制赋值不是同一成本

`T b = a`；初始化新对象，`b = a`；必须处理 `b` 的旧状态。已有容量可能复用，旧资源可能需要释放，`allocator` 条件也可能改变路径。`benchmark` 若把二者混测，就无法解释差异。

O(1) 不等于 Cheap

裸指针复制、共享计数更新、大型但固定大小的结构复制，都可能相对某个 `N` 是 `O(1)`，常数和机器行为却不同。报告成本时说明 `N` 是什么，并观察字节、分配、元素操作、同步、缓存和延迟分布。

2. 实验 · G3-L1: 复制的是完整值，还是访问关系？

预测：修改 `vector` 副本、`span` 副本、`shared_ptr` 副本各会影响谁？完整运行例使用完整范围比较，既检查内容，也检查长度，避免空结果被前缀比较误判通过。

[完整实验 · G3-L1 · main.cpp]

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <span>
#include <vector>

int main() {
    const int expected[]{1, 2, 3};
    std::vector<int> source{1, 2, 3};
    auto copy = source;
    if (!std::ranges::equal(copy, expected)) return 1;
    if (copy.data() == source.data()) return 2;
    copy[0] = 9;
    if (source[0] != 1) return 3;

    std::span<int> view{source};
    auto view_copy = view;
    view_copy[1] = 8;
    const int changed[]{1, 8, 3};
    if (!std::ranges::equal(source, changed) ||
        view_copy.data() != source.data()) return 4;
```


完整实验 · G3-L1 · main.cpp · continued

```

    auto owner = std::make_shared<int>(4);
    auto shared = owner;
    *shared = 7;
    if (shared.get() != owner.get() || *owner != 7) return 5;
    std::cout << "value-independent; view-aliases; shared-aliases\n";
}

```

原因：vector 副本拥有独立元素；span 副本复制访问范围；shared_ptr 副本复制共同管理同一对象的句柄。三个语法相似的 copy，复制的逻辑值并不相同。

改变条件：把 vector 副本缩成一个元素，范围相等检查必须失败；只比较副本长度的前缀会漏掉这个错误。此变体作为本批执行器的定向自查，不改正文正确实现。

边界：这里元素是 int。若 vector 的元素本身是 span 或 shared_ptr，复制外层容器不会递归消除内层别名。

3. 平凡复制与对象表示

3.1 Triviality

Trivial Copy

例如：

[机制片段 · 不承诺独立编译]

```

struct Sample {
    std::uint64_t timestamp;
    float value;
    std::uint32_t signal_id;
};

```

可以检查：

[机制片段 · 不承诺独立编译]

```

static_assert(std::is_trivially_copyable_v<Sample>);

```

这种类型非常适合：

```

dense arrays
bulk data movement
telemetry
sensor samples

```

trivially copyable 不等于 Wire Format

仍然要考虑：


```
endianness
padding
ABI
alignment
schema version
floating representation
```

所以：

[机制片段 · 不承诺独立编译]

```
send(fd, &sample, sizeof(sample), ...);
```

不自动成为正确跨系统 serialization。

4. 移动、异常保证与复制消除

4.1 Move Semantics

Copy 与 Move 的根本区别

Copy：

[机制片段 · 不承诺独立编译]

```
T b = a;
```

要求：

```
a logical value preserved
b obtains equivalent logical value
```

Move：

[机制片段 · 不承诺独立编译]

```
T b = std::move(a);
```

允许：

```
a logical state changes
b reuses/transfers a's existing state/resource
```

所以 move 的性能潜力来自：**Source value no longer needs to be preserved.**

从转换到真正操作，中间还有重载决议

问题：如果类型只有复制构造，写 `std::move` 会强行生成移动构造吗？

不会。对对象表达式，`std::move` 提供 `xvalue`（值类别定义见 [G1 §5](#)）；其概念形式是 `static_cast<std::remove_reference_t<T>&&>(x)`，其中 `T` 是推导出的模板参数。后续初始化、赋值或消费函数才选择并执行实际操作。

不能把“调用者允许消费源状态”说成语言已经证明可以安全消费。候选中可能有移动、有可绑定右值的复制，或者有被删除的最优函数而直接报错。“没声明移动”和“显式删除移动”不是同一个重载集合。下面实验先验证最常混淆的 `const` 和 `copy-only` 两条路径。

为什么普通 **vector** 移动构造很便宜？

vector 的元素通常在外部分配的连续存储中，普通 **vector(vector&&)** 可以交接这份存储，复杂度为常数；复制构造需复制元素，工作量随 **N** 增长。

边界：显式带 **allocator** 的移动构造，以及移动赋值，在 **allocator** 不兼容或不能传播时可能逐元素处理；不能把所有带 **move** 字样的操作都标成 **O(1)**。目标原有资源的清理也可能是成本。**array** 的元素内嵌，不能只交接一个外部缓冲区句柄。

为什么 **Array Move** 仍可能 **O(N)**

[机制片段 · 不承诺独立编译]

```
std::array<T, N>
```

elements inline。

没有：

```
allocation handle
```

可以直接 transfer。

所以：

```
move array
≈ move N elements
```

对于 scalar **T**：

```
move ≈ copy
```

因此：**Move syntax does not imply O(1).**

Move Cost 来自 **Representation**

```
Indirect owner
→ transfer handle
→ often cheap

Inline payload
→ transfer payload itself
→ often similar to copy
```

Moved-from ≠ Dead

移动后，源对象通常仍在生命周期内，但值可能改变。标准库类型通常给出 **valid-but-unspecified** 的移动后合同，具体类型可给更强保证，例如相应 **unique_ptr** 转移后源为空。

对用户类型，语言不会自动修复错误的移动实现；要自行维持承诺的不变量和清理能力。操作还有自身前置条件：不能因为 **vector** 可析构，就无条件调用它的 **front()**。也不要要求所有 **moved-from** 容器都为空。

区分三件事：对象是否仍活着、哪些操作允许、逻辑值是否保持。它们不是同一个问题。

const 与 **Move**

[机制片段 · 不承诺独立编译]

```
const T value;
T copy = std::move(value);
```

`std::move(value)` 产生:

```
const T&&
```

而普通 `move constructor`:

[机制片段 · 不承诺独立编译]

```
T(T&&)
```

通常无法接受 `const source`。

因为 `move` 经常需要: 修改 `source state`。

所以经常回退:

```
copy via const T&
```

必须记: **`std::move(const T)` 经常不会真正 `move`。**

4.2 noexcept Move

noexcept 不只是 Optimization Hint

[机制片段 · 不承诺独立编译]

```
T(T&&) noexcept;
```

表达: `move` 不会让 `exception` 逃出。

这会直接影响:

```
generic algorithms
containers
vector relocation
```

为什么 **Vector** 会关心?

reallocation:

```
allocate new storage
↓
construct old elements in new storage
```

若 `move`:

```
可能修改 source
并可能 throw
```

做到一半失败: `rollback` 困难。

若 `copy`:

```
source unchanged
```

新 `storage` 失败: 可以直接丢弃 `partial copies`。

所以：

```
noexcept move
```

通常让 container 更安全地选择 cheap move path。

Move-only + Throwing Move

这是非常棘手组合：

```
copy impossible
move may throw
```

container 不能 fallback copy。

因此对 resource owner：

```
move-only
O(1)
noexcept
simple empty state
```

是很好的设计目标。

4.3 Copy Elision / RVO / NRVO

最好的 Move 是没有 Move

现代性能层级经常是：

```
Copy
↓
Move
↓
Direct Construction / Elision
```

C++17+ Same-type Prvalue

[机制片段 · 不承诺独立编译]

```
T make() {
    return T{};
}
```

现代模型：

```
result object
↓
T constructed directly there
```

不是：

```
temporary T
↓ move
result T
```

因此：move constructor 甚至可以不存在。

Guaranteed Case

[机制片段 · 不承诺独立编译]

```
class Immovable {
public:
    Immovable() = default;
    Immovable(const Immovable&) = delete;
    Immovable(Immovable&&) = delete;
};

Immovable make() {
    return Immovable{};
}
```

C++17+ 可以成立。

这证明：guaranteed copy elision 不是“更快的 move”。

NRVO

[机制片段 · 不承诺独立编译]

```
T make() {
    T value;
    return value;
}
```

value 是 named local。

这是：Named Return Value Optimization。

NRVO 成功：

```
local value
和
function result
实际上对应同一个最终 object
```

Guaranteed Prvalue Case ≠ NRVO

必须区分：

[机制片段 · 不承诺独立编译]

```
return T{};
```

和：

[机制片段 · 不承诺独立编译]

```
T value;
return value;
```

前者 C++17+ same-type result construction 有更强语义保证。

后者：NRVO 仍是 permitted/expected，但不是同等级 guaranteed case。

NRVO Failure → Implicit Move

[机制片段 · 不承诺独立编译]

```
T make() {
    T value;
    return value;
}
```

如果 NRVO 不发生，eligible local 可走：

```
implicit move
```

所以通常不需要：

[机制片段 · 不承诺独立编译]

```
return std::move(value);
```

return std::move(local) 是典型 Smell

[机制片段 · 不承诺独立编译]

```
T make() {
    T value;
    return std::move(value);
}
```

std::move(value) 是 xvalue。

通常：不再保持简单 NRVO candidate 形式。

因此可能从：

```
0 transfers
```

退化成：

```
1 move
```

所以 ordinary local return 默认：

[机制片段 · 不承诺独立编译]

```
return value;
```

5. 实验 · G3-L2 ~ L4: 转换、重载与结果对象

G3-L2: const 与 copy-only 的反直觉路径

先写出四次构造分别调用什么。计数来自明确定义的构造函数，不测 vector 的实现选择。

[完整实验 · G3-L2 · main.cpp]

```
#include <iostream>
#include <utility>

struct Tracked {
    static inline int copies = 0, moves = 0;
    Tracked() = default;
```

完整实验 · G3-L2 · main.cpp · continued

```

    Tracked(const Tracked&) { ++copies; }
    Tracked(Tracked&&) noexcept { ++moves; }
};

struct CopyOnly {
    static inline int copies = 0;
    CopyOnly() = default;
    CopyOnly(const CopyOnly&) { ++copies; } // 没有声明移动构造。
};

int main() {
    Tracked source;
    Tracked a{source};
    Tracked b{std::move(source)};
    const Tracked fixed;
    Tracked c{std::move(fixed)};
    CopyOnly original;
    CopyOnly d{std::move(original)};
    if (Tracked::copies != 2 || Tracked::moves != 1 ||
        CopyOnly::copies != 1) return 1;
    std::cout << "copies=2 moves=1 copy-only=1\n";
}

```

a 复制, b 移动, c 的 `const` 右值不能绑定普通 `Tracked&&` 因而复制; d 可绑定 `const CopyOnly&`, 所以复制。显式删除 `CopyOnly(CopyOnly&&)` 会让它成为更优但不可调用的候选, 这是手动扩展, 不能继续期待复制回退。

G3-L3: 不需要复制/移动的 `prvalue` 返回

[完整实验 · G3-L3 · main.cpp]

```

#include <iostream>

struct Immovable {
    int value;
    explicit Immovable(int n) : value(n) {}
    Immovable(const Immovable&) = delete;
    Immovable(Immovable&&) = delete;
};

Immovable make() { return Immovable{7}; }

int main() {
    auto value = make();
    if (value.value != 7) return 1;
    std::cout << "direct-result=7\n";
}

```

这里同类型 `prvalue` 直接初始化结果对象, 不需要先建立一个源临时再移动。删除复制/移动仍可成立; 这不同于“优化器碰巧删掉了一次 `move`”。

G3-L4: 具名局部返回不能靠可选 NRVO 挽救非法程序

完整编译失败反例:

[反例 · 编译失败 · G3-L4 · main.cpp]

```
struct Immovable {
    Immovable() = default;
    Immovable(const Immovable&) = delete;
    Immovable(Immovable&&) = delete;
};

Immovable make() {
    Immovable local;
    return local;
}

int main() { auto value = make(); }
```

NRVO 是允许的省略，不让本来选到已删除构造函数的程序变合法。预期是针对 `Immovable` 的 `deleted-constructor` 诊断，不是任何编译失败。

反思: `return local;` 符合 NRVO 条件时应保留机会; 改成 `return std::move(local);` 会使这条 NRVO 规则不再适用。不要用一个编译器、一次 `-O0` 计数结果宣称所有返回都复制或都不复制。
N4950: [class.copy.elision](#)

6. 返回值、参数与借用边界

6.1 Return-by-Value Cost Model

看到:

[机制片段 · 不承诺独立编译]

```
T result = make_t();
```

不要问: “复制几次?”

而应该拆成:

1. ConstructCost(T)
2. Result placement
3. Transfer cost
4. Dynamic resource work
5. ABI lowering

现代理想:

```
TransferCost = 0
```

但:


```
construct payload
allocate
I/O
decode
```

这些真实工作不会因为 elision 消失。

6.2 Parameter Passing

参数类型首先是 Semantic Contract

不是：

```
哪个写法最快？
```

而是：

```
callee 要：
borrow?
own?
consume?
mutate?
view?
share?
```

T

[机制片段 · 不承诺独立编译]

```
void f(T value);
```

表示：callee 获得自己的 T value。

caller lvalue：

```
copy
```

caller xvalue：

重载决议可能选择 move、copy，或因无可行操作而失败

same-type prvalue：

direct parameter construction 可能消除不必要 transfer

const T&

[机制片段 · 不承诺独立编译]

```
void f(const T& value);
```

表示：required read-only borrow。

适合：

```
large expensive-to-copy value
callee 只同步读取
```

T&

[机制片段 · 不承诺独立编译]

```
void f(T& value);
```

表示：required mutable borrow。

函数修改：caller-visible object。

T&&

非模板：

[机制片段 · 不承诺独立编译]

```
void consume(T&& value);
```

表示：rvalue reference to a consumable object。

但：reference 本身不是 ownership transfer。

真正 transfer 通常发生在：

[机制片段 · 不承诺独立编译]

```
store(std::move(value));
```

Named T&& 仍然是 Lvalue Expression

[机制片段 · 不承诺独立编译]

```
void consume(T&& value) {
    sink(value);           // lvalue
    sink(std::move(value)); // xvalue
}
```

类型：

```
T&&
```

与表达式 category：

```
lvalue
```

必须分开。

6.3 Sink Parameters

如果函数最终需要：保存自己的 value。

典型：

[机制片段 · 不承诺独立编译]

```
class Robot {
public:
    explicit Robot(std::string name)
        : name_(std::move(name)) {}

private:
    std::string name_;
};
```

这里：

```
T by value
+
move into member
```

常适合：

```
copy expensive
move cheap
```

的 type。

例如：

```
string
vector
Frame
```

Lvalue Caller

```
copy into parameter
+
cheap move into member
```

Rvalue Caller

```
move/direct construct parameter
+
cheap move into member
```

所以：by-value sink 是 API simplicity 与性能之间很好的折中。

6.4 Small Values

例如：

[机制片段 · 不承诺独立编译]

```
struct RobotId {
    std::uint64_t value{};
};
```

通常：

[机制片段 · 不承诺独立编译]

```
Robot* find_robot(RobotId id);
```

优于：

[机制片段 · 不承诺独立编译]

```
Robot* find_robot(const RobotId& id);
```

原因：

```

small
cheap copy
register-friendly
no aliasing relation
no lifetime dependency

```

Strong Small Types

包装：

```

uint64_t → RobotId
int64_t → Timestamp

```

通常不产生有意义性能损失。

却带来：

```

type safety
API clarity
semantic distinction

```

这是典型 zero-overhead abstraction。

6.5 Views

`std::span<T>`

[机制片段 · 不承诺独立编译]

```
void process(std::span<const Sample> samples);
```

表达：

```

non-owning
contiguous
bounded
read-only sequence view

```

它不要求 caller 是：

```
vector
```

所以 API 表达的是：data shape

而不是：owner/container representation。

为什么 Span By Value

span 本身通常只是：

```

pointer
+
extent

```

cheap to copy。

所以：

[机制片段 · 不承诺独立编译]

```
void process(std::span<const T> values);
```

而不是：

[机制片段 · 不承诺独立编译]

```
void process(const std::span<const T>& values);
```

std::string_view

同样：

[机制片段 · 不承诺独立编译]

```
void parse(std::string_view text);
```

适合：character-sequence borrow。

但如果函数/class 需要长期保存：string_view 并不拥有 characters。

需要重新做 ownership decision。

6.6 Borrow vs Ownership Boundary

同步：

[机制片段 · 不承诺独立编译]

```
void inspect(const Frame& frame);
```

owner 在 caller, borrow 只覆盖 call。

异步：

[机制片段 · 不承诺独立编译]

```
submit(...)
```

调用返回后 task 继续运行。

因此：**Async boundary often becomes ownership boundary**。

不能把：

[机制片段 · 不承诺独立编译]

```
const Frame&
span
T*
```

随便保存进 async callback。

必须选择：

```
copy
move
shared ownership
lease
```

之一。

6.7 Aliasing 与 Parameter Performance

Reference:

[机制片段 · 不承诺独立编译]

```
void f(const Vec3& a, const Vec3& b);
```

允许潜在:

a and b alias

Value:

[机制片段 · 不承诺独立编译]

```
void f(Vec3 a, Vec3 b);
```

parameters 是独立 local values。

对 small scalar/aggregate, by value 可能:

更容易 registerize

减少 aliasing constraints

减少 lifetime relationships

所以: reference 不只是“省 copy”。

它也改变 optimizer 的 memory model。

7. 容器迁移与稳定身份

7.1 Container Relocation

size vs capacity

size

=

live T object count

capacity

=

storage capacity for up to N T objects

所以:

[机制片段 · 不承诺独立编译]

```
values.reserve(100);
```

不是: 构造 100 个 T。

而是: 准备 storage。

Reallocation

capacity 不足:

```
old storage
[T][T][T][T]
```

↓ allocate bigger storage

```
new raw storage
```

↓ construct new T objects from old values

↓ destroy old T objects

↓ deallocate old storage

重点：这是 **object lifetime migration**，不只是 **bytes copy**。

Move Constructor ≠ Object Teleportation

[机制片段 · 不承诺独立编译]

```
T b = std::move(a);
```

是：

```
new T object b begins lifetime
a remains alive
```

不是：同一个 object 直接改地址。

所以：

```
Move
≠
Relocation
```

Reallocation Invalidation

old element lifetimes 结束。

所以：

```
T*
T&
iterator
```

若指向旧 vector elements：reallocation 后失效。

pointer object 自己并没改变。

是：backing object relationship 失效。

7.2 reserve()

reserve() 价值不仅是：

```
reduce allocations
```

还可以：

```

reduce element moves/copies
reduce address invalidation
move latency out of hot path
improve tail-latency predictability

```

但：

```

[机制片段 · 不承诺独立编译]
values.reserve(n);

```

本身如果 capacity 不足，也会：reallocate 并立即 invalidate 旧 element references。

7.3 Amortized Complexity

vector 几何增长使：

```

push_back
→ amortized O(1)

```

但单次 expansion：

```

O(N)

```

所以：amortized O(1) ≠ every operation O(1)。

对于 realtime：

```

average latency

```

可能不重要。

更关心：

```

worst-case / bounded latency

```

因此：

```

reserve
fixed-capacity storage
ring buffer
pool

```

经常重要。

7.4 vector<T> vs vector<unique_ptr<T>>

Dense Value Storage

```

[机制片段 · 不承诺独立编译]
std::vector<Detection>

```

布局：

```

[D][D][D][D]

```


优点：

```
contiguous
few allocations
prefetch-friendly
cache-friendly
```

缺点：

```
element addresses can change on reallocation
```

Pointerized Storage

[机制片段 · 不承诺独立编译]

```
std::vector<std::unique_ptr<Detection>>
```

布局：

```
[p][p][p][p]
|  |  |  |
▼  ▼  ▼  ▼
D  D  D  D
```

优点：

```
pointee address stable across outer vector growth
dynamic polymorphism friendly
```

代价：

```
N allocations
N indirections
fragmentation
poor locality
```

因此：不要仅为地址稳定就 pointerize small value types。

7.5 Stable ID vs Stable Address

如果 domain identity 是：

[机制片段 · 不承诺独立编译]

```
TrackId{42}
```

那么 Track object storage 可以：

```
move
compact
reallocate
```

只要 TrackId 不变。

因此：**Address is a storage property, not necessarily domain identity。**

许多系统更适合：

```
stable ID
+
central owner
```

而不是:

```
heap everything to get stable addresses
```

8. 对象表示策略

8.1 Representation Strategies

Inline

```
payload inside object
```

例如:

```
int
Vec3
array<T,N>
```

优点:

```
no allocation
good locality
simple lifetime
```

缺点:

```
object large
move may be same as copy
```

Indirect

```
small wrapper
→ external payload
```

例如:

```
vector
PImpl
heap buffer
```

优点:

```
small object
stable payload address possible
cheap move
```

代价:

```
allocation
indirection
cache miss
```

Hybrid

```
small runtime value
→ inline

large runtime value
→ indirect
```

例如常见：

```
std::string SSO
small-function/SBO wrappers
small-vector abstractions
```

8.2 SSO / SBO

Small String Optimization

常见 `std::string` 实现：

```
short string
→ characters inline

long string
→ heap storage
```

因此：

```
short copy
→ maybe no allocation

long copy
→ allocation + O(N)

short move
→ small inline transfer

long move
→ handle transfer
```

SSO Threshold 不是 Portable Guarantee

不要依赖：

```
<= 15 / 22 / 23 chars
```

这种具体值。

它属于：

```

stdlib implementation
ABI
toolchain

```

SBO Trade-off

```

larger wrapper
↕
fewer small-object allocations

```

如果 inline buffer 太大：

```

cache density ↓
vector relocation bytes ↑
object footprint ↑
ABI constraints ↑

```

所以：SBO 不是越大越好。

8.3 Performance Cliff

Hybrid representation 存在 threshold：

```

size <= N
→ inline

size > N
→ heap

```

因此 runtime cost 可能：

```

N
→ no allocation

N+1
→ allocation + migration

```

产生：performance cliff。

所以 benchmark 不能只测：

```

one tiny string

```

应看真实：

```

value-size distribution
histogram
percentiles

```

8.4 optional<T>

std::optional<T> 概念：

```
storage for T
+
engaged state
```

empty:

```
T storage exists
T lifetime inactive
```

engaged:

```
T object lifetime active
```

再次体现：Storage exists $\neq$ object alive。

optional<Large> Empty 仍可能很大

因为 wrapper 必须：随时能够 inline 容纳一个 Large。

所以：

```
empty optional
≠
pointer-sized
```

大量 sparse large optionals 可能并不适合 inline representation。

8.5 variant

[机制片段 · 不承诺独立编译]

```
std::variant<A, B, C>
```

需要：

```
storage for largest alternative
+
active tag
```

因此 size 通常主要由：

```
max(sizeof(A), sizeof(B), sizeof(C))
```

决定。

运行时 operation cost：取决于当前 active alternative。

还要处理可能的 `valueless_by_exception` 状态：某些抛异常的类型改变操作之后可能没有 active alternative。存储足够容纳候选类型，不等于一直存在其中一个活对象。

所以：

```
same static type
different runtime state
different operation cost
```

9. 值导向架构与生命周期解耦

9.1 Value-oriented Architecture

高性能 C++ 不等于：

```
everything inline
everything by value
```

真正目标：

```
small metadata
→ value

large unique payload
→ movable owner

synchronous algorithm input
→ borrow/view

independent immutable generations
→ narrow shared ownership

pooled resource
→ lease

hot collections
→ dense value containers

identity
→ stable ID when possible
```

9.2 Large Logical Value 可以仍然是 Small Movable Object

例如：

[机制片段 · 不承诺独立编译]

```
struct InferenceRequest {
    RequestId id;
    Timestamp timestamp;
    std::shared_ptr<const Model> model;
    FrameLease frame;
};
```

它代表：

```
large model
large frame
```

但 object representation 可能只有：

```
small values + small handles
```

Move:

```
RequestId
→ cheap

Timestamp
→ cheap

shared_ptr
→ handle transfer

FrameLease
→ lease transfer
```

所以: **Logical payload size $\neq$ object size $\neq$ move cost.**

9.3 Copy as Decoupling

Copy 不只是: data movement.

它还可以购买:

```
lifetime independence
thread independence
storage abstraction
generation independence
```

例如 small metadata:

[机制片段 · 不承诺独立编译]

```
FrameMetadata metadata = frame.metadata();
```

几十 bytes copy。

换: Detection result 不再依赖 Frame lifetime。

这可能是优秀 trade-off。

9.4 Zero-copy

Zero-copy 通常真正表示: 避免主要 large payload duplication。

仍会复制:

```
metadata
pointer
size
lease
handle
```

所以: Zero-copy $\neq$ zero data movement。

Zero-copy 的代价

```
less memory bandwidth
↓
more lifetime coupling
aliasing
reuse coordination
possibly synchronization
```

必须记: **Zero-copy turns bandwidth cost into coordination cost.**

Shared Immutable Data

如果多个 consumers 共享大 payload, 最好:

```
shared
+
immutable
```

这样主要解决:

```
lifetime
```

而不是额外解决:

```
data race
synchronization
consistency
```

9.5 Pool + Lease

Pool:

```
owns backing storage
```

Lease:

```
owns temporary usage right
```

不是:

```
lease owns allocation
```

cleanup:

```
last/use lease ends
→ return slot to pool
```

而不是:

```
delete buffer
```

Pool Stale View

Pool reuse 时:


```
same memory address  
new generation
```

旧 span/pointer 可能：

```
address still mapped  
but logical data generation changed
```

因此：address-valid $\neq$ lifetime-valid $\neq$ generation-valid。

ASan 也未必能发现。

10. 成本图与接口决策

10.1 Cost Graph

高性能代码不要只画 Ownership Graph。

还应该画：

10.2 Value/Copy/Move Graph

```
where values are:  
copied  
moved  
directly constructed  
borrowed
```

10.3 Allocation Graph

```
startup allocations  
per-frame allocations  
per-item allocations  
control blocks  
temporary storage
```

10.4 Indirection Graph

```
object
↓
pointer
↓
object
↓
PImpl
↓
payload
```

每个 pointer 可能增加: cache-miss risk。

10.5 Lifetime/Invalidation Graph

```
who borrows?
what invalidates?
reallocation?
erase?
pool reuse?
generation replacement?
shutdown?
```

10.6 Performance Cost Axes

完整 cost model:

1. Bytes moved
2. Allocation count
3. Deallocation count
4. Object construction/destruction
5. Element copy/move
6. Atomic/refcount operations
7. Locks
8. Pointer indirections
9. Cache footprint
10. Reallocation
11. Invalidation
12. Lifetime coordination
13. Tail latency
14. ABI/layout constraints

所以: **Big-O is necessary but not sufficient.**

10.7 Copy Cost Equation

工程近似：

```
CopyCost(T)
≈
InlineBytesMoved
+
Allocations
+
OwnedPayloadCopy
+
Σ MemberCopyCost
+
Synchronization
+
Cache/MemoryTraffic
+
RepresentationTransition
```

Move：

```
MoveCost(T)
≈
StateTransfer
+
DestinationOldStateCleanup
+
Σ MemberMoves
+
AllocatorConstraints
+
InvariantRepair
+
Synchronization
```

10.8 Parameter Decision Model

先问语义：

```
Does callee:
borrow?
own?
mutate?
consume?
view?
share?
```

然后才问性能。

默认：

语义	C++ 首选起点
small independent value	T
callee needs own value	T
large read-only borrow	const T&
required mutable borrow	T&
optional observer	T*
contiguous sequence borrow	std::span<T>
text borrow	std::string_view
unique owner transfer	std::unique_ptr<T>
shared lifetime acquisition	std::shared_ptr<T>
move-only lease/value transfer	T
explicit consume-only existing value	sometimes T&&

10.9 Container Review Model

看到：

[机制片段 · 不承诺独立编译]

std::vector<T>

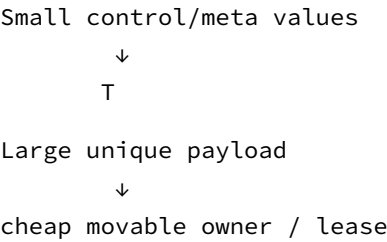
问：

- T copy cost?
- T move cost?
- T move noexcept?
- T address-sensitive?
- T value-like?
- stable address required?
- growth frequency?
- reserve possible?
- what borrows elements?
- what invalidates them?

Container performance: 是 element semantics 与 storage topology 的组合。

10.10 Healthy High-performance Pipeline

典型：



流程示意 · continued

Synchronous stage access

↓

span / const& / T&

Independent large immutable generation

↓

shared_ptr<const T>

Long-lived identity

↓

stable logical ID

Hot collections

↓

dense contiguous values

11. 跨语言回查

11.1 C++ vs Zig

C++

copy/move/destructor
→ heavily encoded in type operations

value syntax
→ can hide expensive semantic work

RVO/NRVO/elision
→ powerful destination construction

Zig

ordinary value operations simpler
clone/resource duplication generally explicit
allocator commonly explicit
result-location semantics

所以：

C++
more semantic machinery behind ordinary syntax

Zig
more operational cost visible in APIs/control flow

11.2 C++ vs Rust

Rust:

```
non-Copy T assignment  
→ ownership move  
  
source usually becomes statically unusable
```

C++:

```
lvalue  
→ copy if copyable  
  
std::move(x)  
→ explicitly expose xvalue  
source remains live moved-from object
```

Rust 静态保证更多 lifetime 规则。

但:

```
cache  
layout  
allocation  
Vec relocation  
Arc refcount  
AoS/SoA
```

仍然必须由工程师设计。

12. 工程审查与常见误判

12.1 Code Review Protocol

对 performance-sensitive code:

Step 1 — Identify Logical Values

这个 type 到底表示什么?

Step 2 — Classify Representation

```
inline  
indirect  
hybrid
```

Step 3 — Locate Large Payloads

```
vectors  
buffers  
images  
matrices  
strings  
models
```

Step 4 — Mark Every Copy

判断：

```
small value copy?  
deep payload copy?  
view copy?  
shared ownership copy?
```

Step 5 — Mark Every Move

问：

```
who consumes?  
what is transferred?  
cheap?  
O(1)?  
noexcept?
```

Step 6 — Review Returns

[机制片段 · 不承诺独立编译]

```
return T{};  
return local;  
return std::move(local);
```

明确：

```
guaranteed direct construction  
NRVO  
pessimizing move
```

Step 7 — Review Parameters

确认接口真正表达：

```
borrow  
own  
consume  
view  
share
```

Step 8 — Review Containers

问：

```

why this container?
why pointerized?
why not dense values?
reserve?
invalidation?

```

Step 9 — Count Allocations

特别查：

```

per item
per request
per frame
per callback

```

Step 10 — Count Indirections

每一层 pointer 都问：why?

Step 11 — Review Lifetime of Views

```

span
string_view
reference
pointer
iterator

```

谁 backing?

何时 invalid?

Step 12 — Measure

使用：

```

benchmark
allocation counters
profilers
assembly
hardware counters

```

验证 hot path。

12.2 高频 Smells

1

[机制片段 · 不承诺独立编译]

```
return std::move(local);
```

普通 local return：NRVO pessimation smell。

2

[机制片段 · 不承诺独立编译]

```
std::vector<std::unique_ptr<SmallValue>>
```


没有 polymorphism/stable address 原因：pointerization smell。

3

[机制片段 · 不承诺独立编译]

```
const TinyStruct&
```

机械 const-ref: cargo-cult borrow。

4

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<T>
```

只因为 many users: ownership uncertainty。

5

[机制片段 · 不承诺独立编译]

```
const std::vector<T>&
```

函数实际只需要：

```
contiguous sequence
```

考虑 span。

6

保存：

```
span  
string_view  
T*  
T&
```

却没有 backing lifetime contract。

7

为了“zero-copy”：

```
shared_ptr everything
```

可能将简单 memory copy 换成更昂贵 lifetime/refcount/cache 成本。

8

已知 batch size，但 hot path vector 不 reserve。

9

为了 stable address 而 heap allocate 所有 small objects。

10

使用 copy/move constructor side effects 承担业务 correctness。

Elision 可能让这些 operations 根本不发生。

13. 术语与统一模型

13.1 Core Terminology

English	中文核心含义
Logical Value	类型对外表示的语义值
Representation	object 在机器中的内部表示
Value Semantics	object copy/move 所定义的逻辑值关系
Copy Construction	从已有对象创建新对象值
Copy Assignment	用已有值替换已存在对象状态
Move Construction	从可消费 source 建立新对象
Move Assignment	用可消费 source 替换已有 target 状态
Moved-from State	move 后 source 仍存活的合法状态
Copy Elision	不建立/不转移某些中间对象
RVO	Return Value Optimization
NRVO	Named Return Value Optimization
Result Object	函数调用返回值最终对应的对象
Inline Representation	payload 直接存在 object 内部
Indirect Representation	object 通过 handle/pointer 管理外部 payload
Hybrid Representation	runtime 按状态在 inline/indirect 之间切换
SSO	Small String Optimization
SBO	Small Buffer/Object Optimization
Relocation	将值从一处 storage 迁移到另一处并结束旧 object lifetime
Invalidation	之前合法的 access path 变为非法
Amortized Complexity	多次操作平均后的复杂度保证
Zero-copy	避免主要 payload duplication
Stable ID	与物理 storage address 解耦的 logical identity

13.2 最终统一公式

```

Performance(T, operation)
=
Logical Semantics
×
Representation
×
Runtime State
×
Storage Topology
×
Ownership/Lifetime Contract

```

或者更工程化：

```

Cost
=
Bytes Moved
+
Allocations
+
Element Operations
+
Indirections
+
Synchronization
+
Cache Effects
+
Lifetime Coordination

```

因此：**C++ performance is semantic.**

13.3 G1 + + Unified Mental Model

任何 access/value/resource 问题，现在都可以按三层判断：

```

Layer 1 — Validity

Does the object exist?
Is storage valid?
Is access in bounds?
Has it been invalidated?


Layer 2 — Ownership

Who keeps it alive?

```

字面文本 · continued

Who cleans it up?

Who borrows?

Who shares?

Layer 3 — Cost

What is copied?

What is moved?

What is allocated?

How many indirections?

What layout reaches the CPU?

压缩:

Correct?

↓

Owned correctly?

↓

Efficiently represented and moved?

14. Final Gate

14.1 Final Gate

必须能够独立解释：

Value

1. Logical value 和 object representation 区别是什么？
2. Value semantics 为什么不等于 deep copy？
3. span/shared_ptr 为什么也拥有自己的 value semantics？
4. move-only type 为什么仍能成为优秀 value abstraction？

Copy

1. 为什么 `sizeof(vector)` 不能预测 copy 成本？
2. `inline` 与 indirect value copy 成本有什么不同？
3. $O(1)$ 为什么不等于 cheap？
4. copy 为什么有时是 lifetime-decoupling primitive？

Move

1. `std::move` 真正做什么？
2. 为什么 vector move 常 $O(1)$ 而 array move $O(N)$ ？
3. moved-from object 为什么仍然 alive？
4. 为什么 `const` 常阻止真正 move？
5. 为什么 cheap resource-owner move 应尽量 `noexcept`？

Elision

1. `return T{};` 与 `return local;` 有什么区别？
2. guaranteed copy elision 与 NRVO 区别是什么？
3. 为什么 `return std::move(local)` 通常是错误？
4. 为什么 return-by-value 不等于 copy-heavy API？

Parameters

1. `T / const T& / T& / T&&` 分别是什么 semantic contract？
2. small value 为什么常 by value？
3. sink 为什么常适合 by value？
4. span/string_view 为什么通常 by value？
5. 为什么跨 async boundary 必须重新做 ownership decision？

Containers

1. vector size 与 capacity 区别是什么?
2. reallocation 为什么开始新 object lifetimes?
3. 为什么 pointer/reference/iterator 失效?
4. 为什么 noexcept move 影响 vector relocation?
5. reserve 为什么同时是 performance 和 lifetime 工具?
6. amortized $O(1)$ 为什么不适合直接推断 realtime latency?

Representation

1. inline / indirect / hybrid 分别交换什么?
2. SSO/SBO 为什么能降低 allocation?
3. SBO 为什么可能伤 cache density?
4. 为什么 hybrid representation 会产生 performance cliff?
5. optional/variant 如何再次体现 storage 与 object lifetime 分离?

Architecture

1. zero-copy 真正消除了什么?
2. zero-copy 新增什么协调成本?
3. stable ID 为什么有时优于 stable address?
4. 为什么 dense small values 通常优于 pointerized small values?
5. 如何同时画 ownership/cost/allocation/invalidation graph?

15. Final Gate · 参考答案与常见误判

- **Value 1 ~ 4:** 逻辑值是类型承诺表示的意义，**representation** 是实现它的状态。复制 **view** 复制访问关系，复制 **shared_ptr** 复制拥有关系，不是深复制目标。**move-only** 可表达只能交接的资源权利；误判是把 **value** 一律等同可复制大对象。
- **Copy 1 ~ 4:** **vector** 本体小，元素存储在外；**inline** 按内嵌状态处理，**indirect owner** 可能分配并复制 **payload**。**O(1)** 隐藏常数、同步和缓存成本；真正复制独立数据可以解耦生命周期，复制 **view** 不行。
- **Move 1 ~ 5:** **std::move** 是转换，随后才决议操作。普通 **vector** 移动构造交接存储，**array** 通常逐元素处理；**allocator** 相关重载另论。源仍活着，可用操作由合同限定。**const** 常阻止普通移动参数绑定；真实不抛的移动可帮助容器维持保证，不能为“快”谎报 **noexcept**。
- **Elision 1 ~ 4:** 同类型 **prvalue** 可直接构造结果；具名局部的 **NRVO** 是可选的，相关操作仍须满足语言要求。**return std::move(local)** 排除了该 **NRVO** 条件，通常多余。按值返回可以直接构造或转移所有权，不等于深复制。
- **Parameters 1 ~ 5:** **T** 建立参数值，**const T&** 借用只读路径，**T&** 提供可变路径，具体非推导 **T&&** 接受右值但不自动消费。小值、**view** 通常复制元数据更自然；**sink** 按值可统一接收复制或转移，但要计入额外操作。异步越过调用范围，引用有效性需重新证明。
- **Containers 1 ~ 6:** **size** 数元素，**capacity** 数可容纳空间；重分配建立新元素并结束旧元素，使旧访问失效。复制/移动可行性和异常保证影响迁移策略。**reserve** 可控制部分重分配窗口，但不免除 **erase** 等失效；摊销均值也不限制单次延迟。
- **Representation 1 ~ 5:** **inline** 换局部性，**indirect** 换稳定句柄/可转移存储，**hybrid** 在运行时选择。**SSO/SBO** 减小对象的分配次数但增加 **wrapper** 大小，阈值切换可能突增成本。**optional** 可无值，**variant** 也可能 **valueless_by_exception**；字节空间存在不等于每个候选对象同时活着。
- **Architecture 1 ~ 5:** **zero-copy** 避免 **payload** 重复，不消除同步、保活和复用协调。**ID** 可与地址解耦，但需防陈旧代际；**dense** 小值常改善局部性，并非通用性能定理。为同一数据流分别标 **owner**、分配点、操作成本和失效点，才能看出局部优化是否把成本转嫁给下游。

完成标准：能对 G3-L1 ~ L4 先预测、后解释，并把“复制了什么/谁仍拥有/何时失效”写成三行。性能结论还需目标 **workload** 测量；本批计数实验不是 **benchmark**。

16. 工程原则回查

16.1 如果几个月后只记住十五条

1. 不要从 C++ 语法猜性能；先理解 **type** 的 **logical value** 和 **representation**。
2. **sizeof(T)** 只描述 **object** 本体，不一定描述它拥有或代表的数据规模。
3. **Copy** 要求保留 **source logical value**；**Move** 允许消费 **source state**；**Elision** 可以完全消除 **transfer**。
4. **std::move** 只是 **xvalue cast**，真正 **move** 由后续 **operation** 执行。
5. **Indirect owner** 通常 **copy** 昂贵、**move** 便宜；**inline value** 的 **move** 可能和 **copy** 一样贵。

6. **Moved-from object** 仍然 **alive**；只能依赖其 **type** 明确提供的 **moved-from contract**。
7. **noexcept move** 是 **generic performance contract**，而不仅是异常声明。
8. **Modern C++** 应大胆 **return-by-value**；**return local**；让 **NRVO** 和 **implicit move** 工作，不要机械 **std::move**。
9. 参数首先表达 **borrow/own/consume/view/share**，然后才讨论性能。
10. **Small values** 适合 **value-flow**；**large synchronous input** 适合 **borrow**；**large ownership** 通常适合 **move**。
11. **vector** 用 **growth-time relocation** 与地址不稳定交换 **dense contiguous storage** 和优秀 **cache locality**。
12. **SSO/SBO** 用更大的 **wrapper** 换取减少 **small-value allocation**，因此同时影响 **cache**、**move**、**ABI** 和 **threshold cliffs**。
13. **Zero-copy** 将 **memory-bandwidth** 成本转换成 **lifetime**、**aliasing**、**reuse** 和 **synchronization** 成本。
14. **Small copy** 有时是在购买 **lifetime independence**、**thread independence** 和 **storage abstraction**。
15. 健康的高性能 **C++** 系统通常是 **dense small values + cheap movable owners + narrow borrows + explicit shared islands + measured storage reuse**。

17. G3 → G4

本章按学习稿继续维护；定向实验不等于所有成本判断都已经测量。

我们已经回答：一个 **C++ value** 在程序中流动时，它的 **copy**、**move**、**return**、**parameter passing** 和 **container relocation** 究竟意味着什么成本。

G4 §1 把本章的值与表示模型用于整个数据集。下面是问题之间的依赖，不是需要重新背诵的术语目录。

```

Container
  ↓
Storage Topology
  ↓
Iterator
  ↓
Invalidation
  ↓
Algorithm
  ↓
Ranges / Views
  ↓
Abstraction Cost

```

下一阶段的核心问题将变成：怎样选择、遍历和组合一整个 **value** 集合，同时仍然保持对 **memory layout**、**iterator validity**、**algorithmic complexity** 和 **machine cost** 的控制。

G4 · STL、Ranges 与数据抽象

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.2.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G3；容器、迭代器、Ranges 与状态类型按语义和成本选型，G6/G7 用于深化。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G3 · 全系列导航 · 下一章：G5

阅读入口

本章的问题是：同一批数据，谁拥有、谁借用、怎样查询，才能同时说清正确性和成本？

首次学习不需要先读 G5 ~ G7：

1. 抽象模型与序列容器：用 `vector` 建立 `owner`、`size/capacity` 和失效模型。
2. 视图与算法、Ranges：学习 `span`、字符串借用、算法与 `view`。
3. `SignalBatch` 完整实验：把排序、查询、统计和只读接口连起来。
4. 容量与 `view` 实验：验证边界，再做 `Final Gate`。

其他容器、布局与状态类型用于回查；API、性能与跨语言材料用于第二遍对照。旧主题锚点保留，正文按下面的章节目录定位，不要求逐条背诵。只带 `g-lab` 标记的代码属于完整运行例；其余按上下文作片段、反例或自主练习。

选型的推理顺序是：需求语义 → 所有权/有效期 → 访问与更新模式 → 算法能力 → 表示与测量。不要从“某容器是 $O(1)$ ”倒推全部设计。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 标准库的抽象模型
- 2. 序列容器与失效
- 3. 连续、多维与字符视图
- 4. 迭代器、算法与 Ranges
- 5. 关联表示与容器选择
- 6. 状态建模

- 7. SignalBatch: 完整组件实验
- 8. 数据修改与 API 边界
- 9. 内存资源与性能决策
- 10. 实验与观察
- 11. 工程审查与常见误判
- 12. 跨语言回查
- 13. 工程原则回查
- 14. Final Gate
- 15. Final Gate · 参考答案与常见误判
- 16. 统一模型
- 17. 接下来怎么用

1. 标准库的抽象模型

1.1 STL 的正确 Mental Model

1. STL 不只是容器目录

[机制片段 · 不承诺独立编译]

```
std::vector<int> values{3, 1, 4, 1, 5};
std::ranges::sort(values);
```

`vector` 拥有元素, `range/iterator` 暴露访问能力, 算法通过这些能力工作。`sort` 需要随机访问和可排序等约束, 而不是要求容器名必须叫 `vector`。

2. 容器与算法怎样解耦?

自由算法可以用于多个满足约束的表示, 不必通过共同基类和虚函数。能力不满足时, 不能靠“反正有 `begin/end`”调用; `list` 有适合自身结构的成员 `sort`。抽象的作用是表达必要能力, 不是强迫一切操作使用同一形式。

3. Owner、View 与 Algorithm

角色	问题	例子
Owner	谁管理元素与资源清理?	<code>vector</code> 、 <code>string</code> 、 <code>map</code>
View / 借用接口	暴露哪种访问能力, 依赖谁存活?	<code>span</code> 、 <code>string_view</code> 、默认 accessor 的 <code>mdspan</code>
Algorithm	对范围要求什么能力与前置条件?	<code>sort</code> 、 <code>find</code> 、 <code>lower_bound</code> 、 <code>transform</code>

这是学习分工, 不是互斥的标准类型分类: `Ranges` 的 `owning_view` 同时是 `view` 并持有底层 `range`。函数对象、投影和捕获也可能带自己的依赖。

2. 序列容器与失效

2.1 vector：动态序列的默认起点

4. 先考虑 vector，再寻找反要求

如果要拥有运行时数量的同类型值，vector 提供连续元素、常数下标和较低的逐元素元数据成本，适合顺序遍历。这里排除 vector<bool> 的特殊表示；连续也不等于可直接作为 wire format, padding、字节序、对象语义仍需处理。

需要稳定节点、两端高频增长等反要求时，再比较其他容器。默认起点不等于无须测量的赢家。

5. size 与 capacity 数的不是同一件事

[机制片段 · 不承诺独立编译]

```
std::vector<int> values;
values.reserve(1024);
// size() == 0, capacity() >= 1024; 还没有可用下标元素。
```

size 是现有元素数；capacity 是当前无需重分配能容纳的元素数。保留容量不许可访问 size 之外的元素。

6. reserve 与 resize

reserve 准备容量，不改变 size；resize 改变元素数，增长时初始化新元素，必要时还会分配。不是“一个快一个慢”的同义操作。想通过下标填充 N 个元素，应先建立这些元素，不能只 reserve(N)。

7. clear 保留容量

clear 销毁元素，使 size 为 0，并保留 capacity。因此“clear 后重新填充”可以复用 vector 的存储。元素自身析构仍可能释放其内部资源；这不证明每次迭代零分配。

[机制片段 · 不承诺独立编译]

```
// 循环片段：fill/process 的合同由应用定义。
std::vector<T> buffer;
buffer.reserve(max_count);
for (;;) {
    buffer.clear();
    fill(buffer);
    process(buffer);
}
```

8. shrink_to_fit 是请求

它不保证 capacity 最终等于 size。若发生重分配，会使旧访问失效；频繁收缩还可能抵消存储复用收益。除非确有内存压力和使用阶段边界，不要把它机械加在每次 clear 后。

9. push_back 与 emplace_back

已有对象时，push_back 清楚表达加入一个值；直接给构造参数时，emplace_back 清楚表达就地构造。emplace_back(make_value()) 已经先求值出一个实参，并不能凭名字承诺完全免去转移；它不是高级版 push_back。

10. 重分配是元素迁移，不只是字节 resize

当成功插入使新 `size` 超过原 `capacity`, `vector` 必须取得更大容量, 并按元素规则建立新位置、结束旧元素和释放旧存储。增长因子不是固定语言承诺。用非平凡类型时, 不能把迁移理解为通用 `realloc/memcpy`。

11. `noexcept` 为什么影响迁移选择?

若移动可能修改源后抛出, 而复制可用, 复制可能更有利于保持异常保证。选择还受操作、类型要求和 `allocator` 条件影响; 不能根据一次计数推出所有 `vector` 都按同一路径实现。必须真实满足合同才声明 `noexcept`。FM-5 展开这些边界。

2.2 Iterator / Pointer / Reference Invalidation

12. 从操作前提判断, 而不是从地址外观判断

[机制片段 · 不承诺独立编译]

```
// 有条件危险的片段: 若发生重分配, 最后的读取非法。
std::vector<int> values{1, 2, 3};
int* p = &values[0];
values.push_back(4);
use(*p);
```

必须查原 `capacity` 与操作规则; 不能依据 `p` 非空或一次运行没崩溃判断。本章 G4-L2 用可移植条件验证增长, 不读取已失效指针。

13. 重分配使旧元素的访问路径失效

指向旧元素的 `pointer`、`reference`、`iterator`, 以及旧 `past-the-end iterator` 都失效。后续要重新从 `owner` 获取合法访问, 不能仅因为数值地址看似相同就继续用旧关系。

14. 没有重分配, 也有失效

`vector::erase` 使擦除点及之后的迭代器和引用失效, 旧尾后迭代器也失效; 原位置可能被后续元素占据。指针地址与逻辑元素身份不能混为一谈。`clear` 更是结束全部元素, 哪怕容量仍在。

15. 长期身份与物理地址分开

Vehicle #1234 是逻辑身份, 不必永久绑定一个 `vector` 元素地址。ID/handle 可通过查找映射到当前表示, 但还需范围检查和代际策略, 防止回收后把旧 ID 误认成新对象。地址稳定和 ID 安全各有合同, 不是换成整数就万事大吉。

16. 回查表: 记住问题, 使用时查精确操作

容器	首先检查的稳定性边界
<code>vector</code>	重分配全部失效; 无重分配的插入/删除另有位置相关规则
<code>array</code>	无结构性增长; 对象寿命与元素赋值的语义仍需区分
<code>deque</code>	分段随机访问; 端点/中间操作对 <code>iterator</code> 与 <code>reference</code> 的影响不同
<code>list</code>	通常保持未删除节点, <code>splice</code> 须满足 <code>allocator</code> 等前提
<code>map / set</code>	插入不使已有迭代器/引用失效; 擦除使被删元素失效
<code>unordered_map</code>	<code>rehash</code> 使 <code>iterator</code> 失效, 但不使元素 <code>pointer/reference</code> 失效
<code>flat_map</code>	默认平面序列; 插入/擦除及底层容器决定移动与失效

此表不是完整操作合同，不能据它省略对 `swap`、`move`、`allocator` 等具体情形的检查。

2.3 `std::array`

17. `std::array<T, N>`

当 `element count` 是：compile-time fixed

优先考虑：

[机制片段 · 不承诺独立编译]

```
std::array<T, N>
```

而不是裸：

[机制片段 · 不承诺独立编译]

```
T[N]
```

它提供正常 `container interface`：

[机制片段 · 不承诺独立编译]

```
size()
begin()
end()
data()
```

并保持：inline contiguous storage。

18. `array` 的 Representation

[机制片段 · 不承诺独立编译]

```
std::array<float, 16> values{}; // 本例初始化元素，避免读取不确定值。
```

逻辑上接近：

```
object itself
```

```
16 inline float values
```

没有单独 heap allocation。

因此非常适合：

```
small fixed buffers
SIMD blocks
protocol headers
compile-time tables
```

19. `array<T, N>` 的 `N` 个 `T` 都是活对象

这一点 G5 已经强调过：

[机制片段 · 不承诺独立编译]

```
std::array<T, 1024>
```

不是：capacity 1024 的 raw storage。

而是：1024 个正常 `T` objects。

如果需要：

```
capacity N
but size runtime
```

真正的 fixed-capacity vector 需要：

```
raw storage
+
manual lifetime management
```

是另一种 abstraction。

2.4 std::deque

20. deque 是 Segmented Sequence

概念不是：

```
one contiguous block
```

而更接近：

```
[block][block][block][block]
```

加一层 indexing structure。

因此：

```
[机制片段 · 不承诺独立编译]
deque[i]
```

仍然支持 random access,

但：

```
[机制片段 · 不承诺独立编译]
&deque[i + 1] == &deque[i] + 1
```

不能作为一般连续存储假设。

21. deque 什么时候有价值？

典型：

```
frequent push_front
frequent push_back
random access required
don't need contiguous storage
```

相比 vector：

```
front insertion
```

不需要整体移动全部 elements。

22. deque 的代价

相比 `vector`:

```
segmented
extra indirection
poorer linear locality
not representable as one span
```

所以对于:

```
high-throughput contiguous numeric processing
```

通常仍然优先 `vector`。

2.5 `list` / `forward_list`

23. `Linked List` 的抽象优势

[机制片段 · 不承诺独立编译]

```
std::list<T>
```

典型能力:

```
stable node addresses
O(1) insertion/erase with known iterator
splice
```

这些是真正优势。

24. 但 “`O(1) Insert`” 很容易骗人

对 `vector`:

```
insert middle
→ O(N)
```

对 `list`:

```
known iterator insert
→ O(1)
```

于是有人直接得出: `list` 更快。

但寻找 `insertion point` 可能仍然:

```
O(N)
```

而 `node traversal`:

```
pointer chasing
cache misses
allocation
```

实际成本很高。

所以: `Big-O` 必须和 `G6 machine model` 一起分析。

25. 什么时候应该真的考虑 `list` ?

当下面要求是真实核心需求：

```
node address/reference stability
frequent splice between lists
frequent insert/erase at known positions
large/non-movable nodes where relocation is unacceptable
```

而不是：“听说中间删除是 $O(1)$ 。”

26. forward_list

单链：

```
Node → Node → Node
```

比 `list` 少一个 backward pointer。

适合非常特定的：

```
forward-only intrusive-ish topology
```

但普通 application/system data 默认通常仍不应该优先 linked list。

3. 连续、多维与字符视图

3.1 std::span

27. `std::span<T>` 是 G4 最重要的 API 类型之一

`span` 表示：**non-owning contiguous sequence view**

例如：

[机制片段 · 不承诺独立编译]

```
void decode(std::span<const std::byte> input);
```

它表达：

```
borrow
+
contiguous
+
extent available
```

比：

[机制片段 · 不承诺独立编译]

```
void decode(
    const std::byte* data,
    std::size_t size);
```

更完整。

28. Span 不拥有数据

[机制片段 · 不承诺独立编译]

```
std::vector<int> values{1, 2, 3};

std::span<const int> view = values;
```

Ownership:

```
vector
→ owns ints

span
→ borrows ints
```

如果:

```
vector dies
```

span: dangling。

29. Span 不延长 Lifetime

访问时必须满足:

```
每次通过 view 使用元素
→ 目标元素仍在生命周期内
→ 没有使该访问路径失效的操作
```

例如:

[机制片段 · 不承诺独立编译]

```
std::span<const int> get_view() {
    std::vector<int> values{1, 2, 3};
    return values;
}
```

错误。

返回后:

```
values destroyed
↓
span dangling
```

30. span 也可能因 Reallocation 失效

[机制片段 · 不承诺独立编译]

```
std::vector<int> values{1, 2, 3};
std::span<const int> view = values;

values.push_back(4); // may reallocate

use(view);
```

如果 reallocation: span 指向旧 storage。

所以:

owner 活着与地址未移动都只是必要审查项，还要检查元素生命周期、范围及具体操作的失效规则；不能把这句话当作完整安全公式。

31. `span<T>` vs `span<const T>`

[机制片段 · 不承诺独立编译]

```
std::span<T>
```

表示：mutable borrowed elements。

[机制片段 · 不承诺独立编译]

```
std::span<const T>
```

表示：read-only access path。

注意：

```
const span
```

和：

```
span<const T>
```

也不是一个概念。

通常 API 输入：

[机制片段 · 不承诺独立编译]

```
std::span<const T>
```

非常自然。

32. Static Extent

[机制片段 · 不承诺独立编译]

```
std::span<const std::byte, 8>
```

表示：extent = 8 是类型的一部分。

相比：

[机制片段 · 不承诺独立编译]

```
std::span<const std::byte>
```

动态 extent。

固定协议字段：

[机制片段 · 不承诺独立编译]

```
std::uint64_t decode(
    std::span<const std::byte, 8> bytes);
```

可以把：“必须恰好 8 bytes”

编码进 API type。

从运行时范围构造静态 extent span 时，调用者仍须满足相应长度前置条件；类型中的 8 不是对任意输入自动执行的长度验证。外部输入应先验证，再建立这种受约束的 view。

33. span 是非常好的 Representation Boundary

假设内部：

[机制片段 · 不承诺独立编译]

```
std::vector<float> values_;
```

对算法不必暴露：

vector ownership/capacity API

可以暴露：

[机制片段 · 不承诺独立编译]

```
[[nodiscard]]
std::span<const float> values() const noexcept {
    return values_;
}
```

算法得到的是：

contiguous read-only range

而不是：修改 owner representation 的能力。

3.2 std::string_view

34. string_view 是字符领域的 Borrowed View

[机制片段 · 不承诺独立编译]

```
void parse(std::string_view input);
```

表示：

```
character sequence
+
length
+
no ownership
```

非常适合：

```
parsing
lookup
read-only string parameters
```

35. 最危险的 string_view

[机制片段 · 不承诺独立编译]

```
std::string_view make_name() {
    return std::string{"vehicle"};
}
```

编译可能成功。

但 temporary string：

```
destroyed
```

返回 `view`: `dangling`。

因此:

```
string_view
```

和:

```
span
```

一样首先是: `lifetime contract`。

36. `string_view` 不保证 Null Termination

[机制片段 · 不承诺独立编译]

```
std::string_view view;
```

只表示:

```
pointer + length
```

不能默认:

[机制片段 · 不承诺独立编译]

```
C_api(view.data());
```

安全。

因为底层 `range` 可能:

```
not NUL terminated
```

或者 `view` 只是原字符串子区间。

与 C API 交互时必须明确 `contract`。

这会在 G8 再出现。

3.3 `std::mdspan` — C++23 多维 View

37. `mdspan` 是 G4 与高性能计算的重要连接

C++23:

[机制片段 · 不承诺独立编译]

```
std::mdspan
```

表达: 对多维数据的 `non-owning structured view`。

例如概念:

[机制片段 · 不承诺独立编译]

```
std::mdspan<float,
            std::extents<std::size_t,
                        std::dynamic_extent,
                        3>>
            points;
```

表示：

$N \times 3$

二维 logical view。

38. 默认 **mdspan** 不负责底层元素所有权

使用默认 accessor/data handle 的常见模型是（自定义 accessor 可能改变句柄与资源管理语义）：

```
owner
→ vector / array / external buffer

mdspan
→ multidimensional indexing view
```

这让：

storage policy

与：

logical multidimensional indexing

解耦。

39. Layout Policy

mdspan 的重要价值是可以表达：

```
layout_right
layout_left
layout_stride
```

也就是说：logical (i, j) 如何映射到 linear storage offset。

这直接连接 G6：

```
row-major
column-major
strided layout
```

和 cache locality。

40. **mdspan** 的系统意义

以前函数可能：

[机制片段 · 不承诺独立编译]

```
void process(  
    const float* data,  
    std::size_t rows,  
    std::size_t cols,  
    std::size_t stride);
```

现在可以将：shape + layout mapping + element access

打包成 view abstraction。

同时不要求算法：拥有数据。

这非常适合：

```
robotics  
linear algebra  
image processing  
tensor kernels  
sensor matrices
```

4. 迭代器、算法与 Ranges

4.1 Iterator：不是“高级 Pointer”

41. Iterator 的核心是 Traversal Capability

最简单：

```
input iterator  
forward iterator  
bidirectional iterator  
random-access iterator  
contiguous iterator
```

能力逐渐增强。

不要首先背继承层级。

应该理解：**Container topology** 决定 **iterator** 能力。

42. Examples

```
forward_list
→ forward

list
→ bidirectional

deque
→ random access

vector
→ contiguous
```

所以：

```
[机制片段 · 不承诺独立编译]
std::ranges::sort(range);
```

需要：random-access sortable range。

因此不能：

```
[机制片段 · 不承诺独立编译]
std::ranges::sort(std::list<int>);
```

这不是标准库“故意刁难”。

而是：sort algorithm 的操作需求与 linked-list iterator capability 不匹配。

std::list 有自己的：

```
[机制片段 · 不承诺独立编译]
list.sort();
```

可以利用 node relinking。

43. Contiguous Iterator

vector iterator 具有：contiguous semantics。

所以可以和：

```
pointer
span
C APIs
SIMD
```

自然衔接。

这是 vector 与 deque 虽然都 random access，却存在的一个重要能力差异。

4.2 Algorithm First

44. 优先表达“我要做什么”

比如寻找：

[机制片段 · 不承诺独立编译]

```
auto it = std::ranges::find(values, target);
```

而不是首先：

[机制片段 · 不承诺独立编译]

```
for (...) {  
    if (...) {  
        ...  
    }  
}
```

原因不是：loop 不好。

而是：

```
find
```

直接表达 semantic intent。

这有利于：

```
review  
genericity  
correctness  
implementation optimization
```

45. 但不要迷信“算法永远优于 Loop”

如果 hot loop 需要：

```
one traversal  
simultaneously:  
min  
max  
sum  
valid count  
error flags
```

将其拆成：

```
min_element  
max_element  
accumulate  
count_if
```

意味着多次 traversal。

这时一个 fused loop：

[机制片段 · 不承诺独立编译]

```
for (const Sample& sample : samples) {  
    ...  
}
```

可能更合理。

因此原则：**Prefer algorithms when they express the operation naturally; fuse work when one-pass semantics/performance matters.**

46. Ranges Algorithms

传统：

[机制片段 · 不承诺独立编译]

```
std::sort(values.begin(), values.end());
```

现代 C++：

[机制片段 · 不承诺独立编译]

```
std::ranges::sort(values);
```

优点：

```
less iterator boilerplate
concept-constrained API
projection support
range-based composition
```

47. Projection

非常好用：

[机制片段 · 不承诺独立编译]

```
struct Signal {
    std::uint32_t id;
    float value;
};
```

```
std::ranges::sort(
    signals,
    {},
    &Signal::id);
```

而不是：

[机制片段 · 不承诺独立编译]

```
std::ranges::sort(
    signals,
    [](const Signal& a, const Signal& b) {
        return a.id < b.id;
    });
```

Projection 表达：“使用这个字段作为排序 key。”

语义更清楚。

48. lower_bound

如果数据已经按 id 排序：

[机制片段 · 不承诺独立编译]

```
auto it = std::ranges::lower_bound(
    signals,
    target_id,
    {},
    &Signal::id);
```

复杂度：

```
O(log N)
comparisons
```

但对于 random-access contiguous storage,
memory behavior 通常也很好。

49. Sorted Vector 是一种非常强的 Associative Representation

如果：

```
read-heavy
updates infrequent
```

可以：

```
vector
↓
sort once
↓
binary search many times
```

而不是直接：

```
map
unordered_map
```

因为：

```
contiguous iteration
low metadata
cache locality
```

经常非常优秀。

这就是：**Flat Representation**

4.3 Ranges Views

50. View 是 Lazy Transformation

例如：

[机制片段 · 不承诺独立编译]

```
auto positive =
    values |
    std::views::filter([](int value) {
        return value > 0;
    });
```

`positive` 通常不是：一个新 `vector`。

它是：对原 `range` 的 `lazy traversal adapter`。

51. Lazy 意味着什么？

[机制片段 · 不承诺独立编译]

```
auto view =
    values |
    std::views::transform(f);
```

通常：

没有立即 `transform` 所有 `elements`

而是 `iterator traversal` 时：

```
read element
↓
apply f
↓
produce transformed value/reference
```

因此：

temporary allocation

通常可以避免。

52. View 是否拥有数据，要看具体类型

问题：下面两种写法都得到 `view`，能否认为都只借用？

[机制片段 · 不承诺独立编译]

```
auto borrowed = std::views::all(values);           // values 为 vector 左值
auto owned = std::views::all(std::vector<int>{1, 2, 3}); // vector 右值
```

前者通常为 `ref_view`，依赖外部 `values`；后者在 C++23 中可形成 `owning_view`，把 `vector` 持有在内部。`view` 是带特定语义/复杂度要求的 `range`，不是“不拥有”的同义词，也不保证每一种 `view` 都执行延迟转换。

边界：即使底层 `range` 被拥有，`filter/transform` 中按引用捕获的其他对象仍可能悬挂。借用型 `view` 也仍受底层容器失效规则影响；`borrowed_range` 不能替你证明底层元素活着。N4950: [Ranges](#)

53. Range Pipeline

例如：

[机制片段 · 不承诺独立编译]

```
auto ids =
    signals |
    std::views::filter([](const Signal& signal) {
        return signal.value > 0.0F;
    }) |
    std::views::transform(&Signal::id);
```

这表达：

```
signals
↓
only positive values
↓
extract IDs
```

非常接近数据流语义。

54. Range Pipeline 不等于自动最快

Lazy pipeline 可能：

```
inline completely
```

得到非常好机器码。

也可能因为：

```
complex predicates
poor vectorization
multiple abstraction layers
```

产生不同 codegen。

因此 G6 原则保持：优先写清晰 abstraction，然后 profile/assembly 验证 hot path。

55. View 的 Lifetime Trap

尤其不要随意：

```
store a view
```

长期超过其 source owner。

例如 class member：

[机制片段 · 不承诺独立编译]

```
std::span<const T> view_;
```

意味着 class 的 lifetime invariant 必须包含：被 view_ 借用的 owner 永远活得更加久，而且不会使 storage invalid。

这是很强的 contract。

4.4 Borrowed Range

56. 为什么 Ranges 要有 borrowed_range ?

假设:

[机制片段 · 不承诺独立编译]

```
auto it =
    std::ranges::find(
        std::vector<int>{1, 2, 3},
        2);
```

temporary vector 在 full-expression 后销毁。

如果算法直接返回:

iterator into temporary vector

就会立即 dangling。

Ranges framework 会通过:

borrowed_range

等机制表达: iterator 的有效性是否不依赖 range 对象本身的生命周期。

这不保证底层元素无限存活。例如 span 满足 borrowed_range, 但它指向的 vector 已销毁时, iterator 仍会悬挂。

57. std::ranges::dangling

对于不安全的 rvalue range:

某些 range algorithm 的返回类型会成为:

[机制片段 · 不承诺独立编译]

```
std::ranges::dangling
```

而不是给你一个立即悬空的 iterator。

这是一种非常漂亮的: **Lifetime-aware generic API design**

与 G1/G5 完全呼应。

5. 关联表示与容器选择

5.1 std::map / std::set

58. Ordered Associative Containers

[机制片段 · 不承诺独立编译]

```
std::map<K, V>
std::set<K>
```

提供:

```
ordered keys
O(log N) lookup/insertion/erase
sorted iteration
range queries
stable node-like element addresses/iterators across ordinary insertion
```

实现通常基于 balanced tree，但标准 contract 并不要求你依赖具体树类型。

59. map 的真正优势

不是：O(log N) 听起来不错。

而是：

```
ordering is part of required semantics
```

例如：

```
lower_bound
upper_bound
range query
ordered traversal
```

以及 node stability 可能有价值。

60. operator[] 的语义

[机制片段 · 不承诺独立编译]

```
map[key]
```

如果 key 不存在：会插入默认构造的 mapped value。

所以只想查询时不要机械：

[机制片段 · 不承诺独立编译]

```
if (map[key] == ...)
```

可能无意修改 container。

查询优先：

[机制片段 · 不承诺独立编译]

```
map.find(key)
```

或者 C++20：

[机制片段 · 不承诺独立编译]

```
map.contains(key)
```

61. try_emplace

例如：

[机制片段 · 不承诺独立编译]

```
map.try_emplace(
    key,
    constructor_args...);
```

如果 key 已存在：不需要构造 mapped object。

但调用的参数表达式仍会先求值；`try_emplace(key, expensive())` 不会因为 `key` 已存在就自动跳过 `expensive()`。

对于昂贵 `value`：

```
lookup + conditional construction
```

很自然。

5.2 `std::unordered_map`

62. Hash Table

[机制片段 · 不承诺独立编译]

```
std::unordered_map<K, V>
```

通常由：

```
hash  
bucket selection  
collision handling
```

组成。

平均 `lookup` 常描述为：

```
O(1)
```

但这绝不意味着：“一定比 `map` 快”。

63. Hash Lookup 的真实 Cost

至少包括：

```
hash computation  
bucket access  
collision handling  
pointer/metadata traversal  
key comparison
```

如果 `key` 是：

[机制片段 · 不承诺独立编译]

```
std::string
```

`hash` 本身可能扫描整条 `string`。

所以：

```
O(1)
```

指 `container size N` 的平均渐进复杂度，

不是：one CPU instruction。

64. Load Factor

[机制片段 · 不承诺独立编译]

```
map.load_factor();  
map.max_load_factor();
```

粗略：

```
elements / buckets
```

load factor 太高：

```
collisions ↑
```

太低：

```
bucket memory ↑  
cache footprint ↑
```

65. reserve

如果预先知道约有：

```
N entries
```

可以：

[机制片段 · 不承诺独立编译]

```
table.reserve(N);
```

减少未来：

```
rehash
```

次数。

这和 vector reserve 是同一思想：使用 domain knowledge 提前稳定 storage topology。

66. Rehash Invalidation

Rehash 会：重新组织 buckets。

因此：

```
iterators invalidated
```

但对既有 elements 的：

```
references/pointers
```

通常不会仅因为 rehash 而失效。

这和 vector reallocation 完全不同。

仍然要对具体 erase 等 operation 单独分析。

67. unordered_map 最大的误区

“Lookup 是 $O(1)$ ，所以这是最快字典。”

如果：


```
N = 20
```

一个线性 scan:

[机制片段 · 不承诺独立编译]

```
std::ranges::find(...)
```

都可能更快。

原因:

```
contiguous memory
simple comparisons
no hashing
no node chasing
```

因此: **Small N changes the game.**

5.3 std::flat_map / std::flat_set – C++23

本主题 的连续存储分析以默认底层容器为前提。flat_map 默认使用分别保存 key 和 mapped value 的 vector, 不是固定的 vector<pair>; 替换底层序列后, 要重新检查连续性、代理引用与失效规则。

N4950: [flat.map.overview](#)

68. Flat Associative Containers

C++23:

[机制片段 · 不承诺独立编译]

```
std::flat_map
std::flat_set
```

提供 associative-container-style interface,

但 representation 更接近: sorted contiguous sequence(s)

而不是 node tree。

69. Flat Map 的 Cost Model

Lookup:

```
O(log N)
```

通常通过 binary search。

Iteration:

```
contiguous / dense underlying storage
```

非常 cache-friendly。

但 insert/erase:

```
O(N)
```

因为 elements 可能需要移动。

因此特别适合：

```
read-heavy
build once
query many
moderate size
```

70. flat_map vs Sorted Vector

如果只是内部实现：

[机制片段 · 不承诺独立编译]

```
std::vector<Entry>
```

然后：

[机制片段 · 不承诺独立编译]

```
std::ranges::sort
std::ranges::lower_bound
```

已经非常好。

flat_map 的价值：提供更明确 associative semantics/API。

选择取决于：

```
representation control
API expectations
mutation pattern
```

71. Dense Integer Key: 别忘了 Array / Vector

假设 key：

```
0 <= id < 64
```

不要第一反应：

[机制片段 · 不承诺独立编译]

```
std::unordered_map<int, Entry>
```

直接：

[机制片段 · 不承诺独立编译]

```
std::array<Entry, 64>
```

或：

[机制片段 · 不承诺独立编译]

```
std::vector<Entry>
```

可能更合理。

Lookup：

[机制片段 · 不承诺独立编译]

```
entries[id]
```

无需：

```
hash
tree
node
```

这是一种非常重要的：**Exploit Domain Constraints**

5.4 Container Selection

72. 不要按 Big-O 选 Container

先问 semantics:

```
Do I need ordering?
Do I need contiguous storage?
Do I need stable addresses?
Do I insert in middle often?
Are mutations rare?
Is key domain dense?
Is N small?
```

之后再看 performance。

73. 一个实用选择矩阵

Requirement	默认候选
动态拥有连续 values	vector
固定编译期数量	array
两端高频增长	deque
需要稳定 node / splice	list
ordered associative	map
average hash lookup	unordered_map
read-heavy sorted associative	flat_map / sorted vector
dense integer ID	array / vector
borrowed contiguous input	span
borrowed string input	string_view
multidimensional borrowed data	mdspan

默认不是定论。

是：starting point。

6. 状态建模

6.1 optional / variant / expected

74. STL 也提供“状态建模”类型

容器之外，现代标准库的重要抽象还有：

[机制片段 · 不承诺独立编译]

```
std::optional<T>
std::variant<...>
std::expected<T, E>
```

这些不是主要 storage containers,

但对系统 API 非常重要。

75. optional<T>

表达：

```
T exists
or
no T
```

例如：

[机制片段 · 不承诺独立编译]

```
std::optional<Job> try_pop();
```

比：

[机制片段 · 不承诺独立编译]

```
bool try_pop(Job& out);
```

有时更 value-oriented。

但如果 T 很大：返回 optional<T> 仍然要分析 representation/move cost。

76. variant

[机制片段 · 不承诺独立编译]

```
using Command =
    std::variant<
        Start,
        Stop,
        Update>;
```

表示：正常有值时持有若干候选之一；部分抛异常的类型改变操作可能使 variant 进入 valueless_by_exception，不能无条件假定一直有值。

比：

```

type enum
+
void*
+
manual union

```

更安全。

非常适合：

```

message passing
state-machine commands
protocol AST

```

77. std::visit

[机制片段 · 不承诺独立编译]

```

std::visit(
    [](auto&& command) {
        handle(command);
    },
    cmd);

```

配合 variant 实现：closed set static sum type dispatch。

它不是 virtual inheritance hierarchy。

C++ compiler 可以基于 alternative set 静态生成 dispatch machinery。

78. expected<T, E> — C++23

表示：

```

success: T
or
failure: E

```

例如：

[机制片段 · 不承诺独立编译]

```

std::expected<Message, ParseError>
parse(std::span<const std::byte> input);

```

比：

```

sentinel
global errno
out parameter

```

更明确。

79. expected 不会自动带来 Strong Exception Safety

如果函数内部：

```

modify shared state
↓
later return unexpected(error)

```

已经发生的 side effects: 不会因为返回 expected 自动 rollback。

所以:

```

error representation

```

和:

```

state transaction guarantee

```

仍然是不同问题。

G2 的 exception-safety model 依旧适用。

7. SignalBatch: 完整组件实验

7.1 Practical SignalBatch

80. G4-L1: 从拥有一批数据到可查询的只读组件

需求: 构建时取得一批采样, 之后多次读取; 按 signal_id 查询, 重复 ID 返回输入中第一条; 空批次没有统计值。数据只允许有限 float, 避免把 NaN 的业务规则藏在 min/max 里。

为使查询不依赖调用者“记得先排序”, 本学习例在构造中排序, 之后不提供 push/clear。它不是生产时序系统: 单位、溢出策略、并发和吞吐目标均不在这个最小例子中。

先预测空输入、缺失 ID、小于/大于所有 ID、重复 ID 各会得到什么。下面是完整运行例:

[完整实验 · G4-L1 · main.cpp]

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <limits>
#include <optional>
#include <span>
#include <stdexcept>
#include <utility>
#include <vector>

struct SignalSample {
    std::uint64_t timestamp_ns{};
    std::uint32_t signal_id{};
    float value{};
    bool operator==(const SignalSample&) const = default;
};

```

完整实验 · G4-L1 · main.cpp · continued

```
struct Stats {
    std::size_t count{};
    float min{}, max{};
    double sum{};
};

class SignalBatch {
public:
    explicit SignalBatch(std::vector<SignalSample> samples)
        : samples_(std::move(samples)) {
        for (const auto& sample : samples_) {
            if (!std::isfinite(sample.value))
                throw std::invalid_argument("finite sample required");
        }
        std::ranges::stable_sort(samples_, {}, &SignalSample::signal_id);
    }

    std::span<const SignalSample> samples() const & noexcept {
        return samples_;
    }
    std::span<const SignalSample> samples() const && = delete;

    std::optional<SignalSample> find(std::uint32_t id) const {
        const auto it = std::ranges::lower_bound(
            samples_, id, {}, &SignalSample::signal_id);
        if (it == samples_.end() || it->signal_id != id)
            return std::nullopt;
        return *it;
    }

    std::optional<Stats> stats() const {
        if (samples_.empty()) return std::nullopt;
        Stats result{0, samples_.front().value, samples_.front().value, 0};
        for (const auto& sample : samples_) {
            ++result.count;
            result.min = std::min(result.min, sample.value);
            result.max = std::max(result.max, sample.value);
            result.sum += sample.value;
        }
        return result;
    }

private:
    std::vector<SignalSample> samples_;
};

int main() {
```

完整实验 · G4-L1 · main.cpp · continued

```

const SignalBatch batch{{{20, 4, 2}, {10, 1, 1}, {30, 4, 3}}};
const SignalSample expected[]{{10, 1, 1}, {20, 4, 2}, {30, 4, 3}};
if (!std::ranges::equal(batch.samples(), expected)) return 1;
const auto hit = batch.find(4);
if (!hit || *hit != expected[1]) return 2;
if (batch.find(0) || batch.find(2) || batch.find(9)) return 3;
const auto stats = batch.stats();
if (!stats || stats->count != 3 || stats->min != 1 ||
    stats->max != 3 || stats->sum != 6) return 4;
const SignalBatch empty{std::vector<SignalSample>{}};
if (!empty.samples().empty() || empty.find(1) || empty.stats()) return 5;
bool rejected = false;
try {
    SignalBatch invalid{{{0, 1, std::numeric_limits<float>::infinity()}}};
} catch (const std::invalid_argument&) {
    rejected = true;
}
if (!rejected) return 6;
std::cout << "sorted; first-duplicate; misses; stats; empty; finite-only\n";
}

```

81. 为什么返回 `span`，而不是 `const vector&`？

消费者只需“连续只读元素”，不需容量和 `allocator` API。`span` 提供较窄的接口，但仍有连续存储承诺，不能透明换成 `list`。它不保活；本例禁止直接从右值 `batch` 提取 `span`，也仍要求使用期间 `owner` 未销毁、未被移动/赋值、未通过外部别名改变元素。

82. 为什么排序属于构建不变量？

如果先发布 `span`，再允许任意 `push`，排序和借用有效性都可能破坏。此例在构造中用 `stable_sort`，保证重复 `key` 保留输入先后顺序。不是所有业务都该排序；时序读取和高频追加需要另一份明确接口合同。

83. `lower_bound` 不等于“找到了”

必须先满足与比较器/投影一致的分区前提，本例用排序建立它；再检查 `end` 与 `key` 相等。测试分别覆盖低端缺失、间隙缺失和高端缺失。只测一个命中会漏掉“返回第一个不小于目标的错误记录”。

84. 重复记录与分组

相同 ID 排在一起便于 `equal_range` 分组。本例 `find` 明确返回第一条的**值副本**；完整范围比较保留 `timestamp`、ID、`value` 的归属，不能只检查排序后的 ID 数量。更多聚合策略由需求定义，不暗中去重。

85. 统计、预测与边界

一次 `loop` 同时计算 `count/min/max/sum`，是有明确理由的手工融合；空批次返回 `nullopt`，而不是制造一个伪造的最小值。

实验使用可精确表示的小数值比较 `sum`，不把它推广成任意浮点求和的精确相等方案。只检查了一种非有限输入；没有注入分配失败或证明所有浮点误差界。改变条件练习：允许构建后追加会破坏哪些合同？应重新排序、失效旧 `view`，还是改用分阶段 `builder`？先说明选择，再写代码。

8. 数据修改与 API 边界

8.1 Erase / Filter

86. 现代 Erase

以前典型：

[机制片段 · 不承诺独立编译]

```
values.erase(  
    std::remove_if(  
        values.begin(),  
        values.end(),  
        pred),  
    values.end());
```

现代：

[机制片段 · 不承诺独立编译]

```
std::erase_if(values, pred);
```

表达更清楚。

87. Filter View vs Physical Removal

[机制片段 · 不承诺独立编译]

```
auto valid =  
    samples |  
    std::views::filter(is_valid);
```

只是：logical filtered view。

底层 samples 仍然全部存在。

而：

[机制片段 · 不承诺独立编译]

```
std::erase_if(samples, is_invalid);
```

真正：修改 owner、结束 object lifetime、移动后续 elements。

这两个 semantics 完全不同。

88. Lazy Filter 的 Cost

filter view 遍历时每个 element：

```
predicate  
↓  
possibly skip
```

如果同一个 filtered subset 被重复处理很多次：每次都重新 predicate。

有时一次 materialize：

```
filter once
↓
new compact vector
↓
many passes
```

更划算。

再次是：

```
transformation cost
vs
reuse count
```

8.2 API Design

89. Owner API

如果函数：接管一份 `vector` 所有权

可以：

[机制片段 · 不承诺独立编译]

```
void consume(std::vector<Record> records);
```

by-value 明确：ownership enters function。

Caller：

[机制片段 · 不承诺独立编译]

```
consume(std::move(records));
```

90. Borrowed Read-only

[机制片段 · 不承诺独立编译]

```
void process(
    std::span<const Record> records);
```

表达：

```
borrow
read-only
contiguous
```

91. Borrowed Mutable

[机制片段 · 不承诺独立编译]

```
void normalize(
    std::span<float> values);
```

表达：caller owns；function 可修改 elements。

92. 不要滥用 `const vector&`

[机制片段 · 不承诺独立编译]

```
void process(
    const std::vector<Record>& records);
```

如果算法真正依赖：

```
contiguous sequence
```

则：

[机制片段 · 不承诺独立编译]

```
std::span<const Record>
```

通常更 general。

这样 caller 可以传：

```
vector
array
C array
subspan
```

93. 但也不要机械全部换 Span

如果 API 的 semantic contract 就是：“这是一个 vector owner，我需要 capacity、growth、push_back。”

那就应该：

[机制片段 · 不承诺独立编译]

```
std::vector<T>&
```

或者更好的 owner abstraction。

例如：

[机制片段 · 不承诺独立编译]

```
void fill(std::vector<T>& output);
```

如果目的明确是：重用 caller-owned dynamic storage。

这是合法设计。

关键是：参数类型应该表达所需 capability。

8.3 Return Values

94. 返回 vector 很正常

[机制片段 · 不承诺独立编译]

```
std::vector<Record> load_records();
```

现代 C++：

```
copy elision
move semantics
```

使：return container by value

成为非常自然的 ownership transfer。

不要为了“怕 copy”机械：

[机制片段 · 不承诺独立编译]

```
void load_records(
    std::vector<Record>& out);
```

除非：

```
storage reuse
allocation control
bounded environment
```

确实是 API requirement。

95. 不要返回指向 Local Container 的 View

错误：

[机制片段 · 不承诺独立编译]

```
std::span<const Record> load() {
    std::vector<Record> result;
    // ...
    return result;
}
```

返回 owner：

[机制片段 · 不承诺独立编译]

```
std::vector<Record> load();
```

才正确。

这是： **Owner must cross boundary if data must outlive the function.**

8.4 Container Choice 与 ABI

96. 不要轻易把 STL Container 放进稳定 Binary ABI

例如 public dynamic library C++ ABI：

[机制片段 · 不承诺独立编译]

```
std::vector<Record>
get_records();
```

会把：

```
stdlib implementation
allocator
layout
exception ABI
template instantiation
```

带进 binary boundary。

这不是说：永远不能。

而是：container-rich C++ API 会形成很强 ABI coupling。
G8 会正式展开。

8.5 Container Choice 与 Concurrency

97. STL Container 本身通常不是“自动线程安全”

例如：

[机制片段 · 不承诺独立编译]

```
std::vector<int> values;
```

多个 threads 只读：在正确 publication 后很自然。

一个 thread 修改、另一个同时读取：需要同步。

不能因为：

vector implementation internally complicated

就认为标准库会自动锁。

98. Different Elements 也要分析 Container Mutation

例如：

Thread A modifies values[0]

Thread B modifies values[1]

对于普通 `vector<int>`，如果 storage 本身不发生结构性 mutation，两个不同 elements 是不同 memory locations，可以设计成合法并发访问。

但：

false sharing

仍可能存在。

而：

[机制片段 · 不承诺独立编译]

```
std::vector<bool>
```

是特殊 bit-packed representation。

不同 logical bool elements 可能共享同一个 underlying word。

所以不要把普通 vector element reasoning 直接套给：

[机制片段 · 不承诺独立编译]

```
vector<bool>
```

8.6 vector<bool>

99. 为什么它特殊？

为了 bit packing：

```
1 bool
~
1 bit
```

而不是普通：

```
sizeof(bool)
```

排列。

因此：

```
[机制片段 · 不承诺独立编译]
values[i]
```

通常返回：proxy reference

而不是普通：

```
[机制片段 · 不承诺独立编译]
bool&
```

100. 什么时候可以用？

如果真实需求就是：

```
dense bitset-like dynamic flags
```

它并不是“禁止使用”。

但必须知道：

```
different semantics
proxy references
bit-level operations
concurrency implications
```

如果你只是想要正常 byte-addressable flags：

```
[机制片段 · 不承诺独立编译]
std::vector<std::uint8_t>
```

可能更直观。

如果需要固定 bitset：

```
[机制片段 · 不承诺独立编译]
std::bitset<N>
```

也可能更适合。

8.7 Flat vs Node-based

101. 一个统一机器模型

Node-based：

```
[node] → [node] → [node]
```

优点：

```
address stability
cheap relinking
```

代价：

```
allocation
metadata
pointer chasing
cache/TLB
```

Flat：

```
[value][value][value][value]
```

优点：

```
density
iteration
prefetch
SIMD
```

代价：

```
insert/erase moves
address instability
```

所以： **Topology is a trade-off, not ideology.**

9. 内存资源与性能决策

9.1 STL 与 Memory Resource

102. Containers 与 `std::pmr`

G6 已经学习：

```
[机制片段 · 不承诺独立编译]
std::pmr::vector<T>
std::pmr::string
```

重要的是：

```
container semantic role
```

和：

```
allocation strategy
```

是两个维度。

如果：

```
vector is the right topology
```

但默认 heap allocation 不是最适合当前 lifetime, 不应该因此放弃 vector。

可以:

```
vector semantics
+
arena resource
```

组合。

103. 不要让 Allocator Policy 泄漏过远

如果 subsystem 内部需要:

[机制片段 · 不承诺独立编译]

```
std::pmr::vector<Temp>
```

不代表: 整个 domain model 所有 API 都需要传 memory_resource*。

与 G5 genericity 一样: **Policy propagation should stop at a deliberate boundary.**

9.2 STL Performance Review

104. 看到 Container 时间问什么?

不要只问 Big-O。

至少问:

```
Who owns elements?
Inline or indirect?
Contiguous or node-based?
How many allocations?
How many indirections?
How stable are addresses?
What invalidates views?
What is the access pattern?
How large is N?
How often mutate?
How often iterate?
```

105. map vs unordered_map vs flat_map vs vector

真正比较:

表示: **map**

- Ordered traversal: ✓
- Lookup: $O(\log N)$
- Insert: $O(\log N)$
- Locality: poor/moderate

- Address stability: high
- Range query: excellent
- Small/read-mostly: often overkill

表示: **unordered_map**

- Ordered traversal: —
- Lookup: avg $O(1)$
- Insert: avg $O(1)$
- Locality: poor/moderate
- Address stability: relatively high for elements
- Range query: poor
- Small/read-mostly: often overkill

表示: **flat_map / sorted vector**

- Ordered traversal: $\checkmark$
- Lookup: $O(\log N)$
- Insert: $O(N)$
- Locality: high
- Address stability: low
- Range query: excellent
- Small/read-mostly: strong

表示: **dense vector**

- Ordered traversal: by index/order
- Lookup: $O(1)$ dense key
- Insert: depends
- Locality: very high
- Address stability: low
- Range query: domain-specific
- Small/read-mostly: strongest if applicable

没有一个 universal winner。

9.3 Practical Selection Rules

106. Rule 1 — vector Until Proven Otherwise

动态 sequence: 先 vector。

换掉它需要明确 requirement。

107. Rule 2 — Dense Key $\rightarrow$ Direct Index

如果 key domain:

```
small
integer
dense
```

优先:

```
array/vector
```

而不是 hash/tree。

108. Rule 3 — Read-heavy Lookup → Consider Sorted Flat Data

如果:

```
build/update rare
lookup frequent
iteration frequent
```

考虑:

```
sorted vector
flat_map
```

109. Rule 4 — Address Stability 必须是 Requirement 才值得付费

不要为了: “pointer 一直有效挺方便”

就默认 node containers。

长期 identity 更推荐:

```
ID / handle
```

110. Rule 5 — Borrow with Span/View, Own with Container

API boundary 上:

```
ownership
→ vector/string/owner type

borrow
→ span/string_view/mdspan
```

这是非常好的默认。

10. 实验与观察

10.1 Practical Lab

111. Lab 1 — Container Selection

给四个 workload:

A

10M float 顺序扫描。

B

1000 个 key, build once, 查询 10M 次, 需要 sorted iteration。

C

动态 queue, 两端 push/pop。

D

需要把 node 从一个 sequence O(1) splice 到另一个, 地址必须稳定。

闭卷选 container, 并从:

```
semantics
+
machine topology
```

两方面解释。

112. G4-L2 — 用实际 capacity 判断失效

问题: reserve(4) 之后第五次 push 一定重分配吗? 不一定, capacity 只保证至少为 4。以下实验先填满实际 capacity, 再成功追加, 确保跨过容量边界。

[完整实验 · G4-L2 · main.cpp]

```
#include <algorithm>
#include <iostream>
#include <span>
#include <vector>

int main() {
    std::vector<int> values;
    values.reserve(4);
    if (!values.empty() || values.capacity() < 4) return 1;
    const auto old_capacity = values.capacity();
    values.resize(old_capacity, 7);
    {
        std::span<const int> view{values};
        if (view.size() != old_capacity ||
            !std::ranges::all_of(view, [](int n) { return n == 7; })) return 2;
    } // 不跨过失效点保存旧 view。
    values.push_back(9);
    if (values.size() != old_capacity + 1 ||
        values.capacity() <= old_capacity || values.back() != 9) return 3;
    const auto new_capacity = values.capacity();
    values.clear();
    if (!values.empty() || values.capacity() != new_capacity) return 4;
    std::cout << "reserve-no-elements; growth; clear-keeps-capacity\n";
}
```

为什么不读取扩容前的 p 来“证明它失效”? 因为那样引入 UB, 结果不是可移植判据。本实验检查触发条件, 失效结论来自 N4950: [vector.capacity](#)。erase 即使没有重分配, 也有自己的失效规则。

113. G4-L3 / L4 — 借用、拥有与算法能力

下面的完整运行例同时检查 `owning_view`、`borrowed_range` 与 `list` 自有排序。先预测三个 `range` 的元素由谁保活。

[完整实验 · G4-L3 · main.cpp]

```
#include <algorithm>
#include <iostream>
#include <list>
#include <ranges>
#include <span>
#include <type_traits>
#include <vector>

int main() {
    std::vector<int> source{3, 1, 2};
    auto borrowed = std::views::all(source);
    auto owned = std::views::all(std::vector<int>{3, 1, 2});
    static_assert(std::ranges::view<decltype(owned)>);
    static_assert(std::ranges::borrowed_range<std::span<int>>);
    static_assert(!std::ranges::borrowed_range<std::vector<int>>);
    static_assert(std::is_same_v<
        decltype(std::ranges::find(std::vector<int>{1}, 1)),
        std::ranges::dangling>);
    source[0] = 9;
    const int borrowed_expected[]{9, 1, 2};
    const int owned_expected[]{3, 1, 2};
    if (!std::ranges::equal(borrowed, borrowed_expected)) return 1;
    if (!std::ranges::equal(owned, owned_expected)) return 2;
    std::list<int> nodes{3, 1, 2};
    nodes.sort();
    const int sorted[]{1, 2, 3};
    if (!std::ranges::equal(nodes, sorted)) return 3;
    std::cout << "borrowed-view; owning-view; list-member-sort\n";
}
```

原因：`view` 不排除拥有，`borrowed_range` 不保证底层元素永远活着。`list` 不能满足 `ranges::sort` 所需的随机访问能力，但有适合节点结构的成员 `sort`。

以下是**编译失败反例**，须出现 `sort` 的约束不满足诊断：

[反例 · 编译失败 · G4-L4 · main.cpp]

```
#include <algorithm>
#include <list>
int main() {
    std::list<int> values{3, 1, 2};
    std::ranges::sort(values);
}
```

自主练习（未计入本批运行结果）：写 `std::optional<double> average(std::span<const float>)`，规定空范围返回 `nullopt`；让 `vector`、`array`、C 数组和连续子范围

调用它。指出借用不保活，列出浮点非有限值与求和误差策略，不把“加个 `span`”当作全部安全合同。

114. Lab 4 — Sorted Vector vs Hash

构造：

```
N = 16
64
1024
1M
```

比较：

```
linear vector lookup
sorted vector + lower_bound
unordered_map
map
```

不要只测 `lookup`。

还测：

```
build cost
memory footprint
iteration
```

本练习未在本批测量。先校验四种表示的查询结果相同，再分开计时构建、命中/缺失查找和遍历；固定输入种子、优化选项和工具链，重复运行并保留分布。`memory footprint` 必须说明测的是元素字节、分配量还是进程 RSS，不能混为一项。不要在看到结果前指定赢家。

这个实验用于检验：“ $O(1)$ 总比 $O(\log N)$ 快”

这种直觉。

115. Lab 5 — View Lifetime

逐个判断：

[机制片段 · 不承诺独立编译]

```
std::string text = "hello";
std::string_view a = text;
```

合法。

然后：

[机制片段 · 不承诺独立编译]

```
std::string_view b = std::string{"hello"};
```

为什么危险？

再：

[机制片段 · 不承诺独立编译]

```
text += very_large_suffix;
```

之后 `a` 是否仍然可靠？

根据：string storage invalidation

分析。

116. Lab 6 — Range Projection

定义：

[机制片段 · 不承诺独立编译]

```
struct Record {  
    std::uint32_t id;  
    std::uint64_t timestamp;  
};
```

要求：

```
sort by timestamp  
find by id
```

用：

[机制片段 · 不承诺独立编译]

```
std::ranges::sort  
std::ranges::find
```

以及 projection 表达，而不是全部写 comparator boilerplate。

11. 工程审查与常见误判

11.1 Review Protocol

看到一段 STL 代码，按这个顺序问。

1. Ownership

Who owns elements?

2. Lifetime

Any span/view/string_view/iterator?
What invalidates it?

3. Required Semantics

ordered?
contiguous?
stable address?
front/back growth?
range query?

4. Access Pattern

- iterate?
- random lookup?
- batch?
- hot/cold?

5. Mutation Pattern

- build once?
- frequent insert?
- frequent erase?
- append only?

6. Scale

- N = 8?
- 1000?
- 10M?

7. Storage Topology

- flat?
- segmented?
- node-based?

8. Invalidation

- which operation invalidates:
 - pointer?
 - reference?
 - iterator?
 - view?

9. Algorithm Capability

- what iterator/range category does algorithm actually require?

10. Measurement

如果是 hot path:

- profile / benchmark

而不是仅凭 Big-O 判断。

11.2 高频错误

11.7. 错误 1

list insert 是 O(1)，所以中间插入很多时应该用 list。

忽略了：

```
finding position  
allocation  
cache misses  
pointer chasing
```

118. 错误 2

`unordered_map` $O(1)$, 所以比 `map/vector` 快。

错。

Big-O 不表达:

```
hash cost  
memory topology  
N
```

119. 错误 3

`reserve(N)` 创建 N 个 objects。

错。

它主要准备 capacity。

120. 错误 4

`clear()` 会释放 `vector` 内存。

不对: 它销毁元素并保留 capacity。成员析构可能释放各自拥有的资源, 不要与 `vector` 的元素存储混淆。

121. 错误 5

`span` 更安全, 所以可以忽略 lifetime。

错。

`Span` 不拥有也不延长 lifetime。

122. 错误 6

`const vector&` 永远比 `span` 好, 因为类型更具体。

通常 abstraction 反而过强。

123. 错误 7

`Ranges/View` 会产生很多临时 containers。

许多 adaptor 是惰性处理, 但 `view` 不等于 `non-owning`; 既要查底层 `range`, 也要查 `predicate/capture` 的生命周期。

124. 错误 8

Lazy 总比 materialize 快。

不一定。

重复多次 traversal 时 materialization 可能更好。

125. 错误 9

Iterator 没变成 null，所以仍然有效。

Invalid iterator/pointer 不需要变 null。

Dangling address 可以看起来完全正常。

126. 错误 10

Stable address 就应该保存 pointer 作为长期 identity。

长期逻辑 identity 通常更适合 ID/handle。

127. 错误 11

返回 vector 会 copy 很多。

现代 C++ 的 value return + copy elision/move 通常非常自然。

128. 错误 12

vector<bool> 和 vector<T> 没区别。

它是 bit-packed specialization，reference semantics 特殊。

12. 跨语言回查

12.1 C++ / Zig / Rust 对照

129. Dynamic Contiguous Owner

C++:

[机制片段 · 不承诺独立编译]

```
std::vector<T>
```

Rust:

```
Vec<T>
```

Zig 通常通过:

```
allocator-backed ArrayList  
or explicitly managed slices/storage
```

表达。

共同机器模型:

```
contiguous owner
capacity growth
relocation
borrow invalidation
```

130. Borrowed Contiguous View

C++:

```
[机制片段 · 不承诺独立编译]
std::span<T>
```

Rust:

```
&[T]
&mut [T]
```

Zig:

```
[]const T
[]T
```

一个重要差异: Rust borrow checker 会静态约束大量 lifetime/aliasing 问题; C++ span 的 lifetime discipline 主要由程序员/API architecture 维护。

131. String View

C++:

```
[机制片段 · 不承诺独立编译]
std::string_view
```

Rust:

```
&str
```

Zig:

```
[]const u8
```

机器上都接近:

```
pointer + length
```

但 Unicode/text semantics 各语言 abstraction 不同。

132. Iterator / Range

C++:

```
iterator concepts
ranges
lazy views
```

Rust:

```
Iterator trait
iterator adapters
```

Zig: 更常见显式 loops/custom iterators, 没有与 C++/Rust 同等规模的统一高级 iterator 生态。

C++ STL 的突出特点是: container topology + iterator capability + generic algorithm

形成一个历史久而庞大的统一模型。

133. Ownership Difference

C++:

```
container/view distinction
```

主要通过类型约定和 lifetime discipline。

Rust: ownership/borrow 关系更强地进入 type system。

Zig: lifetime 更显式但主要靠 programmer discipline。

底层问题仍然一样:

```
owner dies
→ view must not remain usable
```

13. 工程原则回查

13.1 Final Fifteen Axioms

如果半年后只记十五条:

1. STL 的核心不是容器目录, 而是 **Owner** → **Range/Iterator** → **Algorithm** 的抽象分层。
2. 动态拥有一组同类型 values 时, **std::vector** 应该是默认起点, 除非存在明确反要求。
3. **size** 表示活对象数量, **capacity** 表示 storage 容量; **reserve** 与 **resize** 分属 storage 与 lifetime 两个层次。
4. 任何长期保存 iterator/pointer/reference/span 前都必须明确 container 的 invalidation rules。
5. **span**、**string_view** 与默认 accessor 的 **mdspan** 不拥有底层元素; **Ranges view** 则须另查其 ownership 合同。
6. **Iterator capability** 来源于 **storage topology**; 算法应依赖所需 **capability**, 而不是具体 container name。
7. **Ranges algorithms** 主要改善表达、constraints 和 projection, 不改变 underlying machine fundamentals。
8. **Views** 通常 lazy; lazy 不自动等于 faster, 是否 materialize 取决于 reuse 与 access pattern。
9. **map** 的核心价值是 **ordered semantics/stability**; **unordered_map** 的核心是 **hashing semantics**, 而不是“一个 $O(\log N)$ 、一个 $O(1)$ ”这么简单。

10. **Read-heavy、build-rare 的 associative workload 应认真考虑 sorted flat representation / flat_map。**
11. **Dense integer key 最优 representation 很可能只是 array / vector direct indexing。**
12. **Linked node stability 要付出 allocation、pointer chasing、cache/TLB 的机器成本；只有真实需要时才值得。**
13. **API 应按 capability 表达：owner 用 owner type, contiguous borrow 用 span, string borrow 用 string_view。**
14. **现代 C++ 返回 container by value 是正常 ownership transfer，不应因为旧式“怕 copy”直觉滥用 output parameter。**
15. **Container 选择最终是 Semantics × Lifetime × Access Pattern × Mutation Pattern × Scale × Machine Topology 的联合决策。**

14. Final Gate

14.1 Final Gate

应该能闭卷回答以下问题。

Vector

1. `size` 和 `capacity` 根本区别是什么?
2. `reserve` 为什么不创建 `N` 个 `objects`?
3. `reallocation` 为什么会使旧 `reference/pointer/iterator` 失效?
4. 为什么 `noexcept move` 会影响 `vector growth`?

Views

1. `span` 为什么不是 `owner`?
2. `span<const T>` 与 `const vector<T>&` 的 `abstraction boundary` 有什么区别?
3. 为什么 `string_view` 很容易从 `temporary string` 产生 `dangling`?
4. `mdspan` 解决的是什么 `abstraction` 问题?

Containers

1. 为什么 `vector` 通常优于 `list`?
2. 什么情况下 `list` 的 `node stability/splice` 真正有价值?
3. `deque` 为什么 `random-access` 却不能当 `contiguous storage`?
4. `unordered_map` 为什么不自动比 `map` 快?
5. 什么 `workload` 适合 `flat_map` / `sorted vector`?
6. `dense integer ID` 为什么经常应该 `direct indexing`?

Algorithms / Ranges

1. 为什么 `STL algorithm` 不属于 `container member`?
2. `iterator capability` 与 `container topology` 有什么关系?
3. `projection` 有什么价值?
4. `lazy view` 与 `materialized vector` 应如何选择?
5. 什么情况下 `manual fused loop` 比多个 `algorithm passes` 更合理?

Lifetime

1. `vector mutation` 后, 如何判断已有 `span` 是否仍有效?
2. `stable physical address` 与 `logical identity` 为什么不同?
3. 为什么 `std::ranges::dangling` 是 `lifetime-aware API design`?

API

1. 什么时候参数应该是 `std::span<const T>?`
2. 什么时候 `std::vector<T>&` 反而是正确 API?
3. 为什么 `return std::vector<T>` by value 是现代 C++ 的正常设计?

15. Final Gate · 参考答案与常见误判

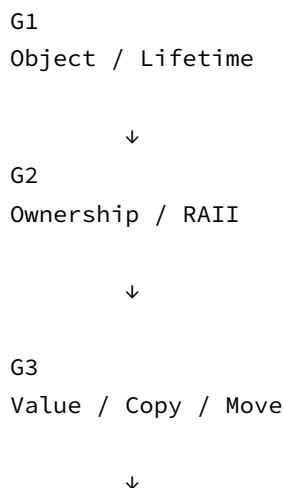
- **Vector 1 ~ 4:** size 是现有元素数, capacity 是不重分配可容纳数; reserve 不增加 size。重分配结束旧元素/存储, 旧路径失效。迁移时的异常保证影响复制/移动选择, 不能把 noexcept 当作一定零成本。
- **Views 1 ~ 4:** span 不管理清理; const span 元素路径与 const vector& 暴露的具体表示/API 不同。临时 string 被销毁, string_view 没有保活。mdspan 组合多维索引、映射和 accessor; 默认 accessor 不拥有元素, 自定义句柄语义另查。
- **Containers 1 ~ 6:** vector 常有更好局部性, 但不是所有任务必胜。list 在确需节点稳定、已知位置和合法 splice 条件时有价值。deque 的随机访问通过分段实现, 不承诺连续。hash、分配和数据规模决定常数; 读多构建少可考虑排序平面表示; 稠密有界 ID 可用直接索引, 仍要检查范围与空槽。
- **Algorithms 1 ~ 5:** 独立算法复用满足能力约束的表示, 不意味着所有算法都必须是自由函数; list::sort 是反例。投影统一抽取字段。lazy 避免部分中间物化, 但反复遍历会重做工作, predicate 还可能涉及缓存和生命周期。需要多项 reduction 时单次融合合理, 是否更快仍需测量。
- **Lifetime 1 ~ 3:** 逐项查操作规则: 重分配使全部旧元素路径失效, erase 等即使不重分配也可失效。地址没变不等于还是同一个逻辑元素; ID 也需代际/失效策略。dangling 阻止部分临时非 borrowed range 的迭代器结果误用, 不是完整借用检查器。
- **API 1 ~ 3:** 同步连续只读输入可用 span<const T>, 须声明不保存及有效期。确需改变容量/拥有序列结构时 vector<T>& 合理。按值返回表达拥有结果, 可直接构造或移动; 为避免想象中的复制而返回局部借用才危险。

完成标准: 能解释 SignalBatch 的排序前提、重复 ID 合同、三种缺失查询和空统计; 能指出四个 G4 实验分别没有证明什么。选型练习与 benchmark 保留为回访题, 不伪记已完成。

16. 统一模型

16.1 与整个 Track 的统一

现在:



流程示意 · continued

G4

Representation Abstraction / STL

↓

G5

Generic Programming

↓

G6

Machine Performance

↓

G7

Concurrency

G4 在中间承担的角色变得非常清楚：**把正确的 object/value/lifetime model，包装成可复用且具备机器成本意识的数据结构与算法接口。**

以后看到：

[机制片段 · 不承诺独立编译]

```
std::unordered_map<
    VehicleId,
    std::shared_ptr<State>>
```

不应该只识别：“hash map + shared_ptr”。

而应该连续追问：

```
为什么需要 hash？
VehicleId 是不是 dense？
为什么 State 不能 inline？
为什么需要 shared ownership？
lookup 是否 hot？
多少 allocation？
是否 pointer chasing？
是否多线程修改？
地址稳定是否真的需要？
能不能 shard by VehicleId？
```

这就是 STL 从“库知识”转化为：**Systems Design Vocabulary** 的标志。

17. 接下来怎么用

本章形成可学习的主线和定向实验，不宣称全部标准库能力冻结。下一次按 [G5 泛型与编译期编程](#) 继续；布局与实测回查 [G6 §4、§9](#)，共享访问回查 [G7 §11](#)，二进制暴露范围回查 [G8 §6](#)。容器选择不本章替代这些专门论证。

[返回全系列导航](#)

G5 · 泛型、类型推导与编译期抽象

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G3；覆盖推导、转发、约束、常量求值及实例化；高级 SFINAE、约束包含与重型元编程仍不展开。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G4 · 全系列导航 · 下一章：G6

阅读入口

本章主线为模板实体、推导与转发、约束、常量求值、实例化控制及泛型边界。泛型编程（generic programming）处理可复用源码中的变化；编译期（compile time）与运行时（runtime）是决策发生的阶段，不是性能等级。首次阅读顺序见下列目录；跨语言、审查和术语用于回查。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 泛型源代码与实例化模型
- 2. 类型推导：实参与形参
- 3. 引用折叠、转发与消费边界
- 4. 类模板与编译期状态
- 5. 约束与语义合同
- 6. 常量求值与编译期分支
- 7. 特化与运行时分派
- 8. 编译模型与实例化控制
- 9. 泛型边界与工程预算
- 10. 跨语言回查
- 11. 工程审查与反模式
- 12. 统一模型与术语
- 13. 实验与验证
- 14. Final Gate
- 15. Final Gate · 参考答案与常见误判

- 16. 工程原则回查
- 17. 参考与验证入口

1. 泛型源代码与实例化模型

1.1 编译期信息的工程边界

本章讨论哪些变化值得成为编译期信息。对象是否合法、谁负责清理以及值如何复制，分别属于 G1 ~ G3；泛型编程（generic programming）则把类型、布局或算法策略的差异交给编译器形成具体实体。工程判断不在于模板写得多，而在于静态差异是否真正改善接口与实现，以及它应在哪一层停止传播。

1.2 Template 的本质

函数模板（function template）是参数化源码，不是运行时根据任意类型解释执行的万能函数。调用 `max_value(1, 2)` 可以推导出 `T = int`；需要定义时，实例化（instantiation）形成相应模板实体。

[机制片段 · 不承诺独立编译]

```
template <typename T>
T max_value(T a, T b) {
    return a > b ? a : b;
}
```

这里还隐含比较和返回值构造要求；并不是任意 `T` 都适用。模板定义、某组实参对应的特化（specialization）和最终二进制符号是三个层次。优化可以内联、常量折叠、合并或删除代码，因此不能通过源代码中的特化数直接推断最终函数符号数。

1.3 Type Template Parameter

类型模板参数（type template parameter）占据的是类型位置。例如 `template<class T>` 中的 `T` 不是运行时变量、对象或类型描述符；当实参是 `int` 时，模板中的 `T` `value`；按 `int` 的规则解释。类型替换并不取消初始化、对象生命周期或表达式合法性的要求。

1.4 Template Argument Deduction

沿用 G1 的值类别与 G3 的移动操作，模板实参推导（template argument deduction）需要分开记录三件事：推导得到的模板实参 `T`、替换并折叠后的形参类型、使用该形参的表达式值类别。三者可能不同：`T` 可以是 `int&`，形参最终也是 `int&`，而源码仍写着 `T&&`。G5-C1 用 `static_assert` 同时观察这些层次，避免只凭变量声明猜结果。

2. 类型推导：实参与形参

2.1 By-value Deduction: T

按值形参 T 建立自己的参数对象。以 `const int x = 42;` 调用 `inspect(x)` 时，推导得到 `T = int`，不会把源对象的顶层 `const` 传播给新参数。

[机制片段 · 不承诺独立编译]

```
template <typename T>
void inspect(T value);
const int x = 42;
// 在函数体中调用 inspect(x): T 为 int。
```

这里忽略的是顶层 `cv` 限定，不是删除类型中的所有 `const`。`const int*` 指向的对象仍受 `const` 限制；把它推导成 `int*` 会错误地授予修改能力。推导规则与之后发生的参数初始化也应分别分析。

2.2 Reference Deduction: T&

`T&` 建立对实参的别名，推导必须保留使引用合法绑定的类型信息。以 `const int` 左值调用 `inspect_ref`，`T` 是 `const int`，最终形参是 `const int&`；以普通 `int` 左值调用，`T` 是 `int`。

[机制片段 · 不承诺独立编译]

```
template<class T>
void inspect_ref(T& value);
```

这里讨论函数模板调用推导，不把 `T&` 理解成“函数必然修改数据”。它说明可绑定的对象与访问能力，实际修改与业务合同还要看函数。

2.3 const T&

`const T&` 中的 `const` 由形参模式提供。以 `int` 或 `const int` 左值调用时，`T` 通常都推导为 `int`，最终形参都是 `const int&`。这与 `T&` 对 `const` 对象推导出 `const int` 不同，不能把最终形参类型直接当作 `T`。

[机制片段 · 不承诺独立编译]

```
template<class T>
void inspect_const_ref(const T& value);
```

该接口可以只读借用不同值类别的对象，但 `const` 引用不是生命周期保险；如果函数保存引用或把它交给异步任务，仍要分析对象何时结束。

2.4 三种最重要 Pattern

在普通调用的核心情况中，可用下表区分三个推导模式。表中 `const int` 是对象本身的 `const`；指向 `const` 的指针、数组等情况应继续按完整规则分析，而不是机械套表。

表 1 · 字段记录视图（列名与原单元格逐项对应）

表 1 · 记录 1 · T	
形参模式	T
int 左值的 T	int
const int 左值的 T	int
接口侧含义	建立独立参数对象

表 1 · 记录 2 · T&	
形参模式	T&
int 左值的 T	int
const int 左值的 T	const int
接口侧含义	保留引用绑定所需限定

表 1 · 记录 3 · const T&	
形参模式	const T&
int 左值的 T	int
const int 左值的 T	int
接口侧含义	形参模式提供只读访问

推导后还要看参数初始化、重载决议及函数体。G5-C1 对表中的关键类型做静态断言，这与运行时打印一个编译器私有类型名相比，更直接地验证目标命题。

3. 引用折叠、转发与消费边界

3.1 普通 Rvalue Reference

普通右值引用（rvalue reference）形参使用已经确定的类型，例如 `Frame&&`。它通常用于接收调用方显式交出的对象，但引用绑定本身不完成资源移动；函数体可以移动、读取，甚至完全不消费。

[机制片段 · 不承诺独立编译]

```
void consume(Frame&& frame);
```

`Frame` 左值不能直接绑定到这个形参。调用方写 `std::move(frame)` 只提供右值绑定选择；资源是否被转移、转移后的状态是什么，仍由选中的操作和 `Frame` 合同决定。

3.2 Forwarding Reference

转发引用是待推导类型模板参数上的特定形式：函数调用推导中的无 `cv` 限定 `T&&` 可以同时接收左值和右值，并把调用方的绑定选择编码到 `T` 中。

[机制片段 · 不承诺独立编译]

```
template<class T>
void relay(T&& value);
```

这个规则不是“&& 能绑定任何东西”的通用规则。`const T&&` 和类模板中已经固定的 `T` 所构成的 `T&&`，都不满足上述条件；应先定位 `T` 在哪里被推导，再谈转发。

3.3 Forwarding Reference Deduction

下面以 `int` 为例逐层展开推导。表达式 `value` 是函数体内普通的具名参数使用；转发后的表达式类别由 `T` 与引用折叠共同决定。

表 2 · 字段记录视图（列名与原单元格逐项对应）

表 2 · 记录 1 · int 左值	
调用实参	int 左值
推导的 T	int&
最终形参类型	int&
std::forward<T>(value)	lvalue

表 2 · 记录 2 · const int 左值	
调用实参	const int 左值
推导的 T	const int&
最终形参类型	const int&
std::forward<T>(value)	const lvalue

表 2 · 记录 3 · int 右值	
调用实参	int 右值
推导的 T	int
最终形参类型	int&&
std::forward<T>(value)	xvalue

右值实参可能原来是 `prvalue`，也可能是 `xvalue`；转发参数并不重建原来的 `prvalue` 求值过程。这是“保留绑定选择”比“恢复一切原始值类别”更准确的原因。

3.4 Reference Collapsing

引用折叠（`reference collapsing`）在通过模板替换、类型别名等形成引用组合时生效。只要组合中有一个左值引用，结果就是左值引用；只有两个右值引用组合才得到右值引用。

组合	结果
T& 与 &	T&
T& 与 &&	T&
T&& 与 &	T&
T&& 与 &&	T&&

这是类型形成规则，不表示运行时存在一个可嵌套的“引用对象”。转发引用把左值实参推导进 T，再通过折叠使最终形参保持左值引用，G5-C1 对这一结果直接断言。

3.5 T&& 不一定是 Forwarding Reference

例如：

[机制片段 · 不承诺独立编译]

```
template <typename T>
class Box {
public:
    void set(T&& value);
};
```

如果：

Box<Frame>

已经确定：T = Frame 那么：

```
void set(Frame&& value);
```

这里是普通 rvalue reference。原因：T 不是由这次 set() 调用推导出来的。

3.6 Named T&& 仍然是 Lvalue Expression

普通表达式中的具名右值引用参数是左值。即使形参类型为 Frame&& 或推导得到 T&&，在函数体中直接写 value 通常仍按左值参与重载决议；需要消费或转发时，再明确转换。

[机制片段 · 不承诺独立编译]

```
template<class T>
void relay(T&& value) {
    consume(value); // 普通具名表达式是 lvalue。
    consume(std::forward<T>(value)); // 按推导结果转发。
}
```

两次调用只是对照片段，不建议实际消费同一参数两次。C++23 的 move-eligible return 等上下文另有隐式移动规则，不能把“具名就永远是 lvalue”当作无例外命题。

3.7 std::move

std::move 对对象表达式提供相应的 xvalue 转换，为后续重载决议开放右值路径。它自身不执行资源搬运，也不删除 const；结果可能选择移动、选择复制，或因没有可行操作而编译失败。

在接口设计中，显式 move 应对应当前代码允许消费源值的边界，而不是用于保证 O(1) 的优化咒语。消费后还想依赖原来的逻辑值，通常意味着合同没有想清楚；类型允许的移出状态操作仍可继续使用。

3.8 std::forward<T>

`std::forward<T>(value)` 使用推导信息恢复调用方的左值或右值绑定选择：左值实参仍以左值转发，右值实参以 `xvalue` 转发。它不会把已经具名的参数表达式重新变成 `prvalue`，也不负责移动资源、延长生命周期或验证所有权。

[机制片段 · 不承诺独立编译]

```
template<class T>
void relay(T&& value) {
    consume(std::forward<T>(value));
}
```

转发引用（forwarding reference）要求相应推导上下文中的无 `cv` 限定类型模板参数右值引用。已经固定的类模板参数 `T` 所形成的成员函数 `T&&` 不是新的转发引用；G5-C2 用编译负例区分这两种模型。规则：N4950 `temp.deduct.call`

3.9 move 与 forward 的根本区别

`move` 与 `forward` 的差别在于绑定选择来自谁。`move` 由当前函数决定开放右值路径，`forward` 使用模板推导记录的调用方选择。两者都不直接执行移动构造。

在普通业务函数中，是否消费由当前层的合同决定；在泛型包装层中，通常应把选择传递给下游。不能因为 `forward` 看起来更通用，就用它掩盖 `owner`、`borrower` 或 `sink` 的职责。

3.10 API Role 决定 move / forward

Borrow

```
void inspect(const Frame& frame);
```

无需 `move/forward`。

Ownership Sink

[机制片段 · 不承诺独立编译]

```
void submit(Frame frame) {
    queue_.push_back(std::move(frame));
}
```

这里函数已经拥有自己的 `Frame value`。可以消费：

```
std::move(frame)
```

Generic Forwarding Layer

[机制片段 · 不承诺独立编译]

```
template <typename T>
void relay(T&& value) {
    target(std::forward<T>(value));
}
```

`wrapper` 不应该擅自改变 `caller` 的 `ownership/value-category` 决定。

3.11 By-value Sink

按值 sink 先取得自己的参数对象，再将其移入成员，可以用一个明确接口同时处理复制调用方和允许被消费的调用方。

[机制片段 · 不承诺独立编译]

```
void set_name(std::string name) {
    name_ = std::move(name);
}
```

左值实参通常复制构造参数，再执行成员移动赋值；右值实参的参数初始化可能移动，某些 prvalue 情况直接形成参数对象。这里的收益取决于 string 等具体表示及赋值状态，不应把它解释为永远少一次操作。它适合合同就是“取得一个值”的接口，不必为每个 setter 引入转发引用。

3.12 Perfect Forwarding 最自然的场景

Factory / emplacement:

[机制片段 · 不承诺独立编译]

```
template <typename T, typename... Args>
std::unique_ptr<T> make_object(Args&&... args) {
    return std::unique_ptr<T>(
        new T(std::forward<Args>(args)...));
}
```

这里 wrapper:

不知道 T constructor 需要什么、也不应该替 caller 决定 copy/move。

所以 perfect forwarding 很合理。

4. 类模板与编译期状态

4.1 Class Template

[机制片段 · 不承诺独立编译]

```
template <typename T, std::size_t N>
class FixedBuffer;
```

这里:

T
→ type template parameter

N
→ non-type template parameter

4.2 Non-type Template Parameter

```
template <std::size_t N>
```

中的 N 是: **compile-time value** 例如:

```
FixedBuffer<float, 16>
```

```
:
```

T = float、N = 16。

4.3 Different Arguments → Different Types

[机制片段 · 不承诺独立编译]

```
FixedBuffer<float, 16>
```

```
FixedBuffer<float, 32>
```

```
FixedBuffer<int, 16>
```

三个都是不同 concrete C++ types。不是:

一个 Buffer type + 不同 runtime configuration

所以:

[机制片段 · 不承诺独立编译]

```
using A = FixedBuffer<float, 16>;
```

```
using B = FixedBuffer<float, 32>;
```

```
static_assert(!std::is_same_v<A, B>);
```

4.4 Template Argument 可以影响 Object Layout

[机制片段 · 不承诺独立编译]

```
template <typename T, std::size_t N>
```

```
struct Buffer {
```

```
    std::array<T, N> storage;
```

```
};
```

于是: Buffer<float,16> 包含 16 个 float。而: Buffer<float,1024> 包含 1024 个。所以:

```
compile-time parameter
```

```
↓
```

```
concrete type
```

```
↓
```

```
object layout
```

```
↓
```

```
sizeof / alignment / generated operations
```

都可能发生变化。

4.5 Compile-time Information vs Runtime State

Runtime:

[机制片段 · 不承诺独立编译]

```
class Buffer {
    std::size_t capacity_;
};
```

每个 object 需要保存: `capacity_ Template:`

`FixedBuffer<T, 16>`

`capacity` 已经由 `type` 表达。可能不需要 object 再保存: `capacity = 16` 这一 runtime field。所以: `Template` 可以把某些 runtime state 提升为 type-level compile-time information。

4.6 但不要把所有数据都 Template 化

适合 compile time:

type、fixed extent、small finite policy、endianness、algorithm mode、protocol layout、compile-time feature。

通常不适合:

timestamp、vehicle ID、request ID、user ID、sample value、大量动态 configuration。

核心判断: 它属于 **program structure**, 还是 **program data** ?

4.7 std::array<T, N> 与 Reserved Storage 的区别

`std::array<T, N>` 将元素存储嵌入对象; 成功构造的数组包含相应的 `N` 个元素对象 (`N` 为零时没有元素)。`vector<T>::reserve(N)` 只保证容量下界, 不把 `size` 增至 `N`, 也不构造 `N` 个元素。固定长度既可进入类型和布局, 也会影响构造、析构及移动成本; 它不是可任意拿来按活对象访问的未初始化缓冲区。

5. 约束与语义合同

5.1 Concepts — 当前阶段只保留 Core

Concept: **Template argument 的 compile-time contract**。例如:

[机制片段 · 不承诺独立编译]

```
template <std::integral T>
T twice(T value) {
    return value + value;
}
```

读成: `T` 必须满足 `std::integral`。

5.2 简单自定义 Concept

[机制片段 · 不承诺独立编译]

```
template <typename T>
concept HasSize = requires(const T& value) {
    value.size();
};
```

表示：对这种 T, `value.size()` 必须是合法表达式。然后：

[机制片段 · 不承诺独立编译]

```
template <HasSize T>
void print_size(const T& value);
```

5.3 Concept 不是什么

Concept 不是：

base class、runtime interface、vtable、runtime type object。

它工作在：compile time 影响：

candidate eligibility、generic contract、instantiation、overload resolution。

5.4 Concept 的边界

Concept 可以检查表达式是否有效、结果类型是否符合限制，以及被编入约束的常量条件。它不能从一个比较表达式可调用就推导出严格弱序，也不能验证未编码的业务守恒关系。语法满足和语义建模应分别验收：编译负例可检验接口拒绝了不具备能力的类型，性质测试和论证才触及语义规律。

5.5 不要 Over-constrain

例如：

[机制片段 · 不承诺独立编译]

```
template <std::copyable T>
void inspect(const T& value);
```

如果函数根本不 copy T: `std::copyable` 就可能是多余限制。原则：**Concept 应表达算法真正需要的最小充分 contract**。

5.6 Concrete Interface 有时比 Concept 更好

如果 parser 只需要：contiguous bytes 那么：

```
void parse(std::span<const std::byte> bytes);
```

可能明显优于：

[机制片段 · 不承诺独立编译]

```
template <ByteBuffer T>
void parse(const T& input);
```

因为 `span` 已经提供：

明确 `data shape`、简单 API、少 `template instantiation`、更小 `compile surface`。

所以：会 Concepts 不代表所有接口都应该 generic。

6. 常量求值与编译期分支

6.1 constexpr

[机制片段 · 不承诺独立编译]

```
constexpr int square(int x) {
    return x * x;
}
```

不要读成：“这是编译期函数。”应该读成：这个函数可以参与 **constant evaluation**。

6.2 同一个 constexpr Function 可以两种执行

```
constexpr int a = square(4);
```

这里：compile-time evaluation required 而：

[机制片段 · 不承诺独立编译]

```
int x = runtime_input();
int b = square(x);
```

可以：runtime evaluation 所以：

`constexpr` ≠ always compile time

6.3 const vs constexpr

```
const int x = runtime_input();
```

`x` 可以是 runtime value。只是不允许通过 `x` 修改。而：

```
constexpr int x = 42;
```

必须拥有 constant-expression value。压缩：

```
const
→ mutability constraint
```

```
constexpr
→ constant-evaluation constraint/capability
```

6.4 consteval

`constexpr` 声明立即函数 (immediate function)。普通调用上下文中的立即调用必须满足常量表达式要求；将运行时输入传入这种调用应诊断失败。立即函数上下文有进一步规则，因此这里不把

它简化成“函数体中的每个中间调用都能独立在源码位置求值”。G5-C4 用 `argc` 制造明确不满足条件的普通调用。

6.5 if constexpr

`if constexpr` 根据常量条件选择结构分支。在模板实例化中，当条件不再依赖模板参数时，被丢弃分支不会按该特化继续实例化，所以类型相关的不同操作可以分别存在。但整个源码仍须被解析，非依赖错误不会因为位于 `discarded statement` 中就一概消失。

[机制片段 · 不承诺独立编译]

```
template<class T>
constexpr auto scalar(T value) {
    if constexpr (std::is_pointer_v<T>) {
        return *value;
    } else {
        return value;
    }
}
```

指针分支的调用还要求指针指向可合法读取的对象；编译期选择不证明运行时生命周期。普通 `if` 不提供这种模板分支丢弃机制。规则：N4950 `stmt.if`

6.6 if constexpr 可以让分支拥有不同合法表达式

模板分支可以使用不同类型能力，因为选中的条件和实例化规则决定哪些依赖表达式需要成立。

[机制片段 · 不承诺独立编译]

```
template<class T>
void process(T value) {
    if constexpr (std::integral<T>) {
        value += 1;
    } else {
        value.do_something();
    }
}
```

对 `process(42)`，未选的依赖分支不要求 `int` 拥有 `do_something`。若改为普通 `if`，不能以运行时条件为由跳过该特化中的类型检查。对其他 `T`，被选中的 `do_something` 仍必须有效；`constexpr` 分支不是隐藏任意坏代码的容器。

7. 特化与运行时分派

7.1 Compile-time Specialization

[机制片段 · 不承诺独立编译]

```
template <Endian E>
std::uint16_t decode(...);
```

可能形成：

- decode<Endian::Little>
- decode<Endian::Big>

其中：Endian 已经从 runtime state 变成 program structure。这可以：

remove repeated branch、enable constant propagation、enable inlining、specialize algorithm。

7.2 但 Template Version 不一定比 Runtime Version 快

Runtime：

```
decode(bytes, Endian::Little);
```

如果 optimizer 看得到：Endian::Little 是 constant，也可能通过：

inlining、constant propagation、branch elimination。

生成和 template specialization 几乎一样的 machine code。所以：不能仅凭源码有 **template** 就推断性能更好。要看：

assembly、benchmark、profile。

7.3 Dynamic Outside, Static Inside

很多系统现实是：

```
startup/configuration
→ dynamic

steady-state processing
→ stable
```

因此很好的模式：

```
runtime choice once
    ↓
enter specialized implementation
    ↓
hot loop many times
```

例如：

[机制片段 · 不承诺独立编译]

```
switch (config.endian) {
case Endian::Little:
    process_stream<Endian::Little>(stream);
    break;

case Endian::Big:
    process_stream<Endian::Big>(stream);
    break;
}
```

核心思想：低频 **dynamic decision** 移出高频 **hot path**。

8. 编译模型与实例化控制

8.1 Template Compilation Model

普通函数：

[机制片段 · 不承诺独立编译]

```
// foo.hpp
int foo(int);

// foo.cpp
int foo(int x) {
    return x;
}
```

调用方编译时只需要 **declaration**。最终由 **linker** 找 **concrete definition**。Template：

[机制片段 · 不承诺独立编译]

```
template <typename T>
T foo(T x);
```

调用：

```
foo(42);
```

需要：foo<int> compiler 要生成这个 **concrete function**，通常就必须看到：Template definition body。这就是为什么 **template definitions** 经常位于 **header**。

8.2 Header-only Template

[机制片段 · 不承诺独立编译]

```
template <typename T>
T foo(T value) {
    ...
}
```

定义放 header。好处：

open genericity、consumer can instantiate required T、optimizer sees body。

代价：

more parsing、more instantiation、larger dependency surface、rebuild propagation、implementation exposure。

8.3 Zero Runtime Overhead ≠ Zero Engineering Cost

Template 可能拥有非常优秀 runtime performance。但是可能支付：

compile time、binary size、debug symbol size、diagnostic complexity、dependency complexity。

所以：**zero-overhead abstraction** 主要描述 **runtime cost model**，不等于 **template** 是免费机制。

8.4 Explicit Instantiation

显式实例化（explicit instantiation）用于集中提供一组已知模板实参的实现。公共头文件给出声明和适当的 `extern template` 声明，提供方翻译单元持有定义并显式实例化所支持的类型；调用方链接到这些定义。这不是让任意新类型都能在看不到定义时自动实例化。若公共接口允许写出 `scale<double>`，但提供方只生成 `scale<int>`，类型检查可能通过而链接失败。G5-C5 分别检查这种失败和提供 `int` 实现后的运行，并记录 nm 观察。符号拼写和优化后的实体数量不是可移植保证。

9. 泛型边界与工程预算

9.1 Thin Template Front-end + Concrete Core

薄模板前端负责真正依赖类型的检查、适配或策略选择，具体核心实现接收 `span`、视图或普通参数，避免把相同工作按每种类型重新实例化。例如序列化前端可以检查输入类型，但核心只能在明确表示合同后处理字节。

[机制片段 · 不承诺独立编译]

```
void consume_bytes(std::span<const std::byte> bytes);
template<class T>
void inspect_representation(std::span<const T> values) {
    consume_bytes(std::as_bytes(values));
}
```

这里观察的是对象表示，不是通用序列化或逻辑值相等算法；padding、端序、指针及 ABI 条件仍存在。具体核心减少重复实例化，但不能以丢弃类型为代价掩盖必要的语义合同。

9.2 Where Should Genericity Stop?

这是 G5 最重要的架构问题。假设：

[机制片段 · 不承诺独立编译]

```
FixedBuffer<int, 16>
FixedBuffer<int, 32>
FixedBuffer<int, 64>
```

如果算法不关心 N，不要机械写：

[机制片段 · 不承诺独立编译]

```
template <std::size_t N>
void analyze(const FixedBuffer<int, N>&);
```

更合理的可能是：

```
void analyze(std::span<const int>);
```

这样：storage detail N 在 algorithm boundary 被擦除。

9.3 为什么这很好？

减少：

algorithm<16>、algorithm<32>、algorithm<64>。

这些几乎相同的 specializations。同时获得：

simpler API、less compile time、smaller binary、less coupling。

所以：不要让 **compile-time variability** 传播得比必要范围更远。

9.4 Template vs Runtime Parameter

Runtime：

```
decode(bytes, endian);
```

优点：

one function、simple API、runtime flexibility、small code size、faster builds。

Template：

```
decode<Endian::Little>(bytes);
```

优点：

static policy、compile-time structural specialization、more optimization knowledge、possibly no runtime branch。

没有永远正确的一边。

9.5 如何决定 Static / Dynamic Boundary

问四个问题：

1. 信息变化频率？

每次 sample

→ runtime

程序启动后不再变化

→ specialization candidate

2. 它改变算法结构吗？

endianness

→ may change operations

timestamp

→ ordinary data

3. Hot path 会不会重复检查它？

如果每秒数百万次重复同一 decision：提前 bind/specialize 更有吸引力。

4. 会产生多少 Specializations？

2

→ 很轻

8

→ 常常合理

6000

→ 要认真评估

combinatorial explosion

→ 危险

9.6 Combinatorial Instantiation Explosion

例如：

[机制片段 · 不承诺独立编译]

```
template <
    Endian E,
    unsigned StartBit,
    unsigned Size,
    bool Signed,
    int ScaleNumerator,
    int ScaleDenominator>
struct Decoder;
```

参数组合可能爆炸。你可能只是为了消除：

一个 branch、几个 metadata loads。

却换来了：

huge build time、huge code、large debug information、complex diagnostics。

所以：**compile-time** 越多不等于越优秀。

9.7 Template vs External Code Generation

二者都能：

```
known metadata
↓
specialized executable structure
```

Template 更适合：

small policy set、type-oriented variation、compile-time dimensions、generic reusable components。

External codegen 更适合：

DBC、IDL、schema、CSV、hundreds/thousands of definitions、large generated registries。

语言无关核心：**稳定 metadata 可以从 runtime data 转化为 generated program structure。**

9.8 Genericity Budget

任何 template abstraction 都应该做一次成本核算。

收益

static checking、compile-time specialization、inlining opportunities、generic reuse、type-level invariants。

成本

compile time、code size、dependency propagation、diagnostic complexity、API complexity、ABI/binary-boundary complexity。

所以：**Genericity 是 architecture budget。**

9.9 设计 Ladder

遇到一个问题，不要直接 template。从最具体开始：

```
Concrete Function
    ↓
Concrete View / span
    ↓
Runtime Polymorphism
    ↓
Generic Template
    ↓
Compile-time Specialized Template
```

每向下一层增加能力，也增加复杂度。选择：能表达真实需求的最简单 abstraction。

9.10 Concrete Type vs span vs Template

假设算法只需要：连续只读 samples。可以：

[机制片段 · 不承诺独立编译]

```
void process(
    const std::vector<Sample>& samples);
```

但暴露了 vector。更窄：

[机制片段 · 不承诺独立编译]

```
void process(
    std::span<const Sample> samples);
```

如果所有类型都能统一成这种 runtime view：没必要 template。只有当算法真正需要保留：different static types、different operations、different compile-time policies。时，template 才更自然。

9.11 Static Polymorphism vs Dynamic Polymorphism

静态多态在编译期按具体类型选择行为，可能帮助内联和特化；动态多态通过运行时分派保留实现替换边界。两者应按变化发生的层次选择，而不是按“模板一定更快”排名。

[机制片段 · 不承诺独立编译]

```
struct Processor {
    virtual ~Processor() = default;
    virtual void process() = 0;
};
```

虚函数接口可以隐藏某些实现细节，但不会自动形成跨编译器、跨版本的稳定 ABI。对象布局、调用约定、异常和所有权仍须约定。静态分派同样可能因代码膨胀损害指令工作集，实际成本交给 G6 的测量方法。

10. 跨语言回查

10.1 C++ / Zig / Rust 对照

C++

```
template <typename T, std::size_t N>
```

特点：

template subsystem、deduction、specialization、reference collapsing、Concepts、constexpr。能力极强，但历史层次很多。

Zig

```
fn foo(comptime T: type, comptime N: usize) type
```

倾向：compile-time execution integrated with normal language model 没有 C++：

- forwarding reference
- reference collapsing
- `std::forward`

整套机制。

Rust

```
struct Buffer<T, const N: usize>
```

和：

```
fn foo<T: Trait>(...)
```

分别提供：

generics、const generics、trait bounds、monomorphization。

Rust ownership/reference model 和 C++ 不同，因此无需复制 C++ forwarding machinery。

10.2 三种语言共同的机器级事实

无论语言如何表达：

```
static type/policy known
↓
compiler may specialize
↓
more optimization opportunities
```

同时：

```
more concrete specializations
↓
more compile work / code
```

所以真正的 systems trade-off 是共同的。

11. 工程审查与反模式

11.1 Code Review Protocol

看到：

```
template <typename T>
```

按下面检查。

1. 为什么需要 Generic ？

普通 concrete API 不够吗？

2. 什么东西在变化 ？

- type?

- layout?
- algorithm?
- policy?

3. 变化真的需要发生在 **Compile Time** 吗？

还是 runtime data 更合适？

4. 会有多少 **Specializations** ？

5. 每个 **Specialization** 机器代码真的不同吗？

6. 是否在不必要地传播 **Template Parameter** ？

7. 是否能在某层收敛成

span、view、function pointer、ordinary function。

？

8. 是否应该用 **by-value sink**，而不是 **forwarding reference** ？

9. **std::move** 是否真的代表一次合理的 **consumption boundary** ？

10. **std::forward** 是否真的位于 **generic forwarding layer** ？

11. **Template** 定义是否必须全部暴露在 **Header** ？

12. 实际性能收益是否被 **benchmark / assembly** 验证？

11.2 高频 Anti-patterns

Anti-pattern 1: Everything is Generic

[机制片段 · 不承诺独立编译]

```
template <typename T>
void parse(T&& input);
```

实际上：

```
void parse(std::span<const std::byte>);
```

已经足够。

Anti-pattern 2: Cargo-cult Forwarding

[机制片段 · 不承诺独立编译]

```
template <typename T>
void set_model(T&& model);
```

只因为：“perfect forwarding 最快”。而实际上 API 就是：

```
void set_model(Model model);
```

Anti-pattern 3: Cargo-cult **std::move**

```
consume(std::move(value));
```

却没搞清：source 之后是否还需要保持原 value。

Anti-pattern 4: Template Every Runtime Constant

Vehicle<123>、Message<456>、Request<789>。

导致 type proliferation。

Anti-pattern 5: Specialize Thousands of Equivalent Paths

为了一点微小 runtime branch 成本产生大量 machine code。

Anti-pattern 6: Heavy Template Core

[机制片段 · 不承诺独立编译]

```
template <typename T>
void process(...) {
    // 500 lines
}
```

而真正 type-dependent 部分只有几行。考虑：

thin template adapter + ordinary implementation core

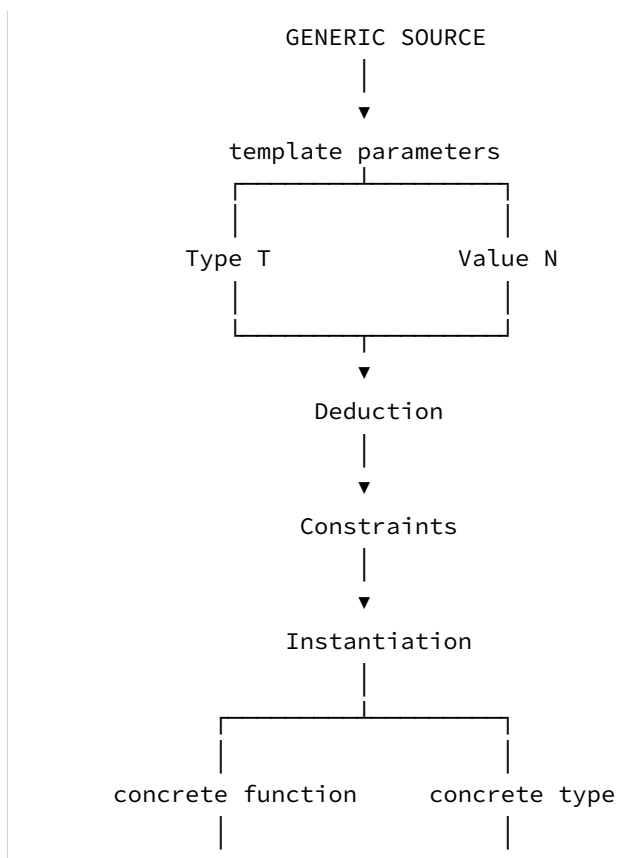
Anti-pattern 7: Compile-time Knowledge Leakage

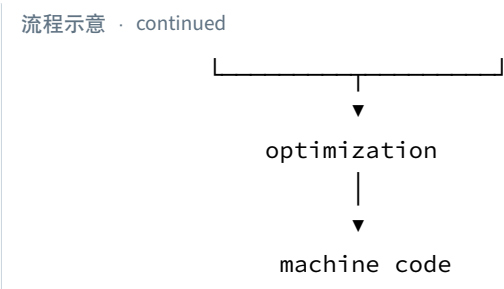
底层只因为：N = 16 整个上层 call graph 都变成：template<N> 导致无意义 specialization 扩散。

12. 统一模型与术语

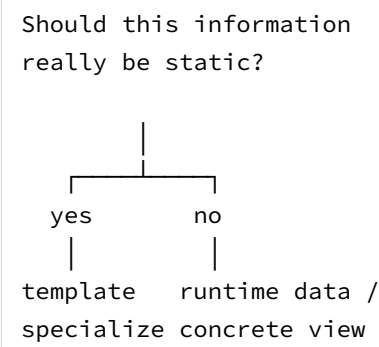
12.1 Final Mental Model

最终可以压缩为：





然后在架构层再问：



12.2 最终术语表

Term	核心含义
Template	编译期参数化的源码抽象
Template Parameter	模板声明中的参数，如 <code>T / N</code>
Template Argument	使用模板时给出的具体参数，如 <code>int / 16</code>
Instantiation	根据模板和 <code>arguments</code> 形成具体模板实体的过程
Specialization	针对具体 <code>arguments</code> 的模板实体
Type Parameter	类型模板参数，如 <code>typename T</code>
Non-type Template Parameter	编译期值模板参数，如 <code>std::size_t N</code>
Template Argument Deduction	从函数调用推导 <code>template arguments</code>
Reference Collapsing	<code>& / &&</code> 组合后的引用折叠规则
Forwarding Reference	推导上下文中的特殊 <code>T&&</code>
Perfect Forwarding	保留 <code>cv/ref</code> 约束及左值/右值绑定选择的转发
<code>std::move</code>	无条件将表达式转换为可消费的 <code>xvalue</code>
<code>std::forward</code>	按推导参数保留左值/右值绑定选择（右值转为 <code>xvalue</code> ）
Concept	对 <code>template arguments</code> 的命名 <code>compile-time constraint</code>
<code>constexpr</code>	可参与 <code>constant evaluation</code> 的语言机制
<code>constexpr</code>	要求 <code>immediate compile-time evaluation</code>
<code>if constexpr</code>	<code>compile-time structural branching</code>
Explicit Instantiation	显式要求生成特定 <code>specialization</code>
Header-only	模板实现对 <code>consumer TU</code> 可见的常见组织方式

Term	核心含义
Code Bloat	多 specializations 导致的代码体积增长
Static Polymorphism	compile-time 根据 concrete type 生成/选择行为

13. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 h-lab/h-file 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

13.1 G5-C1 · 推导、转发与常量分支

命题、观察与边界。用类型断言区分推导得到的 T、最终引用类型和表达式类别；同时让 constexpr 在常量与运行时输入上工作。运行输入为 argc，不把优化器常量折叠误称为语言要求。

[完整实验 · G5-C1 · main.cpp]

```
#include <concepts>
#include <type_traits>
#include <utility>

template<class T> auto by_value(T) -> std::type_identity<T>;
template<class T> auto by_ref(T&) -> std::type_identity<T>;
template<class T> auto by_const_ref(const T&) -> std::type_identity<T>;
template<class T> constexpr bool inspect(T&& value) {
    static_assert(std::is_lvalue_reference_v<decltype((value))>);
    static_assert(std::same_as<decltype(std::forward<T>(value)), T&&>);
    return std::is_lvalue_reference_v<T>;
}

template<class T> constexpr auto scalar(T value) {
    if constexpr (std::is_pointer_v<T>) return *value;
    else return value;
}

constexpr int square(int n) { return n * n; }
template<std::integral T> constexpr T twice(T n) { return n + n; }

int main(int argc, char**) {
    int x = 7;
    const int cx = 8;
    const int* p = &cx;
    static_assert(std::same_as<decltype(by_value(cx))::type, int>);
```

完整实验 · G5-C1 · main.cpp · continued

```

    static_assert(std::same_as<decltype(by_value(p))::type, const int*>);
    static_assert(std::same_as<decltype(by_ref(cx))::type, const int>);
    static_assert(std::same_as<decltype(by_const_ref(cx))::type, int>);
    static_assert(square(4) == 16 && twice(4) == 8);
    static_assert(scalar(4) == 4);
    if (!inspect(x) || !inspect(cx) || inspect(7)) return 1;
    if (scalar(&x) != 7 || scalar(argc) != argc) return 2;
}

```

运行与判据。上述统一命令中的 G5-C1 提取并处理本模块。静态断言、编译与运行均须成立；运行不产生输出且返回 0。

13.2 G5-C2 · 固定 T 的右值引用不能绑定左值

命题、观察与边界。Box<int> 的 T 已固定；成员 set(T&&) 不是转发引用。正例 G5-C1 的推导必须成功，本反例必须因引用绑定失败而被拒绝。

[反例 · 编译失败 · G5-C2 · main.cpp]

```

template<class T> struct Box { void set(T&&) {} };
int main() {
    Box<int> box;
    int value = 1;
    box.set(value);
}

```

运行与判据。上述统一命令中的 G5-C2 提取并处理本模块。只编译不链接，必须匹配指定语义诊断；超时、信号退出和无关错误均不算符合预期。

13.3 G5-C3 · Concept 拒绝不满足约束的类型

命题、观察与边界。std::integral 约束拒绝 double；测试匹配约束诊断，不把缺少头文件或链接失败算作成功。它不检验比较器等概念的语义规律。

[反例 · 编译失败 · G5-C3 · main.cpp]

```

#include <concepts>
template<std::integral T> constexpr T twice(T x) { return x + x; }
int main() { return static_cast<int>(twice(1.5)); }

```

运行与判据。上述统一命令中的 G5-C3 提取并处理本模块。只编译不链接，必须匹配指定语义诊断；超时、信号退出和无关错误均不算符合预期。

13.4 G5-C4 · 立即调用不能使用运行时实参

命题、观察与边界。把 argc 传入普通上下文中的 consteval 调用，必须出现常量表达式相关诊断；对应的 constexpr 正例见 G5-C1。

[反例 · 编译失败 · G5-C4 · main.cpp]

```
constexpr int immediate(int n) { return n + 1; }
int main(int argc, char**) { return immediate(argc); }
```

运行与判据。上述统一命令中的 G5-C4 提取并处理本模块。只编译不链接，必须匹配指定语义诊断；超时、信号退出和无关错误均不算符合预期。

13.5 G5-C5 · 显式实例化与二进制符号

命题、观察与边界。先分别编译 TU，再观察提供方目标文件。`int` 的显式实例化可链接并得到 6；调用未提供的 `double` 特化必须在链接阶段失败。`nm` 输出只作观察，不固定 ABI 拼写、地址或符号个数。

[完整实验 · G5-C5 · scale.hpp]

```
#pragma once
template<class T> T scale(T value);
extern template int scale<int>(int);
```

[完整实验 · G5-C5 · scale.cpp]

```
#include "scale.hpp"
template<class T> T scale(T value) { return value + value; }
template int scale<int>(int);
```

[完整实验 · G5-C5 · main.cpp]

```
#include "scale.hpp"
int main() { return scale(3) == 6 ? 0 : 1; }
```

[完整实验 · G5-C5 · missing.cpp]

```
#include "scale.hpp"
int main() { return scale(1.5) == 3.0 ? 0 : 1; }
```

运行与判据。上述统一命令中的 G5-C5 提取并处理本模块。分别编译 `scale.cpp`、`main.cpp` 与 `missing.cpp`；先记录符号，再检查正例和缺失特化的链接诊断。

14. Final Gate

复习时至少能够闭卷解释这些问题：

14.1 Template

1. Function template 和普通 function 最大的编译模型区别是什么？
2. 为什么一个 template 可以形成多个 concrete functions？
3. Instantiation 和 specialization 分别是什么？

14.2 Deduction

1. T、T&、const T& 的 deduction 有什么区别？
2. top-level const 为什么在 by-value deduction 中通常消失？
3. 为什么 pointer-to-const 中的 const 不能随意消失？

14.3 Forwarding

1. 普通 T&& 与 template<typename T> T&& 有什么不同？
2. 什么条件下 T&& 才是 forwarding reference？
3. Reference collapsing 是什么？
4. 为什么 named T&& expression 仍然是 lvalue？
5. std::move 和 std::forward 的语义区别是什么？

14.4 Class Templates

1. 为什么 Buffer<float,16> 与 Buffer<float,32> 是不同类型？
2. Non-type template parameter 如何影响 object layout？
3. 为什么不应该把普通 runtime identity 全部变成 NTTP？

14.5 Lifetime

1. 为什么 array<T,N> 与 vector<T>::reserve(N) 的 object lifetime 完全不同？

14.6 Concepts

1. Concept 解决什么问题？
2. Concept 是 runtime interface 吗？
3. 为什么 Concept 不能保证 semantic laws？

14.7 Constant Evaluation

1. 为什么 `constexpr function` 不等于“永远编译期执行”？
2. `const` 和 `constexpr` 有什么根本区别？
3. `if constexpr` 与普通 `if` 的核心区别是什么？

14.8 Compilation

1. 为什么 `template definitions` 通常放 `header`？
2. `Explicit instantiation` 解决什么问题？
3. `Header-only generic library` 的主要工程成本是什么？

14.9 Architecture

1. 什么叫 `Dynamic Outside, Static Inside`？
2. 哪些信息适合 `compile time`，哪些更适合 `runtime`？
3. 为什么 `template specialization` 不一定比 `runtime parameter` 快？
4. 什么情况下应该用 `external codegen` 而不是 `template`？
5. 为什么要限制 `genericity` 的传播范围？
6. `Thin template front-end + concrete core` 有什么价值？

15. Final Gate · 参考答案与常见误判

15.1 Template 与 Deduction

Template 1 ~ 3: 普通非模板函数的调用通常只需声明可见，模板特化需要遵守相应定义可达和实例化规则。同一参数化源码可以形成不同实参的实体；实例化是形成实体的机制，**specialization** 指对应具体实参的实体，不限于手写显式特化。最终是否保留独立机器符号还取决于优化与链接，不能把三者画成等号。

Deduction 1 ~ 3: 按值 T 忽略实参的顶层 cv；T& 保留所引用对象的 const；const T& 由形参模式提供 const，不能据最终形参含 const 就断言 T 也含 const。指向 const 对象的指针中的 const 不在顶层，去掉它会授予错误的写权限。G5-C1 分别断言这些类型；编译成功证明的是这些命题在本配置被接受，不是所有推导场景已穷举。

15.2 Forwarding、Class Templates 与 Lifetime

Forwarding 1 ~ 5: 已知 `Frame&&` 只接受相应右值绑定；无 cv 的待推导类型参数 T&& 才可能是转发引用。左值实参使 T 推导为引用，再按“出现 & 则折叠为 &，仅 && 与 && 得 &&”形成最终类型。普通具名参数表达式是左值，C++23 隐式移动上下文另论。`move` 无条件产生相应右值引用转换，`forward` 按推导信息保留左值/右值绑定选择；二者都不搬运资源，也不证明生命周期。

Class Templates 1 ~ 3: 不同 N 属于不同模板实参组合，因而形成不同类型；N 若决定内嵌数组长度，就会影响表示与对象数。运行时 ID 若只用于比较而不改变结构，进入 NTTP 会扩大类型和特化集合，却不一定增加优化机会。Lifetime 1: 成功构造的 `array` 有 N 个元素；`reserve` 只预留容量，不增加 `vector` 的元素数。把 `capacity` 当作活对象数是原有对象模型错误，不是泛型的特例。

15.3 Concepts、Constant Evaluation 与 Compilation

Concepts 1 ~ 3: Concept 在编译期表达被编码的约束，不是虚函数式运行时接口。表达式成立不证明严格弱序、复杂度或业务不变量；约束负例与语义测试承担不同责任。Constant Evaluation 1 ~ 3: `constexpr` 函数允许符合条件的常量求值，也可用运行时输入执行；`const` 仅限制修改，不保证初始化来自常量表达式。`if constexpr` 可丢弃模板中的未选依赖分支，普通 `if` 不能提供同样的实例化选择；非依赖错误和语法仍须合法。不要用“优化后看不到指令”反推语言强制编译期执行。

Compilation 1 ~ 3: 隐式实例化需要相应定义，所以头文件是常见组织方式，但不是唯一手段。显式实例化可以集中提供已知组合；未提供的组合不会因此获得定义。头文件泛型会增加解析、实例化、诊断、依赖和增量构建成本，运行时零额外开销不等于工程零成本。

15.4 Architecture

Architecture 1 ~ 6: `Dynamic Outside, Static Inside` 在外层读取运行时配置，再选有限的静态内核。改变类型、布局或热路径结构且取值集合可控的信息更适合静态化；频繁变化的普通数据应留在运行时。优化器也能常量传播普通参数，特化还可能膨胀指令工作集，因此“模板更快”必须实测。

已有 DBC、IDL 或协议描述适合外部代码生成，尤其当需要多语言产物或独立检查时；这不是再造一个模板元编程框架的理由。限制泛型传播能控制组合数和接口负担；薄前端只保留必要类型信息，

具体核心复用真正相同的逻辑。代价是必须写清类型擦除后的表示合同，不能把任意对象字节当作可移植消息。

16. 工程原则回查

16.1 如果半年后只记住十五条

1. **Template** 是 **compile-time parameterization**，不是 **runtime dynamic genericity**。
2. **Template** 会根据 **arguments** 形成 **concrete functions/types**。
3. 始终分开 **deduced T**、最终 **parameter type** 和 **expression value category**。
4. **By-value deduction** 通常丢弃 **top-level cv**；**reference deduction** 必须保留合法 **alias** 所需的信息。
5. **Forwarding reference** 是 **deduction context** 中特殊的 **T&&**，不是所有 **T&&**。
6. 普通具名右值引用参数表达式是 **lvalue**；C++23 隐式移动上下文另有规则。
7. **std::move** 表示当前代码允许消费 **source**；**std::forward** 表示 **generic wrapper** 保留 **caller** 的决定。
8. **Class template arguments** 可以进入类型和 **layout**；**Buffer<T,16>** 与 **Buffer<T,32>** 是不同类型。
9. 成功构造的 **std::array<T,N>** 包含 **N** 个元素对象；**reserve** 不增加 **vector** 的 **size**。
10. **Concept** 是 **compile-time generic contract**，不是 **runtime interface**。
11. **constexpr** 表示 **constant-evaluation** 能力；**if constexpr** 表示 **compile-time structural selection**。
12. **Template definitions** 通常需要在 **instantiation point** 可见，因此直接影响 **header/TU/build architecture**。
13. **Static specialization** 可以减少 **runtime work**，但会增加 **compile-time/code-size complexity**。
14. 不要让 **compile-time variability** 比真正需要的范围传播得更远。
15. 优秀 **Generic Programming** 的标志不是 **template** 多，而是清楚知道 **genericity** 应该在哪里停止。

17. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

语言规则采用 N4950 的模板推导、模板实例化与 **if constexpr**；编译器实验不是这些规则的替代定义。接下来 G6 §2 把静态表示落实到机器布局与测量；实例化带来的构建传播问题则在 G9 §8 继续。

G6 · 内存布局、机器成本与性能测量

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G3/G5；追踪布局、局部性、分配、虚拟内存与跨核成本，以测量检验假设。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G5 · 全系列导航 · 下一章：G7

阅读入口

主线从对象布局（object layout）追到局部性（locality）、分配（allocation）、虚拟内存（virtual memory）、控制流和跨核协调，再用测量（measurement）与性能剖析（profiling）检验假设。结构体数组（AoS，array of structures）与数组组成的结构体（SoA，structure of arrays）只是表示选择，不能预设赢家。先读第 1 ~ 9 节，再用实验和 Gate 检查推理；其余部分供回查。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 阅读模型
- 2. 对象布局
- 3. 缓存与局部性
- 4. 数据布局：AoS、SoA 与 AoSoA
- 5. 分配、区域与内存资源
- 6. 虚拟内存与地址翻译
- 7. 分支预测与控制流
- 8. 缓存一致性与伪共享
- 9. 测量、剖析与证据
- 10. 统一性能审查
- 11. 跨语言回查
- 12. 术语回查
- 13. 实验与验证
- 14. Final Gate

- 15. Final Gate · 参考答案与常见误判
- 16. 工程原则回查
- 17. G1 ~ G6 的统一系统模型
- 18. 从机器观察转向语言并发模型
- 19. 参考与验证入口

1. 阅读模型

1.1 文档定位

G6 不再主要学习“C++ 有哪些语法”。它研究的是：一个语义正确的 C++ 程序，最终怎样使用 CPU 和内存，以及这些选择为什么会快或慢。前面几个阶段分别解决：

G1

Object 是否存在？

Lifetime 是否有效？

Access 是否合法？

↓

G2

谁拥有？

谁借用？

谁负责 cleanup？

↓

G3

Value 怎样 copy / move / return？

这些操作真实成本是什么？

↓

G5

哪些差异应该由 compiler 在 compile time specialization？

↓

G6

这些 object / value / abstraction

最终如何映射到 CPU、Cache 和 Memory？

因此 G6 的核心链路是：

C++ Type

↓

Object Representation

↓

Storage Topology

↓

流程示意 · continued

Cache Lines

↓

Virtual Pages / TLB

↓

Allocation Topology

↓

Control Flow

↓

Multi-core Coherence

↓

Measured Performance

1.2 G6 的统一性能模型

性能分析先确定指标，再分解成本来源。计算、数据搬运、访存等待、分配、地址翻译、控制流和跨核协调是审查维度，不是可以直接相加的同量纲耗时公式；它们会重叠执行并互相制约。两个 $O(N)$ 算法可能分别连续扫描和依赖式追指针。表示、访问模式、工作集 (**working set**)、输入分布和线程拓扑共同决定机器行为。复杂度给出规模趋势，不能替代具体工作负载的观测；G6-M3 会在保持计算结果一致的前提下记录布局差异，而不规定哪一方必须更快。

2. 对象布局

2.1 `sizeof(T)` 的准确含义

`sizeof(T)`

表示一个 T object representation 所占的 byte 数，并且必须允许：

T values[N]；

中的相邻 objects 正确排列。所以它并不等于：所有成员 `sizeof` 简单相加。例如：

[机制片段 · 不承诺独立编译]

```
struct Sample {
    std::uint8_t flags;
    std::uint32_t id;
    float value;
};
```

成员名义大小：

1 + 4 + 4 = 9 bytes

但常见 ABI 下可能得到：

offset 0	flags	1 byte
offset 1	padding	3 bytes
offset 4	id	4 bytes
offset 8	value	4 bytes

```
sizeof(Sample) = 12
```

具体数字属于 ABI / implementation，不是 C++ 标准固定值。

2.2 Alignment

`alignof(T)`

表示：T object 起始地址必须满足的 alignment requirement。例如某平台：`alignof(uint32_t) = 4` 意味着一个正常 `uint32_t` object 的地址通常必须满足：`address % 4 == 0` Alignment 存在的原因可能包括：

CPU load/store requirements、ABI、atomic operations、SIMD/vector instructions、access efficiency。

重要的是：能够进行 **unaligned access**，不等于 **alignment** 无意义。

2.3 Internal Padding 与 Tail Padding

Internal Padding

成员之间为了使下一个 member 正确对齐而加入的 bytes。例如：

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint8_t a;
    std::uint32_t b;
};
```

可能：[a][pad][pad][pad][b b b b]

Tail Padding

最后一个 member 后面的 padding。例如：

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint32_t x;
    std::uint8_t y;
};
```

可能：[x x x x][y][pad][pad][pad] 这样：

`S values[2];`

中第二个 S 仍然从满足 `alignof(S)` 的地址开始。因此：**Tail padding** 的一个核心作用是保证 **array stride** 正确。

2.4 Member Ordering

考虑：

[机制片段 · 不承诺独立编译]

```
struct A {
    std::uint8_t a;
    std::uint64_t b;
    std::uint8_t c;
};

struct B {
    std::uint64_t b;
    std::uint8_t a;
    std::uint8_t c;
};
```

在常见 64-bit ABI 下，A 可能显著大于 B。如果每个 object 差 8 bytes：

```
10,000,000 objects
× 8 bytes
=
80 MB
```

因此 member ordering 会影响：

RAM footprint、cache density、page density、memory bandwidth、TLB reach。

但不能机械执行：“所有 struct 都从大字段排到小字段。”因为 declaration order 同时决定：

construction order、destruction reverse order、ownership teardown dependency、semantic grouping、ABI/layout。

所以：**Member ordering** 是架构设计，不是格式化规则。

2.5 Inline Representation 与 Indirect Representation

Inline

```
std::array<float, 1024>
```

payload 就在 object 内部。因此：

$\text{sizeof}(\text{object}) \approx \text{logical payload footprint}$

Indirect

```
std::vector<float>
```

vector object 本身通常只是类似：

pointer、size、capacity。

而真正的大 payload 位于 dynamic allocation。因此：

```
sizeof(std::vector<float>)
```

只能说明：vector wrapper 本身的 representation。不能说明：它拥有的数据总 footprint。同理：

unique_ptr、shared_ptr、string、vector、PImpl。

都必须分析完整 object graph，而不只是 sizeof(wrapper)。

2.6 Object Representation ≠ Logical Value

假设：

[机制片段 · 不承诺独立编译]

```
struct S {
    std::uint8_t a;
    std::uint32_t b;
};
```

即使：

```
x.a == y.a && x.b == y.b
```

也不能普遍推出：

```
std::memcmp(&x, &y, sizeof(S)) == 0
```

适合作为 logical equality。因为 object representation 可能包含：

padding、multiple valid representations、pointer representation、floating-point representation details。

所以：

Logical Equality ≠ Byte-for-byte Object Representation Equality

同样：std::is_trivially_copyable_v<T> 不等于 T 可以直接作为 portable wire format。序列化还必须考虑：

endianness、padding、ABI、versioning、schema、type width。

3. 缓存与局部性

3.1 Memory Hierarchy

简化模型：

```
Registers
  ↓
L1 Cache
  ↓
L2 Cache
  ↓
LLC / L3
  ↓
DRAM
```

通常：

```

closer to CPU
→ smaller
→ faster

farther from CPU
→ larger
→ slower

```

因此很多程序不是：CPU arithmetic 不够快。而是：CPU 等不到数据。

3.2 Cache Line

缓存行 (cache line) 是硬件进行缓存填充及一致性管理的重要粒度；其具体大小和层级由处理器决定，C++ 不规定固定为 64 字节。相邻访问可能复用同一行，写入不同对象也可能争用同一一致性单元。用 64 字节推演是一个显式假设，不是本机事实。实验记录操作系统可报告的缓存行和页大小；这些查询也不是完整的缓存拓扑测量。不能仅凭 `sizeof(T)` 和一个假定行宽，就断言实际 miss 数或 false sharing 已经发生。

3.3 Spatial Locality

短时间内访问地址彼此接近的数据。例如：

[机制片段 · 不承诺独立编译]

```

for (const auto& value : values) {
    process(value);
}

```

对于：

```
std::vector<T>
```

通常：

```

[T][T][T][T][T]
→ → → → →

```

非常适合 spatial locality。

3.4 Temporal Locality

短时间内反复访问同一份数据。例如：

[机制片段 · 不承诺独立编译]

```

state.position += velocity;
use(state.position);
state.position += correction;

```

`state.position` 很可能仍位于：register / cache 因此后续访问更便宜。

3.5 Cache-line Utilization

假设: `cache line = 64 B` 而一个算法每条 `line` 只真正使用: `4 B` 则:

```
useful ratio ≈ 6.25%
```

也就是说: 代码只写了一个 4-byte load, 不代表 `memory subsystem` 只付出了 4 bytes 的成本。这成为 SoA / hot-cold split 的主要动机之一。

3.6 Pointer Chasing

Dense layout: `[T][T][T][T][T]` Pointerized layout:

```
[p][p][p][p]
|  |  |  |
▼  ▼  ▼  ▼
T  T  T  T
```

后者可能引入:

extra pointer load、random heap locations、cache misses、dependent loads、poor hardware prefetch、more pages、larger TLB working set。

所以:

```
std::vector<T>
```

与:

```
std::vector<std::unique_ptr<T>>
```

即使遍历都是: $O(N)$ 实际性能仍可能完全不同。

3.7 Dependent Memory Access

链表:

```
Node0 → Node1 → Node2 → Node3
```

下一地址必须等: `Node0.next` 先从 `memory` 返回。这意味着:

```
load Node0
↓
wait
↓
discover Node1
↓
load Node1
↓
wait
```

多个 `memory misses` 很难充分并行。所以 `pointer chasing` 的一个深层问题是: **Memory-Level Parallelism 较差**。

3.8 Hardware Prefetch

规则访问：

```
X
X+64
X+128
X+192
```

硬件容易预测：很可能还会继续向后读。于是提前请求下一些 cache lines。完全随机访问：

0xA...、0xF...、0x21...。

则难得多。因此：

contiguous、regular-strided。

访问模式通常更硬件友好。

3.9 Working Set

某个时间窗口内真正活跃的数据集合。如果 working set 能较好驻留 cache：

```
cache hits ↑
latency ↓
```

如果远大于 cache：

```
replacement ↑
memory traffic ↑
```

因此：Object footprint 不仅影响 RAM，也决定能有多少 hot objects 同时留在 cache。

3.10 Cache Density

假设：cache line = 64 B 如果：sizeof(T) = 8 一条 line 大约可以容纳：8 objects 如果：sizeof(T) = 32 只能容纳：2 objects 因此 object size 会影响：每次 cache fill 能顺便带进多少 useful objects。

3.11 Latency 与 Bandwidth

Latency

等某个特定数据到达需要多久。Random pointer chasing 容易 latency-bound。

Bandwidth

单位时间能够搬多少 bytes。大型连续 scan 更容易 bandwidth-bound。例如：

```
[机制片段 · 不承诺独立编译]

for (float x : huge_values) {
    sum += x;
}
```

虽然会产生很多 cache misses，但：

sequential、predictable、prefetchable、parallel memory requests。

所以仍可能非常高效。这说明：**Cache miss 本身不是性能 verdict**。

3.12 Arithmetic Intensity

粗略定义：

Useful Computation
Bytes Moved

如果：1 add / many bytes 更可能 memory-bound。如果：many FLOPs / few bytes 更可能 compute-bound。因此优化前要问：

- memory-bound?
- compute-bound?
- latency-bound?
- branch-bound?

4. 数据布局：AoS、SoA 与 AoSoA

4.1 AoS — Array of Structures

[机制片段 · 不承诺独立编译]

```
struct SignalSample {
    std::uint64_t timestamp;
    std::uint32_t id;
    float value;
    std::uint8_t valid;
};

std::vector<SignalSample> samples;
```

内存：

[ts id value valid]、[ts id value valid]、[ts id value valid]。

优势：

complete entity contiguous、simple ownership、simple lifetime、simple invariants、natural API。

适合：每次处理一个完整 entity。

4.2 SoA — Structure of Arrays

- timestamps: [ts][ts][ts]
- ids: [id][id][id]
- values: [v][v][v]

- valid: [x][x][x]

适合: 批量扫描某几个字段。例如只计算 values: [value][value][value][value] 每条 cache line 几乎都装当前 kernel 真正需要的数据。

4.3 SoA 的第一收益

不是首先: SIMD。而是: **减少 Overfetch 和无用 data movement**。之后才进一步改善: vector loads、SIMD、masked processing、column compression。

4.4 SoA 的成本

SoA 将一个 logical row 拆成多个 arrays。于是必须保持:

timestamps.size()、==、ids.size()、==、values.size()、==、valid.size()。

产生新的 structural invariant。同时可能增加:

multiple allocations、coordinated reserve/growth、exception-safety complexity、API complexity、borrow invalidation complexity。

所以 SoA 是: 用 representation complexity 换 performance opportunity。

4.5 AoSoA

- Block 0:
 - x[N]
 - y[N]
 - z[N]
- Block 1:
 - x[N]
 - y[N]
 - z[N]

它折中:

```
SoA SIMD friendliness
+
bounded block locality
+
some entity neighborhood
```

常用于:

particles、physics、robotics kernels、HPC。

但实现复杂度高。默认路线应该是:

```
AoS
↓
measured field-wise bottleneck
↓
SoA
↓
clear block/SIMD need
↓
AoSoA
```

4.6 Hot / Cold Split

例如:

[机制片段 · 不承诺独立编译]

```
struct Robot {
    Position position;    // hot
    Velocity velocity;    // hot

    std::string name;     // cold
    DebugInfo debug;      // cold
};
```

physics loop 只需要:

position、velocity。

则拆为:

RobotState、RobotMetadata。

可以同时降低:

cache working set、memory bandwidth、page working set、TLB pressure。

这往往比: 省几 bytes padding 有价值得多。

4.7 Layout 的统一原则

```
Frequently co-accessed data
→ colocate

Independently scanned data
→ separate

Independent high-frequency writers
→ separate cache lines / regions
```

这三条贯穿单线程和多线程 data layout。

4.8 Phase-specific Representation

系统不必只有一种 domain representation。可以：

```
Ingress
→ AoS

Compute
→ SoA

Storage
→ Columnar

External API
→ Row DTO
```

一次 representation transform: $O(N)$ 如果后续：大量重复处理 完全可能值得。因此：**Representation** 可以服务某个 **processing phase**，而不是服务抽象上的“对象纯洁性”。

5. 分配、区域与内存资源

5.1 Allocation 与 Construction

沿用 G1 §1 的对象/存储区分：普通 `new T(args...)` 涉及存储分配 (allocation) 与对象构造 (construction)，匹配的 `delete p`；涉及析构 (destruction) 与存储释放 (deallocation)。本章不重复生命周期规则，只把这四类工作分开计量；资源归还的责任仍由 G2 的所有权合同约束。

5.2 General Heap Allocation 的成本

可能包含：

allocator metadata、size classes、free-list operations、synchronization、cache misses、fragmentation handling、occasionally acquiring more pages。

但：普通 `new` 一般不意味着每次都直接进行 `syscall`。Allocator 通常已经从 OS 获取较大的 memory regions。

5.3 Stack 为什么便宜？

通常的栈帧存储管理利用后进先出的调用结构，可以用栈指针调整批量取得和归还空间，省去一般堆分配器的任意释放搜索与同步。但 **automatic storage duration** 是语言概念，局部对象也可能被放入寄存器或完全消除。“栈便宜”只讨论常见存储管理路径，不包含构造函数、巨大栈帧、触页或栈溢出成本。不能把复杂对象改成局部变量，就据此认定其内部动态分配或清理工作消失。

5.4 Per-item Allocation

例如：

```
std::vector<std::unique_ptr<Detection>>
```

大量 small values 时可能产生：

N allocations、N allocator metadata、N indirections、fragmentation、poor cache locality、more pages、larger TLB working set。

所以如果：

small、value-like、no polymorphism、no address-stability requirement。

通常优先：

```
std::vector<Detection>
```

5.5 Internal / External Fragmentation

Internal Fragmentation

请求：24 B allocator 给：32 B slot 内部没用掉的 8 B。

External Fragmentation

总 free space 足够，但：散成多个小 region 无法满足一块较大的 contiguous request。Fragmentation 不只浪费 RAM，还可能恶化：

page usage、locality、TLB pressure。

5.6 Allocation Optimization Ladder

优先级：

1. Avoid allocation
2. Inline storage
3. reserve + reuse
4. batch allocations
5. Arena
6. Pool
7. Custom allocator/resource
8. Measure again

这是非常重要的顺序。不要：

看到 new
→ 写 memory pool

5.7 reserve + clear

当容量足够时，reserve 加上反复填充与 clear 可以复用 vector 的元素存储，优先于引入自定义分配器。clear 销毁已有元素但保留容量，后续在容量内构造元素不需要重分配这块存储。这个结

论不意味着整个过程没有分配或成本：元素自身可能另行分配，析构和重新构造也必须执行。G6-M2 记录指定 PMR 资源收到的请求，并单独核对活对象数；它不代表全进程 allocator profile。

5.8 Arena / Region Allocation

核心：

```
large region

[A][B][C][free.....]
      ↑
    cursor
```

每次 allocation：

align cursor、return cursor、advance cursor。

最终：reset / release whole region 适合：

request lifetime、batch lifetime、parser pass、frame processing、compiler AST。

即：一批 objects 拥有近似共同 lifetime。

5.9 Arena 的真正来源

Arena 快不是因为“特殊内存”。而是因为它限制：individual arbitrary deallocation。因此不需要一般性的：

free-list search、coalescing、per-object reclaim。

再次体现：**Lifetime constraints** 可以换来更简单、更可预测的 **memory management**。

5.10 Arena 与 Object Lifetime

Arena reset 只是在处理：storage。它不会自动正确销毁 placement-new 出来的：

string、vector、file wrapper、mutex。

等 non-trivial objects。因此：

Arena Storage Lifetime $\neq$ C++ Object Lifetime

5.11 Pool

Pool 适合：

many same/similar-size objects + individual arbitrary lifetimes

结构：

```
[used][free][used][free]
      ↑       ↑
    free list
```

allocate: pop free slot free: push slot back

5.12 Arena vs Pool

Arena

- lifetime grouped
- bulk release

Pool

- slots reused independently
- arbitrary per-object lifetime

这是根本区别。

5.13 Stale Generation

Pool:

```
slot 42
generation 7
→ Task A
```

释放再复用:

```
slot 42
generation 8
→ Task B
```

旧 pointer: same address 但已经:

different object、different lifetime、different generation。

所以: **Stable address** 不等于 **stable logical identity**。Generational handle:

[机制片段 · 不承诺独立编译]

```
struct Handle {
    std::uint32_t index;
    std::uint32_t generation;
};
```

可以检测 stale access。

5.14 std::pmr

核心 abstraction:

std::pmr::memory_resource

它把: container/value behavior 和: runtime allocation strategy 进行一定程度解耦。例如:

std::pmr::vector<T>

可以配:

monotonic_buffer_resource、unsynchronized_pool_resource、synchronized_pool_resource、custom resource。

5.15 monotonic_buffer_resource

`std::pmr::monotonic_buffer_resource` 为一组生命周期相近的分配提供单调增长的存储。单独的 `deallocate` 不归还对应块；`release` 或资源销毁统一释放其管理的存储，但不会代替放在其中的对象执行析构。因此顺序应是结束所有相关对象及借用，再释放资源。即使容器已 `clear`，它通常仍持有 `capacity`；这时释放资源后继续使用该容器仍是错误设计。内存资源必须活得比依赖它的容器久。PMR 抽象的是存储分配策略，不是 GC，也不自动提供线程安全。

5.16 Allocation Profile 要看什么？

至少：

allocation count、allocated bytes、size distribution、lifetime distribution、peak live bytes、thread of allocation、thread of free。

尤其：millions of tiny allocations 往往比少数大 allocation 更可疑。

6. 虚拟内存与地址翻译

6.1 Pointer 通常是 Virtual Address

用户态：

`T* p;`

通常首先代表：virtual address。不是直接的 DRAM physical address。每个 process 有自己的：Virtual Address Space

6.2 Pages

Virtual memory 以 page 为主要映射粒度。例如平台可能使用：

4 KiB、16 KiB、large pages、...。

这是 OS / hardware property。Virtual pages：

VPage 100、VPage 101、VPage 102。

可以分别映射到：不同 physical pages 所以：virtual contiguous 不代表 physical contiguous。

6.3 MMU / Page Table

简化：

```

Virtual Address
    ↓
MMU
    ↓
Page Table
    ↓
Physical Address
  
```

一个 virtual address 可以概念拆：

Virtual Page Number + Page Offset

映射改变 page number, offset 保持。

6.4 TLB

Translation Lookaside Buffer

缓存：

```
Virtual Page
→
Physical Page
```

的 translation。因此 memory access 同时涉及两套 locality：

Address Translation Locality + Data Cache Locality

6.5 TLB Miss 与 Page Fault

必须严格区分。

TLB Miss

```
translation not in TLB
↓
page-table walk
↓
mapping found
↓
continue
```

page 可能完全正常 resident。

Page Fault

CPU 当前无法完成访问：trap to kernel 可能是：

first anonymous-page touch、copy-on-write、mapped file page、protection fault、illegal address。

因此：Page fault 不等于程序一定出错。

6.6 Allocation ≠ Physical Memory Immediately

可能发生：

```
allocator returns virtual range
↓
not every page physically touched yet
↓
first access
↓
page fault / backing / zeroing
```

所以：malloc/new 10 GB 与：touch 10 GB of actual pages 不是同一件事。

6.7 First Touch

第一次真正访问 large memory:

page faults、page zeroing、mapping work。

可能产生 latency spike。因此 latency-sensitive 系统有时会:

preallocate、pre-touch、reuse。

把成本从: hot phase 搬到: initialization 这叫: **Latency Placement** 而不是凭空消除工作。

6.8 Page Locality / TLB Reach

Dense storage: many useful objects / page 意味着一个 TLB entry 可以服务更多 object accesses。Sparse pointer graph: few useful bytes spread over many pages 则可能:

```
TLB working set ↑
page walks ↑
```

因此 object density 同时影响:

cache density、page density、TLB reach。

6.9 Huge Pages

更大 page:

```
translations needed ↓
TLB reach ↑
```

但可能增加:

internal waste、mapping granularity、fault cost、memory management complexity。

所以只有: translation overhead 已被 profile 证明有意义时才值得考虑。

6.10 mmap

核心不是: “读取文件的另一种 API”。而是: 把 virtual address range 映射到某种 backing object。访问:

mapped[i]

看起来只是普通 pointer access, 但可能隐藏:

TLB miss、page fault、filesystem、storage I/O。

所以: **Pointer syntax** 不揭示 **memory latency class**。

7. 分支预测与控制流

7.1 Branch Prediction

CPU 遇到：

[机制片段 · 不承诺独立编译]

```
if (condition) {  
    foo();  
} else {  
    bar();  
}
```

不能总停下来等 condition 完成。因此：

```
predict branch  
↓  
fetch predicted path  
↓  
execute speculatively  
↓  
verify later
```

7.2 Predictable Branch

例如：99.9% true 通常很容易预测。因此：if 并不天然昂贵。真正可能贵的是：**Branch Misprediction**

7.3 Misprediction

预测错误：

```
wrong-path instructions  
↓  
discard/squash  
↓  
restart correct path
```

成本显著高于单纯 compare。所以 branch 成本取决于：
data distribution、branch history、microarchitecture。

7.4 Branch Cost 是 Runtime-state-dependent

同样：

[机制片段 · 不承诺独立编译]

```
if (valid) {  
    process();  
}
```

数据：99.9% true 和：50/50 random 性能可能完全不同。因此：**Input distribution** 是性能 **specification** 的一部分。

7.5 Source Branchless $\neq$ Machine Branchless

```
count += value >= threshold;
```

源码没有 if，compiler 仍决定具体 lowering。反过来：

[机制片段 · 不承诺独立编译]

```
if (value >= threshold) {  
    ++count;  
}
```

也可能被编译成 branchless code。所以：查看 optimized machine code，而不是从 C++ syntax 猜。

7.6 Branchless 的 Trade-off

Branchless:

control uncertainty ↓

但可能：

extra work ↑
extra loads ↑
dependency chain ↑

如果 branch 能跳过：

expensive computation、invalid operation、memory load、rare path。

且容易预测，branch 往往更好。

7.7 SIMD 与 Mask

当一条 vector instruction 同时处理多个 lanes：

lane 0 true、lane 1 false、lane 2 true、...。

单一 scalar control-flow branch 很难表达。因此 SIMD 常用：

mask、select、blend、masked operations。

这也是 SoA 更容易 vectorize 的一个原因。

7.8 Fast Path / Slow Path

高概率场景：

short、predictable、hot。

低概率异常：

large diagnostics、error handling、logging。

可以拆：

fast path + cold slow path

这既可能帮助：branch predictability 也可能改善：instruction-cache working set

7.9 `[[likely]]` / `[[unlikely]]`

它们主要是：compiler optimization hints。不是：runtime branch predictor command 也不是 correctness guarantee。应该来自：

real domain distribution、profile、strong invariant。

而不是凭感觉乱加。

7.10 Direct / Indirect Dispatch

Direct call：

```
decode_msg_100();
```

target compile-time known。Function pointer：

```
fn();
```

target runtime known。Virtual dispatch：

```
obj.virtual_method();
```

通常也是 indirect control flow。Indirect dispatch 可能影响：

target prediction、inlining、interprocedural optimization。

但 target 稳定、每次 useful work 足够多时：成本完全可能很小。

7.11 Static Dispatch

Template specialization：

```
process<Camera>();
```

target 更容易在 compile time 决定。优势：

direct call、inline、constant propagation。

代价：

more specializations、larger code、I-cache pressure、compile time。

所以 static dispatch 不是免费胜利。

8. 缓存一致性与伪共享

8.1 Coherence

多核：

Core 0 Cache、 $\updownarrow$ 、coherence、 $\updownarrow$ 、Core 1 Cache。

当同一 memory region 被多个 cores 访问，hardware 必须保持 cache copies 的一致性。重要粒度通常是：cache line / coherence block，而不是 C++ object。

8.2 False Sharing

[机制片段 · 不承诺独立编译]

```
struct Counters {
    std::atomic<std::uint64_t> a;
    std::atomic<std::uint64_t> b;
};
```

Thread 0: writes a Thread 1: writes b 如果：

a + b

位于同一 cache line，硬件仍会让 line ownership 在 cores 之间反复转移。这叫：**False Sharing** 即：logically separate、but、physically shared coherence block。

8.3 True Sharing vs False Sharing

True Sharing

多个 threads 真正更新同一个：

```
std::atomic<uint64_t> counter;
```

共享不可避免。

False Sharing

多个 threads 更新不同 values，只因为物理 colocated 而产生 coherence contention。因此：

```
Race-free
≠
Contention-free
≠
Fast
```

8.4 memory_order_relaxed 不能解决 False Sharing

relaxed 控制的是：C++ memory-ordering contract。但 atomic write 仍然必须获得：对对应 coherence line 的 writable ownership。因此：

weak ordering + terrible false sharing

完全可能同时存在。

8.5 Shared Immutable Data

多个 cores 只读: immutable Model cache lines 可以保留共享副本。因此: Shared Immutable Data 同时具有:

simpler synchronization + friendly cache-coherence behavior

这也是 immutable snapshot architecture 强大的原因。

8.6 优先减少 Shared Mutation

不要遇到 false sharing 就先 padding。更强的优化顺序:

1. Remove sharing、2. Single writer、3. Per-thread / per-worker state、4. Batch aggregation、5. Cache-line isolation。

例如:

```
shared atomic ++ per event
```

通常远差于:

```
thread-local ++
↓
one aggregate update per batch
```

8.7 Parallel Partition

较好:

```
Thread 0 → contiguous block A
Thread 1 → contiguous block B
```

较差:

```
Thread 0 → even indexes
Thread 1 → odd indexes
```

因为 interleaved writes 很容易让每条 cache line 被多个 cores 同时修改。因此: Parallel partition 应尽可能和 physical ownership region 对齐。

8.8 alignas 与 Interference Size

对齐与填充可以帮助分离频繁写入的状态, 但增大 footprint 也可能破坏局部性。std::hardware_destructive_interference_size 是实现提供的干扰间距建议; 可用性、值和目标配置必须实测, 不能当作所有处理器的真实缓存行枚举。若使用显式 alignas 值, 要说明目标假设, 并检查数组步长和实际地址关系。单凭两个字段的地址不同不能排除 false sharing; 反过来, 记录了对齐也不证明该工作负载一定更快。减少共享写、线程本地聚合和批量合并往往比无限加 padding 更重要。

9. 测量、剖析与证据

9.1 性能首先要定义 Metric

可能是：

Latency、Throughput、p99、CPU utilization、Memory footprint、Allocation rate、Startup time、Energy。

“更快”不是完整 requirement。

9.2 Latency vs Throughput

Latency：一次 operation 多久完成。Throughput：单位时间完成多少工作。增加 batch size 可能：

```
throughput ↑  
latency ↑  
memory ↑
```

所以必须知道优化目标。

9.3 Tail Latency

平均值可能隐藏极端慢请求。因此系统常看：

p50、p90、p95、p99、p99.9、max。

例如：p99 = 10 ms 表示：大约 99% observations 不超过 10ms。不是：最慢 1% 平均值是 10ms。

9.4 Benchmark / Profiler / Tracing

Benchmark

回答：某个操作有多快？

Profiler

回答：CPU 时间花在哪里？

Tracing

回答：一条 request/job 的事件时间线是什么？例如：

```
queued  
↓  
download  
↓  
decompress  
↓  
decode  
↓  
produce
```

它们解决的是不同问题。

9.5 End-to-end First

先看整个系统：

Download 20%、Decompress 40%、Decode 5%、Produce 10%、Waiting 25%。

如果 decode 只占 5%，即使加速很多：end-to-end 也不会提升很多。这就是 Amdahl's Law 的工程直觉。

9.6 Microbenchmark

适合回答：

AoS vs SoA kernel、runtime dispatch vs static dispatch、allocator A vs allocator B。

但必须控制：

setup、allocation、RNG、logging、dead-code elimination、constant folding、cache state、input distribution。

最大的风险是：非常精确地测错问题。

9.7 Optimized Build

生产性能实验必须接近 production：-O2 / -O3 不能以：-O0 作为主要性能结论。因为：

templates、span、iterators、small wrappers。

很多 abstraction 成本只有 optimizer 后才真正体现。

9.8 Dead-code Elimination

如果 benchmark result 从未被使用，compiler 可能删掉整个 computation。所以 benchmark 必须：让结果具有 observable relevance。不要机械用 volatile 解决所有问题。专业 benchmark framework 通常提供专门 optimization barriers。

9.9 Warm-up

第一次运行可能包括：

cold caches、page faults、branch-predictor cold state、dynamic-linking initialization、filesystem cache effects。

所以必须明确：测 cold-start，还是 steady-state？两个都是合法目标，但不能混。

9.10 Dataset Size

Benchmark 应覆盖：

small、medium、large。

使 working set：

fits small cache、fits larger cache、exceeds cache、streams from memory。

否则小 dataset 的结果可能完全不能代表 production。

9.11 Input Distribution

例如：

```
branch:
99% true
vs
50/50 random

hash:
uniform
vs
collision-heavy

string:
SSO short
vs
heap-backed long
```

都会改变结果。所以：Input distribution 是 benchmark specification 的一部分。

9.12 Hardware Counters

可以帮助观察：

cycles、instructions、branch misses、cache misses、TLB misses、loads/stores。

它们解释：为什么慢。但最终是否优化成功仍然由：

end-to-end latency、throughput、tail、memory。

决定。

9.13 Profiler Before Assembly

正确顺序：

```
profile
↓
find hotspot
↓
inspect hotspot assembly
```

Assembly 是：microscope。Profiler 是：map。不要研究一个只占 0.1% CPU 的函数三小时汇编。

9.14 Bottleneck Moves

优化前：

allocator 40%、decode 30%。

优化 allocator 后：

allocator 3%、decode 65%。

说明：新瓶颈浮现。因此必须：

```
profile
→ optimize
→ profile again
```

而不是一次 profile 后一路凭经验优化。

9.15 One Variable at a Time

如果同时：

```
AoS → SoA
heap → arena
branch → branchless
runtime → template
```

最终快 30%，你不知道是谁贡献的。所以尽可能：一次改变一个主要因素。保留：

baseline、change、result、interpretation。

9.16 Performance Evidence Ladder

```
Level 0
Intuition

    ↓

Level 1
Timing

    ↓

Level 2
Profiling / Tracing

    ↓

Level 3
Assembly / Hardware Counters

    ↓

Level 4
Production-like / Real System Metrics
```

越往下层越解释 mechanism，越往高层越证明真实系统价值。

10. 统一性能审查

10.1 G6 九层 Review Protocol

任何性能敏感代码都可以按下面顺序检查。

Layer 1 — Object Representation

问：

- sizeof(T)?
- alignof(T)?
- padding?
- inline or indirect?

Layer 2 — Access Pattern

问：

- sequential?
- strided?
- random?
- pointer chasing?

Layer 3 — Working Set

问：

- 多少 bytes?
- 多少 cache lines?
- 多少 pages?
- 是否反复使用?

Layer 4 — Data Layout

问：

- 哪些 fields 是 hot?
- 哪些 fields 总一起访问?
- AoS / SoA?
- Hot/cold split?

Layer 5 — Allocation

问：

- 多少次 allocation?
- size distribution?

- lifetime distribution?
- 是否能 reuse?

Layer 6 — Virtual Memory

问：

- first touch?
- page faults?
- TLB working set?
- dense or sparse pages?

Layer 7 — Control Flow

问：

- branch distribution?
- predictable?
- indirect dispatch?
- branch 能跳过多少工作?

Layer 8 — Multi-core

问：

- 谁写什么?
- shared mutation?
- same cache line?
- single writer possible?
- batching possible?

Layer 9 — Evidence

问：

- 优化目标是什么?
- baseline 是什么?
- profile 证据是什么?
- 改完实际改善多少?

10.2 G6 Performance Smell Catalogue

Smell 1 — Pointerized Small Values

```
std::vector<std::unique_ptr<SmallValue>>
```

却没有：

polymorphism、stable-address、independent lifetime。

等真实需求。

Smell 2 — Per-item Allocation

[机制片段 · 不承诺独立编译]

```
for (...) {  
    std::make_unique<T>();  
}
```

位于百万级 hot path。

Smell 3 — Cold Data in Hot Struct

```
hot numeric state  
+  
strings  
+  
debug metadata  
+  
rare fields
```

全部混排。

Smell 4 — Random Access on Huge Dataset

却只因为 container 是 vector 就认为 cache-friendly。

Smell 5 — Premature SoA

没有 profile 就把简单 data model 拆成复杂 columns。

Smell 6 — Branchless Cargo Cult

把 highly predictable branch 改成每次都执行昂贵工作。

Smell 7 — Shared Atomic per Event

高频业务事件全部更新同一个 global atomic。

Smell 8 — Per-thread Counters Adjacent

逻辑独立，却共享 cache line。

Smell 9 — Custom Pool Before Reuse

连：

reserve、clear、reuse。

都没试，就自己维护 allocator。

Smell 10 — Huge Template Specialization

只为了消除一个 branch，生成大量近似函数，增加 I-cache/code-size 压力。

Smell 11 — Benchmarking -00

用 debug lowering 得出 production 性能结论。

Smell 12 — Optimizing Without Baseline

只有：“我感觉现在快了。”没有可重复 benchmark / profile。

11. 跨语言回查

11.1 C++

C++ 的性能优势来自：

explicit representation control、value semantics、contiguous standard containers、compile-time specialization、low-level memory access、RAII。

但相应要求工程师真正理解：

lifetime、aliasing、layout、allocator behavior、cache、memory model。

语言不会自动替你选择最优 representation。

11.2 Zig

Zig 更显式暴露：

allocator、array/slice distinction、layout、comptime。

使：

- where does allocation happen?
- who owns allocator?

通常更容易从 API 看出。但：

cache、TLB、AoS/SoA、false sharing、memory bandwidth。

这些机器规律完全一样。

11.3 Rust

Rust 更强地静态约束：

ownership、borrows、thread safety traits。

但同样不能自动解决：

bad locality、too many allocations、Arc contention、false sharing、poor AoS layout。

因此：**Memory safety** 与 **memory performance** 是不同维度。

11.4 跨语言共同事实

无论：

C++
Zig
Rust

都无法绕开：

cache-line granularity、page granularity、TLB、memory bandwidth、branch predictor、coherence、working set。

这些属于：machine architecture 而不是某种语言哲学。

12. 术语回查

12.1 G6 核心术语表

Term	核心含义
Object Representation	一个 object 在内存中的 byte-level representation
Alignment	object address 必须满足的边界要求
Padding	为 alignment / layout 插入的非成员 value bytes
Tail Padding	object 末尾用于保证相邻 array element 正确对齐的 padding
Cache Line	cache hierarchy 中的重要数据传输/coherence 粒度
Spatial Locality	短时间访问相邻地址
Temporal Locality	短时间重复访问同一数据
Working Set	一段时间内活跃访问的数据集合
Cache Density	单个 cache line 能容纳多少 useful data/object
Pointer Chasing	通过一个 pointer load 的结果决定下一次 memory address
Memory-Level Parallelism	同时存在多个独立 memory requests 的能力
Bandwidth-bound	性能主要由 bytes/sec 限制
Latency-bound	性能主要由单次 memory wait 限制
Arithmetic Intensity	Useful computation / bytes moved
AoS	Array of Structures
SoA	Structure of Arrays
AoSoA	Array of Structures of Arrays
Hot/Cold Split	将高频和低频访问数据分离
Arena	group-lifetime / region allocator
Pool	固定或少数 slot size 的 reusable allocator
Fragmentation	allocation 布局导致的内部/外部空间浪费
PMR	Polymorphic Memory Resource
Virtual Address	process 看到的虚拟地址
Page	virtual-memory mapping 粒度
TLB	virtual→physical translation cache
TLB Miss	address translation cache miss
Page Fault	当前 memory access 需要 kernel/VM 处理

Term	核心含义
First Touch	首次真正访问某 virtual page
Branch Prediction	CPU 预测 branch control flow
Misprediction	branch outcome 预测错误
Speculative Execution	按预测路径提前执行
Indirect Call	target address 在 runtime 决定的 call
Cache Coherence	多核 caches 对 shared memory 保持一致的机制
False Sharing	不同 logical data 因同 cache line 上的写而互相干扰
True Sharing	多线程真正读写同一个 logical state
Benchmark	测量“多快”
Profiling	定位“时间花在哪”
Tracing	描述“事件什么时候发生”
Hardware Counter	CPU 暴露的微架构事件计数
Tail Latency	延迟分布高 percentile 部分
Amdahl's Law	局部优化受未优化部分比例限制的总体收益规律

13. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 h-lab/h-file 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

13.1 G6-M1 · 对象布局不是语言常量

命题、观察与边界。记录 sizeof、alignof、offsetof，并用静态断言检查数组大小与元素大小的关系；只对语言要求检查，绝不硬编码 Sample 必须为 12 字节。成员顺序的成本意义要结合实际目标及大量对象推理。

[完整实验 · G6-M1 · main.cpp]

```
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <type_traits>

struct Sample { std::uint8_t flags; std::uint32_t id; float value; };
```

完整实验 · G6-M1 · main.cpp · continued

```

struct Reordered { std::uint32_t id; float value; std::uint8_t flags; };
static_assert(std::is_standard_layout_v<Sample>);
static_assert(sizeof(Sample) % alignof(Sample) == 0);
int main() {
    Sample values[2]{};
    static_assert(sizeof(values) == 2 * sizeof(Sample));
    std::cout << "{\"size\":\" << sizeof(Sample)
                << "\",\"align\":\" << alignof(Sample)
                << "\",\"id_offset\":\" << offsetof(Sample, id)
                << "\",\"value_offset\":\" << offsetof(Sample, value)
                << "\",\"reordered_size\":\" << sizeof(Reordered) << "\"" << std::endl;
}

```

运行与判据。上述统一命令中的 G6-M1 提取并处理本模块。程序的语义不变量须成立，JSON 数值作为 OBSERVED 保存；不要求固定布局或分配字节数。

13.2 G6-M2 · 资源请求与对象生命周期分别计数

命题、观察与边界。指定 PMR 资源只统计本容器的请求；reserve 后在容量内重复构造与 clear，不增加该资源请求，但仍执行元素析构。不用这个局部计数冒充全进程分配 profile。

[完整实验 · G6-M2 · main.cpp]

```

#include <cstddef>
#include <iostream>
#include <memory_resource>
#include <vector>

struct Counting : std::pmr::memory_resource {
    std::size_t allocations = 0, deallocations = 0, bytes = 0;
    void* do_allocate(std::size_t n, std::size_t a) override {
        void* p = std::pmr::new_delete_resource()->allocate(n, a);
        ++allocations;
        bytes += n;
        return p;
    }
    void do_deallocate(void* p, std::size_t n, std::size_t a) override {
        ++deallocations;
        std::pmr::new_delete_resource()->deallocate(p, n, a);
    }
    bool do_is_equal(const std::pmr::memory_resource& r) const noexcept override {
        return this == &r;
    }
};

struct Item {
    static inline int live = 0;
    Item() { ++live; }
    Item(const Item&) { ++live; }
}

```

完整实验 · G6-M2 · main.cpp · continued

```

    Item(Item&&) noexcept { ++live; }
    ~Item() { --live; }
};
int main() {
    Counting resource;
    {
        std::pmr::vector<Item> items{&resource};
        items.reserve(64);
        const auto allocated = resource.allocations;
        const auto capacity = items.capacity();
        if (allocated == 0 || Item::live != 0) return 1;
        for (int round = 0; round < 20; ++round) {
            for (int n = 0; n < 64; ++n) items.emplace_back();
            if (Item::live != 64) return 2;
            items.clear();
            if (Item::live != 0 || items.capacity() != capacity ||
                resource.allocations != allocated) return 3;
        }
    }
    if (Item::live != 0 || resource.allocations != resource.deallocations) return 4;
    std::cout << "{ \"allocations\": " << resource.allocations
                << ", \"deallocations\": " << resource.deallocations
                << ", \"requested_bytes\": " << resource.bytes << "}" << std::endl;
}

```

运行与判据。上述统一命令中的 G6-M2 提取并处理本模块。程序的语义不变量须成立，JSON 数值作为 OBSERVED 保存；不要求固定布局或分配字节数。

13.3 G6-M3 · 同一求和工作负载的 AoS / SoA 观察

命题、观察与边界。两种布局都读取 key，AoS 另带冷字段。三个规模、一次预热、七轮交替顺序，分配与初始化不计入计时；每次求和必须符合完整期望值。分离编译且不启用 LTO，避免调用者把重复工作折叠；生成汇编供回查。输出 `elapsed_ns` 只是当前进程观察，不能归因于某个缓存层或当作固定性能排名。

[完整实验 · G6-M3 · kernel.hpp]

```

#pragma once
#include <cstdint>
#include <cstddef>
struct Record { std::uint64_t key; std::uint64_t cold[7]; };
std::uint64_t sum_aos(const Record*, std::size_t);
std::uint64_t sum_soa(const std::uint64_t*, std::size_t);

```

[完整实验 · G6-M3 · kernel.cpp]

```
#include "kernel.hpp"

std::uint64_t sum_aos(const Record* p, std::size_t n) {
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < n; ++i) sum += p[i].key;
    return sum;
}

std::uint64_t sum_soa(const std::uint64_t* p, std::size_t n) {
    std::uint64_t sum = 0;
    for (std::size_t i = 0; i < n; ++i) sum += p[i];
    return sum;
}
```

[完整实验 · G6-M3 · main.cpp]

```
#include "kernel.hpp"
#include <chrono>
#include <iostream>
#include <vector>

int main() {
    using Clock = std::chrono::steady_clock;
    for (std::size_t n : {1024U, 65536U, 1048576U}) {
        std::vector<Record> aos(n);
        std::vector<std::uint64_t> soa(n);
        std::uint64_t expected = 0;
        for (std::size_t i = 0; i < n; ++i) {
            aos[i].key = soa[i] = i % 251;
            expected += soa[i];
        }
        if (sum_aos(aos.data(), n) != expected ||
            sum_soa(soa.data(), n) != expected) return 1;
        for (int trial = 0; trial < 7; ++trial) {
            for (int position = 0; position < 2; ++position) {
                const bool use_aos = ((trial + position) % 2 == 0);
                const auto begin = Clock::now();
                const auto sum = use_aos ? sum_aos(aos.data(), n)
                                         : sum_soa(soa.data(), n);
                const auto end = Clock::now();
                if (sum != expected) return 2;
                const auto elapsed = std::chrono::duration_cast<
                    std::chrono::nanoseconds>(end - begin).count();
                std::cout << "{\n\n": " << n << ",\ntrial\n": " << trial
```

完整实验 · G6-M3 · main.cpp · continued

```
        << ",\layout\":" << (use_aos ? "aos" : "soa")
        << "\",\elapsed_ns\":" << elapsed
        << ",\checksum\":" << sum << "}" << std::endl;
    }
}
}
```

运行与判据。上述统一命令中的 G6-M3 提取并处理本模块。以 -O3 分离编译两个 TU，不启用 LTO；记录 42 条测量及对应校验和，汇编输出属于代码生成观察。**测量解释边界。**G6-M3 的计时含单次函数调用和计时器开销，小规模可能接近分辨率；每轮交替顺序并不消除系统噪声。这里记录的是进程墙钟间隔，不是 CPU 周期、实际 DRAM 流量、缓存/TLB miss 或端到端吞吐。CPU 采样 profile 和硬件计数器本批未采集；汇编不是其替代证据。后续真实优化应另以热点和输入分布为依据。

14. Final Gate

14.1 Object Layout

应能解释：

1. 为什么 `sizeof(struct)` 不等于 `members size sum`?
2. Alignment 到底限制什么?
3. Internal padding 与 tail padding 有什么区别?
4. 为什么 member reordering 可以改变大数组 footprint?
5. 为什么 `memcmp` 通常不是普通 struct equality?
6. 为什么 `sizeof(vector<T>)` 不代表真实 payload footprint?

14.2 Cache

应能解释：

1. 什么是 cache line?
2. 什么是 spatial locality?
3. 什么是 temporal locality?
4. 什么叫 cache-line utilization?
5. 为什么 pointer chasing 不只是多一次 pointer dereference?
6. 为什么 sequential streaming 可以 cache miss 很多却仍然快?
7. latency-bound 与 bandwidth-bound 区别是什么?
8. working set 为什么重要?

14.3 Data Layout

应能解释：

1. AoS 和 SoA 物理布局区别是什么?
2. 哪种 workload 更适合 AoS?
3. 哪种 workload 更适合 SoA?
4. 为什么 SoA 往往更容易 SIMD?
5. SoA 会引入哪些新的 invariant?
6. AoSoA 在交换什么?
7. 为什么 hot/cold split 可能比 member padding 优化更重要?
8. 为什么 domain entity 不必等价于实际 C++ struct object?

14.4 Allocation

应能解释：

1. new T 至少包含哪两个阶段？
2. 为什么 new 不等于每次 syscall？
3. 为什么 per-item heap allocation 常常非常昂贵？
4. stack 为什么通常 storage management 很便宜？
5. Arena 的核心 lifetime constraint 是什么？
6. Pool 和 Arena 有什么不同？
7. 为什么 arena reset 不能自动替代 destructor？
8. 为什么 stable pool address 仍然会产生 stale pointer？
9. reserve + clear 为什么应该优先于 custom allocator？
10. PMR 真正抽象的是什么？

14.5 Virtual Memory

应能解释：

1. 用户态 pointer 为什么通常是 virtual address？
2. virtual contiguous 与 physical contiguous 有什么区别？
3. page table 做什么？
4. TLB 做什么？
5. TLB miss 与 page fault 有什么区别？
6. 为什么 allocation 与 physical backing 不是一回事？
7. first touch 为什么会造成 latency spike？
8. dense layout 为什么也改善 TLB locality？
9. huge pages 在优化什么？
10. mmap 为什么可能让普通 pointer access 隐含 I/O latency？

14.6 Control Flow

应能解释：

1. CPU 为什么需要 branch prediction？
2. branch misprediction 真正浪费什么？
3. 为什么 if 不是天然昂贵？
4. 为什么 50/50 random branch 往往难预测？
5. 为什么 source-level branchless 不代表 machine branchless？
6. branchless 为什么可能做更多无用工作？
7. function pointer 为什么是 indirect control flow？
8. 为什么 static dispatch 不一定最终更快？

9. instruction working set 与 code bloat 有什么关系?

14.7 Multi-core

应能解释:

1. cache coherence 解决什么?
2. 什么是 true sharing?
3. 什么是 false sharing?
4. 为什么不同 variables 仍然可能互相拖慢?
5. 为什么 memory_order_relaxed 不解决 false sharing?
6. 为什么 shared immutable data 非常友好?
7. 为什么 per-thread aggregation 往往比 global atomic 强?
8. 为什么 contiguous block partition 比 even/odd interleave 更适合并行写?
9. 为什么 race-free 不代表 fast?

14.8 Measurement

应能解释:

1. latency 和 throughput 有什么区别?
2. p99 表示什么?
3. benchmark / profiler / tracing 分别回答什么?
4. 为什么先做 end-to-end measurement?
5. 什么是 Amdahl's Law 的工程含义?
6. 为什么 microbenchmark 容易测错?
7. 为什么 benchmark 必须使用 optimized build?
8. 为什么 input distribution 是 benchmark specification?
9. 为什么 assembly 应该在 profiling 后看?
10. 为什么优化后必须再次 profile?
11. 为什么 hardware counter 不是最终 verdict?
12. 为什么性能收益必须和 complexity cost 一起评估?

15. Final Gate · 参考答案与常见误判

15.1 Object Layout 与 Cache

Object Layout 1~6: 成员对齐造成内部空隙, 对象尾部填充保证数组中下一元素对齐, 所以成员大小之和不等于 `sizeof`。对齐限制可用地址; 调整成员次序可能减少 `padding`, 重复到大数组时改变 `footprint`。普通 `struct` 的 `padding` 与表示差异使 `memcmp` 不能普遍替代逻辑相等; `vector` 本体通常只保存控制状态, 元素存储另计。具体布局以目标 ABI 与 G6-M1 的输出为证, 不能背固定字节数。

Cache 1~4: 缓存行是传输/一致性的重要粒度; 空间局部性复用邻近地址, 时间局部性复用近期数据, 行利用率描述一行中本次计算真正使用的部分。5~8: 追指针把下一地址依赖于前次加载, 限制并行访存和预取; 连续流式读取可利用并发请求和带宽, 即使 `miss` 很多也未必慢。 `latency-bound` 等待依赖链, `bandwidth-bound` 受数据速率限制; 工作集变化会改变驻留行为。单次耗时不能独立判定是哪种瓶颈。

15.2 Data Layout 与 Allocation

Data Layout 1~3: `AoS` 把同一对象字段放一起, 适合逐对象使用多字段; `SoA` 把同字段排成连续数组, 适合跨对象列扫描。4~6: `SoA` 的规律访问有利于向量化, 但要维持长度、索引和更新一致性; `AoSSoA` 以块为单位折中行列局部性、`SIMD` 宽度与尾部处理。7~8: 热冷拆分避免每次搬运大段不用的数据, 收益可能超过几个 `padding` 字节; 领域实体可映射为多个存储区域, 代价是显式维护关联不变量。G6-M3 只观察一种列扫描, 不为全部工作负载选择 `SoA`。

Allocation 1~4: 普通 `new-expression` 结合取得存储与初始化; 通用分配器可从用户态缓存取块, 并非每次系统调用。逐项分配增加管理、碎片和间接访问; 常见栈帧利用后进先出结构批量管理存储, 但对对象构造并不免费。5~8: `arena` 依赖成组存活/释放, `pool` 依赖槽位复用; 归还存储不替代析构, 地址复用也可能让旧指针指向错误代次。9~10: `reserve/reuse` 用简单的容量合同减少请求; `PMR` 替换分配策略, 不替代对象生命周期与资源依赖顺序。G6-M2 的计数只覆盖指定资源。

15.3 Virtual Memory 与 Control Flow

Virtual Memory 1~5: 普通用户态地址由页表映射, 虚拟连续不要求物理连续; `TLB` 缓存翻译, `miss` 可由页表遍历解决, `page fault` 才涉及当前访问需内核处理的异常, 后者也不一定发生磁盘 I/O。6~10: 分配虚拟区间、取得物理 `backing` 和实际触页不是同一事件; `first touch` 可能引入清零/分配延迟。紧凑布局减少活跃页, `huge pages` 用更大映射范围减少翻译压力, 但有碎片等代价。`mmap` 文件可使首次普通读取触发缺页和 I/O。本批未采集 `TLB/缺页` 硬件事件, 不能从这些机制解释直接推出已经实测。

Control Flow 1~4: 预测帮助流水线提前取指, 预测错误会丢弃投机工作; 稳定分布的 `if` 可预测, 独立随机的 50/50 分支通常更困难, 但实际相关性由数据决定。5~6: 编译器可把源码 `if` 转为选择指令, 也可反向生成跳转; 无分支方案可能计算两侧和增加访存。7~9: 函数指针目标运行时决定, 静态分派可能利于优化, 也可能增加代码体积与指令工作集。先看热点, 再读相关汇编; 不能按源码关键字估算周期。

15.4 Multi-core 与 Measurement

Multi-core 1 ~ 5: 一致性协调核间对共享内存的缓存状态; true sharing 是同一逻辑状态竞争, false sharing 是不同状态碰到同一干扰粒度。relaxed 不取消原子写的所有权协调, 所以不是 false sharing 修复。6 ~ 9: 真正不可变的共享数据减少写失效; 本地聚合减少全局 RMW。连续块分区通常比交错写更易分离写入行, 但要检查边界和实际布局。race-free 只排除一类语言错误, 不保证没有等待、争用或坏局部性。

Measurement 1 ~ 3: 延迟是单次完成时间, 吞吐是单位时间产出; p99 是给定样本分布的百分位, 不是最大值。benchmark 测量工作负载, profile 归因热点, trace 重建事件时间线。4 ~ 6: 端到端指标保证局部热点确实相关; 未优化部分限制总体收益。微基准会受优化消除、输入、热身和计时开销影响, 不能机械外推。

Measurement 7 ~ 12: 测量目标发布配置时使用相应优化选项; 调试构建可用于诊断, 但不能代表优化构建性能。输入分布、规模与顺序都是实验合同。profile 指导汇编阅读, 优化后瓶颈可能转移所以要再测。计数器是可解释信号, 不是因果裁决; 收益还要与维护、内存和构建复杂度比较。未采集的 CPU profile 或硬件 counter 必须登记 NOT RUN, 不用汇编代替。

16. 工程原则回查

16.1 如果半年后只能记住十五条

1. C++ 性能不能从语法直接判断, 必须追到 **object representation**、**access pattern** 和 **machine behavior**。
2. **sizeof(T)** 描述 **object stride**; **alignment** 和 **padding** 会决定大量 **dense objects** 的真实 **footprint**。
3. CPU 的核心成本之一是 **data movement**, 而不是单纯 **arithmetic instruction count**。
4. **Cache line utilization** 决定一次 **memory transfer** 中有多少 **bytes** 真正为当前 **kernel** 服务。
5. **Contiguous dense storage** 的优势来自 **cache locality**、**prefetch**、**memory-level parallelism**、**page locality** 和 **TLB reuse**。
6. **Pointer chasing** 的代价远不止“多一次解引用”: 它会制造 **dependent loads**、**cache misses**、**page spread** 和 **TLB pressure**。
7. **AoS** 与 **SoA** 没有绝对 **winner**; **representation** 必须由 **hot access pattern** 决定。
8. **Hot/cold splitting** 往往比微调几个 **padding bytes** 更值得优先考虑。
9. 最好的 **allocation** 优化首先是不 **allocation**: **inline**、**reserve**、**reuse**、**batch**。
10. **Arena** 用 **grouped lifetime** 换廉价 **allocation**; **Pool** 用固定 **slot topology** 换廉价重复复用; 两者都不会取消 **C++ lifetime rules**。
11. **Virtual address**、**cache**、**TLB**、**page fault** 和 **physical memory** 是不同层次的问题。
12. **Branch** 不天然昂贵; 不可预测的 **control flow** 才可能昂贵, **branchless** 同样可能做更多无用工作。

13. 多线程 **correctness** 和多线程 **performance** 是两个不同问题；**false sharing** 可以让完全 **race-free** 的程序严重退化。
14. 高性能设计优先减少高频跨边界动作：**allocation**、**page fault**、**shared mutation**、**cache-line ownership transfer**、**runtime dispatch**。
15. 任何性能优化都必须经过 **Measure** → **Explain** → **Change** → **Measure Again** 的证据闭环。

17. G1 ~ G6 的统一系统模型

现在我们已经可以用同一个流程分析绝大多数系统 C++。假设看到：

```
std::vector<std::shared_ptr<Record>>
```

不要只看类型名字。按顺序问：

G1 — Object Model

Record / shared_ptr / vector 哪些 objects 存在？

哪些 references/pointers 可能失效？

G2 — Ownership

为什么 Record 需要 shared ownership？

谁延长 lifetime？

G3 — Value Cost

shared_ptr copy 有没有 refcount？

Record 是否可以直接 value-store？

move/copy 成本如何？

G5 — Genericity

哪些行为需要 compile-time specialization？

这里的 template 是否真的必要？

G6 — Machine

多少 allocations？

多少 indirections？

Record 散落多少 pages？

iteration locality 怎样？

shared_ptr control blocks 是否跨线程竞争？

是否存在 false sharing？

真正 profile 热点在哪里？

这份清单把 G1 的有效访问、G2 的共享所有权、G3 的表示成本与本章测量接起来；前几章负责语义，本章只追问这些选择在给定机器上的代价。

18. 从机器观察转向语言并发模型

G6 的成本分析不能替代同步正确性；下一章 G7 §3 以普通数据发布为例，区分硬件一致性与 C++ 可依赖的顺序。机器观察不替代语言顺序；本轮源稿状态与历史本地证据按全书口径分开。

19. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

布局 and 分配结论区分语言合同与目标 ABI；生成汇编的工具选项见 [Clang 用户手册](#)。本批输出不构成处理器性能排名。

G7 · 并发协议与 C++ 内存模型

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G6；从数据竞争、同步和回收到线程停机与并发架构；工程论证不等于形式化证明。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G6 · 全系列导航 · 下一章：G8

阅读入口

本章先区分数据竞争（data race）、原子性（atomicity）和不变量（invariant），再建立顺序与生命周期协议。先发生关系（happens-before, HB）指语言允许依赖的顺序关系，不是墙钟上的先后。首次阅读第 1 ~ 11 节；跨层关系图、审查和反模式用于回查。读写改操作（read-modify-write, RMW）、比较并交换（compare-and-exchange, CAS）都是实现协议的手段，不是设计的出发点。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 阅读模型
- 2. 数据竞争与语言内存模型
- 3. 顺序与同步关系
- 4. 原子性与内存序
- 5. 互斥锁与条件变量
- 6. 原子状态转换与 CAS
- 7. ABA、进展保证与安全回收
- 8. 并发队列的交接协议
- 9. 线程生命周期与停机
- 10. 任务执行、线程池与背压
- 11. 并发架构与写入权限
- 12. 跨层关系图
- 13. 统一并发审查
- 14. 并发反模式

- 15. 工程原则回查
- 16. 实验与验证
- 17. Final Gate
- 18. Final Gate · 参考答案与常见误判
- 19. G0 ~ G7 的统一模型
- 20. 实际代码的跨层审查
- 21. 本章范围与后续阅读
- 22. 参考与验证入口

1. 阅读模型

1.1 G7 到底解决什么问题？

G6 讨论缓存、地址翻译和一致性如何影响成本；本章讨论两个线程何时可以合法访问并观察同一状态。这是语言模型问题，不能由“处理器最终会把缓存同步”代替。

工程上应先确定对象存活、写入权限与跨线程交接，再选择 `atomic`、`mutex` 或队列。协议若没有正确的同步和回收关系，某个平台上长期运行正常也不是其合法性的证明。

1.2 G7 的统一链路

本章的推理顺序是：内存位置与冲突访问 → 数据竞争检查 → 原子性及顺序关系 → 对象所有权和生命周期 → 并发协议、进展及容量约束。原子 API 只解决链条中的一部分。

先写清谁拥有数据、谁能修改、哪些字段必须保持一致、何时交接权限及哪些操作需要 HB；之后才选同步原语。队列、线程池和快照都应回到这组问题，而不是从某种“更高级的无锁结构”倒推业务模型。

2. 数据竞争与语言内存模型

2.1 Data Race 是 G7 的第一道门槛

考虑：

[反例片段 · 未定义行为：无同步的冲突访问；不要用于生产代码]

```
int value = 0;
bool ready = false;

// Thread A
value = 42;
ready = true;

// Thread B
while (!ready) {
}

use(value);
```

人类直觉：

```
Thread B 看到了 ready == true
      ↓
Thread A 肯定已经执行了 value = 42
      ↓
所以 value == 42
```

C++ 并不允许这样推理。这里首先存在：**Data Race** 因此：**Undefined Behavior**

2.2 Data Race 的核心条件

数据竞争（data race）的判断对象是潜在并发的冲突操作：修改同一内存位置，或者开始/结束与访问重叠对象的生命周期，都可能形成冲突；至少一方非原子，且没有所需的 happens-before 次序时，会进入未定义行为。只读且生命周期稳定的同一对象通常不构成数据竞争。不同成员是否对应独立内存位置还需留意位域等情况。这个规则不是“读到旧值”的概率模型，更不能用在某次机器运行中恰好没出错来否定。规则：[N4950 intro.races](#)

2.3 read/read 与 read/write

通常：

```
read / read
→ 不冲突
```

而：

```
read / write
write / write
→ conflicting
```

所以：

```
const Config config;
```

在正确 publication 后被多个 threads 只读：是非常自然的模型。真正复杂的是：Shared Mutable State。

2.4 Data Race 不等于 Race Condition

数据竞争（data race）采用 §2.2 的语言判据；竞态条件（race condition）则是更广义的逻辑问题：结果依赖不受控制的交错。前者涉及未定义行为，后者即使只使用原子操作也可能破坏业务不变量。例如：

[机制片段 · 不承诺独立编译]

```
std::atomic<int> balance{100};

if (balance.load() >= 80) {
    balance.fetch_sub(80);
}
```

两个 threads 都可能：

load 100、load 100、subtract、subtract。

最后：-60 所有 atomic operations 本身合法，但业务 invariant 被破坏。所以：

Data-race-free $\neq$ Race-condition-free

2.5 volatile 不是并发同步

C++ volatile 不提供原子性、线程间同步或 happens-before。把共享 ready 标志改成 volatile，不能修复普通 payload 的发布协议。

[反例片段 · 未定义行为风险：volatile 不建立线程同步]

```
volatile bool ready = false;
```

它与 atomic、mutex 承担不同语义责任；应使用适合协议的同步机制，而不是借 volatile 阻止某种优化来模拟线程通信。

3. 顺序与同步关系

3.1 sequenced-before

描述：同一 thread 内 C++ abstract machine 的顺序关系。例如：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true);
```

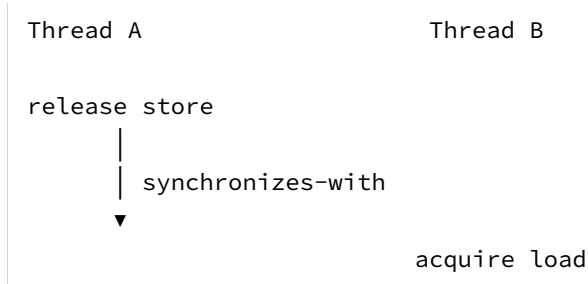
有：

```
data = 42
  |
  | sequenced-before
  ▼
ready.store(true)
```

注意：这不是“CPU 一定先把 data 写入 DRAM”。它是：语言级 ordering relation。

3.2 synchronizes-with

跨线程 synchronization primitive 可以建立：**synchronizes-with** 例如 release/acquire：



前提是：acquire 真正接到了对应的 publication。不是只因为源码中写了两个关键词。

3.3 happens-before

happens-before 是建立合法跨线程访问的重要关系。在本章不使用 consume 的协议中，可以沿线程内 sequenced-before 与跨线程 synchronizes-with 组成传递链。必须逐条说明同步边由哪一次实际观察建立，而不是只标注两端使用了 acquire/release。

如果普通 payload 的冲突访问之间存在所需 HB，且对象生命周期和其余访问也合法，就可以推理这次读写。单条 HB 边不等于整个系统无数据竞争，更不决定所有无关原子操作的全局顺序。

3.4 最重要的一张图

单次发布的关键不是时间先后，而是可组合的关系。生产者写 payload，随后 release-store 标志；消费者的 acquire-load 必须读到该 store（或它引领的适用 release sequence）的值，才能建立 synchronizes-with。结合线程内 sequenced-before，payload 写入 happens-before 消费者读取。

```

producer: payload write --SB--> ready.store(release)
                                     |
                                     SW (load reads this publication)
                                     |
consumer: payload read <--SB-- ready.load(acquire)
  
```

G7-D1 仅做一次发布，不重置标志，也不在发布后并发修改 payload。若复用同一槽位，还需消费者完成读取到生产者下一次写入的反向交接；看到 true 不自动授予永久读取许可。

3.5 为什么 data 可以不是 Atomic ?

普通 payload 不必全部改为 atomic：在上述单次发布协议中，写入与读取已由 release/acquire 的 HB 链排序。原子标志承担交接责任，payload 在交接后不再被生产者修改。

这个结论依赖完整协议，不是“有一个 atomic 就保护附近变量”。若另有写者、重置标志后立即覆盖、或对象在读者使用前已销毁，原来的论证必须重做。

4. 原子性与内存序

4.1 Atomicity ≠ Ordering

原子性 (atomicity) 描述某个访问或 RMW 作为不可分割的操作被观察；内存序 (memory ordering) 说明它与其他相关操作建立什么顺序约束。原子对象本身的操作合法，不自动把一组业务操作合成事务。

例如对余额先 load 再 store，两步各自原子却可能丢失并发更新。需要根据不变量选择互斥区或条件 RMW，而不是把每个字段换成 atomic 就结束审查。

4.2 Atomic Operation 三类

Load

```
x.load(order);
```

Store

```
x.store(value, order);
```

Read-Modify-Write

[机制片段 · 不承诺独立编译]

```
x.fetch_add(...);  
x.exchange(...);  
x.compare_exchange_weak(...);
```

同时有：

read side + write side

4.3 Memory Order 属于 Operation

不是：“这个 atomic variable 是 acquire atomic”而是：

[机制片段 · 不承诺独立编译]

```
x.load(std::memory_order_acquire);  
  
x.store(  
    value,  
    std::memory_order_release);  
  
x.fetch_add(  
    1,  
    std::memory_order_relaxed);
```

同一个 atomic object：不同 operations 可以使用不同 memory order。

4.4 memory_order_relaxed

保证：

atomicity + 该 atomic object 自己的 modification order

但不自动建立：surrounding ordinary data 的跨线程 synchronization。典型：

[机制片段 · 不承诺独立编译]

```
processed.fetch_add(
    1,
    std::memory_order_relaxed);
```

适合：

statistics、independent counter、ticket generation。

前提：不依赖它去发布其它 state。

4.5 Modification Order

每个 atomic object 都拥有：对其所有 modifications 的一致总顺序。例如：

```
x:
0
↓
1
↓
2
↓
3
```

所以 relaxed 并不是：“atomic value 本身也完全乱序。”它仍然具有该 object 自己的 atomic coherence。

4.6 memory_order_release

典型角色：**Publish**

[机制片段 · 不承诺独立编译]

```
payload = build_payload();

ready.store(
    true,
    std::memory_order_release);
```

表示：release 之前 sequenced-before 的相关 operations，可以通过合适 acquire 建立跨线程 HB。
不要翻译成：flush cache to RAM

4.7 memory_order_acquire

典型：

[机制片段 · 不承诺独立编译]

```
if (ready.load(
    std::memory_order_acquire)) {
    use(payload);
}
```

如果 acquire 观察到对应 release publication，则 release 之前的 writes：HB acquire 之后的 reads。

4.8 Release / Acquire 的核心不是 Cache Flush

release/acquire 是语言排序合同，不是“先把所有缓存刷到 DRAM，再让另一核从 DRAM 重读”。编译器针对目标架构把所需约束映射为适当指令和屏障；硬件也可能直接在缓存之间传递数据。

因此协议正确性在语言关系层证明，成本才在目标代码与机器层测量。二者分开，才能避免因某架构的强顺序或某次汇编看起来简单，就削弱可移植程序的必要关系。

4.9 memory_order_acq_rel

主要用于 RMW：

Acquire previous publication + Publish new state

例如：

[机制片段 · 不承诺独立编译]

```
state.compare_exchange_weak(
    expected,
    desired,
    std::memory_order_acq_rel);
```

但：**RMW 不自动意味着一定需要 acq_rel**。如果只需要 atomicity：relaxed 可能就够。如果只需要 acquire：acquire 也可能够。

4.10 memory_order_seq_cst

memory_order_seq_cst 除相应 acquire/release 作用外，还对 seq_cst 操作（包括相应 fence）提供符合标准约束的单一总序。它不是把所有非原子访问、混合弱序操作和外部事件都排成一个全局时钟。选择 seq_cst 可以简化部分推理，但不能补救对象已销毁、复合不变量被拆散或存在非原子数据竞争的问题。混合内存序时仍需逐条说明同步关系，而不是引用“最强”两个字。规则：N4950 atomics.order

4.11 Memory Order 不是性能等级表

内存序不是从 relaxed 到 seq_cst 的固定性能排行榜。目标架构、具体操作、编译结果和争用都会改变成本；相同关键字在不同场景中不保证相同开销。

选择顺序应是业务不变量、所需 HB、同步协议、足够的内存序，最后才是测量。无证据地降序可能破坏正确性；无分析地全部升到 seq_cst 也不能弥补生命周期或进展缺陷。

5. 互斥锁与条件变量

5.1 Mutex 保护的是 Invariant

不是：“锁住一个变量”。例如：

[机制片段 · 不承诺独立编译]

```
struct Account {
    int balance;
    int reserved;
};
```

真正需要保护的是：

$balance \geq 0$ 、 $reserved \geq 0$ 、 $reserved \leq balance$ 。

也就是：多个字段构成的 logical invariant。

5.2 Mutex 提供两件事

Mutual Exclusion

同一时刻：只有一个 owner 进入 critical section。

Memory Synchronization

一个 thread：

```
writes
↓
unlock
```

另一个：

```
successful lock
↓
reads
```

可以建立相应 synchronization / HB。因此 mutex 内保护的数据：

```
int value_;
```

不需要全部改 atomic。

5.3 RAII Locking

默认：

```
std::lock_guard lock{mutex};
```

需要：

unlock/relock、condition_variable、deferred locking。

时使用：

```
std::unique_lock lock{mutex};
```

多个 mutex：

```
std::scoped_lock lock{a, b};
```

优先避免手写：

[机制片段 · 不承诺独立编译]

```
mutex.lock();  
...  
mutex.unlock();
```

5.4 Critical Section 原则

不是：“越短越好。”而是：覆盖维护 **invariant** 所需的最小完整 **logical transaction**。不能为了缩短锁：

check、unlock、...、lock、update。

把本应 **atomic** 的业务操作拆开。同时不要无必要地把：

I/O、network、sleep、large computation、callbacks。

放在 hot shared lock 内。

5.5 Deadlock

经典：

- Thread A:
- holds A
- waits B
- Thread B:
- holds B
- waits A

解决方法：

- consistent lock order
- std::scoped_lock
- architecture redesign

如果系统出现：

大量 **mutex**、随机组合获取。

应该考虑：是否 shared-state topology 本身已经过度复杂。

5.6 Condition Variable 等待的是 Predicate

不是 Notification。正确：

[机制片段 · 不承诺独立编译]

```
cv.wait(lock, [&] {
    return closed_ || !queue_.empty();
});
```

真正业务事实：

queue non-empty、or、closed。

notify_one() 只是：“state 可能已经改变，请重新检查。”

5.7 Notification 是 Edge，Predicate 是 State

条件变量（condition variable）不保存一张可消费的通知清单。正确性依赖受同一互斥协议保护的谓词：生产者持锁改变状态；消费者持锁检查状态，并通过 wait 原子地释放锁进入等待。唤醒后重新持锁检查谓词。若通知早于消费者检查，持久状态使它无需等待；若消费者需要等待，锁与 wait 的协议避免“检查完但尚未等待”的空窗丢失状态改变。仅把标志改成 atomic，再在锁外修改并通知，不自动提供这个保证。G7-D2 把队列、closed 和等待条件放在同一把 mutex 下。

5.8 Spurious Wakeup

wait() 可以在没有目标业务事件时返回。另外多个 waiters 竞争：

A wakes and consumes item、B wakes afterwards、queue empty。

因此：

[机制片段 · 不承诺独立编译]

```
if (!predicate()) {
    cv.wait(lock);
}
```

通常错误。应：

```
cv.wait(lock, predicate);
```

或：

[机制片段 · 不承诺独立编译]

```
while (!predicate()) {
    cv.wait(lock);
}
```


6. 原子状态转换与 CAS

6.1 为什么 load + store 不等于 Atomic Transition ?

[机制片段 · 不承诺独立编译]

```
if (state.load() == Ready) {  
    state.store(Running);  
}
```

两个 individually atomic operations, 仍然可能:

Thread A load Ready、Thread B load Ready、Thread A store Running、Thread B store Running。

两个 threads 都认为自己抢到了 transition。

6.2 CAS

[机制片段 · 不承诺独立编译]

```
value.compare_exchange_strong(  
    expected,  
    desired);
```

语义:

- if current == expected:
- current = desired
- return true
- else:
- expected = current
- return false

关键: CAS 把 check + update 合成一个 atomic RMW。

6.3 expected 是 In/Out Parameter

成功:

atomic = desired、expected unchanged、return true。

失败:

atomic unchanged、expected = actual observed value、return false。

这就是为什么 CAS loop 不需要每次重新 load。

6.4 CAS Loop

```

load current
↓
compute desired
↓
CAS
├ success → done
└ failure
    ↓
    current updated
    ↓
    retry

```

这是：**Optimistic Concurrency** 观察:current state 尝试:conditional commit 冲突:retry

6.5 Weak vs Strong

`compare_exchange_weak` 允许伪失败，适合本来就要重试的循环；`compare_exchange_strong` 不允许这种伪失败，适合需要解释单次失败含义的用法。两者仍须处理值确实改变的情况。

`weak/strong` 描述 CAS 的失败模型，`relaxed/acquire/release` 描述排序与同步，不能互相替代。循环中还要根据更新后的 `expected` 重算 `desired`，并检查进展和重复副作用。

6.6 Success / Failure Ordering

双内存序 CAS 必须分别解释成功的 `read-modify-write` 和失败的 `load`。失败不执行写入，因此 `failure order` 不得是 `release` 或 `acq_rel`；应按失败后是否读取发布数据选择允许的顺序，不能把成功侧的 `release` 要求机械复制过来。

[机制片段 · 不承诺独立编译]

```

while (!state.compare_exchange_weak(expected, desired,
                                     std::memory_order_acq_rel,
                                     std::memory_order_acquire)) {
    desired = next(expected); // 依据更新后的 expected 重算。
}

```

`next` 必须与实际状态转换合同匹配，循环内也不能不经处理地重复外部副作用。`weak CAS` 的伪失败与 `memory order` 的强弱是两个问题。

7. ABA、进展保证与安全回收

7.1 CAS 只比较 Current Value

CAS 比较当前值表示 (value representation)，不是调用用户定义的 `operator==`，也不记录对象经历过的历史。指针或索引可能从 A 变为 B 再回到相同表示 A；CAS 看到相等仍无法判断它是否属于同一逻辑代次，这就是 ABA。需要历史身份时，可以把 `generation/tag` 纳入被比较状态；还要考虑宽度、回绕和原子实现条件。`tag` 并不保护已经取得的指针所指对象，内存回收协议仍是独立责任。

7.2 ABA 不是 Use-after-free

ABA: state/version/history 问题。Reclamation: object lifetime 问题。两者经常同时出现，但必须分开。

7.3 Unlinked ≠ Reclaimable

一个 node:

```

Linked
↓
Unlinked
↓
Retired
↓
Reclaimable
↓
Destroyed

```

从 lock-free structure 中删除: 只解决 Logical Membership。不代表: 没有 reader 仍持有 pointer。

7.4 Pointer Acquisition ≠ Safe Dereference

并发:

```
Node* p = head.load();
```

只得到: pointer value。并没有自动保证: *p object lifetime 仍然 active。另一个 thread 可能: remove、delete。

发生在: load p 和: p->next 之间。

7.5 Tagged Pointer / Generation

把: pointer 扩展成: (pointer, generation) 例如:

```

(A, 41)
→
(B, 42)
→
(A, 43)

```

旧: (A, 41) 不再等于: (A, 43) 可以检测一类 ABA。但: 不自动解决 pointee lifetime。

7.6 Reclamation Strategies

Reference Counting

```

reader holds ownership
→ node cannot die

```

简单组合, 代价:

atomic refcount、control block、coherence traffic。

Hazard Pointer

reader 显式发布: I currently protect Node A reclaimer:

```
A appears in hazard set
→ cannot free
```

流程:

```
load pointer
↓
publish hazard
↓
revalidate source
↓
dereference
↓
clear hazard
```

Epoch-based Reclamation

reader 表示: I am active in epoch E retired node: 等所有可能仍在旧 epoch 的 readers 离开后再 reclaim。

RCU / QSBR

```
build new generation
↓
publish
↓
wait grace period
↓
reclaim old generation
```

特别适合:

read-heavy、write-rare。

系统。

7.7 Lock-free 不是性能等级

Lock-free 是: Progress Guarantee。不是: “一定比 mutex 快”。CAS contention 可能产生: retry storm、cache-line ping-pong。

一个低 contention mutex 可能更快、更简单。

7.8 Progress Guarantees

粗略:

Wait-free、 $\Rightarrow$ 、Lock-free、 $\Rightarrow$ 、Obstruction-free。

Lock-free

系统整体保证：持续有某个 operation 取得进展。不保证每个 thread 都不饿死。

Wait-free

更强：每个 operation 都在有限步骤内完成。

7.9 atomic<T>::is_lock_free() 不是算法证明

它只回答：这个 atomic object 的 implementation 是否 lock-free。不代表：整个 data structure / allocator / reclamation / operation path 是 lock-free。

8. 并发队列的交接协议

8.1 Queue 首先按 Topology 分类

Queue	Producer	Consumer
SPSC	1	1
MPSC	N	1
SPMC	1	N
MPMC	N	N

Topology 直接决定：

- 谁写 enqueue cursor?
- 谁写 dequeue cursor?
- 哪里需要 CAS?
- 哪里可能 single-writer?

8.2 SPSC 的关键简化

- Producer:
- only writer of tail
- Consumer:
- only writer of head

因此：不需要 CAS 去争抢 cursor ownership。只需正确 publication。

8.3 SPSC 的两条 HB Edge

Producer → Consumer

```

construct slot
↓
tail.store(release)
↓
tail.load(acquire)
↓
consume slot

```

因此：

slot write、HB、slot read。

Consumer → Producer

```

finish consuming
↓
head.store(release)
↓
head.load(acquire)
↓
reuse slot

```

因此：

old generation use、HB、new generation construction。

这两条就是 SPSC correctness 骨架。

8.4 MPSC 的关键新问题

多个 producers 可以：reserve slots 但 reservation completion 可以 out-of-order：

```

P0 reserves slot 10
pause

P1 reserves slot 11
writes + publishes

```

于是：

reservation frontier $\neq$ ready frontier

简单 global tail 不够。

8.5 MPMC 为什么需要 Per-slot Generation

常见有界 MPMC 环形队列用每槽 sequence/generation 区分空闲、已占位、已发布及可复用的代次。多生产者的 reservation 不等于元素构造和发布已经完成；消费者必须按设计好的槽状态取得元素。这不是所有 MPMC 算法都必须采用同一种布局的定理。序号回绕、失败路径、对象构造/析构及消费者回收都需要单独证明。本章解释协议问题，不把一个 per-slot 数字当作完整队列正确性证明。

8.6 Bounded Queue = Backpressure

如果 producer rate:

、consumer rate。

无界 queue:

memory ↑
latency ↑

bounded queue: full 显式告诉上游: 系统已达到当前 processing capacity。Full 是: 正常系统状态。不是异常。

8.7 Queue 是 Ownership Boundary

成功:

```
queue.push(std::move(job));
```

自然语义:

```
Producer owns Job
↓
Queue owns Job
↓
Consumer owns Job
```

因此跨线程 queue 中优先:

owned value、move-only handle、lease、unique ownership。

而不是未经约束的 borrowed pointer。

9. 线程生命周期与停机

9.1 std::thread

如果 destructor 时: joinable == true 会: std::terminate()。所以 thread lifetime 必须显式解决:

join、or、detach。

9.2 detach() 不解决 Lifetime

它只是: 解除 std::thread object 与执行线程的 join relationship。不会解决:

thread still accesses this、thread still accesses queue、thread still accesses stack object。

所以 detach 常常只是: 隐藏 ownership 问题。

9.3 `std::jthread`

核心：

RAII join + cooperative stop

Destructor 大体：

```
request_stop
↓
join
```

但：stop 是 request，不是 kill。Worker 必须 cooperate。

9.4 `stop_token`

表示：观察 cancellation request 的 capability。不是：pause/resume state 而是 monotonic：

```
not requested
↓
requested
```

9.5 Cancellation 必须唤醒 Blocked Thread

合作取消必须同时解决请求可见、阻塞可唤醒和退出后可 join。`request_stop()` 只提出请求，不会强制终止任意 wait、I/O 或持锁操作。

一种方案是在同一 mutex 下设置 closed 等终止谓词并通知所有等待方；另一种是使用 `condition_variable_any` 提供的 stop-token 等待重载。仅注册一个在锁外调用 `notify_all` 的 stop callback，仍可能遗漏检查与等待之间的竞争窗口。普通条件变量实验 G7-D2 使用显式 close；它不冒充 stop-token 重载的验证。规则：N4950 `condition_variable_any`

9.6 Cancellation Point 必须保持 Invariant

不能：

```
update half of shared state
↓
see stop
↓
return
```

如果状态因此不一致。Cancellation 和 Exception Safety 有同样结构：

```
prepare
↓
safe cancellation point
↓
commit
```


9.7 Member Destruction Order

成员按声明的逆序析构，因此让工作线程成员最后声明，可以使它先于其访问的依赖成员析构。但声明顺序只是前提之一：线程必须收到可执行的停止协议，被阻塞时能唤醒，且 `join` 不与被持有的锁形成死锁。`jthread` 析构在仍为 `joinable` 时请求停止并 `join`，却不能替代用户协议。若工作函数不响应停止、继续等待外部资源，或者回调重新进入已在析构的 `owner`，仅调整成员顺序并不能解决问题。

9.8 Shutdown 基本顺序

1. 停止接收新工作。
2. 发布关闭或停止状态。
3. 唤醒阻塞线程。
4. 按合同排空已接收工作，或取消并处理未完成工作。
5. 等待线程函数退出。
6. 完成 `join`。
7. 销毁共享状态及其资源。

具体依赖可以要求更细的步骤，但不得在仍有访问者时先销毁依赖。

9.9 Drain vs Abort

Drain

```
reject new work
↓
process already accepted work
↓
exit
```

Abort

```
reject new work
↓
request cancellation
↓
discard/cancel according to contract
↓
exit
```

两者绝不能用一个模糊：

`stop();`

让 `caller` 猜。

10. 任务执行、线程池与背压

10.1 Thread ≠ Task

Thread: execution resource. Task: unit of work. Thread pool 的本质:

```
many tasks
↓
bounded worker threads
```

把: logical concurrency 和: physical parallelism 解耦。

10.2 C++23 Task Representation

非常适合:

[机制片段 · 不承诺独立编译]

```
using Task =
    std::move_only_function<void()>;
```

因为 Task 经常捕获:

unique_ptr、socket、buffer lease、move-only handle。

这和 ownership transfer 模型一致:

```
Submitter
→ Queue
→ Worker
```

10.3 Worker 不能持 Queue Lock 执行 Task

正确:

```
lock queue
↓
move task out
↓
unlock
↓
execute task
```

Queue mutex 只保护: queue invariant. Task execution 已经变成: worker-local ownership。

10.4 Pool 必须考虑两种 Bound

Worker count 限制: physical parallelism. Queue capacity 限制: backlog。只有 bounded workers、unbounded queue: overload 仍然可以把 memory/latency 推爆。

10.5 CPU-bound 与 I/O-bound Pool

CPU-bound:

```
worker count
≈ available compute resources
```

从此附近开始 benchmark。I/O-bound: threads 可能大量 blocked, worker count 可能更大。但更成熟可能使用: async I/O 避免大量 blocked OS threads。

10.6 Resource Isolation

不要把:

CPU tasks、blocking database calls、fsync、network waits、background cleanup。

全部塞进一个 global pool。否则: blocking work 会 starve CPU work。典型:

CPU pool、blocking pool、async I/O runtime。

分离。

10.7 Thread Pool Starvation Deadlock

Pool 只有一个 worker:

```
Task A runs
↓
submit Task B to same pool
↓
A waits future B
↓
B waits for a worker
↓
the only worker is occupied by A
```

没有 mutex cycle, 仍然 deadlock。这是: execution-capacity dependency cycle。

10.8 Bounded Submit 也能 Deadlock

多个 workers:

```
each running task
↓
blocking submit child task
```

queue full。所有 workers: wait queue capacity 但 queue capacity 只能通过: worker consuming tasks 来释放。于是:

```

Workers
↓ wait for
Queue Capacity
↓ requires
Workers

```

形成 cycle。Backpressure 必须结合：resource graph 分析。

10.9 Global Queue vs Work Stealing

Global MPMC:

simple、good natural balancing、central contention。

Per-worker queue:

locality、less global contention。

空闲 worker: steals Work stealing 本质: common case local ownership, rare case cross-worker coordination。

11. 并发架构与写入权限

11.1 并发设计最重要的问题：有几个 Writer ?

不是：有几个线程。而是：

- 每一份 mutable state
- 有几个 writer?

复杂度通常随着：multiple writers 急剧上升。

11.2 Reasoning Complexity Ladder

粗略：

```

Thread-local mutable
      ↓
Single-writer mutable
      ↓
Immutable shared
      ↓
Ownership transfer
      ↓
Mutex-protected shared mutable
      ↓
Atomic shared mutable
      ↓
Multi-object lock-free shared mutable

```

不是性能排名。而是：reasoning complexity。

11.3 Privatize First

例如 metrics：差：

```
32 workers
→ one global atomic counter
```

更好：

```
each worker
→ own counter
↓
periodic aggregation
```

这可以显著减少：

coherence traffic、atomic contention。

所以：**Privatize first, synchronize later.**

11.4 Single Writer Principle

单写者（single writer）使状态转换集中在一个序列中，减少多写者竞争和不变量证明分支。线程本地状态、按实体分片及消息传递，都可以用来构造这种所有权边界。单写者不意味着其他线程可无同步读取同一可变对象。读者仍需快照、锁、消息回复或经过论证的发布协议；消息本身也要说明转移后谁可以继续访问底层数据。

11.5 Message Passing 的真正价值

不是：“不用 shared memory。” Queue 本身仍然共享。真正变化的是：**Mutation Authority** 从：

```
Many writers
→ same state
```

变成：

```
Many producers
→ queue
→ one owner
→ state
```

11.6 Sharding

如果 single writer 成为 throughput ceiling：

```
key
↓
shard
↓
single owner
```

例如：

```
EntityId → shard 0
EntityId → shard 1
...
```

得到：

- within shard:
- single writer
- across shards:
- parallelism

11.7 Shard Key = Ownership Key

Sharding 的本质不是：“分几个 queue”。而是：哪个 key 决定谁拥有写入权限 (mutation authority)? 例如：

VehicleId、AccountId、PartitionId、ConnectionId。

选错 shard key：cross-shard coordination 会非常多。

11.8 Immutable Snapshot

不可变快照 (immutable snapshot) 把读者需要的一组字段作为一致版本发布，适合读多写少且允许读到稍旧版本的场景。读者取得快照所有权后不需要逐字段争锁，但发布与旧版本回收仍需协议。

```
std::atomic<std::shared_ptr<const Model>> current;
```

const Model 只约束这条访问路径，不自动证明深层对象不可变；若写者仍持有可变别名并同时修改内容，快照承诺便被破坏。shared_ptr 保证的是所管理对象的生命周期，不是所有关联状态的线程安全。

11.9 Snapshot Consistency vs Freshness

Reader 可能持有：V1 而 current 已经：V2 这可能是：stale but internally consistent。是否允许由 product/system contract 决定。不要把：latest 和：consistent generation 混为一谈。

11.10 Old Generation 的 Lifetime

not current ≠ safe to destroy

Old reader 可能还在使用 V1。因此仍需要：

shared_ptr、epoch、RCU。

等 generation reclamation。这和 lock-free node reclamation 完全同构。

12. 跨层关系图

12.1 Ownership Graph

所有权图标出对象的 owner、borrower、转移点与销毁者。同步边并不自动延长存活时间，所以线程开始工作之前，就要知道谁保证它访问的对象仍然存在。

12.2 Synchronization Graph

同步图标出每条 HB 的来源：读到发布的 acquire、相应 mutex 的解锁/加锁、线程完成后的 join 或已论证的队列交接。不能只画“线程 A → 线程 B”而省略触发该边的实际操作。

12.3 Mutation Graph

写入权限图（mutation authority graph）为每份状态列出全部写者、允许的读者与权限交接。多写者是重点审查位置，单写者也必须解释跨线程读取如何安全发生。

12.4 Progress Graph

进展图描述谁等待谁、谁有能力使谁继续，检查死锁、线程池饥饿和背压环。只有容量限制而没有释放容量的可执行路径，系统仍可能停住。

12.5 Lifetime / Reclamation Graph

生命周期与回收图区分逻辑移除和实际销毁。标出最后可能访问该对象的读者，以及证明这些访问已经结束的条件；不要把从容器 erase 当作跨线程的回收屏障。

12.6 Cache Ownership Graph

缓存所有权图追踪哪些核心高频写哪些物理邻近数据。它用于 G6 的争用/伪共享分析；race-free 仅是正确性前提，不等于一致性流量低。

12.7 Queue / Capacity Graph

容量图标出压力积累位置、满队列后的行为以及释放容量依赖谁。与进展图联读，可以发现 worker 全部阻塞在 submit、却无人能继续消费的闭环。

13. 统一并发审查

任何并发问题，建议按以下顺序。

Layer 1 — State

共享的 state 到底是什么？

Layer 2 — Ownership

- 谁拥有它?
- 谁控制 lifetime?

Layer 3 — Writers

- 有几个 writer?
- 能不能降到 1?

Layer 4 — Conflict

- read/read?
- read/write?
- write/write?

Layer 5 — Atomicity

- ordinary?
- atomic?
- compound invariant?

Layer 6 — Ordering

HB edge 从哪里来?

Layer 7 — Synchronization

选择:

- mutex?
- CV?
- atomic?
- CAS?
- queue?
- join?

Layer 8 — Lifetime

reader 持有 pointer 时 object 能不能死?

Layer 9 — Progress

- blocking?
- lock-free?
- wait-free?
- 谁会饿死?

Layer 10 — Backpressure

full / overload 时怎么办?

Layer 11 — Shutdown

- drain?
- abort?
- wake?
- join?

Layer 12 — Architecture

最后问：

- 这份 shared mutable state
- 真的需要共享修改吗？

这是最重要的一层。

14. 并发反模式

14.1 Smell 1 — Global Mutable State

```
all threads  
→ one global map/state
```

然后不断增加：

mutex、atomics、flags。

14.2 Smell 2 — Atomic Every Field

[机制片段 · 不承诺独立编译]

```
struct State {  
    std::atomic<int> a;  
    std::atomic<int> b;  
    std::atomic<int> c;  
};
```

但没人定义：multi-field consistency。

14.3 Smell 3 — Relaxed Cargo Cult

看到 atomic：

memory_order_relaxed

只因为：“更快”。却画不出 HB proof。

14.4 Smell 4 — Lock-free Cargo Cult

存在 mutex：第一反应就是 CAS。没有 profile，也没有 progress requirement。

14.5 Smell 5 — Detached Thread

```
std::thread{...}.detach();
```

但 thread lifetime、dependencies、shutdown 全不明确。

14.6 Smell 6 — Unbounded Queue

用：“不阻塞 producer”掩盖 processing capacity mismatch。

14.7 Smell 7 — Worker Waits Same Pool

Pool worker:

```
submit child  
↓  
wait child
```

没有分析 execution-capacity cycle。

14.8 Smell 8 — Notification as State

I got notify、therefore condition true。

错误。Predicate 才是 truth。

14.9 Smell 9 — Remove Then Delete

Lock-free structure:

```
CAS remove node  
↓  
delete immediately
```

却没有 reclamation proof。

14.10 Smell 10 — Shared shared_ptr<MutableT> Everywhere

共享拥有可以延长生命周期，却没有确定写入权限（mutation authority）。

14.11 Smell 11 — All Workers Touch All Entities

随机 dispatch 所有 state，造成：

locks、cache migration、poor affinity。

14.12 Smell 12 — Shutdown as One Bool

`bool running;`

却没有定义：

- `accepting?`
- `draining?`
- `aborting?`
- `stopped?`

15. 工程原则回查

如果半年后只能保留十五条：

1. 在讨论线程“看见什么”之前，先证明程序没有 **Data Race**。
2. **Happens-before** 是 C++ 并发推理的核心关系，不是 **wall-clock** 时间顺序。
3. **Atomicity**、**memory ordering**、**logical transaction** 是三个不同概念。
4. **Release/Acquire** 的本质是建立 **publication** → **consumption** 的 **synchronization relation**，不是刷新 **cache**。
5. **Mutex** 保护的是 **shared-state invariant**，而不是某个孤立变量。
6. **Condition variable** 等待的是 **predicate**；**notification** 只是唤醒提示。
7. **Atomic load + store** 不构成 **atomic state transition**；**CAS** 才能表达 **conditional commit**。
8. **CAS success** 不解决 **pointer lifetime**；**lock-free structure** 必须单独证明 **memory reclamation**。
9. **ABA** 的本质是 **current value** 恢复相同，但 **logical history/generation** 已改变。
10. **Lock-free** 是 **progress guarantee**，不是性能排名。
11. **SPSC/MPSC/SPMC/MPMC** 的区别本质上是 **writer/claimer ownership topology** 不同。
12. **Thread** 本身具有 **lifetime**；**shutdown** 必须遵循 **stop accepting** → **wake** → **drain/abort** → **exit** → **join** → **destroy**。
13. **Bounded queue/backpressure** 是系统稳定性 **contract**，不是容器细节。
14. 并发复杂度主要来自 **Shared Mutable State + Multiple Writers**，因此优先 **thread-local**、**single-writer**、**sharding**、**immutable snapshot** 和 **ownership transfer**。
15. 优秀并发架构的目标不是使用更聪明的 **synchronization**，而是让绝大多数业务 **state** 根本不需要 **concurrent mutation**。

16. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 h-lab/h-file 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

16.1 G7-D1 · 单次发布的不变量与动态观察

命题、观察与边界。每轮创建新的 payload 和 ready，写者只写一次；读者只在 acquire 观察到发布后读取。写入 SB release、release SW acquire、acquire SB 读取给出 HB 链。join 完成后才销毁状态；有限 stress 与 TSan 无诊断仅支持本次路径，不证明所有调度、公平性或复用协议。

[完整实验 · G7-D1 · main.cpp]

```
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    for (int round = 0; round < 200; ++round) {
        int payload = 0;
        std::atomic<bool> ready{false};
        bool correct = false;
        std::thread consumer([&] {
            while (!ready.load(std::memory_order_acquire))
                std::this_thread::yield();
            correct = (payload == 42);
        });
        std::thread producer([&] {
            payload = 42;
            ready.store(true, std::memory_order_release);
        });
        producer.join();
        consumer.join();
        if (!correct) return 1;
    }
    std::cout << "publication=200" << std::endl;
}
```

运行与判据。上述统一命令中的 G7-D1 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.2 G7-D2 · 有界通道：排空、拒绝与停机

命题、观察与边界。队列和 closed 由同一 mutex 保护；push 插入与 close 置位是各自线性化点。pop 在 closed 且空时结束，否则按 FIFO 取出；close 后不接受新值，已接受值仍排空。notify 只负责唤醒，谓词决定动作。实验检查完整 FIFO、空通道关闭、满通道关闭及重复 close；等待中的计数器在同锁下登记，保证关闭确实发生在等待路径，不用 sleep 猜时序。

[完整实验 · G7-D2 · main.cpp]

```
#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <thread>

class Channel {
    std::mutex mutex_;
    std::condition_variable changed_;
    std::deque<int> queue_;
    bool closed_ = false;
    unsigned waiting_ = 0;
public:
    bool push(int value) {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.size() == 4) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
        if (closed_) return false;
        queue_.push_back(value);
        changed_.notify_all();
        return true;
    }
    std::optional<int> pop() {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.empty()) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
        if (queue_.empty()) return std::nullopt;
        int value = queue_.front();
        queue_.pop_front();
        changed_.notify_all();
        return value;
    }
    void close() {
```

完整实验 · G7-D2 · main.cpp · continued

```

        std::lock_guard lock{mutex_};
        closed_ = true;
        changed_.notify_all();
    }
    void wait_until_blocked() { // Test observation, not a production API.
        std::unique_lock lock{mutex_};
        changed_.wait(lock, [&] { return waiting_ != 0; });
    }
};

int main() {
    for (int round = 0; round < 50; ++round) {
        Channel channel;
        bool producer_ok = true, consumer_ok = true;
        int count = 0;
        std::thread reader([&] {
            while (auto value = channel.pop()) {
                if (*value != count) consumer_ok = false;
                ++count;
            }
        });
        std::thread writer([&] {
            for (int i = 0; i < 1000; ++i)
                if (!channel.push(i)) producer_ok = false;
            channel.close();
        });
        writer.join();
        reader.join();
        channel.close();
        if (!producer_ok || !consumer_ok || count != 1000 ||
            channel.push(1001) || channel.pop()) return 1;
    }
    Channel empty;
    bool ended = false;
    std::thread reader([&] { ended = !empty.pop(); });
    empty.wait_until_blocked();
    empty.close();
    reader.join();
    if (!ended) return 2;

    Channel full;
    for (int i = 0; i < 4; ++i) if (!full.push(i)) return 3;
    bool rejected = false;
    std::thread writer([&] { rejected = !full.push(4); });
    full.wait_until_blocked();
    full.close();
    writer.join();
    if (!rejected) return 4;
}

```

完整实验 · G7-D2 · main.cpp · continued

```

    for (int i = 0; i < 4; ++i) {
        auto value = full.pop();
        if (!value || *value != i) return 5;
    }
    if (full.pop()) return 6;
    std::cout << "fifo=50000; empty-close; full-close; joined" << std::endl;
}

```

运行与判据。上述统一命令中的 G7-D2 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.3 G7-D3 · 工具阳性对照：缺少同步的写冲突

命题、观察与边界。反例有意让两个线程无同步写入同一普通对象。只在隔离进程中启用 TSan 运行，要求 data race 诊断和指定退出码；不能把结果值、普通崩溃或超时算作成功。它验证工具对这个已知错误的检测能力，不验证 G7-D1/D2 的所有路径。

[反例 · 未定义行为：仅限 TSan 隔离检测 · G7-D3 · main.cpp]

```

#include <thread>
int value = 0;
int main() {
    std::thread writer([] { value = 1; });
    value = 2; // Deliberate data race: no HB edge between the two writes.
    writer.join();
}

```

运行与判据。上述统一命令中的 G7-D3 提取并处理本模块。先运行无竞争的 TSan 探针；可用时要求 data race 诊断与退出码 66。环境不可用明确 SKIP，不将普通崩溃充作检测。**协议论证边界。**D2 假设 mutex/CV 与标准容器正确、工作线程获得调度、内存分配不失败；它不覆盖任务执行异常、强制终止、MPMC 无锁回收或 stop-token API。状态只在锁内转换，所有工作线程先 join 再销毁 channel。有限 stress/TSan 结果不能证明公平性或全程序无竞争；D3 阳性对照也不能提升这一结论。

17. Final Gate

下面这些问题应该能闭卷回答。

A. Data Race

1 为什么：

[机制片段 · 不承诺独立编译]

```
int x = 0;

// A
x = 1;

// B
use(x);
```

在没有 synchronization 时不能只说：“B 可能读到 0 或 1”？

2 为什么：sleep 1 second 不能建立 happens-before？

B. Happens-before

3 解释：

$SB + SW + \text{transitivity} \rightarrow HB$

4 为什么普通 payload 可以通过 atomic flag publication 安全跨线程？

C. Memory Order

5 什么场景适合：

memory_order_relaxed

？

6 为什么：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true, relaxed);
```

不能配合 relaxed load 正确发布普通 data？

7 acq_rel 为什么主要自然出现在 RMW 上？

8 seq_cst 比 release/acquire 多提供的核心是什么？

D. Mutex / CV

9 为什么 mutex 保护的是 invariant 而不是 variable？

10 为什么：

[机制片段 · 不承诺独立编译]

```
if (!pred()) {
    cv.wait(lock);
}
```

通常错误？

11 为什么 notification 可以发生在 waiter 真正等待之前，而正确 predicate-based design 仍然不丢工作？

E. CAS

12 CAS failure 为什么修改 expected？

13 为什么：weak CAS 不等于：weak memory ordering？

14 为什么 atomic：

load + store

不能替代 CAS？

F. ABA / Reclamation

15 解释：

$A \rightarrow B \rightarrow A$

为什么 CAS 可能无法发现中间变化。

16 为什么：

Unlinked $\neq$ Safe to delete

？

17 Hazard Pointer 与 Epoch 的核心区别是什么？

18 为什么 generation/tagged pointer 不能自动解决 lifetime reclamation？

G. Queue

19 SPSC 为什么通常不需要 CAS 更新 head/tail？

20 SPSC 为什么需要两个方向的 release/acquire handoff？

21 MPSC 中为什么：reservation tail 不能直接等价于：published tail？

22 Per-slot sequence number 在 MPMC queue 中解决的核心是什么？

H. Thread Lifetime

23 为什么 joinable `std::thread` 析构会 terminate？

24 为什么 detach 不能解决 object lifetime？

25 为什么 `jthread.request_stop()` 不能强制杀死 thread？

26 为什么 thread member 通常应该最后声明？

I. Thread Pool

27 为什么 worker 不应该持 queue mutex 执行 Task？

28 为什么 bounded pool 中 worker 递归 blocking submit 可能 deadlock？

29 为什么 CPU-bound 与 blocking-I/O task 不应该机械使用同一个 pool？

30 为什么 work stealing 的主要架构价值不是“更复杂”，而是 common-case local ownership?

J. Architecture

31 为什么 single writer 能显著降低 concurrency complexity?

32 Sharding 的本质是什么? 不是: 多几个 queue。而是?

33 Immutable snapshot 主要优化什么 workload?

34 为什么: old snapshot no longer current 仍不代表可立即销毁?

35 为什么“每个字段都 atomic”经常不如一个 immutable snapshot?

18. Final Gate · 参考答案与常见误判

18.1 Data Race、HB 与内存序 (1 ~ 8)

1. 无同步的冲突非原子访问属于数据竞争，语言不把结果限定为旧值/新值二选一。还必须验证对象生命周期，而不只检查 load/store。
2. sleep 改变调度概率，不建立 synchronizes-with；必须由锁、原子发布或其他标准规定的同步机制建立关系。
3. 在线程内用 sequenced-before 排列相关操作，跨线程找出 synchronizes-with，再使用传递性得到所需 HB。箭头必须有规则依据，不能按墙钟时间补画。
4. 非原子 payload 的写与读可由单次 release/acquire 发布建立 HB。读者必须读到相应发布；写者不能在读者使用时再次改写同一 payload。
5. 独立计数且不依赖该计数发布其他数据时可使用 relaxed；还要分析计数溢出、复合不变量和对象存活。
6. relaxed 的读写没有所需发布同步，标志观察本身不使普通 data 读写合法。
7. RMW 既读取此前状态又发布本次更新，可能同时需要 acquire 和 release；纯 load/store 不需要也不接受所有 RMW 组合。
8. seq_cst 提供对相应 SC 操作的受约束总序，不会把其他错误协议变正确；混合弱序操作须另行推理。

18.2 Mutex、CV 与 CAS (9 ~ 14)

9. 相关字段组成一个状态不变量；只给各字段分别上锁可能仍让读者观察到不允许的组合。
10. wait 可伪唤醒，醒来后谓词也可能已被其他线程改变；使用循环或谓词重载，并在同一锁协议下检查状态。
11. 不是因为通知被保存，而是谓词状态受锁保护并持久存在。消费者持锁检查和 wait 的原子解锁/等待配合生产者的持锁修改，消除丢失工作的窗口；锁外 atomic 标志加 notify 不能机械替代。
12. 失败意味着当前值不同或 weak 伪失败；expected 接收观察结果，下一次 desired 应由它重算。不能反复用旧假设提交状态。
13. weak 指允许伪失败，内存序指定同步/排序，二者独立。
14. 独立原子 load 和 store 之间可插入另一修改；CAS 把条件检查与更新合成一个原子转换。

18.3 ABA、回收与队列 (15 ~ 22)

15. 比较当前值表示不能看到 A→B→A 的历史，因此相同表示不保证相同代次。
16. 从结构摘除只阻止某些新获取，不能证明此前读者已不再使用对象。
17. Hazard pointer 公开保护具体对象并要求正确的获取/重验协议；epoch 等待相关读者离开旧时期后批量回收。读者停顿、注册成本与回收延迟的权衡不同。
18. tag 可区分某些历史变化，却不阻止对象被释放；还要处理回绕并提供真正的存活保护。
19. 常见 SPSC 中每个游标各有唯一写者，不需要多个写者竞争同一个增量；交叉观察仍要同步。

20. 正向发布元素供消费，反向发布消费完成供生产者安全复用槽位；缺少反向边会让下一轮写覆盖仍在读取的数据。
21. 取得位置不等于元素已经构造完成；多生产者可按不同速度完成，消费者不能跨过未发布位置。
22. 常见有界环中每槽序号区分空闲、就绪与复用代次；这只是协议组成，不是所有 MPMC 算法或内存回收的通用证明。

18.4 线程生命周期与运行时 (23 ~ 30)

23. joinable thread 的析构按标准调用 `terminate`；不能偷偷把仍运行的工作和依赖责任丢掉。
24. `detach` 只分离线程句柄的等待责任，不延长捕获对象生命周期，也不建立停机确认。
25. `stop_token` 是合作机制；工作函数与阻塞操作必须响应请求。不能依靠强杀跳过不变量恢复。
26. 最后声明通常先析构，可使 `jthread` 先于依赖销毁；仍须满足唤醒、退出、`join` 和锁依赖，声明顺序不是完整方案。
27. 在队列锁内执行任务会串行化调度，并可能被回调重入、阻塞和再次提交拖入死锁。
28. 所有 `worker` 都等待有界队列空位时，可能已没有线程能消费腾出位置。需非阻塞/拒绝、`helping` 或不同依赖拓扑。
29. 阻塞 I/O 占住执行槽，会饿死需要 CPU 或完成回调的工作；线程数、队列和资源隔离须按负载决定。
30. `Work stealing` 保留常见路径的本地队列操作，仅在需要时跨线程协调；仍有窃取、任务依赖与停机复杂度，并非自动更快。

18.5 架构 (31 ~ 35)

31. 单写者集中状态转换和不变量；其他线程的读取仍需同步或快照，不能据此无锁读取可变状态。
32. 分片按稳定的 `ownership key` 划分写入权限、执行位置与关联数据；跨片事务、迁移和热点仍须协议，不只是多建几个队列。
33. 不可变快照适合读多写少、允许一定陈旧度的负载，以发布和版本保留换取简单读取。
34. “非当前”不等于无人持有；必须等全部相关读者释放或到达回收安全点。
35. 各字段 `atomic` 不自动构成一致版本。快照把相关字段作为一个逻辑版本发布，但需要真正不可变及正确生命周期管理。

19. G0 ~ G7 的统一模型

下面只列跨章审查的交接点，概念定义回到各章主讲处；本节保留它们在并发问题中的用途，不另写一套定义。

19.1 G1 — Object Model

由 G1 §1 ~ 2 检查存储、生命周期、类型、边界与对齐，先确认访问合法，再讨论同步。

19.2 G2 — Ownership

由 G2 §1、§9 列出拥有者、借用者、交接与清理；并发新增的问题是使用者是否真的都已退出。

19.3 G3 — Value Semantics

由 G3 §4、§8 区分复制、移动、分配与表示，不由指针大小推断跨线程交接的完整成本。

19.4 G5 — Genericity

由 G5 §9 界定哪些变化留在编译期，避免调度与生命周期策略无必要地扩散成模板参数。

19.5 G6 — Performance

由 G6 §8 ~ 9 分析布局、缓存、分配、TLB、分支和一致性成本；这些观察不替代本章的语言顺序论证。

19.6 G7 — Concurrency

- Who may access or mutate this state concurrently,
- and what ordering/lifetime guarantees make that legal?

最终统一：

```
Object
  ↓
Lifetime
  ↓
Ownership
  ↓
Representation
  ↓
Machine Cost
  ↓
Mutation Authority
  ↓
Synchronization
  ↓
Concurrent Architecture
```

这已经是一套相当完整的 systems reasoning model。

20. 实际代码的跨层审查

看到：

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<MutableModel> model;
std::mutex mutex;

void worker() {
    std::lock_guard lock{mutex};
    model->update(...);
}
```

以前可能只问：“mutex 会不会慢？”现在应该按完整链路分析：

G1

model pointee lifetime 是否有效？

G2

为什么是 shared ownership？

G3

shared_ptr copy/refcount 成本是什么？

G6

model working set 是否在 cores 间迁移？
mutex/cache-line 是否 contention？

G7

为什么有多个 writers？
能否 single writer？
能否 shard by model/entity？
readers 是否可以使用 immutable snapshot？

这项分析的目的在于：把 **synchronization problem** 向上提升成 **ownership architecture problem**。

21. 本章范围与后续阅读

数据竞争、内存序、锁与等待、CAS、回收、队列、线程停机、任务运行时和并发架构均保留为本章主题。协议论证是明确假设下的工程推理，不是全程序形式化验证。下一阶段按总导航进入 [G8 ABI](#) 与 [C 互操作](#)；接口一旦交给独立组件，线程与销毁合同也必须穿过边界。需要完整关闭案例时回查 [G10 §7](#)。

22. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

语言模型参见 [N4950 数据竞争、原子内存序](#)；工具边界参见 [Clang ThreadSanitizer](#)。

G8 · ABI、原生库与 C 互操作

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。C 接口实验使用 C11；二进制兼容和调用合同按具体平台与配置审查。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G7 · 全系列导航 · 下一章：G9

阅读入口

本章的问题是：库升级了，而调用者没有重新编译，原有调用还能成立吗？先读 §1~8 建立二进制模型，再读 §10~14 设计接口，最后以 §20 的实验检验符号、错误与输出合同。兼容审查可直接查 §9、§15~16；FFI、插件与版本演进保留为深入回查，不要求一次全部读完。

首次出现的核心术语在主线中解释；代码标为机制片段时不承诺独立编译。只有完整实验标记纳入执行器。原稿的编号主题通过 `g8-topic-N` 锚点映射到对应主题组；新章节按工程问题组织，不再逐项复制数十个 Part。原稿的 Complete / Frozen 不沿用为技术验收。

- 1. API 与二进制合同
- 2. 符号、链接属性与 C 入口
- 3. 调用约定与数据模型
- 4. 对象布局、PImpl 与继承
- 5. RTTI 与异常边界
- 6. 分配域、标准库与不透明句柄
- 7. 结构版本与存储表示
- 8. 可见性、库与插件协商
- 9. 兼容性变更与构建配置
- 10. C 接口设计规则与错误输出
- 11. 回调、并发与销毁
- 12. 包装层、跨语言与借用
- 13. 二进制检查与语义合同
- 14. 库接口的分层实现
- 15. 兼容策略与边界分级
- 16. 接口审查清单

- 17. 实践路线与验证选择
- 18. 常见误判
- 19. C++、Zig 与 Rust 对照
- 20. 完整实验：符号与 C 二进制接口
- 21. 边界复核协议
- 22. Final Gate
- 23. Final Gate · 参考答案与常见误判
- 24. 工程原则与全链回查
- 25. 参考资料与验证边界

1. API 与二进制合同

1.1 两种兼容性要分别验证

应用编程接口（application programming interface, API）规定源码如何使用组件，包括名称、类型、前置条件、错误与所有权；应用二进制接口（application binary interface, ABI）规定独立编译组件对符号、传参、返回、布局和运行时的共同理解。源码重新编译后仍成立，不代表旧机器代码能消费新库。

例如 `Config` 从一个 `int mode` 变成再含一个 `int flags`，旧调用者已经按原大小分配对象。新库若直接读取新增字段，即使函数名完全未变，也可能越过旧对象。具体大小不能无条件写死为 4/8；那只是特定数据模型下的常见观察。正确审查比较的是双方真实的大小、对齐、偏移和传递方式，而非只看头文件是否还能编译。

1.2 从语言模型追到二进制合同

G1 ~ G7 的问题没有在库边界消失：对象是否存活、谁负责清理、指针借用多久、异常如何处理、线程能否并发使用，都继续约束调用。所有权和线程规则不全属于狭义机器 ABI，但属于完整接口合同。二进制兼容、源码兼容和语义兼容应分别陈述。

首次阅读先建立“声明 → 类型/表示 → 符号 → 调用约定 → 链接/加载”的链路；审查现有 SDK 时则从边界暴露的类型逆向检查承诺。C++23 是语言基线，不是统一的跨编译器 ABI 证书。

机制示意。一个 Source-compatible 但 ABI-breaking 的例子；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
struct Config {  
    int mode;  
};
```


[机制片段 · 不承诺独立编译]

```
struct Config {  
    int mode;  
    int flags;  
};
```

2. 符号、链接属性与 C 入口

2.1 函数实体、符号与名字改编

有跨翻译单元引用的普通非内联函数通常需要可链接定义，但“每个源函数必定成为最终符号”不成立：内联、未使用实体消除和 LTO 都可能改变结果。模板特化、源码函数与最终机器代码也不是一一对应。

C++ 重载、命名空间、类、模板实参和成员 `cv/ref` 限定需要区分实体，具体 ABI 常用名字改编（`name mangling`）编码必要信息。不是所有信息都以直观方式进入名称，例如一些 ABI 的普通非模板函数名不编码返回类型。G8-B1 以分离编译观察 `add(int,int)`、`add(double,double)` 和 C 函数，而不规定所有平台必须采用相同拼写。

2.2 C 语言链接属性的实际边界

语言链接属性（`language linkage`）作用于相应函数类型及具有外部链接的名称。C++ 中的 `extern "C"` 不是导出宏、动态加载指令或表示转换器；C 头文件须在 `__cplusplus` 条件下才出现这段语法。实现通常按目标平台 C 约定处理名称，但 Mach-O 目标文件可能带前导下划线，其他平台也可能有装饰。

把返回类型写成 `std::string`，不会因入口是 C 名字就消除标准库布局、分配器、异常和生命周期要求。稳定跨语言接口常选择标量、指针、显式长度、简单结构和函数指针；优势是可描述、易绑定，不是“C ABI 在所有体系结构上都相同”。N4950 [dcl.link](#)。

机制示意。C++ 为什么不能简单把 Symbol 叫 `add`。以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
int add(int, int);  
double add(double, double);
```

3. 调用约定与数据模型

3.1 调用约定与隐藏参数

调用约定（`calling convention`）至少涉及参数/结果所在的寄存器或栈位置、聚合体分类、保存寄存器责任、栈对齐及间接传递。大对象可能拆分传参，也可能通过调用者准备的存储传递；不能仅凭源码类型推断所有目标机器的指令序列。

一些返回约定让调用者分配结果存储，再传入隐藏地址，常称结构返回 (sret)。这是 ABI lowering，不等于语言层 RVO 或保证复制消除。非静态成员调用还常有对象地址及子对象调整；概念上的 `f(object_address, x)` 不应伪装成合法显式 `this` 参数签名。虚调用涉及运行时分派元数据，优化器也可能消除实际间接调用。

3.2 数据模型与明确宽度

LP64、LLP64 等数据模型使 `long` 与指针宽度关系不同。边界字段采用 `uint32_t`、`int64_t` 可明确数值宽度，但这些精确宽度类型以目标实现提供为前提；它们不同时保证结构对齐、端序、填充或语义兼容。

`size_t` 适合当前进程的对象大小和索引，在同一目标 ABI 的指针+长度接口中很自然；它不是跨架构的固定宽度线格式。持久文件和网络协议应另定整数宽度、字节序与范围检查，不把机器 ABI 当作存储模式。

4. 对象布局、PImpl 与继承

4.1 公开类的私有表示仍可能暴露

调用者按值构造公开类时，必须知道大小与对齐；内联函数可能进一步固化成员偏移。给 `private` 区域增加字段、替换容器或改变基类，都可能影响旧调用者。`private` 是源码访问控制，不是二进制隔离。

指向实现 (pointer to implementation, PImpl) 把真实表示和依赖藏进实现文件。公开类只保留稳定约定的指针成员及非内联操作，可降低布局变化和头文件传播，但公开类自身仍有 ABI。`unique_ptr<Impl>` 并非由语言保证永远等于一个裸指针大小；涉及删除 `Impl` 的操作要在类型完整可见处定义。

4.2 PImpl 的收益与成本

PImpl 同时服务布局稳定、编译隔离和依赖隔离。其代价可能包括分配、间接访问、非内联调用、优化可见性下降与特殊成员函数维护。它不是所有类的默认高级写法：一起重编译的内部数据类型可能更适合直接表示，独立演进的 SDK 则更值得付出隔离成本。

4.3 布局性质、继承与虚表

现代 C++ 应区分标准布局 (standard-layout)、平凡可复制 (trivially copyable)、特殊成员平凡性及隐式生命周期相关性质，而非仅用历史 POD 一词概括。标准布局提供特定布局与 `offsetof` 等推理条件，不保证任意平台上的相同表示。平凡可复制给出满足条件的对象表示复制保证，不授予任意资源对象按字节搬迁和遗忘源对象的权限，也不保证可移植序列化。N4950 basic.types。

继承可能引入基类子对象、虚基类、指针调整与虚表结构。常见实现通过对象中的虚表指针找到函数槽位，但那是实现 ABI，不是 C++ 标准规定的物理图。插入/重排虚函数或改变基类布局可能破坏旧槽位、偏移和转换；公开多态类因此形成较强承诺。

机制示意。为什么 PImpl 存在。以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
class Decoder {
public:
    Decoder();
    ~Decoder();

    Decoder(Decoder&&) noexcept;
    Decoder& operator=(Decoder&&) noexcept;

    void decode();

private:
    class Impl;
    std::unique_ptr<Impl> impl_;
};
```

[机制片段 · 不承诺独立编译]

```
class Decoder::Impl {
    std::vector<float> buffer_;
    Config config_;
    // 其他实现成员省略。
};
```

5. RTTI 与异常边界

5.1 RTTI 与异常需要兼容的运行时

`dynamic_cast`、`typeid` 和类型身份比较依赖相应实现的类型元数据；插件与宿主若使用不兼容编译器、标准库或选项，不能假定类型信息互通。禁用 RTTI、隐藏符号和混用运行时也要纳入支持配置，而非只检查类名相同。

异常传播不仅传递一个 `Error` 对象，还依赖异常存储、展开表、`personality routine`、类型匹配及析构执行。统一受控的 C++ 组件可以选择支持跨库异常，但须维护该运行时合同；第三方 SDK、长期插件和跨语言边界不宜隐含依赖它。

5.2 异常翻译和 `noexcept` 各负其责

本章的 C 接口合同禁止 C++ 异常逸出，入口负责捕获并转换成显式状态。写 `extern "C"` 不会自动插入 `catch`；写 `noexcept` 也不自动生成错误码。异常试图离开 `noexcept` 函数会终止，转换和诊断路径本身也应避免再次抛出。

头文件和定义的异常说明必须一致。G8-B2 用条件宏给 C++ 声明与定义均加 `noexcept`，而 C 编译端看不到 C++ 专有语法。受控异常只验证翻译路径，不代表分配失败、损坏指针或所有终止路径都经过实验。

6. 分配域、标准库与不透明句柄

6.1 分配域与调用者缓冲区

创建方若使用不同运行时、自定义 arena 或专门分配器，消费者的 free/delete 未必匹配。稳健默认是由同一分配域 (allocation domain) 释放：create/destroy、make_buffer/free_buffer 成对，明确句柄和缓冲区的拥有者。虚析构能够帮助选择析构逻辑，却不自动修复所有分配来源和运行时不匹配。

调用者提供输入和输出缓冲区，库只在调用期间借用，可避免跨库转移分配所有权。接口仍要说明元素单位、容量、零长度、空指针、重叠以及失败后计数。自定义分配器回调还须规定 context 生命周期、大小/对齐、失败值、线程安全和释放匹配，不能仅传一对函数指针就算完整协议。

6.2 标准库接口与源代码包装层

vector、string、shared_ptr 等类型会把表示、模板实例化、分配器、异常和标准库 ABI 带到边界。同一产品统一工具链、同步重编译时可以接受；独立升级的 SDK 要显式控制。span 不分配，但仍是 C++ 库类型，不因此成为通用 C 参数。

C++ 源接口可以使用 span、expected 和 RAII，底层二进制接口使用指针+长度、状态码及成对创建销毁。包装层负责相互转换；“二进制接口简单”不要求 C++ 用户体验也退化为手工管理。

6.3 不透明句柄隐藏布局，不自动赋予所有权

公开头文件只声明 typedef struct decoder decoder;，消费者持有 decoder* 而不知道大小，库实现定义真正结构。内部可更换容器、arena 或调度器，不必把变化直接暴露给消费者。

拥有还是借用必须由函数合同规定：create 返回拥有型句柄，destroy 结束其生命周期，get_global 可能只借用。任意地址 reinterpret_cast 成指针不会创造对象；空检查也不能识别悬挂、类型混淆或重复销毁。销毁前无在途调用是常见且必要的前提。

7. 结构版本与存储表示

7.1 结构版本、前缀和保留字段

C 结构体比复杂 C++ 类容易约定，但增加字段仍可能改变大小、对齐、步长和传参。结构中携带 struct_size 与 abi_version，可声明调用者理解的版本和范围；库仍须先确认公共前缀可读，再逐项检查新增字段的完整字节范围，不能先按新版 sizeof 复制再检查旧大小。

尾部追加可选字段比重排旧字段更容易兼容，但安全性取决于指针/按值传递、默认值、数组步长和目标 ABI。保留字段可提前预算扩展空间，同时应规定零初始化、禁止使用位及未来协商规则。G8-B2 仅执行 v1 的大小/版本拒绝，不声称完成 v1/v2 升级验证。

7.2 枚举、布尔与位域

内部 `enum class` 很适合类型安全。公开二进制字段应明确底层宽度和数值含义；C 接口也可用固定宽度状态类型加常量。已经发布的数值不能随意复用，消费者对未来未知值要有明确处理，不能无条件落入 `unreachable`。

`bool`、`char` 的语义与表示应按目标 ABI 或接口规则约定；长期跨语言字段可使用规定了 0/非零规则的整数。位域分配和布局依赖实现，不应直接用作可移植协议布局。明确整数宽度仍不等于明确结构布局。

7.3 内存布局、packing 与线格式

`pragma pack` 可能改变对齐/填充，却不能解决端序、生命周期、未对齐访问、版本或语义。将字节缓冲区强转为结构体指针可能同时违反多个访问前提；“本机恰好能读”不是格式定义。

持久与网络格式应规定字节范围和端序，显式编码/解析并检查长度。领域对象和稳定模式之间建立转换，不把含指针、填充或实现布局的内存直接当长期文件。ABI 常用于同进程目标平台约定，`wire/storage schema` 面向更长时效和更宽环境，二者需要独立设计。

机制示意。 `Size-prefixed Struct`；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
typedef struct decoder_config {
    uint32_t struct_size;
    uint32_t api_version;

    uint32_t mode;
    uint32_t flags;
} decoder_config;
```

机制示意。 `#pragma pack(1)` 不是“让 Struct 可以传网络”；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
#pragma pack(push, 1)
struct Packet {
    std::uint8_t type;
    std::uint32_t length;
};
#pragma pack(pop)
```

8. 可见性、库与插件协商

8.1 控制导出面与静态/共享库边界

共享库通常宜默认隐藏内部符号，显式导出稳定入口。暴露大量辅助函数、模板和运行时类型信息，会增加偶然耦合、冲突与演进成本。Windows 常用 `dllexport/dllimport`，Clang/GCC 系平台常用 `visibility` 属性；具体宏须考虑静态构建和平台支持，不能把示意宏当作所有编译器通用代码。

静态库通常是对象文件归档，最终链接器按需要选择成员；它本身不是运行时独立加载组件，但对象的调用/布局和运行时仍须兼容。LTO 可能进一步跨翻译单元优化，不保证每个静态链接都有收益。共享库则由加载器映射、绑定和定位依赖，形成更明显的独立升级边界。ELF .so、Mach-O .dylib 和 Windows DLL 的加载细节不同。

8.2 插件函数表与显式协商

常见插件流程是加载库、查找单一稳定入口、请求版本化函数表。函数表由应用协议规定字段顺序、大小、可选函数和调用类型，不同于编译器控制的 C++ 虚表。只把工厂名字改成 `extern "C"`，却返回多态 `Base*`，仍会暴露 C++ 对象、析构、RTTI 和运行时。

软件版本描述产品演进，ABI 版本描述二进制合同代际。入口可以明确接受或拒绝 `requested_version`，而不依赖版本号“看起来接近”。函数表、回调、对象及线程都可能引用插件代码；卸载前应停止新调用、等待在途工作、销毁对象并解除回调。加载成功不能证明卸载安全。

机制示意。Export Macro；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
#if defined(_WIN32)
#   if defined(MYLIB_BUILD)
#       define MYLIB_API __declspec(dllexport)
#   else
#       define MYLIB_API __declspec(dllimport)
#   endif
#else
#   define MYLIB_API __attribute__((visibility("default")))
#endif
```

[机制片段 · 不承诺独立编译]

```
MYLIB_API
int decoder_create(...);
```

机制示意。Function Table；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
typedef struct decoder_api_v1 {
    uint32_t struct_size;

    decoder_handle* (*create)(
        const decoder_config*);

    void (*destroy)(
        decoder_handle*);

    decoder_status (*process)(
        decoder_handle*,
```

机制片段 · 机制片段 · continued

```
    const uint8_t*,
    size_t,
    float*,
    size_t,
    size_t*);
} decoder_api_v1;
```

9. 兼容性变更与构建配置

9.1 哪些变化需要兼容性复核

字段、基类、虚函数、对齐/packing、枚举表示、参数或返回类型、异常说明以及标准库/编译器 ABI 配置，都可能影响边界。参数类型变化不必然改变每个 ABI 的链接名；C 名字通常不编码类型，反而可能在错误签名下仍链接成功。某些普通 C++ 返回类型也不进入名字改编，所以“符号没变”远远不够。

noexcept 同时涉及类型系统和失败语义，不能当无关装饰。Debug/Release 标签本身不决定兼容性；真正需要记录的是调试迭代器、运行时选择、RTTI/异常开关、调用约定、packing、标准库 ABI 选项和插桩配置。不是所有选项都会改变布局，但都应在支持配置中有去向。

9.2 头文件逻辑也会分发行

inline 函数和模板实例化可能已编进调用者，更新共享库不能自动更新旧指令。公共头文件改变语义，即使新的库符号兼容，也可能让新旧消费者保留不同逻辑。模板定义通常需在实例化点可达，集中显式实例化是受控选择，不意味着任意组合都由二进制库提供。

薄泛型前端可以校验约束、规范化输入，再进入非模板核心；稳定外部边界继续降为明确的 C 类型。这样保留类型友好的源码接口，同时控制实例化传播与二进制变化面。

9.3 ODR 与配置一致性

不同组件因宏、不同头文件或选项而看到不同 inline、模板或类定义，可能违反单一定义规则 (ODR)。共享库不是语言规则的自动隔离层；加载器符号绑定行为还会增加诊断难度。

升级审查应同时记录头文件身份、导出符号、类型表示、依赖运行时与消费者构建配置。只保留 -std=c++23 不能重建 ABI 上下文；一起重编译与保留旧消费者的验证是不同命题。

10. C 接口设计规则与错误输出

10.1 十二条 C 接口约束的用途

本章的外部接口 profile 包括：C 链接入口；异常止于边界；状态对象采用不透明句柄；显式拥有/借用；输入用指针+长度；适用时由调用者拥有输出；稳定字段使用明确宽度；版本与范围协商；不暴露 STL；不使用 C++ 引用参数；明确线程安全；明确生命周期。

这些规则缩小外部合同，不要求整个产品内部都用 C 风格。实现内部仍可使用容器、泛型、`span`、`expected` 和 `RAII`。每条规则都应有对应前提，例如指针 + 长度须说明空范围是否允许空地址，以及库是否会在返回后保留输入。

10.2 错误状态要可独立解释

状态码应有稳定数值与明确成功/失败含义。`last_error` 可以提供详细消息，但必须规定是线程局部、句柄局部还是全局状态，何时覆盖以及并发读取是否安全。把错误对象或消息缓冲区作为调用者提供的输出，能减少隐式状态，却仍需要说明截断、所需长度和终止字符。

创建接口采用状态 + `out_handle` 时，失败将有效输出位置清空，成功转移一个可销毁句柄。不能把一个仍拥有旧对象的槽位交给 `create` 后期待自动替换；没有声明的清理和事务语义不会由输出参数形式自动产生。

10.3 两次调用不自动形成快照

先用空缓冲区查询 `required`，再分配并读取，是可变长度输出的常见模式。两次调用之间数据可能增长或改变，因此需定义容量不足重试、版本号、快照句柄或锁定协议。

还要规定 `required` 与 `written` 的单位、查询是否有副作用、失败是否部分写入。消费者不能把第一次查询到的长度当成第二次必然足够的永久保证。

11. 回调、并发与销毁

11.1 函数指针与 context 构成回调协议

C 回调通常组合 `function pointer` 与 `void* context`，C++ 包装层再经静态 `trampoline` 恢复对象。协议须说明回调是只在当前调用中借用，还是保存到未来；由哪些线程执行；能否并发；数据指针有效多久；注销是否等待已开始的回调结束。

若 `context` 已销毁而库稍后调用，仍是生命周期错误，类型签名不能保护它。回调应遵守不抛异常的边界约定；如果支持回调错误返回，必须定义它如何影响当前操作的输出和状态。

11.2 重入、锁与销毁

持有内部锁调用不可控用户代码，可能因回调重入而死锁，也可能放大锁持有时间。接口需明确允许的重入集合；避免在热锁内调用外部代码是稳健默认，而不是单靠“回调很短”的假设。

线程合同至少区分同一句柄是否可并发、不同句柄是否独立、读者/写者规则和销毁前提。内部 `mutex` 不能让“另一线程已销毁的对象”重新合法；`destroy` 通常要求没有在途操作。异步接口还应定义 `close/cancel/join` 与销毁次序。G8-B2 故意采用同步回调，未验证异步注销协议。

机制示意。 C Callback；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
typedef void (*decoder_log_fn)(
    void* context,
    uint32_t level,
    const char* message,
    size_t message_len);
```

[机制片段 · 不承诺独立编译]

```
decoder_set_log_callback(
    decoder*,
    decoder_log_fn,
    void* context);
```

12. 包装层、跨语言与借用

12.1 RAII 包装不改变底层合同

C++ 包装器可以持有 `decoder*`，析构调用 `destroy`，禁用复制并在移动时转移句柄；返回 `expected`，将 `span` 转为指针 + 长度。包装层仍要定义创建失败、移动后状态和自移动/替换行为，不能因为类有析构就假定所有权完整。

丰富的 C++ 源接口与小而稳定的 C 二进制接口可同时存在。包装器通常随消费者编译，因此其模板、`inline` 语义和所需标准库仍属于源码分发合同。

12.2 Rust 与 Zig 的表示约定仍有边界

Rust 默认布局不是通用 C ABI。 `repr(C)` 请求相应 C 表示规则，但嵌套成员的表示、枚举有效值和调用约定仍要单独审查； `borrow checker` 不会替外部原始指针证明有效期或线程安全。

Zig 能导入和导出 C 接口，但 `slice`、`error union`、`allocator` 和 `comptime` 类型也不能未经设计直接当 C 类型。C++ `span`、Rust/Zig `slice` 通常展开为地址 + 长度，并在语言包装层恢复检查。分配器抽象不会自动跨语言互通；由哪侧分配、哪侧释放仍需明确。

12.3 拥有、借用和 `const` 是不同维度

`create` 返回拥有型句柄；只在调用期间使用输入是短借用；保存输入指针到返回之后则是长期借用，必须额外约束寿命、可变性和并发。若不能建立这些约束，应复制数据或显式转移所有权，不能暗中延长借用。

`const T*` 限制经该访问路径修改，不说明其他别名或线程不能改动。`const decoder*` 可以表达逻辑只读，但实现仍可能更新缓存、指标或同步状态；`const` 不是完整线程安全协议。

机制示意。 C ABI 不意味着 C++ 用户体验差；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
decoder* decoder_create(...);
void decoder_destroy(decoder*);
```

[机制片段 · 不承诺独立编译]

```
class Decoder {
public:
    explicit Decoder(const Config& config);

    ~Decoder() {
        decoder_destroy(handle_);
    }

    Decoder(const Decoder&) = delete;
    Decoder& operator=(const Decoder&) = delete;

    Decoder(Decoder&& other) noexcept
        : handle_{
            std::exchange(
                other.handle_,
                nullptr)} {}

private:
    decoder* handle_{};
};
```

13. 二进制检查与语义合同

13.1 从二进制中读取证据

使用 `nm`、反改编工具、`objdump/readelf`、`otool` 或 `dumpbin`，分别观察目标符号、动态导出和依赖。ELF 的完整符号表与动态符号表要区分；本批 Mach-O 实验记录原始/反改编符号与 `otool` 依赖，不把其中一个工具当成所有 ABI 属性的验证器。

升级前比较导出集合、函数合同、类型大小/对齐/偏移、虚接口、枚举数值、版本协商和依赖 ABI。自动 ABI 比较工具可以辅助，但语义、生命周期和性能合同仍需有针对性的审查与实验。

13.2 语义、有效期与运行约束

同一个签名若把米改成厘米，二进制布局可能未变而调用语义已破坏。返回 `const char*` 还必须说明是静态存储、句柄内借用、下次调用失效还是调用者负责释放。同步入口若改为保存输入供异步处理，也改变了关键合同。

同样的函数名可以对应不同的线程安全和实时行为。无动态分配、不阻塞或有界执行若是产品承诺，就应单独版本化并测试；不能由固定签名或“C ABI 简单”推导。本批不做实时性和性能验收。

14. 库接口的分层实现

14.1 头文件、实现与包装分工

公开 C 头文件只保留不透明类型、状态常量、配置、条件链接属性、导出声明及回调类型。实现文件定义对象，转换配置、检查调用前提并捕获可恢复异常；C++ 源包装提供 RAII、span 和 expected。本章完整可编译接口集中维护在 G8-B2，不让多份签名不同的示例成为并行事实源。

创建时先清空有效的输出槽，再检查配置版本/大小并分配；处理时先置零有效计数位置，检查空范围和容量，再写数据。示例要求容量不足与受控异常发生时输出不变，成功时输出长度和全部元素正确。span 本身并不会验证外部指针的可读写性。

14.2 插件与跨模块析构

extern "C" BasePlugin* create_plugin() 只稳定入口名字，宿主仍需理解对象的虚表、析构和运行时。更强隔离可采用版本化 C 函数表和不透明句柄，显式约定 create/destroy/process。

由创建域提供 destroy 可减少释放不匹配；虚析构的正确分派不等于任意自定义内存都能交给宿主 delete。无论采用哪种方案，都必须让对象先于提供析构代码的模块卸载。

15. 兼容策略与边界分级

15.1 三种策略按边界选择

一起重编译全部组件的内部产品，可接受较丰富的 C++ 类、模板和容器接口，独立 ABI 演进压力较小。稳定 C++ ABI 则需要受控工具链、PImpl、可见性策略、谨慎虚接口和兼容检查。长期 SDK、插件及跨语言边界常采用稳定 C 接口加语言包装，但仍须控制平台 ABI 和语义版本。

不能把最后一种策略机械推广到所有内部模块。边界越独立，显式合同越重要；本地算法和容器不必为想象中的跨语言需求退化为 void*。

15.2 六类边界及审查强度

同一翻译单元主要由单次编译控制；分离翻译单元增加声明、定义与 ODR 一致性；静态库增加预编译对象和最终链接兼容要求。同一产品中的共享库可以统一版本，但仍应理解加载和运行时耦合。

第三方二进制 SDK/插件需要明确旧消费者与新提供者的支持矩阵；跨语言接口再增加表示、错误和生命周期映射。分级服务风险识别，不意味着前几类没有 ABI，也不意味着 C 接口可以跨不同位宽直接互调。

16. 接口审查清单

16.1 从签名到生命周期的审查顺序

先标明哪些类型穿过边界、是否含编译器或标准库特有表示；再逐项列拥有者、分配域、释放入口、借用期和是否保存指针。错误模型要区分状态、异常、部分输出及终止路径，不能只写“出错返回非零”。

接着检查线程安全、回调线程、重入和销毁并发；核对版本协商、结构范围、未知值和保留位。最后检查异步句柄的停止、取消、等待与销毁顺序。没有这些文字合同的 `pointer signature` 不算完整接口设计。

17. 实践路线与验证选择

17.1 本批验证与可选深化

原有七条实践路线保留：名字改编、布局变化、PImpl、C 包装、跨语言调用、回调和版本演进。G8-B1 落实符号与链接负例；G8-B2 落实 C11 调用者、C++ 共享提供者、同步回调和 v1 拒绝合同。二者的完整代码和命令在后文集中提供。

布局深化应分别在合法的 v1/v2 翻译单元观察表示，不把不兼容旧对象交给新代码执行并期待固定崩溃。PImpl 深化可比较实现字段变化前后的公开布局，但需固定工具链。Rust/Zig 调用、异步回调与 v1/v2 前缀协商仍是可选、未执行路线；不要把练习题列出等同于已经验证。

18. 常见误判

18.1 兼容性误判

“public API 没变，所以 ABI 没变”遗漏私有布局；“都用 C++23”遗漏实现 ABI；“平凡可复制可以直接永久存盘”遗漏指针、填充、端序和版本；“pack(1) 就是网络格式”混淆布局和编码。

“extern C 让 vector 变成 C 类型”以及“无分配的 span 适合直接跨所有 C ABI”都混淆名称/成本与表示。factory 返回 Base* 仍暴露 C++ 多态对象；exception 跨 DLL 总是安全也不是可支持的普遍结论。

18.2 生命周期与演进误判

“库分配、调用者 free 就行”遗漏分配域；“opaque 只是保密实现”遗漏布局隔离；“inline 或模板逻辑随共享库更新”遗漏消费者已编译代码。

“符号仍兼容所以语义兼容”遗漏单位、所有权、借用与线程合同。狭义 ABI 主要约束机器交互，但工程接口审查必须覆盖这些非位级条件，不能借术语边界把责任删掉。

19. C++、Zig 与 Rust 对照

19.1 语言内部丰富，边界保持明确

C++ 的类、模板、STL、异常和 RAII 适合内部建模，但在独立边界上需要评估运行时耦合。Zig 的显式布局、分配和 C 互操作便于集成，却仍有不直接等于 C 表示的 slice 和 error union。Rust 的所有权、枚举、trait 与 Result 也需通过受约束的 FFI 包装连接外部世界。

共同结构是语言原生 API → 薄适配层 → 小而明确的 C 接口。它是常见工程策略，不是所有项目必须采用的唯一架构。学习时比较的是合同在哪里表达、哪些条件由语言检查、哪些只能由跨边界协议和测试承担。

20. 完整实验：符号与 C 二进制接口

这些实验由正文提取到新临时目录，完整文件是唯一维护来源。执行器只运行已审阅的本地代码，不是安全沙箱；不修改历史 PDF、FM 或出版系统。

```
python3 c++/learning/verify_native.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

20.1 G8-B1 · 语言链接、符号与目标链接诊断

待验证命题。分别编译 C 定义、C++ 重载与调用者；观察 C/C++ 名称差异。缺少 C 链接声明的调用者可以编译，但必须因目标符号未解析而链接失败。

范围与前提。只观察本次 Clang/Mach-O 符号，不规定所有平台名字；不执行不匹配布局，不把每个源函数等同于最终符号。

[完整实验 · G8-B1 · sum.c]

```
int native_sum(int a, int b) { return a + b; }
```

[完整实验 · G8-B1 · overloads.cpp]

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

[完整实验 · G8-B1 · main.cpp]

```
extern "C" int native_sum(int, int);
int add(int, int);
double add(double, double);
int main() {
    return native_sum(2, 3) == 5 && add(4, 5) == 9 &&
        add(1.25, 2.5) == 3.75 ? 0 : 1;
}
```

[完整实验 · G8-B1 · missing.cpp]

```
// Intentional link-negative: declaration has C++ linkage.
int native_sum(int, int);
int main() { return native_sum(2, 3) == 5 ? 0 : 1; }
```

运行与解释。本节开头的统一命令按 G8-B1 提取全部文件；具体编译、链接、运行及负例命令保存在 results.json。先预测结果，再用完整诊断核对，不能把任意构建失败或超时当作预期反例。

20.2 G8-B2 · 真正的 C 消费者与共享库接口

待验证命题。头文件以 C11 编译，库以 C++23 编译，C 调用端链接并加载共享库。检验配置大小/版本、拥有句柄、完整输出、容量失败不写入、同步回调与异常到状态码的转换。

范围与前提。非空配置指针必须指向完整可读 config 对象；struct_size 只是声明，不是内存验证。其他非空地址也必须有效。输入/输出/计数所指的对象范围不得重叠，也不能与句柄对象重叠；dec_create(config, out) 的配置对象与输出句柄存储同样遵守此条件，不能只比较首地址是否相等；同一句柄调用与销毁不并发；回调只在本次调用中同步借用，不抛异常、不销毁或重入该句柄。未测试分配失败、历史 v1/v2 混用、插件 dlopen/卸载或 Rust/Zig 调用。

[完整实验 · G8-B2 · decoder.h]

```
#ifndef NATIVE_DECODER_H
#define NATIVE_DECODER_H
#include <stddef.h>
#include <stdint.h>
#ifdef __cplusplus
#define DEC_NOEXCEPT noexcept
extern "C" {
#else
#define DEC_NOEXCEPT
#endif
#define DEC_API __attribute__((visibility("default")))
typedef struct decoder decoder;
typedef uint32_t dec_status;
enum { DEC_OK, DEC_INVALID, DEC_SMALL, DEC_INTERNAL };
typedef struct dec_config {
    uint32_t struct_size;
    uint32_t abi_version;
    uint32_t model;
} dec_config;
typedef void (*dec_callback)(void* context, size_t count);
DEC_API dec_status dec_create(const dec_config*, decoder**) DEC_NOEXCEPT;
DEC_API void dec_destroy(decoder*) DEC_NOEXCEPT;
DEC_API dec_status dec_decode(decoder*, const uint8_t*, size_t,
    float*, size_t, size_t*, dec_callback, void*) DEC_NOEXCEPT;
#ifdef __cplusplus
}
#endif
#endif
```

[完整实验 · G8-B2 · decoder.cpp]

```

#include "decoder.h"
#include <stdexcept>
struct decoder { uint32_t model; };
extern "C" dec_status dec_create(const dec_config* config,
                                decoder** out) noexcept {
    if (!out) return DEC_INVALID;
    *out = nullptr;
    if (!config || config->struct_size < sizeof(dec_config) ||
        config->abi_version != 1 || config->model > 1) return DEC_INVALID;
    try {
        *out = new decoder{config->model};
        return DEC_OK;
    } catch (...) { return DEC_INTERNAL; }
}
extern "C" void dec_destroy(decoder* value) noexcept { delete value; }
extern "C" dec_status dec_decode(decoder* value, const uint8_t* input,
    size_t size, float* output, size_t capacity, size_t* count,
    dec_callback callback, void* context) noexcept {
    if (!count) return DEC_INVALID;
    *count = 0;
    if (!value || (!input && size) || (!output && capacity)) return DEC_INVALID;
    if (capacity < size) return DEC_SMALL;
    try {
        if (value->model == 1) throw std::runtime_error{"controlled fault"};
        for (size_t i = 0; i < size; ++i) output[i] = input[i] * 2.0f;
        *count = size;
        if (callback) callback(context, *count);
        return DEC_OK;
    } catch (...) { return DEC_INTERNAL; }
}

```

[完整实验 · G8-B2 · consumer.c]

```

#include "decoder.h"
#include <stdio.h>
static void counted(void* context, size_t count) { *(size_t*)context += count; }
int main(void) {
    dec_config config = {sizeof(dec_config), 1, 0};
    decoder* handle = NULL;
    if (dec_create(&config, &handle) != DEC_OK || !handle) return 1;
    decoder* rejected = handle; /* borrowed alias used only as an output sentinel */
    config.abi_version = 2;
    if (dec_create(&config, &rejected) != DEC_INVALID || rejected) return 2;
    config.abi_version = 1;
    config.struct_size = sizeof(dec_config) - 1;
    if (dec_create(&config, &rejected) != DEC_INVALID || rejected) return 3;
    const uint8_t input[] = {1, 7, 255};
    float output[] = {-1, -1, -1};
}

```


完整实验 · G8-B2 · consumer.c · continued

```

size_t count = 99, callbacks = 0;
if (dec_decode(handle, input, 3, output, 3, &count, counted, &callbacks) ||
    count != 3 || output[0] != 2 || output[1] != 14 || output[2] != 510 ||
    callbacks != 3) return 4;
output[0] = output[1] = output[2] = -1;
if (dec_decode(handle, input, 3, output, 2, &count, counted, &callbacks) !=
    ↪ DEC_SMALL ||
    count != 0 || output[0] != -1 || output[1] != -1 || output[2] != -1 ||
    callbacks != 3) return 5;
if (dec_decode(handle, NULL, 0, NULL, 0, &count, NULL, NULL) != DEC_OK ||
    count != 0) return 6;
if (dec_decode(handle, NULL, 3, output, 3, &count, NULL, NULL) != DEC_INVALID ||
    count != 0) return 7;
dec_destroy(handle);
dec_destroy(NULL);
config.struct_size = sizeof(dec_config);
config.model = 1;
if (dec_create(&config, &handle) || !handle) return 8;
if (dec_decode(handle, input, 3, output, 3, &count, NULL, NULL) != DEC_INTERNAL
    ↪ ||
    count != 0 || output[0] != -1 || output[1] != -1 || output[2] != -1) return
    ↪ 9;
dec_destroy(handle);
puts("c-abi-contract-ok");
return 0;
}

```

运行与解释。本节开头的统一命令按 G8-B2 提取全部文件；具体编译、链接、运行及负例命令保存在 results.json。先预测结果，再用完整诊断核对，不能把任意构建失败或超时当作预期反例。

未执行路线。前面列出的扩展练习仍可用于学习，但不自动计入本批通过项。执行记录按文档检查、编译/链接/消费、性能观察、并发检测分别说明；后两类在本批不运行。

21. 边界复核协议

从边界类型开始，依次追到消费者实际行为。这里的清单服务审查，不是全部已有自动化证据。

审查层次	核心问题	证据入口
边界 / API / ABI	独立升级还是一起重编译?	支持配置与消费者身份
符号 / 调用	名称、类型、参数约定是否一致?	目标文件、诊断、平台 ABI
布局 / 版本	大小、偏移、范围如何协商?	布局观察与版本矩阵
所有权 / 分配	谁创建, 谁释放, 在哪个域?	成对入口与失败路径
错误 / 生命周期	失败输出是什么, 借用到何时?	状态测试与文字合同
并发 / 回调	可否重入, 谁等待在途工作?	同步/销毁协议
跨语言	表示及语义由哪一层恢复?	真正的异语言消费者

22. Final Gate

应该能闭卷回答：

22.1 API / ABI

1. API 与 ABI 的区别是什么？
2. 为什么 private member 变化也可能 ABI break？
3. 为什么 source-compatible 不等于 binary-compatible？

22.2 Symbol

1. C++ 为什么需要 name mangling？
2. extern "C" 真正改变什么？
3. 为什么它不能让 `std::string` 获得稳定 C ABI？

22.3 Calling Convention

1. Calling convention 至少规定哪些东西？
2. 为什么大型 return object 可能需要 hidden result pointer？

22.4 Layout

1. 为什么 class inheritance/vtable 会扩大 ABI surface？
2. `trivially_copyable` 为什么不等于 portable serialization？
3. 为什么 packing 不能解决 endianness？

22.5 Ownership

1. 为什么 library allocate / caller free 可能错误？
2. caller-provided output buffer 有什么优势？
3. opaque handle 如何表达 owning stateful object？

22.6 Error

1. 为什么 exception 不应穿过 C ABI？
2. `noexcept` 在 FFI boundary 有什么作用？

22.7 STL / Templates

1. 为什么 vector / string 会扩大 binary coupling？
2. 为什么 template implementation 经常实际编进 caller？
3. 为什么 inline implementation 会让旧 caller 保留旧逻辑？

22.8 Versioning

1. `struct_size` 有什么价值?
2. 为什么 `append-only field evolution` 比 `reorder field` 更容易兼容?
3. `Software version` 与 `ABI version` 为什么不是一回事?

22.9 Plugins

1. 为什么 `extern "C" Base* create()` 仍然不是纯 C ABI?
2. `versioned function table` 为什么适合插件?

22.10 Cross-language

1. 为什么 `Rust repr(C) / Zig C interoperability` 仍然不能取消 `ownership/lifetime contract`?
2. 为什么 `pointer + length` 是 `span/slice` 跨 FFI 的自然降级形式?

23. Final Gate · 参考答案与常见误判

23.1 API / ABI

1. API 约束源码可见名称、类型和语义；ABI 约束独立编译组件对名称、调用与表示的共同理解。相同源码语言版本不保证相同 ABI。
2. 调用者若按值构造公开类，编译时已把大小、对齐甚至内联成员访问写入机器代码。`private` 只控制源码访问，不消除布局承诺；`PImpl` 减少这一承诺，但仍保留指针成员及运行时合同。
3. 改动后重新编译可能成功，而旧机器代码仍按旧偏移读写。常见误判是拿“新头文件+新库一起构建通过”证明旧调用者兼容；那其实没有测试旧调用者。

23.2 Symbol

1. C++ 重载、命名空间和模板需要区分实体，具体 ABI 常用名字改编编码必要信息。并非所有函数都在优化后的制品中保留独立符号。
2. `extern "C"` 指定相应名称/函数类型的语言链接属性；实现按平台 C 约定处理，不是统一跨平台符号拼写，也不是自动导出。
3. `std::string` 的布局、分配、异常和生命周期不因链接名改变。G8-B1 的链接负例只说明名字约定不匹配，不证明任意 C 链接签名都适合 C 调用。

23.3 Calling Convention

1. 参数和返回值位置、寄存器保存责任、栈对齐、聚合体分类及隐藏参数都可能属于约定。不能仅检查双方函数名一致。
2. 返回大对象时，某些 ABI 让调用者提供结果存储地址；是否采用该方式取决于类型和目标 ABI。它不是语言层 RVO 的同义词，优化后的汇编也不反向定义语言保证。

23.4 Layout

1. 继承会引入子对象、指针调整、可能的虚表与运行时类型信息。虚函数槽位和基类偏移改变可能破坏旧调用者，不能只审查显式字段。
2. 平凡可复制只提供有条件的表示复制保证，不能把指针、填充、端序和目标布局固定成长期文件格式。
3. `packing` 影响布局/对齐，端序决定多字节数值的字节次序。二者独立；压紧布局也不解决任意字节地址的生命周期和合法访问。

23.5 Ownership

1. 创建方可能使用不同运行时或自定义分配器，调用者的 `free/delete` 未必匹配。让同一分配域提供 `destroy`，或明确协商分配器。
2. 调用者拥有输出缓冲区可以避免跨域转移，但还必须定义容量不足、部分输出、重叠和计数规则。指针+长度不是内存安全认证。

3. `create/destroy` 及文字合同赋予句柄所有权，不透明类型本身只隐藏布局。G8-B2 检查成功非空、失败清空和同步借用；不验证悬挂地址或重复销毁。

23.6 Error

1. C 调用者不理解任意 C++ 异常运行时；跨语言接口应把异常止于边界，输出可解释的状态。统一受控 C++ 运行时下跨库异常是另一种可选合同，不能和纯 C 接口混同。
2. `noexcept` 限制传播而不自动 `catch`；异常离开会终止。声明/定义必须匹配，转换路径也不能再次失败。G8-B2 的 `model=1` 只注入受控异常，不代表实际分配失败已测试。

23.7 STL / Templates

1. `vector/string` 暴露标准库布局、分配器和运行时要求；同一产品统一构建可接受，不等于第三方 SDK 长期稳定。
2. 模板定义通常在调用者可达处实例化，显式实例化是受控例外。3. `inline` 逻辑也可能编入旧调用者；只换共享库无法更新这些机器指令。常见误判是把头文件实现当成库端可随时替换的细节。

23.8 Versioning

1. `struct_size` 让库知道调用者声明的可读/可写范围，但不能验证任意指针或谎报大小；先检查公共前缀，再读可选字段。
2. 尾部追加可能保留已有偏移，但仍需检查传递方式、大小、对齐、默认值及数组步长，不自动兼容。3. 软件版本记录产品演进，ABI 版本记录特定二进制合同；二者可以独立变化。

G8-B2 仅拒绝未知版本与过小声明尺寸，没有执行 v1 调用者对 v2 实现的兼容实验。把版本检查成功写成全面 ABI 演进验收，是最需要避免的误判。

23.9 Plugins

1. 返回 `Base*` 仍暴露 C++ 多态对象、析构、类型信息和运行时，C 名称仅稳定了入口名字。
2. 版本化函数表让字段顺序、可选能力和协商显式化，但宿主仍要核对函数类型和可用范围。表和函数指针在插件卸载后不可继续使用；必须先停止调用、等待回调并销毁对象。动态加载成功不证明卸载协议安全。

23.10 Cross-language

1. Rust `repr(C)` 或 Zig C 互操作约定表示/调用的一部分，不替外部原始指针证明有效期、别名规则、线程访问或分配域。各语言的安全包装必须从同一文字合同恢复这些限制。
2. `pointer + length` 显式表达连续缓冲区地址和元素数，便于不同语言重建视图；还需规定元素单位、空范围、可写性及借用时长。本批 C11 消费者是跨语言的有限实例，不能替代未运行的 Rust/Zig FFI 检查。

24. 工程原则与全链回查

如果半年以后只能记十五条：

1. API 是 source-level contract, ABI 是 independently compiled binaries 之间的 machine-level contract。
2. 相同 C++ 标准版本不自动意味着相同 binary ABI; mangling、calling convention、layout 和 runtime 属于 implementation/platform ABI。
3. C++ public class 的 private representation 也可能进入 ABI, 因为 caller 需要知道 sizeof、alignment 和 layout。
4. PImpl 的核心价值之一是把 private representation 从 public binary layout 中移除。
5. extern "C" 主要建立 C language linkage; 它不会让 C++ object、STL、exception 突然变成 C ABI。
6. 稳定跨语言/native SDK boundary 的默认最小公分母是: C linkage + opaque handles + primitive scalars + pointer/length + explicit ownership。
7. 谁 allocate, 通常就应该由同一 allocation domain 负责 free; allocator ownership 是 ABI contract。
8. C++ exception 不应穿过 C ABI; boundary 应将异常转换为显式 error representation。
9. 不要把 in-memory C++ layout 当 wire/storage schema; packing、endianness、versioning、lifetime 是不同问题。
10. STL types 在同一受控 toolchain 内可以很好用, 但暴露它们会显著扩大 stable binary ABI coupling。
11. Opaque Handle 能把内部 class layout、allocator、container 与实现演进隐藏在 stable ABI 后面。
12. Plugin 边界优先使用 versioned C function table, 而不是依赖 compiler-controlled C++ vtable/RTTI ABI。
13. ABI versioning 必须显式考虑 struct size、field order、enum values、symbol set 与 semantic behavior。
14. Ownership、lifetime、thread safety、callback threading 和 shutdown 虽然不全是 bit-level ABI, 却都是完整 binary interface contract 的一部分。
15. 优秀 native library architecture 通常是 Rich C++ Internals + Small Stable Binary Boundary + Idiomatic Language Wrappers。

从 G0 的符号/重定位/加载到 G1 ~ G7 的对象、所有权和同步, 最终都落实到接口合同。内部返回 `std::vector<std::shared_ptr<Foo>>` 可以合理; 若作为第三方 SDK 边界, 则同时承诺容器、标准库、引用计数、分配域、对象表示与线程行为。

25. 参考资料与验证边界

语言规则采用 N4950 链接属性、对象表示 和 异常终止。Itanium C++ ABI 是具体 ABI 资料，不是 C++ 标准或所有平台通用实现；Apple run-path 文档 用于理解本次 @rpath/@loader_path 配置。

跨语言表示参见 Rust 1.85.0 Reference 和 Zig 0.15.2 C 互操作文档。这些链接支持概念回查，不代表运行了对应编译器；版本线与本轮读取身份见全书引用约定。

历史验证与限制仍保留编译、链接与消费的定向范围。当前编辑修订及源码字节对应见全书一致性记录，不将旧结果冒充重跑。从本章的二进制合同进入 G9 §7 安装与导出：接口可描述，还要让独立消费者取得正确的依赖与配置。

G9 · 构建、打包与原生生态

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。完整 CMake 实验声明最低 3.28，实测版本另记；最低要求不是各版本均已测试。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G8 · 全系列导航 · 下一章：G10

阅读入口

本章的问题是：库在自己的仓库能构建，为什么安装后另一项目仍不能使用？先读 §1 ~ 7 建立 target、配置与包的模型，以 §18 的完整实验走通安装消费；再按实际问题阅读模块、工具、交叉编译和构建成本。后三类不是未执行就自动通过的功能清单。

首次出现的核心术语在主线中解释；代码标为机制片段时不承诺独立编译。只有完整实验标记纳入执行器。原稿的编号主题通过 g9-topic-N 锚点映射到对应主题组；新章节按工程问题组织，不再逐项复制数十个 Part。原稿的 Complete / Frozen 不沿用为技术验收。

- 1. 构建图与三棵目录树
- 2. Target 与依赖传播
- 3. 产物类型、语言要求与文件集
- 4. 构建阶段与生成器
- 5. 预设、工具链与交叉编译
- 6. 依赖发现与供应策略
- 7. 安装、导出与包配置
- 8. 代码生成与模块依赖
- 9. 构建优化、告警与检测配置
- 10. 开发工具、测试与加载路径
- 11. 项目组织与安装库示例
- 12. 构建反模式
- 13. 可复现性、矩阵与构建成本
- 14. 工程配置的五层职责
- 15. 实践路线与验证选择
- 16. 常见误判

- 17. C++、Zig 与 Rust 生态对照
- 18. 完整实验：安装消费与生成依赖
- 19. 构建复核协议
- 20. Final Gate
- 21. Final Gate · 参考答案与常见误判
- 22. 工程原则与全链回查
- 23. 参考资料与验证边界

1. 构建图与三棵目录树

1.1 CMake、后端与编译器的职责

CMake 配置项目并生成后端构建描述，不是 C++ 编译器，也不是完整包管理器。Ninja、Make 或 IDE 后端执行依赖图，编译器驱动编译、汇编和链接工具。CMakeLists 描述目标、输入、依赖、属性、使用要求及安装规则，而非逐行复述 shell 命令。

产物图中的边表示生成依赖或使用关系，不等于字节包含关系。静态库依赖另一静态库时，普通 archive 不自动嵌入依赖对象；最终链接仍须解析这些符号。G9-P1 让 core PRIVATE 依赖 helper，由消费者验证 CMake 是否保留必要链接关系。

1.2 源码、构建和安装是三种视图

源码树保存维护入口，包括公开头文件、实现、测试、配置模板与生成输入。构建树保存缓存、后端规则、编译数据库、对象和产物，应与源码分开，并可由记录的输入重新生成。清理前要核对具体路径及其中是否含手工材料，不把宽泛递归删除作为通用教程命令。

安装树才是另一项目得到的接口：头文件、库、可执行工具、包配置与导入目标。仓库内构建成功可能依赖源码路径、缓存或全局设置；它不证明安装包完整。学习闭环应是 `configure` → `build` → `test` → 临时安装 → 独立 `find_package` → `link` → `run`。迁移前缀和隐藏原生产者路径，可以暴露一部分偶然依赖。

2. Target 与依赖传播

2.1 Target 表达需求的归属

构建目标 (target) 是现代 CMake 的核心对象：可包含产物、源码、编译定义/选项、语言要求、链接依赖、可见性、PIC 与头文件/模块集合。使用要求 (usage requirements) 描述消费者编译、链接或使用该目标需要什么，不只是一条 linker 参数。

例如 `decoder` 的公开头文件引用 `fmt` 类型，消费者也必须理解它；该依赖应进入公开接口。若 `fmt` 仅用于实现文件，编译接口通常可以保持私有。先从 C++ 接口事实推导 `scope`，再检查实际链接需求，不靠“这是公开库”猜 `PUBLIC`。

2.2 PRIVATE、PUBLIC 与 INTERFACE

PRIVATE 给目标自身使用；PUBLIC 同时给自身和消费者；INTERFACE 只描述消费者的需求。纯头文件库常用 INTERFACE 表达 include、特性和依赖，但三者都不是发布权限标记。

静态目标没有完成最终链接。PRIVATE bar 不会把 bar 的普通编译要求传播给消费者，却仍可能保留仅链接依赖，常见表达为 LINK_ONLY。包导出可能需要包含 bar 或在 Config 中找到它，不能因为 PRIVATE 就删掉。G9-P1 同时检查 helper 宏对 core 可见、对消费者不可见，以及最终链接确实解析 helper。链接传播规则。

2.3 将配置附着到最小负责目标

全局 include_directories、CMAKE_CXX_FLAGS 字符串拼接和 link_directories 容易制造目录级隐式状态，污染第三方依赖和消费者。优先使用 target_include_directories、target_compile_options、target_compile_definitions 和实际/导入目标。

目录级配置并非语言上非法；问题是它往往不能清楚说明需求归属。已有 Foo::Foo 时优先消费目标，避免退回裸库名搜索；绝对库路径也不是完整依赖模型，因为它未描述头文件、定义、标准模式与传递库。

机制示意。不要把 CMake 理解成“设置一堆变量”；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
add_library(vehicle_core
    src/core.cpp
)

add_executable(vehicle_app
    src/main.cpp
)

target_link_libraries(vehicle_app
    PRIVATE
        vehicle_core
)
```

3. 产物类型、语言要求与文件集

3.1 选择正确产物类型

STATIC 生成对象归档；SHARED 生成可正常链接并在运行时加载的共享库；MODULE 通常供动态加载，不作为普通 target_link_libraries 依赖。OBJECT 复用已编译对象而不先归档，适用于特定组合需求，不宜机械替代普通库。

INTERFACE 目标通常不产生普通库二进制，但可携带使用要求、头文件和导出信息；带源或文件集时仍可能出现在构建图。ALIAS 提供命名空间形式的本地别名，帮助构建树与安装树消费名称接近；别名自身不是安装产物，实际目标导出名称要保持一致。

3.2 语言模式与头文件集合

`target_compile_features(core PUBLIC cxx_std_23)` 让 CMake 选择相应编译器的标准模式参数；它声明模式下限，不检查所有 C++23 库设施已实现。公开头文件使用 `expected` 等类型时，消费者也需要语言和库支持；Concepts 本身是 C++20，不能据“现代特性”统称为 C++23 要求。

`CXX_STANDARD_REQUIRED` 可在采用标准属性时防止模式降级；`CXX_EXTENSIONS NO` 请求不使用对应扩展模式，但不是禁用所有实现扩展的静态审计。`HEADERS` 文件集明确公开头文件成员、基目录和安装关系，比与目标无关的整目录安装更容易维护。

3.3 生成器表达式与两种接口

生成器表达式（generator expression）在生成及相关构建上下文求值，不是普通 `configure-time` 变量。含空格、分号或换行的表达式须按规则作为完整参数引用；为了排版把未引用表达式拆成多段，会改变 CMake 参数解析。

`BUILD_INTERFACE` 可使用源码/构建路径，`INSTALL_INTERFACE` 应相对安装前缀表达自身公开目录，不固化依赖库在开发机的位置。本批完整实例用 `HEADERS` 文件集维护路径关系。复杂条件可以先组织变量，但不应把每一条普通配置逻辑都写成嵌套表达式迷宫。[生成器表达式指南](#)。

机制示意。Header 不只是“手工 install 一个目录”；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
target_sources(core
    PUBLIC
        FILE_SET HEADERS
        BASE_DIRS
            include
        FILES
            include/project/core.hpp
)
```

4. 构建阶段与生成器

4.1 先定位失败阶段

Configure 处理 CMake 语言、选项、依赖发现、特性探测和目标关系；Generate 将其映射成 Ninja、Make 或 IDE 规则；Build 执行编译、归档、链接和自定义命令。后面还有 `test`、`install` 与实际运行加载，各自有不同前提。

`find_package` 找不到配置是配置/发现问题；`undefined symbol` 通常是链接问题；运行时缺少共享库则已经进入加载器。不要在每种错误下都先改 `include path`，应该沿 G0 的阶段模型追到对应输入和工具。

4.2 单配置与多配置

普通 Ninja、Unix Makefiles 通常一个构建树对应一个配置，通过 `CMAKE_BUILD_TYPE` 选择。Visual Studio、Xcode 和 Ninja Multi-Config 可在同一构建树中包含多个配置，并在 `build/test/install` 时选择。

因此到处使用 `if(CMAKE_BUILD_TYPE STREQUAL Debug)` 不能覆盖全部生成器。配置相关要求采用适当的 `target` 属性、`CONFIG` 表达式和命令参数；预设记录支持的调用组合。本批实际运行 Ninja 单配置，不将对多配置模型的说明冒充实测覆盖。

5. 预设、工具链与交叉编译

5.1 预设记录 workflow，工具链记录目标环境

`CMakePresets.json` 宜进入版本控制，描述团队支持的 `configure/build/test` 等流程；

`CMakeUserPresets.json` 承载个人路径和机器差异，通常不共享。`schema` 版本按项目最低 CMake 版本选择，而不是每次复制官方示例中最大的数字。

预设名宜表达 `dev`、`san`、`release` 的用途。`dev` 关注增量构建、调试信息、告警和编辑器；`san` 关注诊断，不是性能基线；`release` 关注生产代码生成，但仍可保留符号以便 `profile`。`LTO`、`assert`、可见性和 `ABI` 选项都应是明确策略，不把 `Release` 等同于单个 `-O3`。G9-P2 只实际运行 `dev` 预设。

5.2 工具链应在探测前确定

Toolchain file 定义编译器、目标架构/系统、`sysroot` 和交叉环境，须在编译器识别及能力探测前生效。项目是否启用示例、测试或业务特性通常放在项目/预设层，而不是全部塞进工具链。

预设回答“如何调用和配置这个项目”，工具链回答“为哪个目标环境构建”。`vcpkg` 等也可借工具链入口集成发现逻辑，但这不取消两层职责的区别；切换编译器/目标时宜使用独立构建树，避免复用不相容缓存。

5.3 交叉构建的两类产物

代码生成器必须能在构建机运行，目标库/应用则要为目标平台生成。切换编译器路径不足以处理头文件、库、`sysroot`、`find_program/find_library` 及依赖包的环境选择。

`try_compile` 可以检验目标编译，`try_run` 还需要执行目标程序；交叉构建可能需模拟器、明确预填结果或重新设计探测。不能静默用宿主程序结果替代目标事实。本批没有配置交叉工具链、执行目标程序或验证 `host/target` 双图，相关路线保留为未运行。

6. 依赖发现与供应策略

6.1 发现机制与导入目标

Module 模式寻找 FindFoo.cmake，由项目、CMake 或其他来源提供查找逻辑，可能依赖启发式路径和版本推断。Config 模式消费包提供的 FooConfig.cmake 等信息，通常更贴近自身目标、组件和依赖；它仍是可执行 CMake 代码，不是安全解析任意不可信数据。

导入目标 (imported target) 把已存在的库、include、定义、语言要求和传递依赖表达为 Foo::Foo。消费者尽量通过 find_package(... CONFIG REQUIRED) 加命名空间目标消费，而非拼 FOO_LIBRARIES/INCLUDE_DIRS。CMake 4.4 还描述 CPS 发现能力，其开关和状态须按版本核对；本批只验证传统 Config package。find_package。

6.2 获取依赖和使用依赖不是同一层

FetchContent 可获取并接入源码，使依赖进入同一次配置/构建环境，适合受控的小型源码依赖；它不是完整包管理器。大量依赖共处一个 configure universe，会产生选项冲突、policy 交互和配置成本。

Conan 2 可通过工具链与依赖配置生成器连接 CMake；CMakeDeps/CMakeConfigDeps 的具体状态要按锁定版本查阅。vcpkg manifest 表达项目依赖并通过相应集成提供发现信息。两者都不应迫使核心 CMakeLists 到处写包管理器分支；理想接口仍是目标和 find_package。本批不下载、不运行这些包管理流程。

6.3 明确四类供应来源

系统包有操作系统整合和安全更新优势，但版本随发行环境变化；包管理器提供依赖声明和二进制/源码流程，仍需锁定配置；源码接入便于统一构建，但增加共同配置压力；vendor 提供离线控制，也带来更新、仓库规模和许可维护责任。

选择时比较组织的配方、二进制缓存、交叉目标、注册表、发布和 CI 需求，不宣布 Conan/vcpkg 存在普遍赢家。同一项目可合理混用来源，但应规定边界，避免各开发机偶然安装的软件决定构建结果。

机制示意。find_package；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
find_package(fmt CONFIG REQUIRED)

target_link_libraries(app
    PRIVATE
        fmt::fmt
)
```

[机制片段 · 不承诺独立编译]

```
include_directories(${FMT_INCLUDE_DIR})
link_libraries(${FMT_LIBRARY})
```


7. 安装、导出与包配置

7.1 安装、导出、配置各解决什么

安装规则决定产物和公开头文件放在哪里；GNUInstallDirs 提供标准安装目录变量，适应 lib/lib64 等布局策略。install(TARGETS ... EXPORT ...) 把目标归入导出集合，install(EXPORT ...) 生成安装后的目标描述。命名空间名称既便于区分来源，也让不存在的目标更早报错。

包 Config 文件负责准备依赖、载入导出目标和处理组件。configure_package_config_file 帮助形成相对安装前缀的可迁移配置，但不能修复任意手写绝对路径。生产者 include/source/build 路径不应泄漏进公开安装接口；依赖应通过自身目标提供信息。

7.2 版本规则与传递依赖

write_basic_package_version_file 生成版本接受策略，SameMajorVersion 是发布者声明的兼容规则，不会自动比较 ABI、符号、布局或语义。G9-P1 实际检查支持的请求能发现包、不支持的主版本被配置诊断拒绝，不把这等同于 G8 的旧二进制升级验证。

导出目标需要外部 Foo::Foo 时，Config 通常先 include(CMakeFindDependencyMacro)、find_dependency(Foo CONFIG)，再加载自己的 targets。静态 PRIVATE 链接依赖也可能需要，不能只对 PUBLIC 依赖做发现。本批选择把 helper 一并安装导出，避免网络或外部包管理器成为实验前提。

7.3 迁移后独立消费

消费者是另一份 CMake 项目，只通过 Config 包和命名空间目标获得使用要求。实验先安装到临时前缀，再移动前缀、隐藏原生产者源码和构建目录，在新构建树配置消费者，并限制注册表及环境搜索的偶然帮助。

这一结果说明本次静态包可独立消费，不说明完全 hermetic、跨机器或逐字节可复现。编译器、SDK、系统库仍来自本机；共享库 RPATH、DLL 搜索、签名和发布事务是其他合同。包配置辅助模块。

机制示意。install(EXPORT ...); 以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
install(  
    EXPORT MyProjectTargets  
    NAMESPACE MyProject::  
    DESTINATION  
        ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject  
)
```

机制示意。Exported Target 依赖另一个 Package；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
include(CMakeFindDependencyMacro)  
find_dependency(fmt CONFIG)
```

[机制片段 · 不承诺独立编译]

```
include(
    "${CMAKE_CURRENT_LIST_DIR}/MyProjectTargets.cmake"
)
```

8. 代码生成与模块依赖

8.1 生成输入和输出必须进入图

如果输入模式变化应导致生成源码变化，`add_custom_command(OUTPUT ...)` 应声明输出、生成命令及输入依赖，必要时说明额外产物。生成文件再列入实际 `target` 的源集合，使后端知道何时运行及何时编译。

`configure_file` 适合由项目配置产生的模板；`execute_process` 在 `configure` 阶段运行，不会自动成为后续构建依赖图。不能把每次配置恰好运行过脚本当作增量依赖完整。G9-P2 修改数据输入后只 `build`，并验证新值；故意漏掉 `DEPENDS` 的变体用陈旧值暴露错误。自定义生成命令。

8.2 模块需要新的依赖顺序

传统 `include` 是文本包含；命名模块还要准备可供 `import` 的编译器模块信息。A `import` B 时，构建图必须安排 B 的相关产物先于 A 的编译，因而需要依赖扫描与工具链集成，不是把 `hpp` 改成 `cppm` 就完成迁移。

CMake 的 `CXX_MODULES` 文件集从 3.28 引入，但可用组合还涉及编译器、扫描器、生成器、安装/导出与工具支持。C++23 `import std` 的语言存在性不等于当前标准库和 CMake 集成可用。此批主验证采用 `headers`；模块路线保留但 NOT RUN，不因设置 `cxx_std_23` 自动升级状态。模块支持矩阵。

9. 构建优化、告警与检测配置

9.1 PCH、Unity、LTO 优化不同阶段

PCH 复用稳定头文件解析产物，可减少前端工作，但引入失效成本与工具链依赖；源码不能依赖 PCH 偶然提供缺失 `include`。Unity 合并翻译单元，可能减少重复解析，也可能造成匿名命名空间、`static` 名称和宏冲突。非 Unity 构建仍应正确。

LTO/IPO 保留中间表示或摘要供链接期跨 TU 优化，可能带来内联、常量传播和去虚化，也增加链接耗时、内存与诊断复杂度。用 IPO 属性前检查工具链支持，并根据目标工作负载决定，不把 `Release` 自动等同于“所有优化全开”。本批未测量这些优化。

9.2 告警是项目策略，不是默认下游合同

告警选项通常 `PRIVATE`，尤其不宜把 `-Werror` 强加给使用不同编译器的消费者。内部 CI 严格告警和第三方接口要求是两件事；也不应让目录级全局告警污染受控范围外的依赖。

接口辅助目标可以复用告警配置，但 `PRIVATE` 链入静态库仍可能影响导出图。未导出的 `project_warnings` 不会仅因名字里有 `private` 就自动消失；需设计 `build-only` 关系并检查 `export`。小型示例直接给实际目标设置 `PRIVATE` 编译选项，更容易看清归属。

9.3 动态检测要覆盖编译与最终链接

ASan、UBSan、TSan 和 coverage 插桩会改变二进制、时序和布局，应由独立 `profile` 管理，不能混入生产性能结论。通常既需要编译选项，也需要在最终链接中带上对应运行时；给静态库配置一个不会发生的普通链接步骤，并不能自动完成这一点。

项目应明确哪些目标和依赖采用插桩、哪些组合受支持，再用 `target_compile_options/target_link_options` 表达。检测器没有报错仍不是所有未定义行为或并发协议正确性的证明；G9 本批没有新增动态检测结果。

机制示意。CMake 中是 IPO Property；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
set_property(
    TARGET core
    PROPERTY
        INTERPROCEDURAL_OPTIMIZATION TRUE
)
```

10. 开发工具、测试与加载路径

10.1 开发工具消费真实构建信息

`compile_commands.json` 记录每个翻译单元实际需要的 `include`、宏、语言模式与命令。`clangd` 应消费对应配置的数据库；诊断缺头文件时先检查数据库是否过期、生成头是否存在及构建描述是否完整。CMake 的编译数据库支持也取决于生成器，并非所有 IDE 后端都相同。

`clang-tidy` 可作为 `target` 编译钩子、独立 CI 检查或编辑器诊断；成本不同，应分配置。`clang-format` 管理格式，不是 C++ 构建正确性证明，宜单独检查，避免把格式修改混入正常编译的副作用。

10.2 测试、基准和覆盖率不要混成一个结论

CTest 负责登记并调度测试，不自动决定其命题是否有效。组件测试应通过真实公开 `target` 消费，明确需要时才绕过接口做白盒测试；还要有独立安装消费路径。`dev test preset` 使调用方式固定，但“零个测试成功退出”不能冒充通过。

Benchmark 需要适当优化配置、输入和噪声控制，不基于 `Debug/ASan` 结果得出生产速度结论。Coverage 也是独立插桩配置；覆盖率数字不能替代有判别力的断言。本批只报告构建/消费与生成结果，不提供性能或并发检测证据。

10.3 可见性、PIC 与运行加载

G8 的导出策略可映射到 `CXX_VISIBILITY_PRESET`、`VISIBILITY_INLINES_HIDDEN` 和显式导出宏。PIC 使用 `POSITION_INDEPENDENT_CODE` 表达；静态对象若将进入共享库，是否需要 PIC 应在该产物组合中检查，不能只看后缀。

链接成功仍不保证运行加载正确。ELF 的 `RPATH/RUNPATH`、Mach-O 的 `install name/@rpath/@loader_path` 和 Windows 的 `DLL` 搜索属于不同平台机制。避免把开发机构建目录写死进交付产物；G8-B2 仅在本机验证相对加载，G9-P1 是静态包，不替代完整共享包迁移检查。

机制示意。Test 也是 Target Consumer；以下仅展示接口或结构，所需头文件、依赖类型及实现须另行补齐。

[机制片段 · 不承诺独立编译]

```
include(CTest)

add_executable(core_tests
    tests/core_test.cpp
)

target_link_libraries(core_tests
    PRIVATE
        MyProject::core
)

add_test(
    NAME core.unit
    COMMAND core_tests
)
```

11. 项目组织与安装库示例

11.1 项目结构服务目标归属

根 CMakeLists 管理最低版本、project、共同项目决策和子目录组织；组件负责自己的源成员、公开头、依赖和测试。`include/src/apps/tests/benchmarks` 等结构是常见手段，不是必须复制的模板。目标边界和所有权明确比目录名字更重要。

选项应表达真实可支持能力。`shared/static`、测试开关、模块、异常或引擎模式会扩大配置空间；列出选项不等于所有组合受支持。发布矩阵应明确实际承诺的集合，避免无测试的 2^N 组合。

11.2 可安装库的完整职责

完整生产者至少需要实际目标、稳定消费名称、公开文件集、语言要求、实现依赖、适当告警、安装规则、导出集合、Config/Version 配置以及测试。相关片段若缺模板、依赖目标或消费者，不应标为“完整可发布工程”。

G9-P1 给出完整文件集合，最低 CMake 声明为 3.28；实际执行工具版本另记。最低版本声明不是从 3.28 起所有版本都实测过。静态 helper 一并导出，告警直接附着实际目标，公开头文件使用 expected 验证消费者获得语言/库能力。

11.3 先定义独立消费者

消费者只写 find_package 和 target_link_libraries，不手工补生产者 include、库路径或编译宏。公开头文件需要的要求应自动传播，而实现 helper 宏不应泄漏。若发现消费者必须复制一串 README flags 才成功，应先审查 target interface。

构建树别名与导出名称保持接近，能减少两种消费方式的差异，但不能让本仓库测试代替安装后验证。完整实验在后文集中维护，避免多个不一致的 CMake 配置成为并行示例。

12. 构建反模式

12.1 隐式状态与路径反模式

把所有 flags 放 CMAKE_CXX_FLAGS、全局 include/link directory、硬编码本机库路径，都会让需求归属和消费关系变模糊。有导入目标时退回裸库名，会丢失相应使用要求；安装接口出现开发机绝对路径，会使包在原仓库之外失效。

file(GLOB) 即使使用 CONFIGURE_DEPENDS 也需要理解生成器行为和源码成员变化；公开库通常显式列源更清楚。问题不是“任何 glob 都违法”，而是不应为省几行清单牺牲可见的图。

12.2 依赖、告警和框架反模式

把所有第三方依赖都 FetchContent 进一个配置环境，会扩大 policy/选项交互和构建成本。到处写 Conan/vcpkg 分支，使供应策略侵入项目语义。全局 -Werror 则可能把不受控消费者或依赖变成项目内部告警政策的受害者。

库从未安装给独立消费者，就不能把仓库构建成功当作包正确。大量动态变量、宏和字符串技巧也不等于专业 CMake；复杂业务生成逻辑宜交给合适工具，CMake 保持可读的输入/输出/依赖关系。

13. 可复现性、矩阵与构建成本

13.1 可追踪、可再构建与位级复现

至少记录源码身份、编译器/标准库/SDK、CMake/后端、依赖版本、目标平台及选项。相同源码不保证相同二进制；manifest、lock 和 recipe revision 帮助约束输入，但路径、时间戳、环境和工具链也可能影响结果。

Hermetic 构建尝试排除偶然系统输入；发行版打包则可能明确选择系统依赖。目标应匹配交付模式，而不是把所有项目都宣布为 hermetic。本批清理部分搜索环境，只检验同机安装消费，不声称位级复现或完全隔离。

13.2 矩阵与构建成本

选择真实承诺的编译器、OS、架构、配置及 shared/static 集合，按快速 PR、主干和周期性检查分层；不要为追求全覆盖做无意义笛卡尔积。性能、安装消费者、ABI 和动态检测各有不同环境要求。

构建成本应区分 configure、clean build、实现改动的增量编译、公开头变动和链接。可用手段包括减少头依赖、前置声明、PImpl、目标划分、模块、PCH、Unity、链接器和分布式/缓存编译；先确认瓶颈，再选择工具。公开 include 的传递影响既是接口设计问题，也是构建图问题。

13.3 缓存与分发包

编译缓存依赖有效的输入键：编译器、选项、预处理内容、路径和环境等。随机宏、时间戳及无意义路径漂移可能降低命中；缓存命中也不能证明源依赖声明完整。

CPack 位于安装树到归档/安装器的分发层，不会替错误 install/export 规则补头文件或修复路径。先完成 build、test、install、独立 consumer，再讨论归档、签名及正式发布。此次不生成发行包。

14. 工程配置的五层职责

14.1 五层职责独立维护

项目语义层规定 targets、源码成员、依赖 scope、语言和安装/导出；工具链层规定编译器、架构和 sysroot；构建配置层区分 dev/san/release/coverage；依赖解析层规定 system、包管理器、FetchContent 或 vendor 来源；制品交付层规定安装前缀、包配置及分发形式。

这些层可以有明确连接，但不应揉成无法复用的巨型 CMakeLists。学习者应能够指认每个选项属于哪一层，以及它改变的是接口、构建环境还是观察方式。

15. 实践路线与验证选择

15.1 保留九条路线，明确本批执行集合

原有练习包括全局配置改 target、推导 PUBLIC/PRIVATE、可安装库、预设、交叉编译、依赖接入、代码生成、模块和构建成本。G9-P1 落实静态 PRIVATE 链接与安装消费；G9-P2 落实 dev 预设与输入变化后的重新生成。完整文件和判据在后文提供。

交叉工具链应能区分宿主生成器和目标产物；依赖练习比较不同来源但保持消费 target 稳定；模块练习关注扫描和排序；构建成本练习应保留基线与配置。上述扩展未在本批执行，不从两个已跑通流程推断其他能力通过。

16. 常见误判

16.1 模型误判

CMake 不是编译器，`target_link_libraries` 不只是 `linker` 命令，`PUBLIC` 也不是“对外发布”。只有 `.a` 路径并没有表达完整依赖；把所有 `flags` 堆成字符串不等于配置简单。`Preset` 和 `toolchain` 分工不同，包管理器不替代项目构建图，`FetchContent` 也不自动成为完整供应系统。

`Release` 不等于单个 `-O3`，模块不会因为替代文本 `include` 就自动让构建系统更简单。`clangd` 错误可能来自编译数据库，而不只是编辑器设置；交叉编译也不只是更换编译器可执行文件。

16.2 验证误判

本仓库能链接不代表安装包完整；`PCH`、`Unity` 和 `LTO` 都有特定收益与代价，不应无测量全开。更长的 `CMakeLists` 不是成熟度指标。

最危险的简化是“`build` 返回 0，所以依赖图正确”。G9-P2 的错误变体仍能成功构建并运行，只是观察到旧数据；需要明确值判据，不能用编译成功替代生成依赖验证。

17. C++、Zig 与 Rust 生态对照

17.1 不同生态组织相同责任

C++ 的原生构建生态由 CMake、其他构建系统、包管理器、CTest 和外部工具组合，历史兼容面广。Zig build 以语言/工具链的产物图接口组织 `target`、优化和模块；Cargo 更集中地组织包、依赖、`feature`、构建和测试。它们的集成程度不同，最终都需说明输入、目标和消费合同。

用 Zig 驱动内部 C++ 构建可以是合理选择，但参与第三方 CMake SDK、机器人/HPC 等生态仍需理解安装/导出与 Config 目标模型。比较不能只停在执行 `clang++` 的语法；本批也不编译 Zig/Rust 构建示例，保留概念对照而非工具能力排名。

18. 完整实验：安装消费与生成依赖

这些实验由正文提取到新临时目录，完整文件是唯一维护来源。执行器只运行已审阅的本地代码，不是安全沙箱；不修改历史 PDF、FM 或出版系统。

```
python3 c++/learning/verify_native.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

18.1 G9-P1 · 静态依赖、安装导出与迁移消费者

待验证命题。生产者静态库 `PRIVATE` 依赖 `helper`，公开传播头文件与 C++23 模式。安装后迁移前缀、隐藏原生产者源码和构建树，再以独立 `find_package` 消费。错误主版本须在配置阶段被拒绝。

范围与前提。同机静态库包，不是共享库 RPATH 测试，不是 hermetic 或逐字节可复现证明。SameMajorVersion 是声明的版本接受策略，不是自动 ABI 判断。实验不访问注册表或下载第三方依赖。

[完整实验 · G9-P1 · producer/CMakeLists.txt]

```
cmake_minimum_required(VERSION 3.28)
project(HandbookNative VERSION 1.2.0 LANGUAGES CXX)
include(GNUInstallDirs)
include(CMakePackageConfigHelpers)
include(CTest)
add_library(native_helper STATIC helper.cpp)
add_library(native_core STATIC core.cpp)
add_library(HandbookNative::core ALIAS native_core)
set_target_properties(native_core PROPERTIES EXPORT_NAME core CXX_EXTENSIONS NO)
target_sources(native_core PUBLIC FILE_SET HEADERS
    BASE_DIRS include FILES include/handbook/core.hpp)
target_compile_features(native_core PUBLIC cxx_std_23)
target_compile_definitions(native_helper INTERFACE HELPER_PRIVATE=1)
target_link_libraries(native_core PRIVATE native_helper)
target_compile_options(native_core PRIVATE
    "$<$<CXX_COMPILER_ID:Clang,AppleClang,GNU>:-Wall;-Wextra>")
if(BUILD_TESTING)
    add_executable(native_test test.cpp)
    target_link_libraries(native_test PRIVATE HandbookNative::core)
    add_test(NAME native.value COMMAND native_test)
endif()
install(TARGETS native_core native_helper EXPORT HandbookNativeTargets
    ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
    FILE_SET HEADERS DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})
install(EXPORT HandbookNativeTargets NAMESPACE HandbookNative::
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/HandbookNative)
configure_package_config_file(HandbookNativeConfig.cmake.in
    ${CMAKE_CURRENT_BINARY_DIR}/HandbookNativeConfig.cmake
    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/HandbookNative)
write_basic_package_version_file(
    ${CMAKE_CURRENT_BINARY_DIR}/HandbookNativeConfigVersion.cmake
    VERSION ${PROJECT_VERSION} COMPATIBILITY SameMajorVersion)
install(FILES ${CMAKE_CURRENT_BINARY_DIR}/HandbookNativeConfig.cmake
    ${CMAKE_CURRENT_BINARY_DIR}/HandbookNativeConfigVersion.cmake
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/HandbookNative)
```

[完整实验 · G9-P1 · producer/HandbookNativeConfig.cmake.in]

```
@PACKAGE_INIT@
include("${CMAKE_CURRENT_LIST_DIR}/HandbookNativeTargets.cmake")
check_required_components(HandbookNative)
```

[完整实验 · G9-P1 · producer/include/handbook/core.hpp]

```
#pragma once
#include <expected>
namespace handbook { std::expected<int, int> answer(); }
```

[完整实验 · G9-P1 · producer/helper.cpp]

```
int helper_answer() { return 42; }
```

[完整实验 · G9-P1 · producer/core.cpp]

```
#include <handbook/core.hpp>
#ifndef HELPER_PRIVATE
#error "private helper requirements missing from producer"
#endif
int helper_answer();
std::expected<int, int> handbook::answer() { return helper_answer(); }
```

[完整实验 · G9-P1 · producer/test.cpp]

```
#include <handbook/core.hpp>
#ifdef HELPER_PRIVATE
#error "private helper compile requirement leaked to consumer"
#endif
int main() {
    const auto result = handbook::answer();
    return result && *result == 42 ? 0 : 1;
}
```

[完整实验 · G9-P1 · consumer/CMakeLists.txt]

```
cmake_minimum_required(VERSION 3.28)
project(IndependentConsumer LANGUAGES CXX)
include(CTest)
set(REQUESTED_VERSION 1 CACHE STRING "Requested package version")
find_package(HandbookNative ${REQUESTED_VERSION} CONFIG REQUIRED)
add_executable(consume main.cpp)
target_link_libraries(consume PRIVATE HandbookNative::core)
add_test(NAME consumer.value COMMAND consume)
```

[完整实验 · G9-P1 · consumer/main.cpp]

```
#include <handbook/core.hpp>
#ifdef HELPER_PRIVATE
#error "private helper compile requirement leaked to installed consumer"
#endif
int main() {
    const auto value = handbook::answer();
    return value && *value == 42 ? 0 : 1;
}
```

运行与解释。本节开头的统一命令按 G9-P1 提取全部文件；具体编译、链接、运行及负例命令保存在 results.json。先预测结果，再用完整诊断核对，不能把任意构建失败或超时当作预期反例。

18.2 G9-P2 · 预设与代码生成依赖的失效检查

待验证命题。通过 dev preset 配置、构建和测试；修改值文件后，不重新 configure，build 必须重新生成并编译。故意漏掉输入 DEPENDS 的错误变体编译成功，但旧值必须被观察判据拒绝。

范围与前提。只检查此生成图的数据依赖与可见结果。没有用耗时阈值证明增量构建快，也没有启用 PCH、Unity、LTO、模块、交叉编译或 sanitizer。

[完整实验 · G9-P2 · CMakeLists.txt]

```
cmake_minimum_required(VERSION 3.28)
project(GeneratedValue LANGUAGES CXX)
include(CTest)
set(generated "${CMAKE_CURRENT_BINARY_DIR}/generated.hpp")
add_custom_command(OUTPUT "${generated}"
    COMMAND "${CMAKE_COMMAND}"
        "-DINPUT=${CMAKE_CURRENT_SOURCE_DIR}/value.txt"
        "-DOUTPUT=${generated}"
        "-P "${CMAKE_CURRENT_SOURCE_DIR}/generate.cmake"
    DEPENDS value.txt generate.cmake
    VERBATIM)
add_executable(generated_value main.cpp "${generated}")
target_include_directories(generated_value PRIVATE "${CMAKE_CURRENT_BINARY_DIR}")
target_compile_features(generated_value PRIVATE cxx_std_23)
add_test(NAME generated.initial COMMAND generated_value 7)
```

[完整实验 · G9-P2 · generate.cmake]

```
file(READ "${INPUT}" value)
string(STRIP "${value}" value)
if(NOT value MATCHES "^[0-9]+$")
    message(FATAL_ERROR "expected nonnegative integer")
endif()
file(WRITE "${OUTPUT}" "#pragma once\ninline constexpr int generated_value =
    ␣ ${value};\n")
```

[完整实验 · G9-P2 · value.txt]

7

[完整实验 · G9-P2 · main.cpp]

```
#include "generated.hpp"
#include <charconv>
#include <cstring>
#include <iostream>
int main(int argc, char** argv) {
    if (argc != 2) return 2;
    int expected = 0;
    const auto end = argv[1] + std::strlen(argv[1]);
    const auto parsed = std::from_chars(argv[1], end, expected);
    if (parsed.ec != std::errc{} || parsed.ptr != end) return 3;
    std::cout << generated_value << '\n';
    return generated_value == expected ? 0 : 4;
}
```

[完整实验 · G9-P2 · CMakePresets.json]

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/dev",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_EXPORT_COMPILE_COMMANDS": true
      }
    }
  ],
  "buildPresets": [
    {
      "name": "dev",
      "configurePreset": "dev"
    }
  ],
  "testPresets": [
    {
      "name": "dev",
      "configurePreset": "dev",
      "output": {
        "outputOnFailure": true
      }
    }
  ]
}
```

运行与解释。本节开头的统一命令按 G9-P2 提取全部文件；具体编译、链接、运行及负例命令保存在 results.json。先预测结果，再用完整诊断核对，不能把任意构建失败或超时当作预期反例。

未执行路线。前面列出的扩展练习仍可用于学习，但不自动计入本批通过项。执行记录按文档检查、编译/链接/消费、性能观察、并发检测分别说明；后两类在本批不运行。

19. 构建复核协议

面对一个 C++ build system，按这个顺序审查。

表 1 · 字段记录视图（列名与原单元格逐项对应）

表 1 · 记录 1 · Target Graph	
Layer	Target Graph
关键问题	实际 artifacts 是什么？

表 1 · 记录 2 · Dependencies	
Layer	Dependencies
关键问题	每条 edge 为什么存在？

表 1 · 记录 3 · Usage Requirements	
Layer	Usage Requirements
关键问题	PUBLIC / PRIVATE / INTERFACE 是否与 C++ API 一致？

表 1 · 记录 4 · Toolchain	
Layer	Toolchain
关键问题	compiler / target platform 从哪里定义？

表 1 · 记录 5 · Profiles	
Layer	Profiles
关键问题	dev/san/release 是否清晰分开？

表 1 · 记录 6 · Dependency Resolution	
Layer	Dependency Resolution
关键问题	package 来源是否明确？

表 1 · 记录 7 · Reproducibility	
Layer	Reproducibility
关键问题	versions/toolchains 是否可追踪？

表 1 · 记录 8 · Generated Code

Layer	Generated Code
关键问题	inputs/outputs/dependencies 是否进入 graph?

表 1 · 记录 9 · Tests

Layer	Tests
关键问题	是否测试真实 targets 与 install tree?

表 1 · 记录 10 · Install

Layer	Install
关键问题	artifact/header layout 是否正确?

表 1 · 记录 11 · Export

Layer	Export
关键问题	external consumer 能否只通过 namespaced target 使用?

表 1 · 记录 12 · ABI

Layer	ABI
关键问题	symbol visibility/shared/static policy 是否与 G8 一致?

表 1 · 记录 13 · Cross Compile

Layer	Cross Compile
关键问题	host/target artifacts 是否分离?

表 1 · 记录 14 · DX

Layer	DX
关键问题	clangd/compile database/incremental build 是否健康?

表 1 · 记录 15 · Performance

Layer	Performance
关键问题	build optimizations 是否基于真实数据?

20. Final Gate

完成 G9 后，应能闭卷回答：

20.1 Build Model

1. CMake、Ninja、Clang 分别做什么？
2. 什么是 artifact graph？
3. Source/Build/Install Tree 为什么必须区分？

20.2 Targets

1. Target 为什么是 modern CMake 核心？
2. Target property 与 usage requirement 区别是什么？
3. Interface Library 为什么可以没有 binary artifact？

20.3 Scopes

1. PRIVATE 的准确含义是什么？
2. PUBLIC 为什么等于 implementation + interface requirement？
3. INTERFACE 什么场景最自然？
4. 如何从 public C++ header 推导 dependency scope？

20.4 Language

1. 为什么 `cxx_std_23` 比裸 `-std=c++23` 更好？
2. public header 使用 C++23 feature 时为什么 standard requirement 可能需要传播？

20.5 Configuration

1. Preset 与 Toolchain File 区别是什么？
2. Single-config 与 Multi-config generator 有什么区别？
3. 为什么不应该到处依赖 `CMAKE_BUILD_TYPE`？

20.6 Dependencies

1. Module-mode `find_package()` 与 Config-mode 区别？
2. Imported Target 为什么比 `FOO_LIBRARIES` 更成熟？
3. FetchContent 为什么不是 package manager？
4. Conan/vcpkg 为什么最好不要污染 project target logic？

20.7 Packaging

1. Build 成功为什么不代表 library package 正确?
2. `install(EXPORT ...)` 解决什么?
3. `<Package>Config.cmake` 有什么作用?
4. 为什么 installed package 必须 relocatable?
5. `find_dependency()` 为什么对 PUBLIC dependency 重要?

20.8 Advanced Build

1. PCH 优化的是什么?
2. Unity Build 的风险是什么?
3. LTO 为什么是跨 TU optimization?
4. Modules 为什么要求 build-system dependency scanning?

20.9 Tooling

1. 为什么 clangd 要依赖 `compile_commands.json`?
2. 为什么 sanitizer/coverage 应该独立 profile?
3. 为什么 benchmark 不应该基于 Debug/ASan build?

20.10 Cross Compilation

1. 什么是 Host Tool 与 Target Artifact?
2. `try_run()` 为什么在 cross build 中危险?
3. Toolchain file 为什么必须很早读取?

20.11 Architecture

1. 为什么 CMakeLists 越长越复杂并不意味着工程越成熟?
2. Modern CMake 的最终目标是什么?

21. Final Gate · 参考答案与常见误判

21.1 Build Model

1. CMake 配置并生成构建描述，Ninja 执行依赖图，Clang 驱动编译/链接工具。阶段出错应分别检查配置发现、编译语义、链接符号和运行加载。
2. 产物图连接输入、派生产物及依赖；target 还携带使用要求。静态库依赖边不表示把依赖库字节塞进自己的 archive。
3. 源码树是维护入口，构建树是配置相关派生状态，安装树是消费者视图。G9-P1 隐藏前两者仍成功，才对后者独立消费提供证据。

21.2 Targets

1. Target 把源码成员、产物、编译配置和依赖关系放在有归属的对象上，减少全局隐式状态。
2. 自身属性参与本目标构建；INTERFACE 属性描述消费者需要什么。PUBLIC 同时设置两侧，不表示“库已经公开发布”。
3. 接口库可只承载使用要求而没有普通库二进制。若含文件集或自定义生成关系，仍可能参与构建图；没有 archive 不等于没有语义。

21.3 Scopes

1. PRIVATE 给自身使用，不把编译使用要求作为普通公开接口；静态目标仍可能向最终链接保留依赖。G9-P1 的 helper 宏对 core 可见、对消费者不可见，但 helper 符号必须参与最终链接。
2. PUBLIC 同时描述自身和消费侧要求。3. INTERFACE 只描述消费者要求，适合纯头文件接口或共享的接口策略。
3. 先独立编译公开头文件：它需要哪些类型、宏和头文件？再看库的未解析符号需要什么。仅据 .cpp 使用就断言安装包无此依赖，会遗漏静态链接边。

21.4 Language

1. cxx_std_23 让 CMake 选择编译器对应的标准模式参数，而不是把某家编译器选项写死。它只声明模式下限，不能保证标准库所有 C++23 组件齐全。
2. 消费者要编译公开头文件，所以语言和库要求也属于接口。G9-P1 用 expected 真实编译消费端；未据此宣称全部 C++23 实现已验证。

21.5 Configuration

1. Preset 描述项目如何配置/构建/测试，toolchain 描述编译器、目标平台和 sysroot；它们可引用彼此但职责不同。
2. 单配置生成器通常每个构建树选一个 CMAKE_BUILD_TYPE，多配置生成器在构建等阶段选配置。3. 因而硬编码 if(CMAKE_BUILD_TYPE STREQUAL Debug) 不能覆盖多配置行为；采用恰当的配置表达式和命令参数。

G9-P2 只运行 Ninja 单配置 dev preset，不把示例中的 san/release 建议算成实际执行。

21.6 Dependencies

1. Module 模式寻找 FindFoo.cmake，常依赖外部查找逻辑；Config 模式由包提供导入信息。两者都不是版本锁定策略。
2. 导入目标还能携带头文件、定义和传递依赖，不仅是一个路径列表。3. FetchContent 负责获取/接入源码，不自动提供完整二进制包管理。
3. Conan/vcpkg 应尽量在包获取与工具链层工作，使核心 target 逻辑仍通过 find_package 消费。常见误判是以为“使用了包管理器”就自动得到 hermetic 或逐字节复现。

21.7 Packaging

1. 本仓库构建可能靠源码 include、缓存、全局路径或已有依赖偶然成功；安装缺头文件、缺目标和硬编码路径要由独立消费者发现。
2. install(EXPORT) 生成安装后的目标描述。3. Config 负责加载目标、准备依赖和处理组件；Version 文件采用声明的版本接受策略。
3. 可迁移包尽量相对安装前缀定位自身；具体交付也可有明确固定路径策略，不能隐含开发机位置。5. PUBLIC 依赖通常要 find_dependency；静态 PRIVATE 链接依赖同样可能需要，不能只检查公开头文件。

G9-P1 验证同机静态包迁移及错误主版本拒绝，没有验证动态库加载路径、签名、发布事务或所有消费者配置。

21.8 Advanced Build

1. PCH 复用稳定头文件的解析产物，不能替代显式 include。2. Unity 合并翻译单元可能引入匿名命名空间、宏和静态名称碰撞；正常非 Unity 构建仍须成立。
2. LTO 让链接阶段看到更多跨翻译单元中间信息，效果和成本需测量。4. 模块导入先后及编译模块产物依赖必须进入构建图，改后缀不能完成迁移。本批没有执行上述优化或模块实验。

21.9 Tooling

1. clangd 需要实际编译命令的头文件路径、宏和模式；数据库过期或缺生成头文件会使编辑器诊断偏离真实构建。
2. Sanitizer/coverage 改变插桩与运行环境，应有独立配置和证据类别。3. Debug/ASan 不代表生产代码生成及时间成本，不能据其微基准得出发布性能结论。G9-P2 的命令日志与编译数据库是构建证据，不是性能测试。

21.10 Cross Compilation

1. 代码生成器运行于构建机，目标库运行于目标平台；必须区分两类工具和依赖。2. `try_run` 需要执行目标程序，交叉构建可能只能通过模拟器或预填结果解决，不能静默用宿主结果代替。
2. `toolchain` 必须在编译器识别与特性探测前影响目标环境。仅换编译器路径而继续使用宿主头文件/库，是典型的混合配置错误。本批交叉编译为 NOT RUN。

21.11 Architecture

1. 构建成熟度来自依赖和支持矩阵清楚，而非函数、宏与选项数量。每个新增选项都增加需要支持的配置集合。
2. 目标是准确表达产物、依赖和使用要求，让工具执行可检查的构建/交付流程。G9-P2 故意漏掉输入 `DEPENDS` 后仍能“构建成功”，但消费者看到旧值；因此成功退出并不等于依赖图正确。

22. 工程原则与全链回查

如果半年以后只保留十五条：

1. Build system 的核心是 Artifact Dependency Graph，而不是 shell command 集合。
2. Modern CMake 的核心 abstraction 是 Target: artifact + build properties + transitive usage requirements。
3. PRIVATE / PUBLIC / INTERFACE 应从 C++ source interface dependency 推导，而不是靠经验背诵。
4. Build requirement 应附着到最窄的 owning target，避免 global include paths、flags 和 linker state。
5. Source Tree、Build Tree、Install Tree 是不同 representations；一个库必须测试真正的 install-consumer path。
6. Preset 描述项目支持的 build workflow，Toolchain 描述 compiler/target environment；两者不能混。
7. Dependency package 应尽量以 Imported Target 的形式进入 CMake graph，而不是裸路径和全局变量。
8. CMake、Conan/vcpkg、Ninja 分别解决 build graph、package resolution、build execution 等不同问题，不应混为一层。
9. 一个可发布 library 的完成条件不仅是 build 成功，而是 install/export/config/version/consumer 全链路成立。
10. Generated code 也必须成为 build graph 中具有明确 input/output/dependency 的 artifact。
11. C++ Modules 需要 build system 参与 dependency scanning；语言 feature availability 不等于整个 compiler/stdlib/build stack 都已成熟。
12. PCH、Unity、LTO 都是特定阶段的 optimization，不是“现代 C++ 必开选项”。
13. Cross compilation 是 Host Universe 与 Target Universe 的分离，不只是换 compiler path。

14. Developer tooling 应消费真实 build truth: clangd 来自 compile database, 测试来自真实 targets, package tests 来自 install tree。
15. 优秀 CMake 的目标不是展示 CMake 技巧, 而是用最少的隐式状态准确表达 C++ artifact architecture。

一条 `target_link_libraries(app PUBLIC decoder)` 不只是链接语法: 先检查源码接口, 再追到使用要求、安装包依赖、ABI 配置和消费者工具链。构建描述是架构的可执行表达; 不是 CMake 越复杂, 工程就越成熟。下一章 [G10 §10](#) 直接复用安装消费模型, 将本章的交付合同与 [G7](#) 的关闭协议组合, 不重新讲一遍 CMake 入门。

23. 参考资料与验证边界

CMake 规则引用固定的 4.4 文档线: 构建模型、链接传播、生成器表达式、包配置、生成命令、预设和模块支持。工具版本和源码摘要以本批记录为准, 不使用“最新稳定版”作为永久事实。

包管理器为范围说明, 参见 [Conan](#) 与 [vcpkg manifest](#)。本批不运行包管理器, 不验证其缓存或依赖复现性。

历史验证与限制仍保留编译、链接与消费的定向范围。当前编辑修订及源码字节对应见全书一致性记录, 不将旧结果冒充重跑。

G10 · 系统运行时工程

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。Systems Runtime Engineering；前置为 G0 ~ G9，组合运行时协议、安装消费与测量。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G9 · 全系列导航 · 下一章：G11

阅读入口

本章把 G0 ~ G9 组合成可执行的工程案例。先读 §1 ~ 7 写出不变量与关闭顺序，再运行 §18；性能分析回查 §8、§12 ~ 13，安装边界看 §9 ~ 10。

原稿的 Complete/Frozen 是历史编辑标记，不沿用为技术验收。编号主题以 g10-topic-N 映射到相应主题组，保留技术去向；章节不再逐个复制原有 Part。完整实验与机制片段有可见身份，Gate 答案是普通章节。

- 1. 项目合同：从语言机制到有界数据流
- 2. 不变量与生命周期状态
- 3. 载荷、借用与工作线程私有存储
- 4. 有界队列的状态与线性化点
- 5. 工作线程、Sink 与 happens-before 论证
- 6. 启动、接收与错误模型
- 7. 背压预算与关闭协议
- 8. 指标与时延的测量口径
- 9. 表示演进、PImpl 与 C 边界
- 10. 工程布局与构建配置
- 11. 验证按协议命题组织
- 12. 从性能观察到 profile 假设
- 13. 优化阶梯与配置发布
- 14. 可观察性、异常保证与交付
- 15. 实现阶段与工程记录
- 16. 常见架构误判及其反例

- 17. 验收层次与本章的完成含义
- 18. 完整实验：运行时协议、安装与观察
- 19. 跨组件复核记录
- 20. Final Gate
- 21. Final Gate · 参考答案与常见误判
- 22. 参考资料与验证边界

1. 项目合同：从语言机制到有界数据流

1.1 问题、产物与非目标

系统运行时（systems runtime）要回答的不是“队列怎样写”，而是多个正确组件组合后，是否仍保留接收、资源、失败与退出合同。本章沿用 `flow-runtime`：输入经有界队列交给工作线程，结果经第二个有界队列进入单写者 Sink。业务采用可核对的合成载荷，不接 Kafka、HTTP、数据库或 ROS；这样可以把生命周期缺陷与第三方协议故障分开定位。

目标包括并发提交、背压（backpressure）、正常排空（graceful drain）、中止（abort）、错误可观察、资源预算以及可安装的原生库。分布式执行、持久队列、跨崩溃 `exactly-once`、动态调度插件、协程、NUMA 和实时调度不是本章实现目标。进程内每个已接收任务恰有一个终态，不等于外部业务副作用“恰好一次”。

1.2 用三张图约束设计

```

Caller → InputQueue → Worker × N → OutputQueue → Sink
      |             |             |             |
      bounded      |             bounded      |
                   |             |             |
                   └─ local scratch ─┘       └─ single writer ─┘

```

Runtime owns: configuration, processor, queues, sink, threads

Thread lifetime is nested inside all borrowed dependencies

输入所有权、线程拓扑与依赖生命周期是三种关系，不能只画一张类图代替。队列负责交接，Processor 只在任务期间借用数据，Sink 在全部线程结束后向调用者交还只读汇总。读本章时先为每条箭头写明“谁可以写、何时交接、失败后谁负责”，再查看实现。

2. 不变量与生命周期状态

2.1 接收是责任转移点

`submit` 返回 `Accepted` 后，运行时必须最终报告 `Succeeded`、`Failed` 或 `Cancelled`。接收前被拒绝不属于 `Cancelled`；同一个 ID 不能被两个终态重复计算。仅验证总数相等还不够，重复一项并遗漏另一项也会使总数相等，因此还要核对每个接收 ID 的终态和结果。

队列满足 $0 \leq \text{depth} \leq \text{capacity}$ ；大型可变载荷沿热路径尽量只有一个逻辑拥有者。依赖对象在访问它的线程退出并被 `join` 后才能销毁。线程入口是异常边界，但其故障报告路径本身也必须有可执行的失败策略；写了 `noexcept` 不意味着锁、分配或报告不会失败。

2.2 不把 `close`、`abort`、`join` 混为一谈

```

Created → start → Running → close → Closing → join → Stopped
                        |
                        |_____ abort → Stopping → join → Stopped
Closing ————— abort ————|

```

`close` 停止接收并保留已接收工作；`abort` 停止接收、唤醒等待者并允许取消尚未完成的工作；`join` 是等待和阶段收口，不应暗中替调用者选择 `drain` 或 `abort`。`close/abort` 必须幂等，重复 `start` 应拒绝。

生命周期操作需要调用方合同。实验规定公开 `start`、`join`、`summary` 和析构由外部 `owner` 串行执行；公开 `submit`、`close()`、`abort()` 的前置条件是 `start()` 已成功返回。在此之后，`close/abort` 可与 `submit` 竞争。终止接收后先等外部提交线程返回，再 `join` 运行时。析构不能与尚在调用成员函数的外部线程竞争，也不能从内部 `worker` 自我 `join`。

实现没有用显式 `RuntimeState` 检查全部非法调用；成功 `start` 前公开 `close/abort` 属于合同外调用，不承诺抛错或特定结果。内部启动失败回滚和析构清理不受这一公开调用前置条件限制，§18 已有部分启动失败与析构测试也不等于测试了所有非法转换。

3. 载荷、借用与工作线程私有存储

3.1 拥有者与处理视图分离

原始项目的通用表示是 `InputItem { id, vector<byte> payload }`。普通 `vector` 移动构造通常转移存储管理状态；带不同分配器的操作则不能无条件推定常数时间。把“控制对象很小”与“载荷很大”分开，有利于跨队列移动，但必须继续检查所有别名和分配器条件，详见 G3。

`Processor` 接收 `span<const byte>`，明确只在调用期间读输入，不保存它。`worker` 持有载荷 `owner`，返回值是自包含的小结果；异步保留 `span` 会越过借用期限。裸指针加长度可以表达视图，却不能独自说明拥有、释放和失效，不能直接作为跨线程任务的默认所有权表示。

3.2 Scratch 是每个 `worker` 的预算

解码缓冲、临时记录和解压空间应由工作线程独占，初始化时 `reserve`，任务间 `clear/reuse`。`clear` 销毁元素而不缩减 `vector` 容量；超过已留容量仍会分配，元素析构也可能承担成本。全局 `scratch` 加 `mutex` 把本可并行的处理串行化，也扩大了共享写入范围。

`BufferLease` 可在测量后替代逐任务 `vector`，成为 `move-only` 的池租约。其 `pool` 必须比所有 `lease` 活得久，归还路径要线程安全且不抛，池耗尽需背压或明确拒绝。实验先用固定 64 字节自包含载荷：这是可测、总量有界的集成切片，不冒充已完成通用变长缓冲池。

4. 有界队列的状态与线性化点

4.1 队列不只是容器

输入是多生产者、多消费者；输出是多生产者、单消费者。第一版可复用 `mutex/CV` 队列。生产者等待“非 `Open` 或有空间”；消费者等待“非 `Open` 或非空”。`Closed` 允许继续 `drain`；`Aborted` 结束消费并取消剩余责任。通知只提示重新检查，状态和谓词必须在同一个 `mutex` 下维护。N4950 条件变量入队提交是接收的线性化点（`linearization point`）：先确保能保存任务，再登记 `Accepted`，最后允许消费者观察。若先公开任务、返回前才另写 `Accepted`，快 `worker` 可能先完成，而提交者再把终态覆盖回 `Accepted`。实验把不抛出的接收登记放在队列锁内，和槽位提交组成一个不可被消费者插入的步骤。

4.2 值参数、异常与实际内存上界

`push(T value)` 是所有权接收端，但调用前的 `move` 可能已经改变源；拒绝入队不意味着原调用者还保留载荷。需要“失败仍保留输入”的 API，应另定义成功才 `move` 的接口或返回未接收的 `owner`。不要靠返回码猜 `moved-from` 状态。

`deque` 支持首删尾增，是通用正确性基线，但条目数有界不等于完全预分配。完整实验改用预分配的 `optional` 槽位环，限制 `T` 的移动构造、析构及提交回调不抛。这样队列提交不需要动态扩容。若泛化为会抛的 `T`，要重新证明半提交恢复；不能删掉约束继续宣称同一保证。处理载荷必须在释放队列锁后进行。

5. 工作线程、Sink 与 happens-before 论证

5.1 借用依赖要活得更久

`worker` 借用队列、处理器和只读配置，独占 `scratch`。线程应在全部依赖初始化后启动；仅把 `jthread` 声明在最后并不足够，因为析构若先等待 `Sink`，而输出又必须等 `worker` 结束才能关闭，仍会死锁。显式 `shutdown` 顺序优先于“成员反序析构恰好正确”。

`stop token` 是协作请求，不会自动改变普通 `condition_variable` 的谓词或唤醒等待。队列 `close/abort` 负责终止阻塞等待；长时间 CPU 任务应另在安全点检查取消。实验处理固定短载荷，不承诺强制抢占不返回的用户处理函数。

5.2 两种同步边与剩余义务

生产者在队列 `mutex` 下写槽位，解锁与消费者之后的锁定建立发布关系；消费者取得独占任务后再处理。`Sink` 是 `checksum` 数组的唯一写者；`owner` 只在 `Sink join` 返回后读取。线程完成与成功 `join` 返回同步，不需要为这份阶段化汇总的每个元素加 `atomic`。N4950 `join`

`ledger` 的跨线程状态用 `atomic`，槽位内容用 `mutex`，`Sink` 数组用单写者与 `join`。这些机制分工不同。仍须论证无重复 ID、输出生产者全部结束后才关输出、`abort` 使每个等待条件成立。`TSan` 无报告不能替代这些协议和进展论证。

6. 启动、接收与错误模型

6.1 分阶段构造与部分启动失败

构造分配状态，不立即暴露 `this` 给线程；`start` 在完整对象上启动 `Sink` 和 `workers`。创建第 `k` 个线程失败时，前 `k-1` 个线程可能已经阻塞在队列上。回滚必须先 `abort` 队列，再唤醒、`join` 已创建线程，最后传播启动错误，不能只依赖 `vector<jthread>` 的析构去“自动解决”。

`submit` 的快速状态检查只是减少无效工作，最终接收判定必须与输入队列状态使用同一提交边界。因此 `close` 与正在等待的 `submit` 竞争时，后者可能被拒绝；不能把“调用发生在 `close` 前”当作已经 `Accepted`。

6.2 区分数据错误、基础设施故障与取消

格式错误属于 `item failure`，仍沿结果通道送给 `Sink`；意外处理异常属于 `fatal runtime failure`，应记录并触发 `abort`；取消是独立控制结果，不伪装成解析错误。通用实现可用 `expected` 表达处理结果、`exception_ptr` 保存首个 `fatal` 异常。低频错误通道不需要为展示技术而设计无锁发布。

实验用可复制错误枚举和 `fatal` 标志，以故意抛出的空 `Injected` 类型测试线程边界；类型没有动态成员，不代表异常运行时不会分配。它没有验证异常文本日志、`bad_alloc` 或 `mutex/OS` 同步设施损坏的恢复。`noexcept abort` 中若同步原语自身失效会终止，不能把灾难性失败解释为已经正常清理。二进制解码的通用版本应显式读取字节和端序，不把字节数组 `reinterpret_cast` 成未建立生命周期的记录对象。

7. 背压预算与关闭协议

7.1 条目数只是资源预算的一部分

队列容量决定突发吸收、等待时延和占用。变长载荷至少需要最大 `payload`、`worker scratch`、输出和调用者阻塞期间持有的载荷预算；无限数量的外部 `submitter` 即使阻塞在有限队列上，也能持有无限份输入。字节预算队列还需处理单项超过总预算与整数溢出，例如用 `cost <= budget - used`，而非未经检查相加。

实验额外限定 `max_items`，`ledger` 和结果存储在启动前分配；它是有限任务批次的运行时，不是无限历史事件数据库。扩大为长期服务时，应把终态流交给有消费确认和保留策略的外部系统，不能任由诊断账本增长。

7.2 正常排空与中止是不同提交路径

Graceful:

close input → workers drain → join workers
→ close output → sink drains → join sink → summary

Abort:

reject input → abort both queues → wake test gates / waiters
→ join external submitters → join workers and sink
→ finalize outstanding accepted IDs as cancelled

Sink 在 join workers 期间必须继续消费，否则 worker 可能因输出满而永远不能结束。close input 时立刻 close output 会丢失已接收任务的结果。

中止后的取消归档只能在所有执行者停止后进行；否则一个“已取消”任务可能随后又发布成功。实验保存未消费槽位至 join，再把仍 Accepted 的账项标为 Cancelled 并清空槽位。正常排空若留下 Accepted，必须报告 unresolved，不能也归为 Cancelled 来掩盖丢失。运行中的外部副作用不能靠这个账本回滚。

8. 指标与时延的测量口径

8.1 先定义计数，再谈速度

硬件线程数只是起点，不是最优 worker 数。独立记录提交尝试、Accepted、Rejected、Succeeded、Failed、Cancelled 和 unresolved；还应观察队列高水位、字节数及各阶段等待。worker 私有计数可在 join 后聚合，实时指标才引入必要的共享访问与缓存行成本。每条记录都打印日志会改变被测系统。

实验同时记录调用者侧成功接收数量和逐 ID 预期，再对照 ledger，避免仅从终态相加得到 accepted 的循环论证。重复 ID 被明确拒绝；该规则只覆盖一个 Runtime 对象生命周期，不处理持久化幂等键。

8.2 duration 不等于 wall time 业务时间戳

用 steady_clock 测持续时间。至少区分“开始 submit”“提交实际成功”“worker 开始”“处理结束”“Sink 消费完成”：入队阻塞、排队、处理与端到端不是同一个量。将处理完成减提交时间称作整个系统的 end-to-end，会遗漏输出队列和 Sink。

§18 基准只测启动完成后的首次提交到 join 完成，给出整个有限批次的总时长和吞吐派生值；不伪造尚未采集的逐任务 p50/p99，也不把它当 profile。需要分位数时为每个阶段定义采样点并评估观测开销。

9. 表示演进、PImpl 与 C 边界

9.1 先复用，再选择池或区域分配

vector 载荷加私有 scratch 是通用起点；固定载荷环是本章实验切片。测量证明分配占比显著后，再考虑 BufferLease、每 worker 的 monotonic_buffer_resource 或批处理。区域 reset 必须晚于区域内对象析构与所有借用结束；PMR 不会自动延长 resource 生命周期。

Processor 初始用具体类型足够。virtual、template Runtime 或 move_only_function 各有接口、代码体积、间接调用和状态所有权成本。只有确有替换需求时才选择，不把整个调度器模板化当作架构完整性的证据。

9.2 运行时身份不随意移动

线程捕获内部状态、队列含 mutex，运行中的 Runtime 默认不复制也不移动。PImpl 降低头文件依赖并隐藏布局，但不把整个 C++ 接口自动变成跨工具链稳定 ABI。公开 C++ 调用仍有标准库、异常和编译器合同，详见 G8。

C 入口用 opaque handle、pointer + size、显式 create/destroy 和状态码。输入仅借用到 submit 返回，包装层复制进运行时拥有的载荷。未来 zero-copy 必须新建租约或释放回调合同，不能悄悄把“调用期借用”改成后台保留。实验提供最小静态库 C 消费者，但没有承诺版本化共享 ABI：配置协商、动态加载、异步回调和旧二进制升级仍需独立验收。

10. 工程布局与构建配置

10.1 库目标比项目目录更重要

公开头文件在 include/flow，队列细节在 src；Flow::runtime 表达语言要求、线程依赖和安装头文件。tests 验证协议，apps/bench 观察批次成本，独立 consumer 只通过安装包 find_package。一个 main.cpp 能运行，不代表库已可消费。

开发、ASan/UBSan、TSan 和优化观察应分开构建目录；ASan 与 TSan 不放进同一个二进制。G9 已解释 presets、生成图与安装导出，本章只把它们用于集成，不复制完整构建教程。实验以执行器的显式命令选择配置，未另外声称实现了一组 dev/san/release presets。

10.2 安装消费的真实边界

源码树内目标成功可能受未声明 include path 帮助。实验安装后迁移前缀，并暂时隐藏临时生产者源码/构建目录，再创建 C11 与 C++23 两个独立消费者。C 编译单元通过 C 头文件调用，但静态 C++ 实现仍由 C++ 链接器提供运行库；“C 消费者”不等于不用 C++ runtime。

源码、编译器、参数和执行结果需要一起保存。本批不下载依赖，不发布包，不写历史 PDF 或 dist。安装隔离是测试目录的消费检验，不是供应链签名或跨平台认证。

11. 验证按协议命题组织

11.1 正例、阻塞点和错误变体

处理核的字节值、错误标记先有确定预期；集成测试再用多提交者、多 worker 和小容量迫使频繁交接。等待一个线程“应该阻塞了”不能靠随便 sleep：实验读取受锁保护的 waiter 计数，确认输入满、输出满或消费者空等具体前置条件后才 close/abort。等待有截止时间；超时是失败，不是成功取消。

正常 drain 检查 128 个 ID 的终态和完整 checksum，重复 20 次；取消分别覆盖输入生产者阻塞、输出生产者阻塞和空输入消费者等待；还覆盖重复关闭、重复 join、析构兜底、部分启动失败及 worker 异常。错误变体提前关闭 output，在保持可编译的前提下必须被接收账本判据拒绝，而不是把任意 crash 当作发现错误。

11.2 动态检测与论证互补

ASan/UBSan 检查本次执行中可检测的内存/未定义行为；TSan 检查插桩执行中的数据竞争。执行器先运行干净启动探针及目标诊断的检测能力对照，再跑本章协议测试。SKIP 与失败分别保存，不用“全部通过”掩盖不支持的工具链。

测试可证明这些输入和调度观察下的断言满足；它不穷举调度，不证明 wait-free，不验证 OS mutex 失败恢复，更不保证有限时间内完成任意用户任务。协议推理、变体拒绝和动态观测是三种不同证据。

12. 从性能观察到 profile 假设

12.1 分层基准避免归因跳跃

Processor 微基准隔离字节解析；队列基准隔离交接；运行时基准观察整体，三者回答不同问题。worker、载荷大小、容量都影响结果。比较时固定输入、工具链、优化配置和功能判据，重复采样并保留原始值，不挑最好一次。

本批基准覆盖 workers 1/2/4、capacity 1/16、每组合 3 次、每次 2000 个固定载荷。checksum 防止把无工作实现当性能改善。秒数和吞吐没有固定 PASS 门槛；只检查观测格式、工作量和结果一致。该小载荷用例很可能主要体现同步成本，不能外推大图像、磁盘或真实解析器。

12.2 测量不是已经做过 profiling

若 allocator 热，先定位分配调用和生命周期；若 queue 热，观察等待、批量与拓扑；若 Processor 热，检查算法、布局和生成代码；若内存带宽饱和，更多 worker 未必有效。必须用实际 profile 支持归因，而非看到某组吞吐低就断言 cache miss 或 false sharing。

本批没有采样调用栈、硬件计数器或生成汇编分析，状态为 PROFILE NOT RUN。后续优化记录应包含假设、基线、一次受控变化、重复测量和 KEEP/REVERT 决定，不把“理论应更快”当证据。

13. 优化阶梯与配置发布

13.1 每项优化都带来新的义务

优先复用存储，再尝试批处理、池、队列分片、AoS/SoA、区域分配，最后才考虑无锁。把 64 项一次交接可减少理想条件下的同步次数，却增加批次等待、部分失败和内存峰值；“通常值得先试”不等于总吞吐一定提高。批次自身的 **vector** 也可能分配。

按 **worker** 分出 SPSC 队列，必须重新讨论负载不均、任务顺序与关闭协调；把 AoS 改成 SoA，要保持列长和索引关系，并证明异常时不留下半更新；引入池，要解决耗尽、归还和借用失效。改变表示不能绕过 G1/G2/G7 的原有责任。

13.2 配置更新也有生命周期

运行期间不变的配置最好只读共享。需要更新时，先私下构造并验证整个 **generation**，再发布，旧读者结束后才回收旧版本。`atomic<shared_ptr<const Config>>` 提供一种拥有型发布方法，但引用计数及最后一个引用释放可能落到读路径，且操作不保证 **lock-free**。

不要让工作线程逐字段读多个独立 **atomics** 后误认得到一致配置。G11 的控制参数更需要整个 **generation** 的一致性和读路径时间边界；跨章复用的是发布/回收论证，不是固定容器选择。

14. 可观察性、异常保证与交付

14.1 无全局单例不等于没有共享状态

运行时显式拥有依赖，避免全局 **Runtime** 指针和随意共享 **counters**。控制路径日志记录 **start/close/abort/fatal/summary**；热路径只写有界指标。账本可以帮助解释责任去向，但必须与调用者接收记录独立对照，不能把自治统计当成端到端结果正确。

通用 **deque** 入队可能分配并抛异常；提交点必须说明异常发生时队列、参数 **owner** 和 **accepted** 状态各自如何。C++ API 可为基础设施故障抛异常、以值表达业务失败；C ABI 则在边界翻译。强保证依赖所有参与操作，不是看到 **noexcept swap** 就自动成立。

14.2 工程完成是多个 Gate 的交集

能安装 **Flow::runtime**、独立 C++ 消费及 C 头文件调用，是本章完成的可执行学习切片。长生命周期 ABI、所有配置组合、真实生产日志、动态模块和分配失败矩阵未实现，不把它们列成已经通过。

资源汇总在 **join** 后是不可变视图；运行时观测若要在后台持续读取，必须新增同步和一致性合同。不能把实验的 **post-join summary** 直接搬到实时监控线程调用。

15. 实现阶段与工程记录

15.1 阶段是内部工具，不是逐补丁审批

合理实现顺序为安装骨架、单线程 Processor、有界队列、单 worker 全链、多 worker、关闭协议、指标基准和可选 C 边界。每一步都保持前一步的可验证命题，最终作为一个完整能力批次交付。本章实验已经给出能编译、运行和安装消费的整体，而不是让读者自行补齐 close/abort 才能使用。

锁自由队列、hazard pointers、SIMD、定制 malloc、协程和 NUMA 不是禁止知识，而是在没有需求或 profile 前不加入集成基线。先使最小协议可解释，才知道后续复杂性解决什么问题。

15.2 记录必须能回到具体源码

架构记录放 ownership/thread/lifecycle 图；不变量记录放 accepted outcome、容量和销毁顺序；性能记录放条件、观察和限制。ADR 只记录有实际替代方案的决策，不为每个小实现动作创建治理负担。

Markdown 中的完整文件是本实验唯一维护源码；执行器提取到新临时目录并记录文件摘要。源稿版本、命令、结果与边界一起构成证据，不拿旧运行的 JSON 为新字节背书。六张跨章审查图在 G12 §15 汇总。

16. 常见架构误判及其反例

16.1 共享与等待常被隐藏在“方便”里

每项都改全局 atomic counter 会造成额外共享写入；到处 shared_ptr 只延长生命周期，不决定写入权限 (mutation authority)。裸借用指针进队列可能指向提交者栈；让 worker 拥有共享队列又会使多个析构责任相互冲突。把依赖注入和 owner 图画清楚，比增加智能指针种类更重要。

无界队列只是把过载从等待改成内存失控。处理时持队列锁把计算与交接串行化。过早关闭 output 会使已接收结果无处提交；只 request_stop 不通知 CV 会使等待者永远看不到退出机会。

16.2 类型复杂性不能代替生命周期

线程在依赖构造前启动、只靠 destructor 决定排空还是取消、未测量就写 lock-free、把所有组件都变为模板参数，分别扩大了失效窗口、隐式行为、证明负担和构建成本。实验让这些问题显式化，但没有声称一种 API 适用所有生产环境。

17. 验收层次与本章的完成含义

17.1 不把学习稿完成改写为生产认证

原稿的 correctness、sanitizer、ownership、shutdown、performance、build、package、ABI、explainability 九类 Gate 继续保留。正文完整意味着读者有规格、推理、实现切片和判据；它不代表全部生产功能、平台或 sanitizer 可检测范围已经覆盖。

本章的硬判据是具体功能、终态和资源上界；性能是条件化观察；并发是协议论证加动态证据；公开 C 接口只验证当前消费者。真实项目若要求旧二进制兼容、长时间可用性或强时限，必须补上对应证据，而不是升级章节状态字符串。

17.2 收口后的学习方式

先闭卷画 ownership、等待依赖和 drain 图，再运行完整实验；预测提前关闭输出会在哪条断言失败。之后读实际时长数据，提出可被 profile 证伪的假设。只有能解释每个类型、容量、同步和失败边界的理由，才算掌握项目，而不是把测试输出复制到笔记。

18. 完整实验：运行时协议、安装与观察

18.1 命题、命令与检查边界

G10-R1 以固定载荷、有限 ID 集合实现双队列 runtime。先根据 §2 ~ 7 预测 drain、abort 和异常的账本，再运行：

```
python3 c++/learning/verify_synthesis.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

命令从仓库根执行，路径可按本机编译器调整。执行器把正文文件提取到新的临时目录，输出其绝对路径和 results.json；记录编译器、标准库宏、参数与每个文件摘要。当前执行路径为 macOS/Clang，其他平台明确 SKIP，不把未运行记为 PASS。

正常程序应输出 runtime protocols verified；错误变体提前关输出，必须先编译成功，再以 invariant=10 和返回码 10 被拒绝。取消测试实际等待 waiter 计数，不以延迟猜阻塞。20 轮并发测试是同一协议的重复观察，不是 20 个新实验。

独立 C/C++ 消费者要求安装迁移后可运行。C 接口只是当前静态库实验：data 必须指向 64 个可读字节，调用期间有效；ID 为 [0,256) 且单实例内唯一；先成功 start，再 submit/close/join；所有生命周期调用按 owner 合同串行，destroy 前不得还有调用。输出指针必须指向有效可写对象，不与 handle 或彼此重叠。无效非空指针不能由 C 包装层检测。destroy(nullptr) 合法，成功创建的 handle 只销毁一次。本批没有旧共享二进制升级承诺。

18.2 完整文件

文件保留为可提取的完整单元，长代码是未来 PDF 续页风险，不用缩小字号或删除实现隐藏。源码中测试 gate 和启动失败参数仅服务确定性反例，不是生产 scheduling API。

[完整实验 · G10-R1 · include/flow/runtime.hpp]

```
#pragma once
#include <array>
#include <cstdint>
#include <cstdint>
#include <memory>
#include <vector>
```

完整实验 · G10-R1 · include/flow/runtime.hpp · continued

```

namespace flow {
enum class Outcome { unseen, pending, accepted, succeeded, failed, cancelled,
    └ rejected };
enum class Submit { accepted, closed, invalid };
enum class Fault { none, malformed, injected };
struct Item {
    std::size_t id{};
    std::array<std::byte, 64> payload{};
    Fault fault{};
};
struct Config {
    std::size_t workers{2}, input_capacity{4}, output_capacity{4}, max_items{256};
    // Laboratory scheduling controls, not a production scheduling API.
    bool hold_workers{}, hold_sink{};
    int fail_start_after{-1};
};
struct QueueStats { std::size_t depth{}, peak{}, push_waiters{}, pop_waiters{}; };
struct Summary {
    std::vector<Outcome> outcomes;
    std::vector<std::uint64_t> checksums;
    std::size_t accepted{}, succeeded{}, failed{}, cancelled{}, unresolved{};
    QueueStats input, output;
    bool fatal{}, protocol_error{};
};
class Runtime {
public:
    explicit Runtime(Config);
    ~Runtime();
    Runtime(const Runtime&) = delete;
    Runtime& operator=(const Runtime&) = delete;
    Runtime(Runtime&&) = delete;
    Runtime& operator=(Runtime&&) = delete;
    void start();
    Submit submit(Item);
    void close();
    void abort() noexcept;
    void join(); // Owner only; first close/abort, then join external submitters.
    Summary summary() const; // Owner only, after join.
    QueueStats input_stats() const;
    QueueStats output_stats() const;
    void release_workers();

```

完整实验 · G10-R1 · include/flow/runtime.hpp · continued

```

    void release_sink();
private:
    struct Impl;
    std::unique_ptr<Impl> impl_;
};
} // namespace flow

```

[完整实验 · G10-R1 · src/queue.hpp]

```

#pragma once
#include <algorithm>
#include <condition_variable>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <vector>
#include "flow/runtime.hpp"

namespace flow::detail {
template<class T> class Queue {
    static_assert(std::is_nothrow_move_constructible_v<T>);
    static_assert(std::is_nothrow_destructible_v<T>);
    enum class State { open, closed, aborted };
    mutable std::mutex mutex_;
    std::condition_variable readable_, writable_;
    std::vector<std::optional<T>> slots_;
    std::size_t head_={}, tail_={}, size_={}, peak_={}, pushes_={}, pops_{};
    State state_{State::open};
public:
    explicit Queue(std::size_t capacity) : slots_(capacity) {
        if (capacity == 0) throw std::invalid_argument("zero capacity");
    }
    template<class Commit> bool push(T value, Commit commit) {
        static_assert(std::is_nothrow_invocable_v<Commit>);
        std::unique_lock lock{mutex_};
        if (state_ == State::open && size_ == slots_.size()) {
            ++pushes_;
            writable_.wait(lock, [&] {
                return state_ != State::open || size_ < slots_.size();
            });
            --pushes_;
        }
        if (state_ != State::open) return false;
        slots_[tail_].emplace(std::move(value));
        commit(); // No throw; acceptance becomes visible before a consumer can pop.
        tail_ = (tail_ + 1) % slots_.size();
        peak_ = std::max(peak_, ++size_);
    }

```

完整实验 · G10-R1 · src/queue.hpp · continued

```

        lock.unlock();
        readable_.notify_one();
        return true;
    }
    bool push(T value) { return push(std::move(value), []() noexcept {}); }
    std::optional<T> pop() {
        std::unique_lock lock{mutex_};
        if (state_ == State::open && size_ == 0) {
            ++pops_;
            readable_.wait(lock, [&] { return state_ != State::open || size_ != 0;
                _   });
            --pops_;
        }
        if (state_ == State::aborted || size_ == 0) return std::nullopt;
        std::optional<T> value{std::move(*slots_[head_])};
        slots_[head_].reset();
        head_ = (head_ + 1) % slots_.size();
        --size_;
        lock.unlock();
        writable_.notify_one();
        return value;
    }
    void close() {
        std::lock_guard lock{mutex_};
        if (state_ == State::open) state_ = State::closed;
        readable_.notify_all();
        writable_.notify_all();
    }
    void abort() noexcept {
        std::lock_guard lock{mutex_};
        state_ = State::aborted;
        // Keep storage until all users exit; join finalizes outstanding outcomes.
        readable_.notify_all();
        writable_.notify_all();
    }
    void clear_after_join() {
        std::lock_guard lock{mutex_};
        for (auto& slot : slots_) slot.reset();
        size_ = 0;
    }
    QueueStats stats() const {
        std::lock_guard lock{mutex_};
        return {size_, peak_, pushes_, pops_};
    }
};

class Gate {
    std::mutex mutex_;

```

完整实验 · G10-R1 · src/queue.hpp · continued

```

        std::condition_variable ready_;
        bool open_;
public:
    explicit Gate(bool held) : open_(!held) {}
    void wait() {
        std::unique_lock lock{mutex_};
        ready_.wait(lock, [&] { return open_; });
    }
    void release() noexcept {
        std::lock_guard lock{mutex_};
        open_ = true;
        ready_.notify_all();
    }
};
} // namespace flow::detail

```

[完整实验 · G10-R1 · src/runtime.cpp]

```

#include "flow/runtime.hpp"
#include "queue.hpp"
#include <atomic>
#include <optional>
#include <stdexcept>
#include <thread>

namespace flow {
struct Result { std::size_t id; std::uint64_t checksum; bool valid; };
struct Injected {};
struct Runtime::Impl {
    Config config;
    detail::Queue<Item> input;
    detail::Queue<Result> output;
    detail::Gate worker_gate, sink_gate;
    std::unique_ptr<std::atomic<Outcome>[]> ledger;
    std::vector<std::uint64_t> checksums;
    std::atomic<bool> accepting{}, aborted{}, fatal{}, protocol_error{};
    bool started{}, joined{}; // Owner-thread lifecycle, never read by workers.
    std::vector<std::jthread> workers;
    std::jthread sink;

    explicit Impl(Config c)
        : config(c), input(c.input_capacity), output(c.output_capacity),
          worker_gate(c.hold_workers), sink_gate(c.hold_sink),
          ledger(std::make_unique<std::atomic<Outcome>[]>(c.max_items)),
          checksums(c.max_items) {
        if (c.workers == 0 || c.max_items == 0)
            throw std::invalid_argument("zero workers/items");
        for (std::size_t i = 0; i < c.max_items; ++i)

```

完整实验 · G10-R1 · src/runtime.cpp · continued

```

        ledger[i].store(Outcome::unseen);
        workers.reserve(c.workers);
    }
    void finish(std::size_t id, Outcome outcome) noexcept {
        auto expected = Outcome::accepted;
        if (!ledger[id].compare_exchange_strong(expected, outcome))
            protocol_error.store(true);
    }
    void abort() noexcept {
        accepting.store(false);
        aborted.store(true);
        input.abort();
        output.abort();
        worker_gate.release();
        sink_gate.release();
    }
    void worker() noexcept {
        std::optional<std::size_t> current;
        try {
            worker_gate.wait();
            while (auto item = input.pop()) {
                current = item->id;
                if (item->fault == Fault::injected) throw Injected{};
                std::uint64_t sum = 0;
                for (auto b : item->payload) sum += std::to_integer<unsigned>(b);
                Result result{item->id, sum, item->fault != Fault::malformed};
                if (!output.push(result) && !aborted.load())
                    protocol_error.store(true);
                current.reset();
            }
        } catch (...) {
            if (current) finish(*current, Outcome::failed);
            fatal.store(true);
            abort();
        }
    }
    void consume() noexcept {
        try {
            sink_gate.wait();
            while (auto result = output.pop()) {
                checksums[result->id] = result->checksum; // Sink is the only
                    └─ writer.
                finish(result->id, result->valid ? Outcome::succeeded :
                    └─ Outcome::failed);
            }
        } catch (...) {
            fatal.store(true);
        }
    }

```


完整实验 · G10-R1 · src/runtime.cpp · continued

```

        abort();
    }
}

void join_threads() {
    for (auto& worker_thread : workers)
        if (worker_thread.joinable()) worker_thread.join();
    output.close(); // Output stays open until every producer has exited.
    if (sink.joinable()) sink.join();
}

};

Runtime::Runtime(Config c) : impl_(std::make_unique<Impl>(c)) {}
Runtime::~Runtime() {
    // Caller must have stopped all external calls before destroying the object.
    if (impl_->started && !impl_->joined) {
        impl_->abort();
        try { join(); } catch (...) { std::terminate(); }
    }
}

void Runtime::start() {
    auto& p = *impl_;
    if (p.started) throw std::logic_error("already started");
    p.started = true;
    try {
        p.sink = std::jthread([&p] { p.consume(); });
        for (std::size_t i = 0; i < p.config.workers; ++i) {
            if (p.config.fail_start_after == static_cast<int>(i)) throw Injected{};
            p.workers.emplace_back([&p] { p.worker(); });
        }
        p.accepting.store(true);
    } catch (...) {
        p.fatal.store(true);
        p.abort();
        p.join_threads();
        p.joined = true;
        throw;
    }
}

Submit Runtime::submit(Item item) {
    auto& p = *impl_;
    if (!p.accepting.load()) return Submit::closed;
    if (item.id >= p.config.max_items) return Submit::invalid;
    auto expected = Outcome::unseen;
    if (!p.ledger[item.id].compare_exchange_strong(expected, Outcome::pending))
        return Submit::invalid; // IDs are unique for one Runtime lifetime.
    const auto id = item.id;
    if (!p.input.push(std::move(item), [&p, id]() noexcept {
        p.ledger[id].store(Outcome::accepted);
    }

```

完整实验 · G10-R1 · src/runtime.cpp · continued

```

        ))) {
        p.ledger[id].store(Outcome::rejected);
        return Submit::closed;
    }
    return Submit::accepted;
}

void Runtime::close() {
    impl_>accepting.store(false);
    impl_>input.close();
}

void Runtime::abort() noexcept { impl_>abort(); }

void Runtime::join() {
    auto& p = *impl_;
    if (!p.started || p.accepting.load()) throw std::logic_error("close/abort
        └ first");
    if (p.joined) return;
    p.join_threads();
    if (p.aborted.load()) {
        for (std::size_t i = 0; i < p.config.max_items; ++i) {
            auto expected = Outcome::accepted;
            p.ledger[i].compare_exchange_strong(expected, Outcome::cancelled);
        }
    }
    p.input.clear_after_join();
    p.output.clear_after_join();
    p.joined = true;
}

Summary Runtime::summary() const {
    const auto& p = *impl_;
    if (!p.joined) throw std::logic_error("join first");
    Summary s;
    s.outcomes.resize(p.config.max_items);
    s.checksums = p.checksums;
    for (std::size_t i = 0; i < p.config.max_items; ++i) {
        const auto state = p.ledger[i].load();
        s.outcomes[i] = state;
        if (state == Outcome::succeeded) ++s.succeeded;
        if (state == Outcome::failed) ++s.failed;
        if (state == Outcome::cancelled) ++s.cancelled;
        if (state == Outcome::accepted || state == Outcome::pending) ++s.unresolved;
    }
    s.accepted = s.succeeded + s.failed + s.cancelled + s.unresolved;
    s.input = p.input.stats();
    s.output = p.output.stats();
    s.fatal = p.fatal.load();
    s.protocol_error = p.protocol_error.load();
    return s;
}

```

完整实验 · G10-R1 · src/runtime.cpp · continued

```

}
QueueStats Runtime::input_stats() const { return impl_->input.stats(); }
QueueStats Runtime::output_stats() const { return impl_->output.stats(); }
void Runtime::release_workers() { impl_->worker_gate.release(); }
void Runtime::release_sink() { impl_->sink_gate.release(); }
} // namespace flow

```

[完整实验 · G10-R1 · tests/runtime_test.cpp]

```

#include "flow/runtime.hpp"
#include <atomic>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <thread>
#include <vector>
using namespace std::chrono_literals;
using flow::Outcome;
void require(bool ok, int code) {
    if (!ok) { std::cerr << "invariant=" << code << '\n'; std::exit(code); }
}
template<class Predicate> void await(Predicate predicate, int code) {
    const auto end = std::chrono::steady_clock::now() + 5s;
    while (!predicate()) {
        require(std::chrono::steady_clock::now() < end, code);
        std::this_thread::sleep_for(1ms);
    }
}
flow::Item item(std::size_t id, flow::Fault fault = flow::Fault::none) {
    flow::Item value{};
    value.id = id;
    value.fault = fault;
    value.payload.fill(std::byte(id % 251 + 1));
    return value;
}
void accounting(const flow::Summary& s, std::size_t accepted) {
    require(s.accepted == accepted && s.unresolved == 0 && !s.protocol_error, 10);
    require(accepted == s.succeeded + s.failed + s.cancelled, 11);
}
int main() {
    // Multi-producer input, multiple workers, independent caller-side acceptance.
    for (int repeat = 0; repeat != 20; ++repeat) {
        flow::Runtime runtime{.workers=3, .input_capacity=2, .output_capacity=1};
        runtime.start();
        std::atomic<std::size_t> accepted{};
        std::vector<std::jthread> producers;
        for (std::size_t p = 0; p != 4; ++p) {
            producers.emplace_back([&, p] {

```

完整实验 · G10-R1 · tests/runtime_test.cpp · continued

```

        for (std::size_t id = p; id < 128; id += 4) {
            auto fault = id % 17 == 0 ? flow::Fault::malformed :
                flow::Fault::none;
            if (runtime.submit(item(id, fault)) == flow::Submit::accepted)
                ++accepted;
        }
    });
}
for (auto& p : producers) p.join();
require(runtime.submit(item(0)) == flow::Submit::invalid, 12);
require(runtime.submit(item(999)) == flow::Submit::invalid, 13);
runtime.close();
runtime.close();
require(runtime.submit(item(200)) == flow::Submit::closed, 14);
runtime.join();
runtime.join();
const auto s = runtime.summary();
accounting(s, accepted.load());
require(accepted == 128 && s.succeeded == 120 && s.failed == 8, 15);
require(s.input.peak <= 2 && s.output.peak <= 1 && !s.fatal, 16);
for (std::size_t id = 0; id != 128; ++id) {
    const auto want = id % 17 == 0 ? Outcome::failed : Outcome::succeeded;
    require(s.outcomes[id] == want
        && s.checksums[id] == 64 * (id % 251 + 1), 17);
}
}
// Prove producer is blocked before close, then allow graceful draining.
{
    flow::Runtime r{.workers=1, .input_capacity=1, .hold_workers=true};
    r.start();
    require(r.submit(item(0)) == flow::Submit::accepted, 20);
    flow::Submit second{};
    std::jthread p([&] { second = r.submit(item(1)); });
    await([&] { return r.input_stats().push_waiters == 1; }, 21);
    r.close();
    p.join();
    require(second == flow::Submit::closed, 22);
    r.release_workers();
    r.join();
    accounting(r.summary(), 1);
    require(r.summary().outcomes[0] == Outcome::succeeded, 23);
}
// Abort while a producer is blocked on full input.
{
    flow::Runtime r{.workers=1, .input_capacity=1, .hold_workers=true};
    r.start();
    require(r.submit(item(0)) == flow::Submit::accepted, 24);

```

完整实验 · G10-R1 · tests/runtime_test.cpp · continued

```

        flow::Submit second{};
        std::jthread p([&] { second = r.submit(item(1)); });
        await([&] { return r.input_stats().push_waiters == 1; }, 25);
        r.abort();
        r.abort();
        p.join();
        r.join();
        const auto s = r.summary();
        accounting(s, 1);
        require(second == flow::Submit::closed && s.cancelled == 1, 26);
        require(s.outcomes[1] == Outcome::rejected, 27);
    }
    // Abort while worker is blocked on full output; sink has not consumed.
    {
        flow::Runtime r{.workers=1, .output_capacity=1, .hold_sink=true};
        r.start();
        for (std::size_t i = 0; i != 3; ++i)
            require(r.submit(item(i)) == flow::Submit::accepted, 28);
        await([&] { return r.output_stats().push_waiters == 1; }, 29);
        r.abort();
        r.join();
        accounting(r.summary(), 3);
        require(r.summary().cancelled == 3, 30);
    }
    // Abort wakes empty-input consumers; destruction also wakes/join threads.
    {
        flow::Runtime r{.workers=2};
        r.start();
        await([&] { return r.input_stats().pop_waiters == 2; }, 31);
        r.abort();
        r.join();
        accounting(r.summary(), 0);
    }
    { flow::Runtime r{}; r.start(); }
    // A fatal processor exception terminates that item and cancels remaining work.
    {
        flow::Runtime r{.workers=1, .hold_workers=true};
        r.start();
        require(r.submit(item(0, flow::Fault::injected)) == flow::Submit::accepted,
            32);
        require(r.submit(item(1)) == flow::Submit::accepted, 33);
        r.close();
        r.release_workers();
        r.join();
        auto s = r.summary();
        accounting(s, 2);
        require(s.fatal && s.failed == 1 && s.cancelled == 1, 34);
    }

```

完整实验 · G10-R1 · tests/runtime_test.cpp · continued

```

        require(s.outcomes[0] == Outcome::failed, 35);
    }
    // Partial startup failure unwinds already-created threads without deadlock.
    {
        flow::Runtime r{.workers=2, .fail_start_after=1};
        bool caught = false;
        try { r.start(); } catch (...) { caught = true; }
        require(caught && r.summary().fatal, 36);
    }
    std::cout << "runtime protocols verified\n";
}

```

[完整实验 · G10-R1 · apps/bench.cpp]

```

#include "flow/runtime.hpp"
#include <chrono>
#include <iostream>
int main() {
    constexpr std::size_t count = 2000;
    for (std::size_t workers : {1U, 2U, 4U})
        for (std::size_t capacity : {1U, 16U})
            for (int repeat = 0; repeat != 3; ++repeat) {
                flow::Runtime r{.workers=workers, .input_capacity=capacity,
                               .output_capacity=capacity, .max_items=count};
                r.start(); // Startup excluded; submit through drain is timed.
                const auto begin = std::chrono::steady_clock::now();
                for (std::size_t i = 0; i != count; ++i) {
                    flow::Item value{};
                    value.id = i;
                    value.payload.fill(std::byte{3});
                    if (r.submit(value) != flow::Submit::accepted) return 2;
                }
                r.close();
                r.join();
                const double seconds = std::chrono::duration<double>(
                    std::chrono::steady_clock::now() - begin).count();
                const auto s = r.summary();
                if (s.succeeded != count || s.unresolved || s.protocol_error) return
                    3;
                std::uint64_t sum = 0;
                for (auto value : s.checksums) sum += value;
                if (sum != count * 192 || seconds <= 0) return 4;
                std::cout << workers << ',' << capacity << ',' << repeat << ','
                    << count << ',' << seconds << ',' << sum << '\n';
            }
}

```

[完整实验 · G10-R1 · include/flow/flow.h]

```

#ifndef FLOW_H
#define FLOW_H
#include <stddef.h>
#include <stdint.h>
#ifdef __cplusplus
extern "C" {
#endif
typedef struct flow_handle flow_handle;
enum flow_status { FLOW_OK, FLOW_CLOSED, FLOW_INVALID, FLOW_ERROR };
int flow_create(flow_handle** out);
void flow_destroy(flow_handle*);
int flow_start(flow_handle*);
int flow_submit(flow_handle*, size_t id, const uint8_t* data, size_t size);
int flow_close(flow_handle*);
int flow_join(flow_handle*, size_t* succeeded, uint64_t* checksum);
#ifdef __cplusplus
}
#endif
#endif

```

[完整实验 · G10-R1 · src/c_api.cpp]

```

#include "flow/flow.h"
#include "flow/runtime.hpp"
#include <new>
struct flow_handle { flow::Runtime runtime{flow::Config{}}; bool started{}; };
extern "C" {
int flow_create(flow_handle** out) {
    if (!out) return FLOW_INVALID;
    *out = nullptr;
    try { *out = new flow_handle; return FLOW_OK; } catch (...) { return FLOW_ERROR; }
}

void flow_destroy(flow_handle* h) { delete h; }

int flow_start(flow_handle* h) {
    if (!h || h->started) return FLOW_INVALID;
    try { h->runtime.start(); h->started = true; return FLOW_OK; }
    catch (...) { return FLOW_ERROR; }
}

int flow_submit(flow_handle* h, size_t id, const uint8_t* data, size_t size) {
    if (!h || !h->started || !data || size != 64) return FLOW_INVALID;
    try {
        flow::Item item{};
        item.id = id;
        for (size_t i = 0; i != size; ++i) item.payload[i] = std::byte{data[i]};
        switch (h->runtime.submit(item)) {
            case flow::Submit::accepted: return FLOW_OK;
            case flow::Submit::closed: return FLOW_CLOSED;
        }
    }
}
}

```

完整实验 · G10-R1 · src/c_api.cpp · continued

```

        case flow::Submit::invalid: return FLOW_INVALID;
    }
} catch (...) { return FLOW_ERROR; }
return FLOW_ERROR;
}

int flow_close(flow_handle* h) {
    if (!h || !h->started) return FLOW_INVALID;
    try { h->runtime.close(); return FLOW_OK; } catch (...) { return FLOW_ERROR; }
}

int flow_join(flow_handle* h, size_t* succeeded, uint64_t* checksum) {
    if (!h || !h->started || !succeeded || !checksum) return FLOW_INVALID;
    try {
        h->runtime.join();
        auto s = h->runtime.summary();
        if (s.fatal || s.protocol_error || s.unresolved) return FLOW_ERROR;
        uint64_t sum = 0;
        for (auto value : s.checksums) sum += value;
        *succeeded = s.succeeded;
        *checksum = sum;
        return FLOW_OK;
    } catch (...) { return FLOW_ERROR; }
}
}

```

[完整实验 · G10-R1 · consumer/main.cpp]

```

#include <flow/runtime.hpp>
int main() {
    flow::Runtime runtime{ {.workers=1} };
    runtime.start();
    flow::Item item{};
    item.payload.fill(std::byte{3});
    if (runtime.submit(item) != flow::Submit::accepted) return 1;
    runtime.close();
    runtime.join();
    auto s = runtime.summary();
    return s.succeeded == 1 && s.checksums[0] == 192 && !s.unresolved ? 0 : 2;
}

```


[完整实验 · G10-R1 · consumer/main.c]

```

#include <flow/flow.h>
#include <string.h>
int main(void) {
    flow_handle* h = NULL;
    uint8_t data[64];
    size_t completed = 999;
    uint64_t checksum = 0;
    memset(data, 3, sizeof(data));
    if (flow_create(&h) != FLOW_OK || flow_start(h) != FLOW_OK) return 1;
    if (flow_submit(h, 0, data, sizeof(data)) != FLOW_OK) return 2;
    memset(data, 9, sizeof(data)); /* Input was copied during submit. */
    if (flow_close(h) != FLOW_OK || flow_join(h, &completed, &checksum) != FLOW_OK)
        return 3;
    flow_destroy(h);
    return completed == 1 && checksum == 192 ? 0 : 4;
}

```

[完整实验 · G10-R1 · CMakeLists.txt]

```

cmake_minimum_required(VERSION 3.23)
project(Flow VERSION 1.0.0 LANGUAGES CXX)
include(GNUInstallDirs)
include(CMakePackageConfigHelpers)
find_package(Threads REQUIRED)
add_library(flow STATIC src/runtime.cpp src/c_api.cpp)
add_library(Flow::runtime ALIAS flow)
set_target_properties(flow PROPERTIES EXPORT_NAME runtime CXX_EXTENSIONS NO)
target_compile_features(flow PUBLIC cxx_std_23)
target_sources(flow PUBLIC FILE_SET HEADERS BASE_DIRS include
    FILES include/flow/runtime.hpp include/flow/flow.h)
target_link_libraries(flow PUBLIC Threads::Threads)
add_executable(runtime_test tests/runtime_test.cpp)
target_link_libraries(runtime_test PRIVATE flow)
add_executable(runtime_bench apps/bench.cpp)
target_link_libraries(runtime_bench PRIVATE flow)
enable_testing()
add_test(NAME protocols COMMAND runtime_test)
set_tests_properties(protocols PROPERTIES TIMEOUT 30)
install(TARGETS flow EXPORT FlowTargets FILE_SET HEADERS
    ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR})
configure_package_config_file(cmake/FlowConfig.cmake.in
    ${CMAKE_CURRENT_BINARY_DIR}/FlowConfig.cmake
    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/Flow)
write_basic_package_version_file(${CMAKE_CURRENT_BINARY_DIR}/FlowConfigVersion.cmake

```

完整实验 · G10-R1 · CMakeLists.txt · continued

```

    VERSION ${PROJECT_VERSION} COMPATIBILITY SameMajorVersion)
install(EXPORT FlowTargets NAMESPACE Flow::
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/Flow)
install(FILES ${CMAKE_CURRENT_BINARY_DIR}/FlowConfig.cmake
    ${CMAKE_CURRENT_BINARY_DIR}/FlowConfigVersion.cmake
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/Flow)

```

[完整实验 · G10-R1 · cmake/FlowConfig.cmake.in]

```

@PACKAGE_INIT@
include(CMakeFindDependencyMacro)
find_dependency(Threads)
include("${CMAKE_CURRENT_LIST_DIR}/FlowTargets.cmake")

```

[完整实验 · G10-R1 · consumer/CMakeLists.txt]

```

cmake_minimum_required(VERSION 3.23)
project(FlowConsumer LANGUAGES C CXX)
find_package(Flow 1 CONFIG REQUIRED)
add_executable(cpp_consumer main.cpp)
target_link_libraries(cpp_consumer PRIVATE Flow::runtime)
add_executable(c_consumer main.c)
set_target_properties(c_consumer PROPERTIES C_STANDARD 11 C_STANDARD_REQUIRED YES)
target_link_libraries(c_consumer PRIVATE Flow::runtime)
enable_testing()
add_test(NAME cpp_consumer COMMAND cpp_consumer)
add_test(NAME c_consumer COMMAND c_consumer)

```

18.3 阅读实际结果

运行时报告先看逐 ID 终态与 checksum，再看计数；查看 abort 是在什么等待状态触发。Sink 可在 abort 竞争前完成已弹出的结果，这类任务允许保留成功，不要求所有 Accepted 一律取消。测试通过并不证明所有调度，尤其不支持任意阻塞 Processor 的强制终止。

基准每行是 workers, capacity, repeat, items, seconds, checksum，共 18 行。时间越小不自动判定“方案更好”：输入小、容量和环境固定，不能推出生产吞吐。原始行保留，派生 items/s 和分布供观察；没有固定性能数值 oracle。

19. 跨组件复核记录

19.1 将代码与论证逐项连接

命题	本章机制	反例或检查
Accepted 有终态	锁内提交 ledger, join 后核对	提前关输出变体
输入/输出有界	固定槽位环	peak 不超过 capacity
取消能解除等待	两队列 abort 加唤醒	full input/output 与 empty input
依赖先于线程存活	完成启动后运行, 显式 join	析构兜底、部分启动失败
处理异常不逃线程	worker catch、fatal、abort	injected fault 后终态
安装接口可消费	导出目标、安装头文件	隐藏生产树后的 C/C++ 消费者

19.2 保留剩余问题

实验不验证无限任务服务、持久队列、真实变长解码、不可撤销副作用、分配失败、OS 同步设施失败、采样 profile 或严格进展保证。当前错误注入是受控处理异常与部分启动失败，不等于所有异常矩阵穷举。学习时应以这些缺口提出后续项目问题，而不是修改记录将它们视为已完成。

20. Final Gate

先闭卷写出条件、机制与反例，再核对下一节；保留原稿问题，不在题干下先给结论。

20.1 Ownership

1. InputItem 从 caller 到 worker 的 ownership 怎样变化？
2. 为什么 Processor 参数应该是 span 而不是 vector？
3. 为什么 queue 中的 raw borrowed pointer 很危险？

20.2 Queue

1. 为什么 queue 必须 bounded？
2. full 是 error 还是系统状态？
3. mutex/CV queue 为什么是合理 baseline？
4. 为什么不能持 queue lock 处理 item？

20.3 Worker

1. 为什么每 worker 应该有自己的 scratch？
2. 为什么 worker thread 必须晚于 dependencies 构造、早于 dependencies 析构？
3. 为什么 thread entry 应该是 exception boundary？

20.4 Shutdown

1. close 与 abort 有什么根本区别？
2. 为什么 close input 后不能立刻 close output？
3. 为什么 output queue 要在 workers 完全结束之后再关闭？
4. 为什么 abort 必须 wake blocked waiters？

20.5 Memory

1. 哪些 allocations 发生在 hot path？
2. reserve + clear 消除了什么？
3. 什么时候值得引入 BufferPool？

20.6 Concurrency

1. 哪些 state 真正 shared mutable？
2. 为什么 Sink 可以不使用 atomic counters？
3. 哪个 synchronization relation 保证 join 后 summary 可安全读取？

20.7 ABI

1. C ABI 为什么使用 opaque handle?
2. 为什么 C submit API 采用 pointer + size?
3. 为什么异常不能逃出 C ABI?

20.8 Build

1. 为什么 runtime 应该成为 installable target?
2. 为什么 external consumer test 是 G9 的真正验收?

20.9 Performance

1. 应先测 processor 还是直接换 lock-free queue?
2. 为什么 batching 通常优先于 CAS 微优化?
3. 为什么 worker_count 必须测量而不是按 CPU 数猜?
4. queue capacity 怎样影响 throughput、latency 和 memory?

20.10 Architecture

1. 如果 profiling 显示 output queue contention 严重，第一反应应该是什么?

21. Final Gate · 参考答案与常见误判

答案按上一节分组和题号对应。重点是推理与适用条件，不把关键词复述当作通过。

21.1 Ownership

1. 成功提交时 **caller** 的任务值进入队列，**pop** 后由 **worker** 独占；**Processor** 只借用，结果再交给输出队列与 **Sink**。失败的值参数可能已从源移动，不能推定原 **owner** 未变。
2. **span** 表达调用期间读取连续范围，不要求 **vector** 的分配策略，也不接管释放。但它不自动证明 **owner** 存活，异步保留必须另设计。
3. 队列延长等待而不延长 **pointee** 生命周期。提交者栈退出、容器扩容或 **owner** 重置都可能使指针悬挂；有明确外部生命周期协议时才可借用。

21.2 Queue

1. 有限内存需要可计算容量，永久生产过快时必须背压、拒绝或按业务降级；无界缓冲只推迟失败。
2. **full** 是正常过载状态，可阻塞、返回暂时拒绝或触发策略，不一定是程序缺陷。必须写清终止等待条件。
3. 它把复合状态放在一个锁域，谓词、提交点与关闭容易论证；不是说 **mutex** 永远最快，而是当前目标先需要可验证正确性。
4. 处理时间扩大锁持有范围，阻塞其他 **push/pop**，可能造成反馈死锁。先交接所有权、解锁，再计算。

21.3 Worker

1. 独占 **scratch** 消除共享写入和锁竞争，并复用容量；仍需预算每线程内存和元素析构成本。
2. 线程可能立即执行，也可能延迟结束；依赖构造完成才启动，停止并 **join** 后才销毁，不能仅靠碰巧的调度。
3. 逃出线程入口的异常会导致终止。边界应区分 **item failure** 和 **fatal**，报告后唤醒退出；报告自身失败仍需明示限制。

21.4 Shutdown

1. **close** 保留已接收工作并 **drain**；**abort** 可将未完成责任归为取消。二者都停止新接收，但业务后果不同。
2. 输入关闭后 **worker** 仍可持有已取出的任务；立刻关输出会拒绝这些结果，留下 **Accepted** 无终态。
3. **join workers** 是不再有输出生产者的阶段屏障。**Sink** 在此期间要持续消费，之后关输出才可结束消费。
4. **stop** 标志若不改变等待谓词并通知，**CV** 上的线程可能永远不醒。输入、输出和其他阻塞点都要覆盖。

21.5 Memory

1. 通用版本可能有输入 `vector`、`deque` 槽、`scratch` 增长及日志分配；实验固定载荷环提前分配，`summary` 的复制在 `join` 后，不属于处理热路径。
2. 已有容量可复用，减少后续扩容；它不保证超过容量时不分配，也不消除元素构造/析构或外部分配。
3. 实际 `profile` 显示大载荷分配是重要成本时。还要处理池上界、耗尽、`lease` 归还和 `pool` 比借用者长寿。

21.6 Concurrency

1. 队列状态/槽位、跨线程 `ledger`、停止与故障标志需要同步；任务和私有 `scratch` 不共享写；Sink 数据在运行期只有一个 `writer`。
2. 只有 Sink 修改，`owner` 等 Sink `join` 后再读，阶段分离使普通字段合法；运行中直接调用同一 `summary` 则不是这个合同。
3. 线程完成与对应成功 `join` 返回 `synchronizes-with`，再经调用线程顺序到达读取。不是 `sleep` 足够久或 `bool` 看起来为 `true`。

21.7 ABI

1. 句柄隐藏 C++ 对象布局和管理方式，让创建/销毁留在实现分配域；但不自动给并发或版本兼容保证。
2. 连续字节无需暴露 `vector`，`extent` 明确。仍要定义空指针、有效范围、是否重叠、是否保留，以及大小上界。
3. 本接口规定 C 状态通道，不能要求外部消费者理解 C++ 展开 ABI。包装层捕获并翻译，`noexcept` 声明本身不会翻译。

21.8 Build

1. 目标传播头文件、语言和链接需求，使应用不是依赖源码内部路径。可安装才有真实消费边界。
2. 独立构建可揭露缺失导出、安装依赖和偶然 `include path`；静态消费者通过仍不证明动态 ABI 升级或所有平台。

21.9 Performance

1. 先测整体和分层成本，再定位主导原因。没有 `profile` 不能认定锁是瓶颈，更不能据此引入回收复杂性。
2. 批处理减少每项交接次数，但增加等待与部分失败语义。应测吞吐、时延和容量，不把理想次数比当实测提速。
3. CPU hint 不包含带宽、`task` 粒度、调度和 Sink 瓶颈；更多线程可引入竞争，所以使用 `worker sweep`。

4. 更深缓冲可吸收突发，也扩大排队时延和内存；持续吞吐最终受最慢阶段限制，不能无限加深解决过载。

21.10 Architecture

1. 先确认 profile 真把成本定位在输出交接，再尝试结果批量、MPSC 专化、worker 本地聚合或分片 Sink，重新核对顺序与记账；不是立即写 lock-free MPMC。

22. 参考资料与验证边界

N4950 的 条件变量、线程 join 用于同步规则回查；Clang ASan、UBSan 与 TSan 说明检测机制。实际版本、结果、SKIP、错误变体和平台限制在批次记录与原始结果。工具文档不是本机已执行证据。引用版本与证据解释统一见全书约定。下一章 G11 §1 ~ 2 在资源与关闭模型上加入物理语义和时钟域；运行时协议通过定向测试，不等于新增了实时保证。

G11 · 机器人系统工程

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。Robotics Systems Engineering；承接 G10，桌面仿真不构成实时或物理安全保证。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G10 · 全系列导航 · 下一章：G12

阅读入口

本章在 G10 的资源与并发模型上加入时间、物理意义和安全状态。先读 §1 ~ 8，再读 §13 与 §16 的仿真实验；ROS/QoS 是 §9 的集成回查，不是学习主线。

原稿的 Complete/Frozen 是历史编辑标记，不沿用为技术验收。编号主题以 g11-topic-N 映射到相应主题组，保留技术去向；章节不再逐个复制原有 Part。完整实验与机制片段有可见身份，Gate 答案是普通章节。

- 1. 物理闭环与实时合同
- 2. 时间域、数据年龄与控制步长
- 3. 调度、干扰与优先级反转
- 4. 内存阶段与故障表示
- 5. 单位、坐标系与数值表示
- 6. 传感器采集、通道语义与快照回收
- 7. 估计、控制与硬件边界
- 8. 看门狗、故障锁存与命令仲裁
- 9. ROS 2、QoS 与借出缓冲区
- 10. 大载荷、配置事务与安全过滤
- 11. 故障隔离、观测与测试阶梯
- 12. 执行等级与集成拓扑
- 13. 输出不变量与工程分层
- 14. 可运行学习案例的范围
- 15. 常见误判与语言选择
- 16. 完整实验：逻辑仿真、快照与时序

- 17. 机器人路径复核记录
- 18. Final Gate
- 19. Final Gate · 参考答案与常见误判
- 20. 参考资料与验证边界

1. 物理闭环与实时合同

1.1 软件结果会改变下一次输入

机器人系统工程 (robotics systems engineering) 把软件正确性放进物理闭环：测量、估计、控制、执行再影响下一轮测量。正确数值若过期才送到执行器，可能已不是正确动作。因此本章关注 `value`、`unit`、`frame`、`time`、`lifetime` 与 `failure` 的联合合同，而不是 ROS 2 API 教程。

```
World → Measure → Estimate → Control → Safety → Actuate
      ▲                               |
      +----- feedback -----+
Non-RT: configuration / planning / ROS / logging
Timing-critical: bounded state / control / safety
Hardware: bus / driver / local watchdog
```

三个执行域可以在部分场景合并，但不能默认共享不可控阻塞。视觉、规划、UI、控制和硬件 I/O 有不同的吞吐、时限与故障需求；“实时”不是给整个仓库贴的标签。

1.2 快、可预测和安全是不同判断

周期 (period) 是计划 `release` 间隔；`deadline` 是完成期限；本章把 `release jitter` 明确定义为实际开始减计划开始，执行时间则是完成减实际开始。1 kHz 对应 1 ms 计划周期，不保证每次实际间隔恰为 1 ms。平均 10 μs 不能覆盖偶发 200 ms 停顿。

Hard real-time 要在明确平台及负载假设下保证截止期限；firm real-time 的迟到结果失去价值但允许受控 miss；soft real-time 的迟到降低服务质量。它们描述需求和保证，不是根据一次最大值自动分类。§16 的桌面仿真只给出本机 `timing observation`，不能证明 WCET 或物理安全。

2. 时间域、数据年龄与控制步长

2.1 时间戳必须有语义和来源

测量时间、设备发送、内核接收、应用接收与处理完成不是同一时间。收到数据后写 `steady_clock::now`，只能给它应用接收时间，不能将通信抖动消掉而称作测量时间。设备计时器、主机单调时钟、UTC、PTP 与 ROS 仿真时钟即使都以 ns 表示，也不能未经映射直接相减。

跨时钟域转换应声明偏移、漂移、同步误差和失效条件；时间戳倒退、设备重启和序列回绕也要处理。仿真时间可以暂停或回拨，主机 `steady_clock` 适合本地 `duration`，不应冒充物理测量时钟。控制的数据新鲜度 (freshness) 先检查时间域和“非未来”，再计算数据年龄 (age)，避免无符号减法下溢。

2.2 固定步长与实际步长不能悄悄替换

固定步长离散控制器基于配置周期；`measured-step` 算法使用实际 `elapsed time`。两者都需要各自稳定性前提，不能将一次调度延迟直接当作大 `dt` 填入原本固定步长的积分器。控制接口应显式携带 `tick/time/duration`，并定义接近零、过大或非有限 `dt` 的策略。

典型循环为 `read` → `estimate` → `control` → `safety` → `write`；每步需审查输入规模、分配、等待、异常和未知回调。配置解析、模型加载与日志移到非实时域，经过验证的规范化配置在安全点发布。本章 `timing` 模块故意用逻辑 1 ms 仿真步长，同时单独记录主机 `wake/finish`；它不是实时物理 `plant` 的保真模拟。

3. 调度、干扰与优先级反转

3.1 优先级是部署条件，不是证明

Linux 的 `SCHED_FIFO/SCHED_RR`、CPU affinity、隔离核心和 IRQ 布局可改变调度干扰，但 `page fault`、驱动、锁持有者、固件和热降频仍影响尾延迟。高优先级线程也可能因等待低优先级持锁者而发生优先级反转（`priority inversion`）。未分析竞争与锁持有时间，不能仅凭“平时不竞争”称其有界。

本章以 `ros2_control Jazzy Controller Manager` 为部署回查线，不给机器设置实时权限，也不更改调度器、内存锁定或系统配置。Linux 部署建议与 macOS 仿真实验是两种证据，不能互相替代。

3.2 把不确定工作移出关键路径

Non-RT 回调可慢慢构建完整 `snapshot`，控制线程在边界处接收。若选择 `try_lock`，读者失败时必须要有“保留旧值并检查 `age`”策略，且仍不能声称底层调用有硬时间上界。若选择无锁发布，必须额外证明存储回收和重试上界。架构是在时间约束、数据新鲜度与实现复杂度间取舍，不是逢 `mutex` 必改 `CAS`。

4. 内存阶段与故障表示

4.1 初始化分配不等于运行期永不分配

`general heap` 的问题是时间和锁/页面行为未必满足合同，而非每次 `new` 必然慢。初始化可预留容器、构造对象、预热和触页；运行期复用有界表示。`reserve` 只保证一定容量，超过容量仍可扩张。C++23 没有标准 `inplace_vector`；可用 `array + size` 或经验证的有界容器，手动 `raw storage` 则重新承担对象生命周期责任。

`memory locking` 与 `pre-touch` 把部分页面成本移出关键阶段，但不把任意程序变为 `hard real-time`。官方 `ROS 2 Jazzy` 实时示例源码文档讨论这些条件；本批没有执行该 Linux 示例或其特权操作。

4.2 故障是受控状态，不只是 throw

关键路径可选择不抛接口，但 `noexcept` 遇到未处理异常会 `terminate`，不会自动生成安全命令。`sensor timeout`、非法状态和 `actuator fault` 应输入显式状态机；日志与报告失败也不能反过来阻塞安全路径。

分配审计必须声明观测面。§16 替换可替换的 C++ `allocation functions`，在当前线程的确定性 `control phase` 计数，并用一次显式分配作检测对照；它不拦截第三方 `malloc`、驱动分配、OS 页面活动或其他线程。观察到 0 只支持此范围内的结论，不等于全进程零分配认证。

5. 单位、坐标系与数值表示

5.1 double 没有物理语义

米、弧度、秒、牛顿和力矩都可能表示为 `double`，却不是可互换量。边界把设备单位规范化到约定单位，接口用 `strong units` 或明确 `schema`。位置还必须属于 `frame`；`T_A_B` 需说明是把 B 坐标变到 A，还是相反。静态关节链可借助类型约束，运行时地图/传感器图则需要动态身份和校验，两者不宜无限模板化。

刚体变换包含旋转和平移，四元数要统一 `wxyz/xyzw`、主动/被动旋转、乘法方向和归一化规则。本章保留这些审查责任，不把单自由度实验扩大成 `SE(3)` 数学验证。

5.2 数值库的表达式仍要检查存储与别名

固定维矩阵适合许多小规模控制状态，动态矩阵应在初始化确定形状并复用。固定尺寸也不意味着包含它的所有高层操作都无分配。`Eigen` 风格表达式可能融合计算，也可能为别名、安全或代价生成临时；`noalias` 只能在确实无别名时使用，不能为了提速掩盖 `A = A * B` 的依赖。

`NaN/Inf`、奇异点、数值条件、近零 `dt` 和漂移属于算法边界。`clamp` 不负责把 `NaN` 变成安全数值；写执行器前应先检查 `finite`，再检查 `limits` 与 `freshness`。具体 `Eigen` 行为应按锁定版本回查；官方 `Aliasing` 说明用于机制解释，是 `nightly` 文档而非本机验证版本。本批不编译 `Eigen`。

6. 传感器采集、通道语义与快照回收

6.1 FIFO 与 latest-value 服务不同需求

驱动适配层把 `vendor packet` 转为规范样本，附测量时间、`clock domain` 和 `sequence`。`timestamp` 回答“何时”，`sequence` 帮助发现丢失、乱序和重启；二者不能互相替代。拥塞策略应按数据意义选阻塞、丢旧、丢新或进入 `fault`。

连续位置状态可能只需最新值；逐个排空百毫秒历史会使控制长期落后。`Stop`、`ResetFault`、交易或边沿事件则不能随意覆盖中间项。`latest-value` 与 `FIFO` 是两种合同，不是同一个 `queue` 调小容量。

6.2 原子发布指针没有证明存储可复用

双缓冲先写 `inactive`，再发布索引，只解决发布顺序的一部分。读者可能仍在读 `A`，写者发布 `B` 后立刻复写 `A` 会形成竞争。需要阶段握手、读者 `pin`、引用拥有或回收协议。对非 `atomic payload` 使用“读版本号—复制—再读版本号”，即使最后丢弃副本，也不能消除复制期间已经发生的 C++ `data race`。

实验使用受 `mutex` 保护的完整 `Snapshot`：writer 在锁内发布，reader `try_lock` 成功才整体复制，失败不修改旧值。它证明 `generation` 字段关系，并明确不是 `lock-free` 或 `hard-RT` 方案。真正无等待回收是独立设计问题，不拿一个双缓冲图冒充实现。

7. 估计、控制与硬件边界

7.1 Single writer 保持算法状态一致

估计器的状态向量、协方差和历史由一个执行上下文修改，多传感器输入先处理时钟映射、排序和迟到策略。摄像头 `t0` 的观测在 `t0+50ms` 才处理，算法应决定回溯、预测、丢弃或乱序更新，不能把 `return` 时间替换状态有效时间。

控制器一次 `update` 固定一份状态与目标 `snapshot`；逐字段读取不同 `generation` 可能组成从未存在过的配置。控制器可持有积分/滤波状态，但不直接发 `ROS`、重新解析配置或打开文件。单写者减少锁和一致性负担，不自动证明数值稳定。

7.2 限幅、速率限制与执行器 I/O

`saturation` 限制数值范围，`rate limit` 限制每单位时间变化，积分器还需 `anti-windup` 协调。它们不能用同一个 `clamp` 替代。实验仅用有界 `PD` 控制和单自由度积分 `plant`，不实现 `PID anti-windup`，也不宣称真实机器人稳定。

硬件接口返回规范状态和写入结果，不把 `vendor` 类型扩散进 `core`。若 `read/write` 有可分析上界，合并到一个循环减少 `handoff`；若可能堵塞，则隔离到 I/O 域并为延迟、丢包和 `freshness` 建模。C SDK/驱动边界继续遵守 `G8` 的分配域、异常与句柄规则。

8. 看门狗、故障锁存与命令仲裁

8.1 安全动作必须由物理系统定义

主机可能 `hang`、`crash` 或失联；执行器不能无限保持上一次危险输出。看门狗 (`watchdog`) 应尽量靠近驱动/MCU，并与独立安全链分层。软件 `E-stop topic` 可以传播意图，但不是硬件安全链的替代，也不构成认证结论。

`safe state` 可能是受控制动、失能、保持或其他动作，不能普遍等同“输出 0”。本章仿真把 0 作为模型内安全命令，只用于状态机实验，禁止直接据此驱动硬件。

8.2 状态、授权与超时共同控制输出

用 Initializing/Standby/Active/Fault/Emergency/ShuttingDown 等明确状态代替互相冲突的 bool。某些 fault 应锁存，直到明确复位条件成立；下一帧正常不能自动恢复动力。多个 planner、teleop、校准和安全源先由 arbiter 决定优先级、模式、租约和有效期，再进入控制器。

RT↔Non-RT 通道传经过验证的完整状态、命令或配置。普通 middleware 发布与日志放到非实时端；realtime_tools Jazzy 提供面向这类边界的工具，但使用工具名不自动满足端到端时限。

9. ROS 2、QoS 与借出缓冲区

9.1 集成层不能接管未声明的控制责任

ROS adapter 依赖普通 C++ core，而非 Controller 继承 Node。这样算法可在 unit、replay、仿真和硬件部署间复用。Executor 决定 callbacks 在何时何线程执行；callback groups、组合进程与多线程配置会改变交错，不能把 ROS timer 当作具有硬期限的 motor scheduler。

本章固定以 Jazzy 文档线解释概念，不追逐“当前最新发行版”。RMW、传输、序列化、进程拓扑和具体版本必须随部署记录；ros2_control 管理控制器与硬件生命周期，但应用的分配、阻塞和回调仍由项目分析。configure/activate/read-write/deactivate/error/cleanup 应与物理状态关联。

9.2 QoS 是数据合同，不是安全保证

History/Depth 决定积压语义，Reliability 决定传输保证，Durability 决定迟加入者的历史数据，Deadline/Lifespan/Liveliness 描述发布间隔、数据有效期及活跃性。配对不兼容可能根本不通信。高频 sensor profile 常选择 best effort 和小队列，但具体关键测量应按需求重选；reliable 不等于及时，不等于 actuator watchdog。ROS 2 Jazzy QoS 文档

9.3 Zero-copy 必须说明减少了哪次复制

intra-process 转移、shared-memory transport、middleware loan 是不同路径。借出缓冲发布后所有权归还 middleware，不能继续读取原 loan；若要保留，需要新的合法拥有/复制合同。底层是否支持 loan、是否分配和是否无锁取决于实现，不能用“zero-copy”一词包办。Jazzy RMW 接口合同还区分参数错误的提前失败与已经归还所有权的路径；调用失败后也必须按具体函数合同处理，而非一律继续使用原指针。

少复制可能降低带宽，也会耦合池压力、回收与慢读者。原稿的 loan 规则保留，但本批没有安装 ROS、运行 RMW、验证 QoS 配对或 loan API，属于文档回查而非运行证据。

10. 大载荷、配置事务与安全过滤

10.1 数据布局由访问模式决定

1920×1080×3 字节约 6.22 MB，一次完整复制在 30 FPS 下约 187 MB/s 的数据量，尚未计读写总线流量和额外拷贝。共享不可变帧可服务感知、录制与可视化，但慢读者可能长期占用池；必须限制租约或退化策略。

关节状态若每次同时用位置、速度、力矩，AoS 可能自然；只扫描一个字段时 SoA 值得测量。固定结论“机器人都应 SoA”与 G6 的访问模式原则冲突。

10.2 一次发布一套合法参数

Kp/Ki/Kd 逐个 atomic 更新不保证读者看到同一套参数。非实时端构造、校验并预计算整个 ControllerConfig，再在安全点发布 generation，旧读者结束后回收。目标位置和速度也应同代消费。

安全过滤不仅接受 desired command，还接受 mode、当前状态、限制、故障和时间。合法数值的过期命令也可能危险；invalid input 应确定性地导致 safe command 或 fault，而不是继续上一次控制输出。

11. 故障隔离、观测与测试阶梯

11.1 不让低关键性失败拖垮高关键性路径

感知超时、UI 崩溃和总线故障是不同 failure domain。控制器可按合同使用有限时间的旧状态、预测或安全模式，但不无限等待视觉。RT telemetry 写有界预分配记录，非实时线程格式化/落盘；缓冲满也要有丢弃或降级策略。

观测包括 wake/执行时间、deadline miss、数据 age、乱序/丢失、队列深度、I/O 耗时和故障。平均值、p99 与 max 都来自有限样本，观测 max 不是理论上界。sequence、generation 与各时钟域时间戳共同支持追踪。

11.2 Simulation、replay、HIL 各有不同覆盖

算法单元测试使用确定输入；replay 固定测量时间和事件序列；SIL 连接软件 plant；HIL 再纳入部分真实硬件、驱动和时序；最终真机还要验证物理安全。仿真时间与主机时钟应分开，因此一个 replay 通过不说明部署 jitter 达标。

边界测试覆盖 NaN、Inf、近零/过大 dt、时间跳变、迟到、掉线、限幅、故障锁存和复位。控制 phase 分配计数也是特定观测，不证明所有输入或库路径都无分配。本批只做单自由度 SIL 风格逻辑与桌面时序观察，HIL、ROS、page fault、真实驱动及安全认证全部未验证。

12. 执行等级与集成拓扑

12.1 RT-safe 必须展开为具体条件

RT_SAFE、INIT_ONLY、NON_RT、BLOCKING 可以用作项目标签，但必须附“何平台、何输入规模、何分配/等待/异常限制”。标注 noexcept 或 high priority 都不能替代这些条件。数据通道应同时写明生产/消费频率、允许丢失、最旧可用 age 和关闭后行为。

线程划分来自阻塞、所有权和时限：采集、估计、控制、ROS、遥测和感知可能分开；若 I/O 有界且任务周期一致，read-estimate-control-write 合并又可能更简单。组件同进程减少某些 IPC 成本，却扩大失败耦合；一节点一进程也不是默认正确。

12.2 时间对齐与命令授权

近似传感器同步的 tolerance 影响物理一致性和丢弃率；插值或积分到目标时间是算法选择，不是只加 mutex。估计 state(t_est) 供 t_now 控制时，需判断 age、预测误差和降级条件。命令也有自身生成时间和有效期。

Disabled → Arming → Active、Active → Fault 与 Emergency 转移应指定发起者和条件。Shutdown 的验收终点是执行器进入定义好的状态，不只是线程退出。实验只涵盖 standby/active/fault/shutdown 简化机，未实现多源仲裁或真实 arming 流程。

13. 输出不变量与工程分层

13.1 先检查是否允许动作，再检查数值

允许 actuation 至少需要：Active、状态有效、clock domain 可比较、时间非未来且足够新、目标未过期、输入输出 finite、数值/速率在限制内、执行器健康。异常数值不能通过 clamp 被“洗白”，写入失败也不能只写日志后继续 Active。

安全过滤决定模型内输出，底层 watchdog 处理上层停止更新；二者覆盖不同故障。真实 brake/drive 状态反馈、不可逆硬件动作和功能安全分析不在本章桌面实验里。

13.2 依赖方向与性能优先级

core 放 units/state/estimation/control/safety，runtime 放通道/生命周期，hardware 与 ROS 是 adapter，simulation 消费相同 core。内部同工具链可用 virtual driver；长期第三方二进制边界应回到小而明确的 C 合同，不能把 C++ 类布局当通用 ABI。

优化先修错误时间域、不可控阻塞和无界通道，再查分配/页面、数据复制、缓存布局，最后才是指令级优化。把最快的数学核放在错误执行域，仍不能满足控制系统合同。

14. 可运行学习案例的范围

14.1 robot-control-core 的阶段产物

原项目阶段从纯数学、固定步长 plant、真实时钟循环，扩展到 sensor thread、非实时命令、故障注入、分配审计，最后 ROS adapter。完整实验保留其学习顺序，但明确划出本批切片：一个有界 PD 控制器、一个单自由度积分 plant、独立的快照并发测试、计数插桩及 1000 周期的主机时序记录。

它不是整机调度器。多速率独立 sensor/control/ROS 线程、真实 bus、复杂估计器、ROS 参数生命周期和 HIL 继续是项目练习要求，不会因为 Markdown Edition 完成而变成已经实现。

14.2 三种问题使用三种判据

确定性注入要求非法输入使 fault 锁存且输出为模型定义的 0，正常帧不能自动恢复；显式 reset 后仍需 activate。快照测试要求观测到的所有字段属于同一 generation。分配测试要求当前线程该阶段的可替换 C++ 分配调用计数为 0，并先确认计数器能检测分配。

时序则完整记录 wake offset、执行耗时和 deadline miss，不以固定 ns 阈值判 PASS。安全逻辑通过与 timing observation recorded 是不同状态。G11 的代码、运行平台和未验证部分见 §16 与批次记录。

15. 常见误判与语言选择

15.1 相近的词往往掩盖不同合同

fast 不是 real-time；高优先级不是期限保证；reserve 不是硬容量；接收时间不是测量时间；单位相同不代表同一时钟域；latest-state 不等于必须处理每一帧；reliable 不等于安全；zero-copy 不等于零生命周期成本；shared_ptr 不等于共享写入安全。

同样，controller 直接依赖 vendor packet、关键循环偶尔日志、最后一次 clamp 代替 safety、软件 topic 代替硬件 E-stop、为“微服务风格”拆出大量 ROS 进程，都绕开了真实合同。应逐条回到时限、失效模式和所有权，而非靠框架名称作答。

15.2 语言是工具选择，不是物理保证

C++ 与既有数值库、ROS 和硬件 SDK 集成常有现实价值；Rust 可降低安全代码中的生命周期/数据竞争风险；Zig 的显式资源与 C 互操作有助于某些底层组件。但三者都不能自动证明 deadline、jitter、驱动行为或物理安全，也不能用语言标签代替具体依赖版本与团队能力评估。

更多语言带来 FFI、构建、调试、错误与资源域成本。先把控制 core 与 adapter 边界设计清楚，再考虑替换局部实现，最后在 G12 §11 ~ 12 统一讨论证明责任。

16. 完整实验：逻辑仿真、快照与时序

16.1 分开运行，分开解释

G11-C1 将确定性控制逻辑和主机时序分为两个程序。使用与 G10 同一条 `verify_synthesis.py` 命令，但分类记录，不把模型故障测试、分配计数和 `jitter` 混成一个 PASS。

`control_test` 要求八类注入进入 `Fault`、保持模型定义的 0 输出、正常帧不自动复位、显式 `reset/activate` 与 `shutdown` 生效。10000 个逻辑步检查 `finite`、幅值/速率和受观测的 C++ 分配调用数。并发快照用 `generation` 与所有字段关联，允许跳过中间代，但不允许混代。

两个错误变体分别删除 `freshness` 判定、在发布时破坏字段关系；均要求编译成功后以目标 `invariant` 和固定退出码拒绝。第三个变体在 `step` 内直接调用 `allocation function`，必须被分配计数拒绝。它们检验判据能辨认错误，不是声称原实现含这些错误。

16.2 完整文件

[完整实验 · G11-C1 · control.hpp]

```
#pragma once
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <mutex>
namespace robot {
enum class Mode { standby, active, fault, shutdown };
struct Snapshot {
    std::uint64_t generation{}, sensor_ns{}, command_ns{};
    double position{}, velocity{}, target{};
};
class Mailbox {
    std::mutex mutex_;
    Snapshot value_{};
public:
    void publish(Snapshot value) {
        std::lock_guard lock{mutex_};
        value_ = value;
    }
    bool try_read(Snapshot& output) {
        std::unique_lock lock{mutex_, std::try_to_lock};
        if (!lock.owns_lock()) return false;
        output = value_;
        return true;
    }
};
class Controller {
    Mode mode_{Mode::standby};
    double previous_{};
    static bool valid(const Snapshot& s, std::uint64_t now) noexcept {
```

完整实验 · G11-C1 · control.hpp · continued

```

        return s.generation != 0 && s.sensor_ns <= now && s.command_ns <= now
            && now - s.sensor_ns <= 5'000'000
            && now - s.command_ns <= 50'000'000
            && std::isfinite(s.position) && std::isfinite(s.velocity)
            && std::isfinite(s.target);
    }
    double fault() noexcept { mode_ = Mode::fault; previous_ = 0; return 0; }
public:
    Mode mode() const noexcept { return mode_; }
    bool activate() noexcept {
        if (mode_ != Mode::standby) return false;
        mode_ = Mode::active;
        return true;
    }
    bool reset(const Snapshot& s, std::uint64_t now) noexcept {
        if (mode_ != Mode::fault || !valid(s, now)) return false;
        mode_ = Mode::standby;
        previous_ = 0;
        return true;
    }
    void shutdown() noexcept { mode_ = Mode::shutdown; previous_ = 0; }
    double step(const Snapshot& s, std::uint64_t now, double dt,
        bool actuator_healthy = true) noexcept {
        if (mode_ != Mode::active) return 0;
        if (!valid(s, now) || !std::isfinite(dt) || dt <= 0 || dt > 0.01
            || !actuator_healthy) return fault();
        const double requested = 20 * (s.target - s.position) - 2 * s.velocity;
        if (!std::isfinite(requested)) return fault();
        const double limited = std::clamp(requested, -5.0, 5.0);
        previous_ = std::clamp(limited, previous_ - 100 * dt, previous_ + 100 * dt);
        return previous_;
    }
};
} // namespace robot

```

[完整实验 · G11-C1 · allocations.hpp]

```

#pragma once
#include <cstdint>
namespace allocations {
extern thread_local bool active;
extern thread_local std::size_t calls;
}

```

[完整实验 · G11-C1 · allocations.cpp]

```

#include "allocations.hpp"
#include <cstdlib>
#include <new>
namespace allocations {
    thread_local bool active = false;
    thread_local std::size_t calls = 0;
}
namespace {
    void count() noexcept { if (allocations::active) ++allocations::calls; }
    void* ordinary(std::size_t n) {
        count();
        if (void* p = std::malloc(n ? n : 1)) return p;
        throw std::bad_alloc{};
    }
    void* aligned(std::size_t n, std::align_val_t a) {
        count();
        void* p = nullptr;
        if (posix_memalign(&p, static_cast<std::size_t>(a), n ? n : 1) == 0) return p;
        throw std::bad_alloc{};
    }
}
void* operator new(std::size_t n) { return ordinary(n); }
void* operator new[](std::size_t n) { return ordinary(n); }
void* operator new(std::size_t n, std::align_val_t a) { return aligned(n, a); }
void* operator new[](std::size_t n, std::align_val_t a) { return aligned(n, a); }
void* operator new(std::size_t n, const std::nothrow_t&) noexcept {
    try { return ordinary(n); } catch (...) { return nullptr; }
}
void* operator new[](std::size_t n, const std::nothrow_t&) noexcept {
    try { return ordinary(n); } catch (...) { return nullptr; }
}
void* operator new(std::size_t n, std::align_val_t a, const std::nothrow_t&)
    noexcept {
    try { return aligned(n, a); } catch (...) { return nullptr; }
}
void* operator new[](std::size_t n, std::align_val_t a, const std::nothrow_t&)
    noexcept {
    try { return aligned(n, a); } catch (...) { return nullptr; }
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete[](void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }
void operator delete[](void* p, std::size_t) noexcept { std::free(p); }
void operator delete(void* p, std::align_val_t) noexcept { std::free(p); }
void operator delete[](void* p, std::align_val_t) noexcept { std::free(p); }
void operator delete(void* p, std::size_t, std::align_val_t) noexcept {
    std::free(p); }

```

完整实验 · G11-C1 · allocations.cpp · continued

```

void operator delete[](void* p, std::size_t, std::align_val_t) noexcept {
    ↪ std::free(p); }
void operator delete(void* p, const std::nothrow_t&) noexcept { std::free(p); }
void operator delete[](void* p, const std::nothrow_t&) noexcept { std::free(p); }
void operator delete(void* p, std::align_val_t, const std::nothrow_t&) noexcept {
    std::free(p);
}
void operator delete[](void* p, std::align_val_t, const std::nothrow_t&) noexcept {
    std::free(p);
}

```

[完整实验 · G11-C1 · control_test.cpp]

```

#include "control.hpp"
#include "allocations.hpp"
#include <atomic>
#include <cstdlib>
#include <iostream>
#include <limits>
#include <new>
#include <thread>
void require(bool ok, int code) {
    if (!ok) { std::cerr << "control invariant=" << code << '\n'; std::exit(code); }
}
int main() {
    using robot::Mode;
    constexpr std::uint64_t now = 100'000'000;
    const robot::Snapshot good{1, now, now, 0, 0, 1};
    // Independent injections: stale sensor, stale command, future, NaN, Inf, bad
    ↪ dt/I/O.
    for (int which = 0; which != 8; ++which) {
        robot::Controller c;
        require(c.activate(), 1);
        require(c.step(good, now, 0.001) > 0, 2);
        auto bad = good;
        double dt = 0.001;
        bool io = true;
        if (which == 0) bad.sensor_ns = now - 5'000'001;
        if (which == 1) bad.command_ns = now - 50'000'001;
        if (which == 2) bad.sensor_ns = now + 1;
        if (which == 3) bad.position = std::numeric_limits<double>::quiet_NaN();
        if (which == 4) bad.target = std::numeric_limits<double>::infinity();
        if (which == 5) dt = 0;
        if (which == 6) dt = 0.02;
        if (which == 7) io = false;
        require(c.step(bad, now, dt, io) == 0 && c.mode() == Mode::fault, 21);
        require(c.step(good, now, 0.001) == 0 && c.mode() == Mode::fault, 22);
        require(!c.activate(), 23);
    }
}

```

完整实验 · G11-C1 · control_test.cpp · continued

```

        require(c.reset(good, now) && c.mode() == Mode::standby, 24);
        require(c.activate(), 25);
        c.shutdown();
        require(c.step(good, now, 0.001) == 0 && c.mode() == Mode::shutdown, 26);
        require(!c.reset(good, now) && !c.activate(), 27);
    }
    // Count replaceable C++ allocation calls in THIS thread / phase only.
    allocations::active = true;
    void* probe = ::operator new(8); // Direct call avoids new-expression elision.
    ::operator delete(probe);
    allocations::active = false;
    require(allocations::calls == 1, 28);
    allocations::calls = 0;
    robot::Controller c;
    c.activate();
    double position = 0, velocity = 0, previous = 0;
    bool valid = true;
    allocations::active = true;
    for (std::uint64_t tick = 1; tick <= 10'000; ++tick) {
        const auto stamp = tick * 1'000'000;
        robot::Snapshot s{tick, stamp, stamp, position, velocity, 1};
        const double command = c.step(s, stamp, 0.001);
        valid = valid && std::isfinite(command) && std::abs(command) <= 5
            && std::abs(command - previous) <= 0.1000001;
        previous = command;
        velocity += command * 0.001;
        position += velocity * 0.001;
    }
    allocations::active = false;
    require(valid && c.mode() == Mode::active, 29);
    require(allocations::calls == 0, 30);
    std::cout << "control verified; tracked_cpp_allocations=" << allocations::calls
        << '\n';
    // Generation consistency: all fields are copied while holding the same mutex.
    robot::Mailbox box;
    std::atomic<bool> done{};
    std::atomic<bool> midpoint_seen{};
    std::jthread writer([&] {
        for (std::uint64_t gen = 1; gen <= 10'000; ++gen) {
            box.publish({gen, gen, gen, double(gen), double(gen * 2), double(gen *
                3)});
            if (gen == 5000)
                while (!midpoint_seen.load()) std::this_thread::yield();
        }
        done.store(true);
    });
    std::size_t observed = 0;

```

完整实验 · G11-C1 · control_test.cpp · continued

```

auto inspect = [&](const robot::Snapshot& s) {
    if (s.generation == 0) return;
    ++observed;
    require(s.sensor_ns == s.generation && s.command_ns == s.generation
        && s.position == double(s.generation)
        && s.velocity == double(s.generation * 2)
        && s.target == double(s.generation * 3), 31);
};
robot::Snapshot snapshot{};
do {
    if (box.try_read(snapshot)) inspect(snapshot);
    if (snapshot.generation == 5000) midpoint_seen.store(true);
} while (!done.load());
writer.join();
require(box.try_read(snapshot), 32);
inspect(snapshot);
require(observed > 0 && midpoint_seen && snapshot.generation == 10'000, 33);
std::cout << "snapshot generations verified\n";
}

```

[完整实验 · G11-C1 · timing.cpp]

```

#include "control.hpp"
#include <array>
#include <chrono>
#include <iostream>
#include <thread>
using Clock = std::chrono::steady_clock;
using Ns = std::chrono::nanoseconds;
struct Trace { long long wake_ns, execution_ns; bool missed; };
int main() {
    constexpr int cycles = 1000;
    constexpr auto period = Ns{1'000'000};
    std::array<Trace, cycles> traces{};
    robot::Controller c;
    c.activate();
    double position = 0, velocity = 0;
    const auto start = Clock::now() + std::chrono::milliseconds{5};
    for (int tick = 0; tick != cycles; ++tick) {
        const auto release = start + tick * period;
        std::this_thread::sleep_until(release);
        const auto wake = Clock::now();
        const auto logical = std::uint64_t(tick + 1) * 1'000'000;
        robot::Snapshot s{std::uint64_t(tick + 1), logical, logical,
            position, velocity, 1};
        const double command = c.step(s, logical, 0.001);
        velocity += command * 0.001;
        position += velocity * 0.001;
    }
}

```

完整实验 · G11-C1 · timing.cpp · continued

```

    const auto finish = Clock::now();
    traces[tick] = {std::chrono::duration_cast<Ns>(wake - release).count(),
                    std::chrono::duration_cast<Ns>(finish - wake).count(),
                    finish > release + period};
}
if (!std::isfinite(position) || c.mode() != robot::Mode::active) return 2;
// All formatting is outside the sampled phase. No deadline threshold is a PASS
└─ oracle.
std::cout << "position=" << position << '\n';
for (int i = 0; i != cycles; ++i)
    std::cout << i << ', ' << traces[i].wake_ns << ', '
                << traces[i].execution_ns << ', ' << traces[i].missed << '\n';
}

```

16.3 观察合同

timing 的首行保留最终 plant 状态，后续 1000 行为 tick, wake_offset_ns, execution_ns, deadline_missed。绝对计划时刻为 $\text{start} + \text{tick} \times 1\text{ms}$ ；迟到时继续执行下一逻辑步，可能出现连续赶进度。本实验没有丢周期、重同步或真实控制补偿策略，记录应据此解释。

执行器计算 p50/p99/max 和 miss count，记录为 OBSERVED；无论本机 jitter 大小都不声称 hard real-time。程序正常返回仅检查模型状态与有限值。没有 affinity、mlock、特权调度、真实传感器、ROS、HIL 或硬件 actuation；也没有检测整个进程 malloc 或 page fault。

17. 机器人路径复核记录

17.1 十六个审查维度

维度	应留下的回答
Physical / units / frame	物理量、规范单位、坐标变换方向
Time / freshness	时钟域、测量时刻、age 与失效界限
Ownership / writer	buffer owner、借用期限、状态修改者
Execution / blocking	关键域、等待点、平台假设
Memory / bounds	分配面、容量、循环上界
Numerics / safety	finite、稳定性条件、fault 与安全动作
Shutdown / middleware	停止后硬件状态、传输与回收合同
Measurement	采样条件、时间分布和未覆盖路径

17.2 本批的结论边界

本章可执行部分检验简化控制模型、锁存、`generation` 和可替换 `C++` 分配调用；采样的是本机调度，不是 `Linux` 实时部署。生成一致性由 `mutex` 整体复制提供，`TSan` 只补充动态观察，不能据无报告证明双缓冲无锁协议。零命令仅是仿真约定，不能当作真机操作建议。

18. Final Gate

先闭卷写出条件、机制与反例，再核对下一节；保留原稿问题，不在题干下先给结论。

18.1 Real-Time

1. Fast 和 real-time 有什么根本区别？
2. Period、deadline、jitter 分别是什么？
3. 为什么 average latency 不能证明 real-time behavior？
4. SCHED_FIFO 为什么不等于 hard-real-time guarantee？

18.2 Time

1. measurement timestamp 和 receive timestamp 有什么区别？
2. 两个相同单位的 timestamp 为什么仍可能不能相减？
3. steady_clock 适合解决什么问题？
4. delayed camera observation 进入 estimator 时为什么不能假装是“现在”？

18.3 Memory

1. 为什么 RT loop 避免 general heap allocation？
2. reserve() 为什么不等于 fixed-capacity guarantee？
3. pre-touch 解决的是哪类成本？
4. 为什么 C++23 中不能直接假定有 `std::inplace_vector`？

18.4 Concurrency

1. 为什么 single-writer state 对机器人特别有价值？
2. latest-value channel 与 FIFO queue 适合的语义有什么区别？
3. 为什么 double-buffer 仍然有 lifetime/reuse 问题？
4. 为什么 RT loop 通常不应等待 non-RT mutex？

18.5 Numerics

1. 为什么裸 double 在 robot API 中容易制造单位错误？
2. coordinate frame 为什么属于 type/API semantics？
3. quaternion 使用时至少要统一哪些 conventions？
4. fixed-size numerical representation 为什么常适合 control core？

18.6 Control

1. 为什么 Controller 应该使用 consistent snapshot?
2. saturation 和 rate limiting 分别限制什么?
3. 为什么 stale command 可能比 numeric out-of-range 一样危险?
4. cancellation/fault 发生时 controller 应该如何保持 invariant?

18.7 Hardware

1. 为什么 vendor SDK type 不应进入 domain core?
2. hardware read 可能阻塞时为什么要重新考虑 thread topology?
3. actuator watchdog 为什么应该尽量靠近硬件?

18.8 ROS 2

1. 为什么 ROS callback executor 不应该自动等价于 motor-control scheduler?
2. QoS 为什么必须由 message semantics 决定?
3. SensorDataQoS 为什么典型采用 best-effort/small depth?
4. loaned message publish 后为什么不能继续使用?
5. 为什么普通 ROS publisher 不适合直接放进 hard-RT update loop?

18.9 Safety

1. 为什么 software topic 不能替代真正安全 E-stop architecture?
2. fault 为什么通常应该进入显式 state machine?
3. 如何定义“允许向 actuator 输出 command”的完整 invariant?

19. Final Gate · 参考答案与常见误判

答案按上一节分组和题号对应。重点是推理与适用条件，不把关键词复述当作通过。

19.1 Real-Time

1. **fast** 只说明某种耗时统计；**real-time** 要在条件下满足时限/可预测性。短平均值不能担保每次截止期限。
2. **period** 是计划 **release** 间隔，**deadline** 是完成期限，**jitter** 是指定事件相对计划时刻的偏移变化；必须说明测的是 **wake** 还是完成。
3. 平均抹去尾部，有限样本 **max** 也不是 **WCET**。需要平台/负载假设、阻塞上界和部署证据。
4. 调度策略不消除中断、固件、**page fault**、锁、驱动和硬件干扰；优先级提升也可能伤害其他关键线程。

19.2 Time

1. 前者描述物理采样时刻，后者描述传输后的接收时刻；用后者替代会把可变通信延迟混入估计。
2. 单位相同仅表示刻度，原点、速度和跳变规则可能不同；先建立 **clock-domain** 转换及误差。
3. 适合进程内持续时间、超时和相对调度，不受 **wall-clock** 调整的同类影响；不能自动对应设备测量时钟。
4. 估计值属于过去，需按算法回溯、预测、乱序更新或丢弃。假装现在会把时延误差变成状态误差。

19.3 Memory

1. 问题在于 **allocator** 及页面路径的时间边界不明，不是每次分配都慢。初始化分配与运行期回收也要分别审查。
2. **reserve** 后还能超过 **capacity**；真正有界接口应拒绝超限或用固定表示。**array** 同时意味着所有元素已构造。
3. 把首次触页等部分成本提前，不证明以后没有页面、缓存或 **OS** 干扰；需在目标平台验证。
4. 它不属于 **C++23** 标准设施。应选 **array+size** 或明确依赖的静态容器，不把未来设施写成本基线。

19.4 Concurrency

1. 估计器/控制器每份可变状态有一个写者，减少复合不变量与锁域；读者仍需合法快照与生命周期。
2. 连续状态允许跳过中间代时适合 **latest**；每个事件都有意义或顺序要求时用 **FIFO**。不能仅凭频率决定。
3. 发布新的索引后旧读者可能仍访问原槽；重用必须等读者结束或有回收协议，**release/acquire** 本身不够。
4. 持锁者可能因非实时工作长期停顿，引入优先级反转。**try_lock**+旧值策略也需 **freshness**，且不是自动硬时限证明。

19.5 Numerics

1. `double` 不区分角度、弧度、长度和力矩；单位进入类型/schema 才能限制误用。数值相等不是物理相同。
2. 同一个三元组在不同 `frame` 含义不同；变换方向和可组合性属于接口正确性，不只是注释。
3. 至少规定元素顺序、乘法方向、主动/被动约定及单位长度维护；否则数学形式相同仍可能旋转反向。
4. 尺寸和工作量容易限定，减少动态 `resize`；但其他表达式、临时或库路径仍需检查，不是全链零分配保证。

19.6 Control

1. 分开读取可能拼出不同 `generation` 的状态/目标，控制计算需固定同代输入；`snapshot` 自身的取得也要合法。
2. `saturation` 限幅值，`rate limiting` 限随时间变化；积分控制还要 `anti-windup`，不是一次 `clamp` 能完成。
3. 目标可能已不适合当前状态，数值合法也可能产生危险动作；有效期与授权是输出条件。
4. 保持 `fault` 锁存和模型定义的安全输出，停止非法更新，显式复位再激活。真实 `safe state` 由物理系统决定。

19.7 Hardware

1. `vendor` 单位、布局与时钟应在 `adapter` 规范化，否则算法/仿真被硬件接口绑住。
2. 阻塞会侵蚀控制期限；可隔离 I/O，但新增 `handoff`、延迟和时钟责任。若 I/O 有界，单循环可能更合理。
3. 主机挂死时上层软件已无法自救，靠近 `actuator` 的独立 `watchdog` 可在失联时执行本地策略；仍需硬件安全设计。

19.8 ROS 2

1. `Executor` 为 `callbacks` 提供执行机制，不自动给出 `motor loop` 期限保证；多线程/group 还会改变并发交错。
2. 不同数据允许的丢失、过期、积压和迟加入行为不同；`reliable` 并非所有场景最合适，兼容性也要核对。
3. 常重视及时最新测量而非补齐每帧；关键低频传感仍可能需要其他合同，不能机械照搬。
4. 发布归还 `loan` 的所有权，继续访问不在原借用范围；需要按该 `RMW/API` 合同重新获得合法对象。
5. 普通发布路径可能涉及不可控分配、锁和 I/O；关键端应通过经过分析的 `handoff` 交给非实时端。

19.9 Safety

1. 软件消息仍依赖进程、调度、通信和供电，无法覆盖它们共同失效；真正 **E-stop** 需要独立、经风险分析的安全链。
2. 状态机把进入、锁存、复位和授权条件显式化，避免多个 **bool** 形成非法组合，也便于注入复核。
3. **Active**、有效且新鲜状态/目标、可比较时钟、非未来时间、**finite**、幅值/速率限制和健康 **actuator** 同时成立；否则进入明确安全策略。

20. 参考资料与验证边界

ROS 概念回查固定 Jazzy 文档线：[QoS](#)、[实时示例](#)、[Controller Manager](#)、[realtime_tools](#)。这些是概念与部署回查，不是本机安装/运行记录。实验工具链、原始时序、覆盖面与未知项见批次记录及原始结果。

引用版本与证据解释统一见全书约定。下一章 [G12 §12](#) 把时间、资源与接口问题合并为审查协议；仿真所得结论仍限于这里声明的模型和平台。

G12 · 统一系统模型与工程审查

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述，不是当前出版状态。正式版出版身份及批准范围以随附发布记录为准。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。Unified Systems Model；承接 G0 ~ G11，Rust/Zig 仅作第二层机制对照。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G11 · 全系列导航

阅读入口

先用 §1 ~ 12 重建 C++ 系统模型，再做 §18 的综合审查。Rust/Zig 用来识别证明责任的差异，不要求并行学习三门语言，也不新增独立语言实验。

原稿的 Complete/Frozen 是历史编辑标记，不沿用为技术验收。编号主题以 g12-topic-N 映射到相应主题组，保留技术去向；章节不再逐个复制原有 Part。完整实验与机制片段有可见身份，Gate 答案是普通章节。

- 1. 全书的终点：分配证明责任
- 2. 对象、存储与清理边界
- 3. 所有权、借用、别名与值传递
- 4. 分配与失败：机制不等于事务
- 5. 泛型、分派与容器抽象
- 6. 机器成本与并发协议
- 7. 二进制与构建：保证到哪里结束
- 8. 编译期元数据与外部生成
- 9. 时限、嵌入式与性能证据
- 10. 安全抽象、状态与 API 合同
- 11. 子系统选择与跨语言成本
- 12. 可迁移的系统审查协议
- 13. 四类项目的对照推理
- 14. 长期维护与常见错误推论
- 15. 三类论证与六张关系图
- 16. 跨章应用与语言反思
- 17. 可带入新项目的最终检查

- 18. 审查练习：把同一套模型用于三个案例
- 19. 全书回查与证据分层
- 20. Final Gate
- 21. Final Gate · 参考答案与常见误判
- 22. 参考资料与验证边界

1. 全书的终点：分配证明责任

1.1 先识别系统问题，再讨论语言

本章不是 C++、Rust、Zig 功能排名。C++ 是正文主线；另两种语言帮助区分偶然的语言机制与真实系统复杂性。C++ 的值类别、重载规则和文本包含有明显语言历史；但所有权转移、借用失效、分配拓扑、跨核顺序、二进制合同和时限并不会因为换语言而消失。

统一问题是：系统要求什么，哪个层次表达它，谁负责证明？类型检查、库封装、架构不变量、动态检测和部署观察不是互相替代的“安全分数”。选择语言是选择一部分证明责任（proof burden）的分配，不是把整个系统正确性外包给编译器。

1.2 比较的基线与限制

C++ 基线为 C++23/N4950；Rust 保留 2024 edition 对照口径，所有权、布局与 unsafe 概念回查固定到 1.85.0 随附官方文档，读取日期与摘要见引用约定；Zig 机制回查固定 0.15.2 文档，不使用含糊的“Modern Zig”版本。本批不运行 Rust/Zig 编译器，所有比较均为解释模型，不构成交叉语言兼容性或性能验证。

C++ 提供 RAII、值类型、模板与机器表示控制，但许多非法访问要由程序员排除；safe Rust 把更多借用和线程共享合法性放进静态约束，仍依赖 unsafe 实现的健全性 (soundness)；Zig 的 allocator、错误和清理表达更显式，却不提供同样的 borrow-checking 保证。不要把示意风格画成优劣坐标。

2. 对象、存储与清理边界

2.1 用 C++ 对象模型打开问题

storage 是存放对象的资源，object 是具有类型与生命周期的语言实体，value 是其状态；三者不能互换。取得足够对齐的存储不普遍意味着已经构造任意 T；手工构造、销毁与复用继续受 G1 的规则约束。Rust 的 value/place 与 Zig 的 typed storage 不是逐字对应 C++ 标准术语，但都要回答初始化、访问有效期和存储回收。

owner.reset 后旧指针仍有地址，并不意味着对象仍活着。RAII 可以可靠安排 owner 的释放，却不能自动证明所有 borrower 都已结束。Rust 静态借用规则能排除许多此类 safe-code 路径；Zig 的 defer 让配对可见，却不会检查整个借用图。

2.2 类型驱动与作用域驱动的清埋

C++ 的析构函数和 Rust Drop 可把资源清理能力绑定到拥有类型；Zig 常在获取资源旁用 defer/errdefer 安排清理。类型携带 deinit 方法不意味着 Zig 会自动调用它，复制一个资源结构体也不自动取消原值的清理责任。

这些是正常语言控制流下的机制，不是“任何进程终止都会执行清理”的保证。C++ terminate、Rust panic=abort、进程被杀或显式泄漏均需单独处理。不要用 cleanup 示例推导持久化事务、设备断电或远端副作用一定回滚；FM 失败模型专门展开这些边界。

3. 所有权、借用、别名与值传递

3.1 可写权限比指针形状更重要

C++ 的 value、unique_ptr、shared_ptr、引用和 span 可表达不同意图；裸 T* 仍可能表示 owner、nullable borrow 或外部句柄，必须有合同。span<const T> 只禁止经该视图修改元素，不保证底层不可被其他别名修改，更不会延长 owner 生命周期；它与仅将视图对象声明为 const span<T> 不同。

Rust &T 的共享访问需考虑 UnsafeCell 内部可变性，不能粗暴说“所有内容永远 immutable”；&mut T 提供受规则约束的独占访问。unsafe/raw pointer 并不免除别名与有效性义务，精确 unsafe 模型也不能被几句口号替代。Rust Reference

Zig 的 []const T、[]T、指针形状表达访问方式与长度，但不静态证明整个 borrower graph。三种 slice 在机器层可能都类似地址+长度，不能据此跨 ABI 假定布局，也不能认为语言保证相同。

3.2 三种“移动”不能直接翻译

C++ std::move 是表达式转换，实际调用由重载决议决定；用户定义的 move 可转移资源，也可没有期望的廉价行为，移后源对象还存在并受其类型合同约束。Rust 非 Copy 值移动使原 binding 不再可用，不是调用用户 move constructor；Copy 也不是“成本必定很小”的性能保证。

Zig 普通赋值没有 Rust 式源值不可用的所有权状态转移。拥有资源的 struct 若普通复制后两边都 deinit，可能重复释放。语言不同，但审查仍应画 owner、借用期限、别名和写入权限（mutation authority），再检查复制/移动究竟改变了哪条边，详见 G2 与 G3。

4. 分配与失败：机制不等于事务

4.1 谁拥有存储策略

C++ vector/PMR 与 Rust Vec/Box 常把普通分配隐藏在 owner 内；Zig 常把 allocator 参数显式传给需要动态内存的函数。显式分配能力允许调用者选择 arena、池或固定缓冲，但 API 出现 allocator 不意味着内部不会分配、不会锁或一定更快。反过来，高层 owner 简化使用，却要求另标注容量、重用和分配失败边界。

分配拓扑要同时写明发生频率、由谁释放、何时回收和能否有界。C++ resource 必须比使用它的 PMR 容器活得久；传入 allocator 和保存 allocator 都有生命周期义务。机器成本取决于实际策略，而非语言名。

4.2 错误通道没有自动回滚

expected、Result 与 Zig error union 都可把预期失败作为值传播；C++ exceptions 提供另一传播通道，panic/terminate/abort 又是不同级别的停止策略。取消是控制结果，不能无条件合并为业务错误；bug/invariant failure 与可恢复失败应分开处理。

先修改 A，再修改 B，最后返回错误，三种语言都不会自动恢复 A。强事务保证仍需要 prepare → commit、补偿或明确部分提交合同。析构/Drop/defer 能处理资源，却不普遍撤销外部动作。跨边界转换错误时还要定义输出是否有效、是否重试以及谁已接收责任。

5. 泛型、分派与容器抽象

5.1 编译期特化共享同一种成本取舍

C++ templates/concepts、Rust generics/traits 与 Zig comptime 都能让静态已知信息形成专门代码；具体机制、约束检查和实例化时机不同。C++ concepts 不证明任意运行时语义，Rust trait bounds 不自动证明业务合同，comptime 也不意味着所有成本免费。

特化可能减少运行时分支并促进内联，同时增加构建时间、二进制体积、调试复杂度与指令缓存压力。动态 virtual、trait object、函数表或类型擦除可能更紧凑，但带来间接分派及生命周期接口。选择依据实际变化维度，参见 G5。

5.2 相似容器背后仍有失效规则

vector、Vec 与 allocator-backed dynamic array 都可能拥有连续存储；span、slice 则借用范围。增长引发重分配会改变地址，稳定地址、稳定索引和稳定逻辑身份不是同一要求。Rust 约束可阻止很多借用期间增长的 safe-code 错误，C++/Zig 需要明确 API 失效规则。

“指针、大小、容量”是理解成本的模型，不是所有容器必须具有这三字段、可以跨语言 memcpy 的布局规范。跨 ABI 只传双方正式约定的表示，不能用容器相似性跳过 G8。

6. 机器成本与并发协议

6.1 类型安全不能取消缓存与同步成本

连续性、对齐、working set、指针追逐、TLB 和 AoS/SoA 取决于具体访问路径。shared_ptr、Arc 或自建原子引用计数都可能付出间接访问和缓存行竞争；安全拥有不是 contention-free。零额外抽象成本也不意味着所选算法、分配和同步本身免费，仍要测具体实现，详见 G6。

6.2 数据竞争安全不等于协议正确

C++ 通过 `mutex/atomic` 建立 `happens-before`；safe Rust 的 `Send/Sync` 与借用约束排除一大类非法跨线程访问，但不能自动排除死锁、错误关闭顺序、过载或原子状态机的逻辑错误。实现 `Send/Sync` 的 `unsafe` 责任不能交给命名猜测。Rustonomicon

单写者、分片、不可变 `generation` 与消息传递能简化三种语言中的共享图。队列交接仍有接收点、失败、背压和析构责任；C++ 的 `std::move` 不像 Rust 一样禁止随后使用源变量。原子 `acquire/release` 只在具体读写关系满足条件时发布数据，不能把“用了 `acquire`”当普遍 `barrier`。

6.3 进展与回收是独立证明

`lock-free` 描述系统进展，不保证每个线程在限定步骤内完成，也不是性能排名。并发节点何时可回收仍需要 `hazard`、`epoch`、引用拥有或其他协议；语言静态借用不能自动表达所有动态回收算法。Rust `unsafe` 将人工义务局部化，但仍须保持有效值、生命周期和别名不变量；C++/Zig 同样应缩小低层实现边界，而非让裸指针遍布业务。

7. 二进制与构建：保证到哪里结束

7.1 C ABI 是可约定的底层边界，不是安全证书

C++ 与 Rust 原生 ABI 都不应被当作跨任意工具链版本的长期通用合同，Zig `native` 表示也不能直接等同 C。明确目标平台的 C ABI、版本化结构、不透明句柄和创建者销毁接口能缩小互操作表面，但仍须规定大小/对齐、空值、范围、借用、回调、线程与错误。

Rust `raw pointer` FFI 无法由编译器推断 C 库是否保留指针、能否并发销毁；错误的 `safe wrapper` 可以把不成立的 `unsafe` 假设暴露给所有调用者。C++ `private vector` 成员仍参与公开类布局，只有真正隐藏对象或合适 `PImpl` 等边界才能隔离，不能把 `private` 误认成 ABI 不可见。参见 G8。

7.2 工具整合度不同，依赖图仍存在

CMake/Ninja 与包管理器分工，Cargo 整合 `package/build/test` 流程，Zig `toolchain` 提供构建图工具。它们都有制品节点、输入与依赖边；遗漏生成器输入、工具身份或链接依赖，仍会造成陈旧或不完整产物。

C++ 文本包含、宏配置、翻译单元与历史 ABI 增加一些语言特有复杂性；Rust `cfg` 与 Zig `comptime/build options` 用不同机制表达配置，但不会消除条件组合测试。源码能编译、能链接、能运行、能安装消费是不同 Gate，详见 G9。

8. 编译期元数据与外部生成

8.1 不把所有 schema 塞进类型系统

静态 DBC/schema、协议表和多车型配置可以用 C++ constexpr/生成源码、Rust const/build script/procedural macro、Zig comptime/外部生成表达。问题不是哪种语法最短，而是源数据身份、诊断位置、生成文件可审阅性与编译成本是否可控。

大量外部元数据用独立生成阶段有利于区分解析错误和语言编译错误，也便于 diff 生成结果。若使用语言内特化，则要限制生成量并记录输入依赖。G9 的 build graph 原则继续适用，不能因为 generator 写在语言工具链里就不声明依赖。

8.2 泛型预算是长期维护预算

每个变体都可能增加诊断、链接、缓存和 code-size 成本。选择表驱动还是专门代码，应把版本更新、测试覆盖与可观察机器结果一起比较。本章不增加新的 decoder 实现；将此前知识压缩成设计选择问题，而不是再建一套特化框架。

9. 时限、嵌入式与性能证据

9.1 没有 GC 只是一个条件

C++ 的析构、Rust 的 Drop、Zig 的 defer 都可能执行有成本的清理；通用 allocator、OS 调度、锁与 I/O 仍可能无可用上界。memory-safe 或 allocation-explicit 都不是 deadline 证明。嵌入式语言选择还受目标工具链、vendor SDK、no_std/runtime 支持、调试与认证约束。

实时需求必须在平台、负载、输入上界与故障模型下分析，不能用桌面 microbenchmark 代替。G11 的 timing observation 与物理时限论证分开，是这个原则的具体例子。

9.2 性能比较必须比较同一个命题

语言不能统一预测最快实现。算法、布局、分配、检查消除、编译器、并发拓扑和工作负载共同决定结果。Rust 的边界检查可能消除也可能保留；C++ 抽象可能优化掉也可能导致额外工作；Zig 显式代码也可能有差的局部性。

用等价功能和失败边界比较，保留原始时间、参数和校验值。specialization 同时影响编译时间、代码大小与 I-cache；不能只比较执行热点忘记交付成本。本批没有三语言 benchmark、嵌入式或跨平台运行。

10. 安全抽象、状态与 API 合同

10.1 将非法状态排除到哪一层

enum/variant、Rust enum 与 Zig tagged union 都能把状态相关数据绑在一个合法分支；但转换是否合法仍需要业务条件。nullable、optional、owned、borrowed 是不同维度，不能让一个裸指针无声地承担所有意义。

Rust 的静态保证依赖正确的编译器、库与 unsafe 封装，并非“任何写着 Rust 的系统都不会出内存错误”；C++ 工程约定与 sanitizer 有价值，却不是同等静态保证；Zig 显式检查也不能自动证明生命周期。低层实现边界应小而可审查，外部 API 必须让安全使用者不容易破坏内部不变量。

10.2 借用输入、拥有输出是同一个系统合同

[机制片段 · C++ 接口合同示意，不承诺独立编译]

```
std::expected<std::vector<float>, DecodeError>
decode(std::span<const std::byte> input);
```

这里输入只借用到调用返回，输出 owner 负责释放，但签名本身仍未说明别名限制、长度上界和分配失败。Rust 的 Result<Vec<_>, _> 以另一套类型约束表达拥有结果；Zig 的 ![]f32 则还需要写清结果由哪个 allocator 释放，或用显式 deinit 的 owner 结构封装。不能把任意 slice 都叫 owned。

内部可以使用丰富语言抽象，稳定外部用小 C adapter 映射 status、输出和释放接口。便利 API 与分配可见性有取舍，关键是用户能知道调用期间哪些资源和责任发生变化。

11. 子系统选择与跨语言成本

11.1 根据依赖与风险选局部边界

已有 C++ SDK、数值库和原生生态可能使 C++ 最直接；复杂不可信输入及共享生命周期可能提高 Rust 静态约束的价值；显式 allocator、C glue 和小型底层工具可能契合 Zig。它们是评估维度，不是“控制一定 C++、服务一定 Rust、驱动一定 Zig”的固定分工。

生态、团队熟悉度、debugger、构建支持、认证和五年维护成本都要进入决策。旧系统若已经满足合同，仅因语言更新而重写没有自动收益；也不能用既有生态为所有内存风险免责。

11.2 FFI 要翻译语义，不只翻译类型

跨语言增加工具链、包生态、CI、调试、错误与所有权翻译。边界宜粗且稳定，减少每几个操作就在语言间往返。Creator destroys 便于维持分配域；跨边界保留内存用显式 handle 或 release callback；调用期 borrow 不能被后台偷偷保存。

exception/Result/error union 可转为 C status 与输出，但输出有效性和重试责任必须一起翻译。Send/Sync 不会穿过 void* 自动生效，同一 handle 是否可并发、destroy 是否需要静默期、callback 在何线程都要公开。语言保证、库抽象、架构不变量是三个层次，不能把其中一个当作全部。

12. 可迁移的系统审查协议

12.1 用可回答的问题建立记录

面对新项目,按如下顺序写出具体对象、边和证据,而不是勾“已考虑”。缺少证据可以记 UNKNOWN,不能根据语言选择补成 PASS。

审查层	必须回答	需要的产物
State / lifetime	什么状态,何时失效	对象与生命周期图
Owner / borrow	谁释放,谁临时访问	拥有/借用期限
Mutation / alias	谁可写,路径是否重叠	writer 与别名合同
Storage / allocation	大小、频率、上界、回收	资源预算与观测
Failure / commit	失败前后谁已负责	状态机与提交点
Concurrency / overload	顺序、等待、容量、唤醒	同步和关闭论证
ABI / build	什么跨独立制品边界	接口与消费实验
Timing / evidence	期限和已知证据范围	条件化观察及缺口

12.2 语言选择放在约束之后

C++ 编译器检查类型、约束与部分静态语义,不完整证明动态指针合法性; Rust 提供更多所有权/共享约束,也不自动检查业务账目或调度; Zig 的类型、comptime 与模式相关检查同样不是全系统证明器。

单位、正确 shard key、过载策略、交易提交和 actuator 安全仍需领域定义。随后才选择哪些义务交给类型系统、哪些交给库、哪些需审查和运行证据。这里的“证明”是明确前提下的论证,不暗示本手册已经完成形式化验证。

13. 四类项目的对照推理

13.1 同一协议,不同风险权重

高吞吐二进制 decoder 应先写清输入不可信程度、边界检查、schema 身份、输出所有权和 C ABI。C++ 的 span/生成表、Rust 的 slice/Result、Zig 的 allocator/comptime 都能表达实现;选择取决于已有系统和风险,不先作速度排名。

并发网络服务还需任务拥有、取消、超时和背压; memory safety 可以降低攻击面,却不能防止永不关闭的连接队列。机器人 core 增加测量时间、单位、硬件 watchdog 与部署时限;小型原生工具增加目标支持、C 库和可分发构建。四者不能由同一“语言得分表”解决。

13.2 成本问题要落到实现

问字节数、分配次数、分支、cache miss 和同步操作，而不是“哪门语言最快”。hash map 的具体实现、安全策略和数据分布可能比语言本身影响更大。原子计数、连续数组和间接图也只有在布局和访问一致时才可比较。

选择语言实际上决定一部分风险由编译器排除、一部分通过清晰 API 限制、一部分靠测试和运行管理。它不是无代价转移，仍要记录替代方案与缺失证据。

14. 长期维护与常见错误推论

14.1 系统由团队和时间共同塑造

代码审查能力、工具链支持、依赖健康、入职成本、调试和升级能力，都影响长期风险。C++ 团队需要明确 owner/borrow 与构建规范；Rust 团队仍需审查 unsafe、async 和 FFI；Zig 团队仍需统一 allocator 与清理协议。语言简洁不自动带来一致架构。

小而版本化的 C 边界可以作为渐进替换点，但替换实现仍要验证语义、并发、分配域和旧消费者。API 长寿、可调试性与迁移成本通常值得和今天的几个百分点性能一起权衡。

14.2 不接受从局部优势推到整体正确

Memory-safe 程序仍可死锁、错算金额或发错力矩；显式代码仍可显式写错 lifetime；RAII 仍允许悬挂 borrow；borrow checker 不能推断所有外部资源合同；无 GC 不等于 real-time；C ABI 不等于安全边界；多语言不必然降低复杂度。

工程上应要求每个强结论指出前提、机制、反例与证据，而非“某语言所以不需要检查”。本章不发布语言选型排名，也不倡导为了风格纯度整体重写。

15. 三类论证与六张关系图

15.1 语义、安全和运行合同分开

语义论证回答输出/状态是否正确；安全论证回答执行是否违反内存、生命周期和共享规则；运行合同论证回答资源、过载、故障与时限是否满足部署需求。编译接受、sanitizer clean、吞吐更高分别只覆盖这些维度的一部分。

六张图分别是 ownership、lifetime、mutation、storage/cost、synchronization 与 binary-boundary。一个 Buffer 的 private vector 管存储，span 借用，扩容使旧路径失效，线程写入需要权限，类是否跨 ABI 暴露仍取决于外部表示。这比只认识 class/vector/span 的语法多出整套工程含义。

15.2 不可变配置仍要发布与回收

C++ atomic<shared_ptr<const Config>>、Rust 受同步保护的 Arc generation、Zig 明确的 pointer/reclamation 都须完成“私下构造 → 发布 → 旧读者结束 → 回收”。没有任何一个语法自动证明控制线程的最后释放有界。

借用输出接口、显式 allocator 及拥有型结果的对照，应回到这些图检查。G12 不再添加重复编译例子，而在 §18 用 G10/G11 的实际协议和证据缺口练习整套审查。

16. 跨章应用与语言反思

16.1 G10 与 G11 共享底层责任

G10 无论由哪种语言实现，仍要保证 accepted work 有结果、队列有界、abort 唤醒、线程先退出再销毁依赖；G11 在此之上增加单位、时间域、freshness、故障锁存与 safe actuation。改变语言不会消除这些架构边。

语言能减少一类非法程序的可表达空间，仍不能决定 capacity 是否符合负载、deadline 是否可满足、业务事件是否允许丢失。比较 Rust/Zig 应帮助发现 C++ 接口中未表达的义务，不是把它们加入本手册当作新课程。

16.2 从比较反过来改进 C++ 设计

从 Rust 学到的是缩短可变借用、使 owner 可见、用 variant/expected 减少非法状态；从 Zig 学到的是分配能力、失败与低层机制应在关键边界可见。C++ 也提醒其他语言：ABI、最终布局、析构顺序和原生工具链都是现实约束。

选择 subsystem 语言时同时看 ecosystem、safety、ABI、performance、timing、ownership、allocation、concurrency、tooling、team、certification 和 evolution，不简单给星级。保持 C++ 主线不是忽略其他设计，而是用同一套问题检验它们。

17. 可带入新项目的最终检查

17.1 十二个问题形成一个可复用入口

对 Buffer、Connection、Job、RobotState、Decoder 或 Queue，回答：它是什么；住在哪里；谁拥有；谁借用；何时结束生命周期；什么使已有路径失效；复制/移动成本；分配在哪里；谁可写；跨线程顺序如何建立；什么跨 ABI；有何时限要求。

回答要包含真实对象和条件，不能只写“RAII”“atomic”“C ABI”。若某题依赖外部模块，需要列出对方的合同及失效后果；若未测量，记录缺口而非猜测。这样语言选择才是实现工具，系统约束才是主线。

17.2 正文到 G12 封顶

本书不再追加 G13。后续维护围绕实际使用、全书一致性和出版视图，不扩成新百科。原稿末尾的全系列 COMPLETE 只表示主题目录闭环，不能覆盖当前审核状态、平台限制和 PDF 未构建事实。

G0 ~ G12 已接受源稿与本轮 Editorial Sweep 修订按系列状态分开：本轮形成待集中审核的内容冻结候选，未启动 PDF Pilot 或正式发布。学习成果应体现为能解释一个真实设计的责任与反例，而不是不断增加章节。

18. 审查练习：把同一套模型用于三个案例

18.1 案例 A：运行时的账本为什么可能自洽却错误

给定实现。提交者把任务放入队列后再写 `accepted`；`worker` 可能已经结束。汇总只用 `succeeded + failed + cancelled` 推算 `accepted`，再验证二者相等。

独立推理。列出提交到消费的线性化点，寻找状态覆盖交错；指出从同一组终态算两边属于循环判据。应保留调用者实际 `Accepted` 返回记录、每个 ID 的预期和结果值；`accepted` 必须在消费者可见前登记。`drain` 后的未决项应暴露，而非悄悄归入 `cancelled`。

对应证据。G10 协议给出锁内提交；运行时实验记录调用者数量、逐 ID 结果与提前关闭 `output` 的错误变体。只支持这些输入与调度观察，不是持久化 `exactly-once` 或形式化证明。

18.2 案例 B：机器人“时间没问题”的错误推论

给定声明。某次 1 kHz 仿真平均执行很快，`allocation counter` 为 0，TSan 没有报告，于是控制器被称为 `hard-real-time safe`。

独立推理。平均耗时没有包含 `release jitter` 和最坏等待；计数器只观察当前线程的可替换 C++ `allocation functions`；TSan 不检查时限、单位或物理动作。还需明确平台、调度、驱动、页面、`freshness`、实际 `safe state` 和独立 `watchdog`。

对应证据。G11 时序与边界区分确定性模型与 `timing observation`。结论应拆为“模型注入满足判据”“观测到该范围分配数”“记录本机 `jitter`”“硬实时/HIL/物理安全 NOT VALIDATED”。没有把 1000 个采样改写为普遍上界。

18.3 案例 C：Rust 包装层为什么仍需审查 C++ 库

给定接口。C 库返回 `opaque handle`，包装者因为指针只是地址而为 `wrapper` 声明可跨线程共享；`destroy` 可能与后台 `callback` 并发。

独立推理。地址可复制不代表被指对象可并发访问；`Send/Sync` 的安全前提必须来自库合同。应确定回调线程、停止注册与静默点、句柄销毁前在途调用是否结束、谁释放 `callback context`，以及异常/`panic` 如何限制。无法确认则不能给 `safe API` 承诺该共享能力。

对应证据。G8 回调与销毁提供审查模型。本批没有执行 Rust FFI、异步 `callback` 或动态卸载测试，案例 C 仅为推理练习。不能把前两例的 C++ 编译结果借来填补此缺口。

19. 全书回查与证据分层

19.1 十三章是一条责任链

章节	应能回答的系统问题
G0 / G1	机器制品如何形成，对象何时合法存在
G2 / G3	谁管理资源，值怎样传递及付出什么成本
G4 / G5	数据如何访问，哪些变化在编译期表达
G6 / G7	机器成本与跨执行者共享如何分析
G8 / G9	二进制合同与可消费构建如何成立
G10 / G11	组件怎样组合，时间和物理约束怎样加入
G12	哪些义务由语言承担，哪些仍留给系统

19.2 四类证据不能压成一格

文档结构与链接只证明可导航和源稿约束；C++ 编译及目标负例检查某些静态/运行命题；性能观察描述特定输入与环境；并发动态检测观察插桩路径。G12 再用审查案例解释这些证据各自能推出什么。

本章没有独立实验执行器，也没有把解释题统计成技术测试数量。原始 G12 的全部 Final Gate 问题保留为下一节，参考答案独立呈现，便于先闭卷推理。

20. Final Gate

先闭卷写出条件、机制与反例，再核对下一节；保留原稿问题，不在题干下先给结论。

20.1 Object / Lifetime

1. C++、Rust、Zig 在 lifetime proof responsibility 上最大的区别是什么？
2. 为什么 Zig defer 和 C++ RAII 不是完全同一种 abstraction？
3. 为什么 Rust borrow checker 并没有消除 lifetime 这个系统问题？

20.2 Ownership

1. C++ unique_ptr、Rust ownership 与 Zig 手动 ownership convention 有什么根本差别？
2. Rust move 和 C++ move 为什么不能简单翻译成同一概念？
3. 为什么 Zig resource struct 普通复制可能需要特别谨慎？

20.3 Borrowing

1. span<const T>、&[T]、[]const T 机器层可能很接近，为什么语言保证却差异巨大？
2. Rust &mut T 对 aliasing 表达了什么重要思想？

20.4 Allocation

1. Zig allocator-as-parameter 的架构价值是什么？
2. 为什么 allocator 显式不自动意味着更高性能？
3. C++/Rust 隐藏 allocator 有什么 API 优势和可观测性代价？

20.5 Error

1. expected<T,E>、Result<T,E> 和 Zig error union 的共同系统模型是什么？
2. 为什么 error-as-value 不能自动提供 strong transactional guarantee？

20.6 Genericity

1. C++ template、Rust generic、Zig comptime 最终有什么共同机器 trade-off？
2. 为什么更多 compile-time specialization 可能伤 I-cache？

20.7 Concurrency

1. Rust safe code 解决了哪些 C++ concurrency 错误？
2. 为什么 Rust 仍然需要理解 acquire/release 和 CAS？
3. 为什么 single-writer 是三门语言共同的重要架构原则？

20.8 ABI

1. 为什么三门语言之间最稳健的长期 ABI 仍然通常是 C ABI?
2. Rust 的安全 guarantees 为什么不能自动穿过 FFI?
3. 为什么 ownership/thread-safety 必须写进 ABI contract?

20.9 Build

1. Cargo、Zig Build、CMake 最大的生态模型差异是什么?
2. 为什么它们底层都仍然可以理解成 artifact graph?

20.10 Real-Time

1. 为什么 C++、Rust、Zig 都不能自动保证 real-time?
2. 为什么没有 GC 不是 sufficient condition?

20.11 Architecture

1. 什么时候应该选择 C++?
2. 什么时候 Rust 的 static guarantees 价值尤其高?
3. 什么类型的问题特别适合 Zig 的显式 allocator/C interop/comptime 模型?
4. 多语言系统的真正成本是什么?
5. 为什么 language choice 应该是 systems-design procedure 的后半部分，而不是第一步?

21. Final Gate · 参考答案与常见误判

答案按上一节分组和题号对应。重点是推理与适用条件，不把关键词复述当作通过。

21.1 Object / Lifetime

1. C++ 主要靠对象规则、RAII 和 API 合同；safe Rust 静态检查更多借用关系；Zig 更依赖显式清理与工程约定。都仍有运行环境和低层边界义务。
2. RAII 把清理绑定到拥有类型的析构，defer 常绑定词法控制流。可调用 deinit 的类型也不自动在离域时执行，二者不能按名字等同。
3. 静态检查限制可表达的借用，但架构仍决定谁拥有、多久有效、外部资源何时失效；FFI/unsafe 又增加人工义务。

21.2 Ownership

1. unique_ptr 是 C++ 库的独占 owner，类型支持但不检查所有借用；Rust move/borrow 是核心规则；Zig 资源责任由 API 和显式清理约定维持。
2. std::move 不本身转移资源，C++ move 调用后源对象仍存活；Rust 非 Copy 值移动使源不可用，且没有用户 move constructor 协议。
3. 普通复制可能产生两个指向同一资源的值，若两边均清理会重复释放。要定义逻辑转移并限制误用，不能以复制语法推断独占。

21.3 Borrowing

1. 相似地址+长度模型不等于相同生命周期/别名静态约束，也不等于正式 ABI 布局兼容。
2. 它表达受规则约束的独占访问；不应再有冲突访问路径。unsafe 与内部可变性需按具体规则审查，不能只背“一个可变引用”。

21.4 Allocation

1. 显式传入动态存储能力，让调用者选择策略、预算与寿命；依然要保证 allocator 比使用它的对象有效得更久。
2. 实际策略可能仍是锁竞争严重的通用堆，也可能有很多小分配；显式只改善可见性，不决定机器成本。
3. 拥有容器封装清理、简化调用，但调用点未必看到分配/回收。性能或时限敏感 API 应另说明成本与复用合同。

21.5 Error

1. 都可表达成功值或预期错误，并通过控制流传播；取消、panic/terminate 和基础设施灾难仍需独立分类。

2. 错误值不撤销已发生的修改。准备所有会失败的工作后提交，或定义补偿/部分提交，才是状态保证。

21.6 Genericity

1. 静态特化可能减少运行时工作，同时增加编译、链接和代码体积；语言机制不同但预算相同。
2. 多个相似专门版本增加代码足迹，可能挤出热点。更易内联也可能更大，需测实际二进制和工作负载。

21.7 Concurrency

1. 在 `sound` 的实现前提下，借用和 `Send/Sync` 约束排除许多非同步共享、悬挂和非法跨线程访问；不保证所有业务并发结果正确。
2. 原子协议仍要证明发布、读到哪个写入、状态转换和回收；`safe` 使用 `atomics` 也能写出逻辑错误或死锁。
3. 减少同一状态竞争写入，使复合不变量与缓存成本更易分析；跨线程读者仍需同步和生命周期合同。

21.8 ABI

1. 目标平台 C ABI 可形成更小、明确的共同合同，避免语言私有对象/异常布局；仍须锁定平台和版本，不是普遍稳定魔法。
2. `raw pointer` 和外部回调没有携带可由编译器验证的完整 `owner`/线程合同，`wrapper` 必须人工证明 `unsafe` 前提。
3. 类型签名不能说明保留多久、谁释放、并发何时合法或何时销毁；漏掉这些会让双方各自“正确”却集成出错。

21.9 Build

1. `Cargo` 更整合包工作流，`Zig` 提供工具链构建模型，`C++` 常由 `CMake`/生成器/包管理器协作；具体能力与版本相关。
2. 都有源、生成物、库、工具与依赖边。集成度高也不会自动知道未声明的输入或外部环境。

21.10 Real-Time

1. 时限取决于工作上界、调度、页面、锁、I/O 和硬件；语言保证只覆盖其中部分表达与安全性。
2. 无 GC 仍可能因 `allocator`、析构、引用回收、缺页、锁或设备阻塞而迟到。`absence of GC` 不是完整时间分析。

21.11 Architecture

1. 既有 C++/原生库、硬件 SDK、数值生态和接口成本主导，且团队能维持所需证明责任时；不是无条件因性能标签选择。
2. 不可信输入或复杂共享生命周期带来高内存风险，依赖与团队又能支持时；仍需设计资源与业务协议。
3. allocator 策略、C 互操作、静态元数据和小型底层组件很关键时值得评估；不忽略生态、目标支持和维护成本。
4. 多套工具链、包生态、调试、部署、FFI、错误/所有权翻译与升级矩阵；边界收益必须大于这些成本。
5. 先知道资源、失败、并发、二进制与时限约束，才知道哪些语言保证真正有价值。个人语法偏好不能替代需求。

22. 参考资料与验证边界

C++ 语义使用 N4950 及前面章节的定向条款；Rust 比较回查 [Reference](#) 的未定义行为边界、[Send/Sync](#)、[Ownership](#)；Zig 回查 0.15.2 文档。Rust 2024 是 edition，不是具体编译器版本。本批未执行 Rust/Zig，不声明这些机制已在本机编译验证。编辑去向、审查练习与平台边界见批次记录。

引用版本与证据解释统一见全书约定。正文在 G12 结束。可沿 §12 的审查协议用于新项目；后续是全书一致性和出版视图，不增加新的技术章。